МОДУЛИ ПО ШАГАМ

Полное учебное руководство по архитектуре и исходному коду

Подробный маршрут каждого модуля, разбор функций внутри этапов, смысл проверок и используемые конструкции языка C. Руководство сверено с фактическими исходными файлами проекта.

Для кого написана глава

Глава рассчитана на читателя, который только начинает изучать C, устройство PE-файла и архитектуру анализатора. Поэтому новое понятие сначала объясняется по смыслу, затем показывается его место в проекте и только после этого разбирается код.

Главный принцип чтения

Сначала следует увидеть движение данных через модуль целиком. После этого внутри каждого этапа разбираются функции, которые технически выполняют действие. Такой порядок не позволяет потеряться в отдельных строках кода и одновременно не скрывает механику реализации.

Формула руководства

```
командная строка
-> байты файла (Buf)
-> модель PE (PeImage)
-> RUNTIME_FUNCTION[] и UnwindSummary[]
-> HeuristicReport и DynamicReport
-> CSV, JSON, HTML и Ghidra CSV
```

Кликабельное оглавление

Название раздела и номер страницы являются ссылкой. В большинстве программ просмотра PDF слева также будет доступна панель закладок.

Кликабельное оглавление 2

Перед кодом: что необходимо понять с нуля 25

1. Что такое файл и байт . . . 25
2. Что означает binary и binary file . . . 25
3. Что такое PE . . . 25
4. Заголовок и секция . . . 25
5. Три вида адреса: file offset, RVA и VA . . . 25
6. Что такое функция, стек и регистр . . . 26
7. Что такое exception и EH . . . 26
8. Что такое unwind и metadata . . . 26
9. Что такое static analysis . . . 26
10. Что такое dynamic analysis . . . 26
11. Что такое parser, decoder, heuristic и report . . . 27
12. Главная идея проекта . . . 27

Часть I. Карта проекта в голове 28

Одна фраза о назначении каждого блока . . . 28

Что означает «строить карту» . . . 28

Главный принцип . . . 29

Полный путь данных: от команды до отчёта 30

Карта main.c: кто управляет последовательностью 33

Маршрут . . . 33

Почему main.c не выполняет эти работы параллельно . . . 33

Какие ветви необязательны . . . 33

Что main.c хранит одновременно . . . 33

Карта util.c: как файл становится безопасными байтами **34**

Две разные роли памяти . . . 34

Почему little-endian выделен отдельно . . . 34

Вторая ветвь util.c . . . 34

Карта pe.c: как строится PE-контейнер **35**

Почему секция имеет два диапазона . . . 35

Что именно является картой . . . 35

Что передаётся unwind.c . . . 35

Карта unwind.c: как внутри PE появляется EH/Unwind **36**

Почему три массива связаны одним индексом . . . 36

Почему одна операция может занимать несколько slots . . . 36

Что проект знает и чего не знает после unwind.c . . . 36

Карта heuristics.c: как карта превращается в findings **37**

Finding и общий verdict - не одно и то же . . . 37

Severity и confidence отвечают на разные вопросы . . . 37

Почему score не является вероятностью . . . 37

Карта dynamic_win.c: две ветви наблюдения **38**

Ветвь A: статически найденные unwind handler-ы . . . 38

Ветвь B: точки наблюдения на Windows API . . . 38

Эти ветви дополняют друг друга . . . 38

Динамика подробно: почему 0xCC не ломает команду **39**

Почему RIP корректируется . . . 39

Зачем нужен single-step . . . 39

Что означает instrumentation_complete . . . 39

Динамика подробно: какие аргументы читаются 40

Что ещё фиксирует debug loop . . . 40

Ограничение вывода . . . 40

Карта report.c: как модели становятся документом 41

Почему форматов несколько . . . 41

Что report.c может сопоставлять . . . 41

Карта app_info.c и заголовочных файлов 42

app_info.c . . . 42

include/*.h . . . 42

Связь обучающего и пофайлового уровней . . . 42

Часть II. Структуры, которыми обмениваются модули 43

Сначала четыре базовых понятия C . . . 43

Типы чисел . . . 44

Ключевая связь массивов . . . 44

Args - проверенная конфигурация одного запуска 45

Место структуры в потоке . . . 45

Поля структуры . . . 46

Buf - ограниченное представление входных байтов 47

Место структуры в потоке . . . 47

Поля структуры . . . 47

StrBuf - расширяемая строка с состоянием ошибки 48

Место структуры в потоке . . . 48

Поля структуры . . . 48

PeSection - одна секция в двух системах координат 49

Место структуры в потоке . . . 49
Поля структуры . . . 49

PeImage - карта внешнего PE-контейнера 50

Место структуры в потоке . . . 50
Поля структуры . . . 51

RuntimeFunction - связь функции с её unwind metadata 52

Место структуры в потоке . . . 52
Поля структуры . . . 52

UnwindParseOpts - политика одного запуска decoder-а 53

Место структуры в потоке . . . 53
Поля структуры . . . 53

UnwindSummary - результат разбора одной UNWIND_INFO 54

Место структуры в потоке . . . 54
Поля структуры . . . 55

HeuristicFinding и HeuristicMitigation 57

Место структуры в потоке . . . 57
Поля структуры . . . 57

HeuristicReport - сводная статическая оценка 58

Место структуры в потоке . . . 58
Поля структуры . . . 59

DynamicEvent - одно наблюдение debug session 61

Место структуры в потоке . . . 61
Поля структуры . . . 61

DynamicReport - модель одной динамической сессии 62

Место структуры в потоке . . . 62

Поля структуры . . . 63

ReportMetadata - происхождение отчёта 65

Место структуры в потоке . . . 65

Поля структуры . . . 65

Как структуры образуют одну модель 66

Почему report.c получает почти все объекты . . . 66

Почему структуры не объединены в один огромный объект . . . 66

Внутренние структуры dynamic_win.c 67

ModInfo и ModList . . . 67

Breakpoint и BpList . . . 67

Внутренние структуры переустановки breakpoint 68

Связь с ASLR и forwarded exports . . . 68

Внутренние структуры heuristics.c 69

Почему используется hash map . . . 69

Почему диапазоны сортируются в копии . . . 69

Внутренние структуры report.c 70

Граница этих структур . . . 70

Внутренняя структура SHA-256 71

Зачем она не находится в util.h . . . 71

Флаги и перечисления, управляющие смыслом структур 72

UnwindFlags . . . 72

UnwindAnom . . . 72

HeuristicEvidenceFamily . . . 72

Результаты чтения . . . 72

Часть III. Пофайловое устройство проекта 73

Как пользоваться этой частью . . . 73

Все модули и их взаимосвязь 74

Главная цепочка данных . . . 74

Три типа связей . . . 74

Что является общим результатом системы . . . 74

main.c - координатор всего анализа 75

Что получает модуль . . . 75

Что модуль делает с полученными данными . . . 75

Что получается на выходе . . . 75

Связь со следующим этапом . . . 75

Граница ответственности . . . 76

Как это выглядит в одном анализе . . . 76

util.c - безопасная основа для байтов, строк и файлов 77

Что получает модуль . . . 77

Что модуль делает с полученными данными . . . 77

Что получается на выходе . . . 77

Связь со следующим этапом . . . 77

Граница ответственности . . . 78

Как это выглядит в одном анализе . . . 78

pe.c - построение координатной карты PE 79

Что получает модуль . . . 79

Что модуль делает с полученными данными . . . 79

Что получается на выходе . . . 79

Связь со следующим этапом . . . 79

Граница ответственности . . . 80

Как это выглядит в одном анализе . . . 80

unwind.c - извлечение и декодирование EH/Unwind 81

Что получает модуль . . . 81

Что модуль делает с полученными данными . . . 81

Что получается на выходе . . . 81

Связь со следующим этапом . . . 81

Граница ответственности . . . 82

Как это выглядит в одном анализе . . . 82

heuristics.c - аналитическая интерпретация структурных наблюдений 83

Что получает модуль . . . 83

Что модуль делает с полученными данными . . . 83

Что получается на выходе . . . 83

Связь со следующим этапом . . . 83

Граница ответственности . . . 84

Как это выглядит в одном анализе . . . 84

dynamic_win.c - наблюдение одного выполнения под Windows 85

Что получает модуль . . . 85

Что модуль делает с полученными данными . . . 85

Что получается на выходе . . . 85

Связь со следующим этапом . . . 85

Граница ответственности . . . 86

Как это выглядит в одном анализе . . . 86

report.c - преобразование внутренних моделей в отчёты 87

Что получает модуль . . . 87

Что модуль делает с полученными данными . . . 87

Что получается на выходе . . . 87

Связь со следующим этапом . . . 87
Граница ответственности . . . 88
Как это выглядит в одном анализе . . . 88

app_info.c - происхождение и воспроизводимость результата 89

Что получает модуль . . . 89
Что модуль делает с полученными данными . . . 89
Что получается на выходе . . . 89
Связь со следующим этапом . . . 89
Граница ответственности . . . 89
Как это выглядит в одном анализе . . . 89

Заголовочные файлы include/*.h - контракты между модулями 90

Что получает модуль . . . 90
Что модуль делает с полученными данными . . . 90
Что получается на выходе . . . 90
Связь со следующим этапом . . . 90
Граница ответственности . . . 90
Как это выглядит в одном анализе . . . 90

ghidra/import_unwind_csv.py - внешний мост к Ghidra 91

Что получает модуль . . . 91
Что модуль делает с полученными данными . . . 91
Что получается на выходе . . . 91
Связь со следующим этапом . . . 91
Граница ответственности . . . 91
Как это выглядит в одном анализе . . . 91

Как модули совместно обрабатывают одну функцию 92

1. pe.c устанавливает координаты . . . 92

2. unwind.c создаёт запись функции . . . 92

3. heuristics.c интерпретирует наблюдения . . . 92

4. dynamic_win.c при необходимости проверяет достижимость . . . 92

5. report.c сохраняет доказательную цепочку . . . 92

Главный вывод . . . 92

1. Модуль как часть системы . . . 93

2. Вход, выход и контракт . . . 94

3. Две структуры результата . . . 95

15. Сервисная функция pe_find_section_for_rva . . . 109

16. Сервисная функция pe_rva_to_fileoff . . . 110

17. Освобождение карты: pe_free . . . 112

18. Ошибочная модель и аналитическая модель . . . 113

19. Как результат используют остальные модули . . . 114

20. Один пример от RVA до байта файла . . . 115

21. Полный маршрут pe.c простыми словами . . . 116

22. Как объяснить модуль на защите . . . 117

23. Проверка понимания . . . 118

Системная архитектура: как модули образуют один анализ 119

Полный маршрут всей системы . . . 119

Этап 1. Получить внешний запрос . . . 119

Этап 2. Создать контролируемое представление файла . . . 120

Этап 3. Доказать структуру PE . . . 120

Этап 4. Построить карту EH/Unwind . . . 121

Этап 5. Сформировать аналитические наблюдения . . . 121

Этап 6. Добавить сведения реального запуска . . . 122

Этап 7. Выдать проверяемый результат . . . 122

Том II. EH/Unwind Analyzer: полная инженерная модель проекта 123

Что именно должно быть понято во втором томе . . . 123

Предметная область проекта . . . 123

Граница продукта . . . 124

Архитектурный тип . . . 124

Модули и их строгие контракты . . . 125

Почему CLI и ядро разделены . . . 127

Главные структуры как состояния анализа . . . 127

Почему используются параллельные массивы . . . 128

Ownership graph . . . 128

Error model . . . 129

Почему analysis продолжается при локальной ошибке . . . 130

Доказательная модель . . . 130

Сквозной инвариант неизменности входа . . . 130

Связь модулей с теоретическим фундаментом . . . 131

Как читать последующие главы . . . 131

Модуль main.c: управление полным жизненным циклом анализа 132

Полный маршрут модуля . . . 132

Этап 1. Разобрать команду . . . 132

Этап 2. Загрузить исследуемый файл . . . 133

Этап 3. Построить PeImage . . . 133

Этап 4. Прочитать таблицу функций . . . 134

Этап 5. Подготовить результаты для каждой функции . . . 134

Этап 6. Разобрать все UNWIND_INFO . . . 135

Этап 7. Выполнить выбранные виды анализа . . . 135

Этап 8. Подготовить происхождение результата . . . 136
Этап 9. Записать отчёты . . . 136
Этап 10. Освободить ресурсы . . . 137

Том II. main.c: полный жизненный цикл одного анализа 138

Роль main.c . . . 138
Входной интерфейс процесса . . . 138
Внутренняя структура Args . . . 139
Почему parsing arguments отделён . . . 139
Позиционный input . . . 140
Разделитель -- . . . 140
Проверка чисел . . . 140
Диапазоны options . . . 141
Exit code CLI parsing . . . 141
Чтение input . . . 141
Error buffers . . . 142
PE parsing . . . 142
Runtime function table . . . 142
Allocations для per-function state . . . 143
Options unwind parser . . . 143
Основной цикл unwind . . . 143
Почему операции могут разбираться повторно . . . 144
Heuristic stage . . . 144
Dynamic stage . . . 145
Outputs CSV . . . 145
Подготовка provenance . . . 145
Redaction arguments target . . . 146

SHA-256 . . . 146
JSON и HTML . . . 146
Финальный cleanup . . . 146
Полный автомат состояний main . . . 147
Tests, относящиеся к main . . . 147
Что можно улучшить . . . 148
Проверка полного понимания main . . . 148

Модуль util.c: безопасные байты, строки, файлы и SHA-256 149

Полный маршрут модуля . . . 149
Этап 1. Открыть файл . . . 149
Этап 2. Выделить и заполнить память . . . 150
Этап 3. Проверять каждое чтение . . . 150
Этап 4. Собрать числа little-endian . . . 151
Этап 5. Создать расширяемую строку . . . 151
Этап 6. Записать файл атомарно . . . 152
Этап 7. Вычислить SHA-256 . . . 152

Том II. util.c: основание безопасного parser и генерации отчётов 153

Почему util.c является частью доказательства корректности . . . 153
Buf как read-only view . . . 153
buf_init_from_file . . . 153
Определение размера . . . 154
Пустой файл . . . 154
Полнота fread . . . 154
buf_free_owned . . . 155
buf_read . . . 155
Typed read functions . . . 155

le16, le32, le64 . . . 156
StrBuf . . . 156
Почему error state sticky . . . 156
Рост capacity . . . 157
sb_appendn . . . 157
sb_appendf . . . 157
Атомарная запись . . . 158
SHA-256 . . . 158
Связь util.c с отчётом и tests . . . 158
Полная роль модуля . . . 159

Модуль unwind.c: от Exception Directory к UNWIND_INFO 160

Полный маршрут модуля . . . 160
Этап 1. Найти физические байты Exception Directory . . . 160
Этап 2. Определить количество записей . . . 161
Этап 3. Прочитать RUNTIME_FUNCTION . . . 161
Этап 4. Найти UNWIND_INFO конкретной функции . . . 162
Этап 5. Разобрать поля заголовка . . . 162
Этап 6. Декодировать все UNWIND_CODE . . . 163
Этап 7. Учесть выравнивание хвоста . . . 163
Этап 8. Интерпретировать tail по Flags . . . 164
Этап 9. Проследовать по цепочке безопасно . . . 164
Этап 10. Сохранить объяснимый сбой . . . 165

Том II. unwind.c: от таблицы функций к модели восстановления стека 166

Почему Windows x64 хранит таблицу функций . . . 166
RuntimeFunction . . . 166
Чтение exception directory . . . 166

Почему reader не проверяет begin/end сразу . . . 167
UnwindSummary . . . 167
Register encoding . . . 168
Flags . . . 168
Вход unwind_parse . . . 168
Инициализация и локализация ошибки . . . 169
Options . . . 169
Alignment UNWIND_INFO . . . 169
Mapping header . . . 169
Header layout . . . 170
Version policy . . . 170
Frame register checks . . . 170
CountOfCodes . . . 170
Structure slot . . . 171
Определение длины operation . . . 171
UWOP_PUSH_NONVOL . . . 171
UWOP_ALLOC_SMALL . . . 172
UWOP_ALLOC_LARGE . . . 172
UWOP_SET_FPREG . . . 172
UWOP_SAVE_NONVOL . . . 173
UWOP_SAVE_NONVOL_FAR . . . 173
XMM saves . . . 173
V1 opcodes 6 и 7 . . . 173
V2 epilog descriptors . . . 173
UWOP_PUSH_MACHFRAME . . . 174
Защита суммы fixed stack . . . 174

Tail после slots . . . 174
Следование по chain . . . 175
valid и anomalies . . . 175
parse_error . . . 176
Сквозной sample . . . 176
Связь с heuristics . . . 176
Tests . . . 177
Ограничения decoder . . . 177

Модуль heuristics.c: от наблюдений к объяснимым findings 178

Полный маршрут модуля . . . 178
Этап 1. Получить готовые факты . . . 178
Этап 2. Проверить карту функций . . . 179
Этап 3. Проверить размещение metadata . . . 179
Этап 4. Проверить результаты декодера . . . 180
Этап 5. Оценить reuse и dominance . . . 180
Этап 6. Создать finding . . . 181
Этап 7. Начислить score . . . 181
Этап 8. Применить факторы ослабления . . . 182
Этап 9. Сформировать итог . . . 182

Том II. Глава 6. heuristics.c: от структурных фактов к оценке свидетельств 183

Место модуля в системе . . . 183
Что означает слово heuristic . . . 183
Выходная модель . . . 184
Валидация arguments и initialization . . . 184
Evidence families . . . 185
Frequency map . . . 185

Первый проход по RUNTIME_FUNCTION 186
Placement unwind metadata 187
Placement handler 187
Parse failure и anomaly 188
Overlap и duplicate begin 188
Ratios 188
Score saturation 189
PE structural findings и веса 189
Empty exception table 189
Процентные integrity thresholds 190
Unwind anomaly threshold 190
Table order findings 190
Strong unwind reuse 191
Handler dominance с benign-context suppression 191
Moderate unwind reuse требует corroboration 192
Placement weights 192
Combination findings 192
Finding storage limit 193
Severity counts и evidence level 194
Confidence calculation 194
Weakening rules 195
Manual review flag 196
Verdict 196
Фактический benign sample 196
Фактический suspicious sample suspicious_unwind_oob.exe 197
Фактический suspicious sample suspicious_unwind_writable.exe 197

Тесты модуля . . . 197
Что следует улучшать в будущей версии . . . 198
Итоговая модель главы . . . 198

Модуль dynamic_win.c: Windows debugger и runtime evidence 199

Полный маршрут модуля . . . 199
Этап 1. Проверить платформу и подготовить команду . . . 199
Этап 2. Создать процесс под управлением debugger . . . 200
Этап 3. Получить базу загруженного образа . . . 200
Этап 4. Найти адреса наблюдаемых API . . . 201
Этап 5. Поставить breakpoint . . . 201
Этап 6. Обрабатывать debug event loop . . . 202
Этап 7. Отличить обычное исключение от собственного breakpoint . . . 202
Этап 8. Выполнить исходную команду после breakpoint hit . . . 203
Этап 9. Переустановить breakpoint после single-step . . . 203
Этап 10. Считать параметры вызова API . . . 204
Этап 11. Зафиксировать handler hit и RIP trace . . . 204
Этап 12. Остановить эксперимент контролируемо . . . 205

Том II. Глава 7. dynamic_win.c: Windows dynamic evidence collector 206

Назначение и граница платформы . . . 206
Defensive intent . . . 206
Модель результата . . . 207
Module map . . . 207
Разрешение forwarded API . . . 208
Наблюдаемые API . . . 208
Breakpoint model . . . 208
Запись bytes в remote process . . . 209

Установка breakpoint . . . 209
Self-check . . . 210
Per-thread pending re-arm . . . 210
Командная строка target . . . 210
Создание процесса . . . 211
Static handler addresses как input dynamics . . . 211
Debug loop и timeout . . . 212
Continuation status . . . 212
CREATE_PROCESS_DEBUG_EVENT . . . 212
LOAD_DLL_DEBUG_EVENT . . . 213
Ordinary exception event . . . 213
Initial system breakpoint . . . 213
Hit собственного INT3 . . . 214
Capture регистрации VEH . . . 214
Capture VCH . . . 215
Capture UEF . . . 215
Capture RtlAddFunctionTable . . . 215
Capture callback table . . . 216
Handler hits . . . 216
EXCEPTION_SINGLE_STEP . . . 216
Thread exit . . . 217
Process exit . . . 217
Session finalization . . . 217
Semantics dynamic signal count . . . 218
Runtime evidence level . . . 218
Structured event limitations . . . 218

Safety and trust boundary . . . 219
Bundled dynamic smoke targets . . . 219
Фактическая Windows validation 8 июня 2026 года . . . 220
Что сделано правильно . . . 220
Что остаётся улучшить . . . 221
Итоговая state machine . . . 221

Модуль report.c: доказательная модель и внешние отчёты 222

Полный маршрут модуля . . . 222
Этап 1. Проверить входную модель . . . 222
Этап 2. Подготовить безопасные строки . . . 223
Этап 3. Записать CSV . . . 223
Этап 4. Собрать JSON . . . 224
Этап 5. Связать статические и динамические данные . . . 224
Этап 6. Собрать HTML . . . 225
Этап 7. Записать результат атомарно . . . 225

Том II. Глава 8. report.c: отчёты, итоговая assessment и интеграционный контракт 226

Отчёт не является форматированным printf . . . 226
Brief console report . . . 226
CSV report . . . 227
Ghidra CSV . . . 227
Anomaly text . . . 228
Report metadata . . . 228
Metadata validation . . . 229
Model validation . . . 229
JSON escaping . . . 229
JSON numeric representation . . . 230

Top-level JSON model . . . 230
pe_validation . . . 230
sections . . . 231
heuristic_analysis . . . 231
dynamic_validation . . . 232
Functions . . . 232
Итоговая static/dynamic correlation . . . 233
Dynamic signal count . . . 233
Runtime evidence level . . . 233
Case strength . . . 234
Final confidence . . . 234
Final verdict . . . 234
Manual validation plan . . . 235
HTML security model . . . 235
HTML sections . . . 236
HTML status nuance . . . 236
Atomic file write . . . 236
Memory failure behavior . . . 237
JSON schema . . . 237
Как backend должен вызывать core . . . 237
Рекомендуемая database model . . . 238
Сравнение analyses . . . 238
API adapter не должен менять core . . . 239
Что сделано правильно . . . 239
Что улучшить . . . 239
Итог . . . 240

Модуль app_info.c: происхождение результата 241

Маршрут модуля . . . 241

Развёрнутое объяснение маршрута . . . 241

Функции версии и схемы . . . 242

Функции параметров сборки . . . 242

Функция app_utc_timestamp . . . 242

Заголовочные файлы: публичные контракты модулей 244

Как компилятор связывает .h и .c . . . 244

include/util.h . . . 244

include/pe.h . . . 244

include/unwind.h . . . 244

include/heuristics.h . . . 244

include/dynamic.h . . . 244

include/report.h . . . 244

include/app_info.h . . . 245

#pragma once и extern C . . . 245

Полный навигатор функций 246

Функции src/main.c 246

Функции src/util.c 249

Функции src/pe.c 254

Функции src/unwind.c 256

Функции src/heuristics.c 262

Функции src/dynamic_win.c 270

Функции src/report.c 278

Функции src/app_info.c 302

Сквозной прогон: один файл от команды до отчёта 304

Шаг 1. Принять команду . . . 304

Шаг 2. Загрузить файл . . . 305

Шаг 3. Построить PE-карту . . . 306

Шаг 4. Прочитать .pdata . . . 307

Шаг 5. Подготовить результаты .xdata . . . 308

Шаг 6. Декодировать UNWIND_INFO . . . 309

Шаг 7. Построить статическую интерпретацию . . . 310

Шаг 8. При необходимости выполнить динамический эксперимент . . . 311

Шаг 9. Сформировать provenance . . . 312

Шаг 10. Сериализовать результаты . . . 313

Шаг 11. Зафиксировать технический статус . . . 314

Шаг 12. Освободить ресурсы . . . 315

Итоговая модель проекта 317

Как формируется аналитический вывод 318

1. От факта к finding . . . 318

2. Почему 27 правил не означают 27 findings в каждом отчёте . . . 319

3. Evidence и семейства доказательств . . . 320

4. Severity: насколько значимо наблюдение . . . 321

5. Score: инженерная сумма весов . . . 322

6. Suspicion: нормированное представление score . . . 323

7. Confidence: насколько убедительна доказательная база . . . 323

8. Mitigation: почему вывод следует ослабить . . . 324

9. Evidence level . . . 324

10. Verdict и requires_manual_review . . . 325

11. Полный расчётный пример . . . 325

12. Пример слабого изолированного сигнала 326

13. Как объяснить результат комиссии 326

Перед кодом: что необходимо понять с нуля

Этот раздел не требует знания C, Windows или анализа бинарных файлов. Его задача - дать значения словам, которые дальше используются при объяснении модулей.

1. Что такое файл и байт

Файл - это конечная последовательность байтов. Байт - целое число от 0 до 255, физически состоящее из восьми битов. Сам байт не сообщает, что он означает. Одни и те же четыре байта можно принять за число, часть текста, машинную команду или поле заголовка. Значение появляется только тогда, когда известны формат и положение байтов внутри этого формата.

Проект сначала видит исследуемую программу именно так: адрес первого байта и общее количество байтов. Он ещё не знает, где код, где таблица функций и где сведения об обработке исключений.

2. Что означает binary и binary file

Binary переводится как «двоичный». Binary file - двоичный файл, внутреннее устройство которого задаётся форматом, а не обычными строками текста. В этом руководстве словом «бинарник» обозначается исследуемый EXE или DLL. Проект не получает исходный код программы: он исследует уже собранный машинный файл.

3. Что такое PE

PE - Portable Executable, «переносимый исполняемый формат». Это формат исполняемых образов Windows. Слово executable означает «исполняемый»: такой файл содержит машинный код и данные, которые загрузчик Windows может разместить в памяти процесса. Проект поддерживает PE32+ AMD64: 64-разрядный вариант PE для архитектуры x86-64.

4. Заголовок и секция

Header переводится как «заголовок». Заголовок - служебная область, описывающая остальные данные: архитектуру, размеры, количество секций и координаты таблиц. Section переводится как «секция». Секция объединяет данные сходного назначения. Например, .text обычно содержит машинный код, .pdata - таблицу RUNTIME_FUNCTION, а .xdata - структуры UNWIND_INFO. Название секции является подсказкой, но проект также проверяет адреса, размеры и права доступа.

5. Три вида адреса: file offset, RVA и VA

File offset - файловое смещение, то есть номер байта от начала файла. Если offset равен 0x200, речь идёт о байте на расстоянии 512 байтов от начала файла. RVA - Relative Virtual Address, относительный виртуальный адрес: расстояние от базы загруженного образа. VA - Virtual Address, полный виртуальный адрес внутри процесса. Эти числа принадлежат разным системам координат и не должны смешиваться.

```
VA = loaded_image_base + RVA
file_offset = section.raw_ptr + (RVA - section.virtual_address)
```

Вторая формула применима только после того, как найдена подходящая секция и доказано наличие соответствующих байтов в файле. Именно поэтому проект сначала строит PeImage, а уже потом читает .pdata и .xdata.

6. Что такое функция, стек и регистр

Функция - участок машинного кода, выполняющий законченную часть работы. Register переводится как «регистр»: очень маленькая и быстрая ячейка внутри процессора. RIP хранит адрес следующей исполняемой команды, RSP указывает на вершину стека. Stack, или стек, - область памяти, используемая для адресов возврата, локальных данных и временно сохранённых значений. Когда функция изменяет RSP или сохраняет регистры, Windows должна знать, как отменить эти действия при раскрутке стека.

7. Что такое exception и EH

Exception переводится как «исключение». Это событие, при котором обычное последовательное выполнение прерывается: например, из-за недопустимого доступа к памяти, деления на ноль, инструкции breakpoint или программного RaiseException. EH - Exception Handling, обработка исключений. Windows ищет подходящий обработчик и при необходимости проходит по кадрам стека.

8. Что такое unwind и metadata

Unwind переводится как «раскрутка стека» или «восстановление предыдущего кадра вызова». Metadata - метаданные, то есть данные, описывающие другие данные или код. UNWIND_INFO не исполняет функцию, а описывает, как функция изменила стек и какие регистры сохранила. Windows использует это описание, чтобы восстановить контекст вызывающей функции.

9. Что такое static analysis

Static analysis - статический анализ. Проект читает файл на диске как данные и не запускает его. Статика отвечает: «что записано в PE и какую карту функций и unwind metadata файл заявляет?» Такой режим одинаково применим на Windows, Linux и macOS, потому что анализируются байты формата.

10. Что такое dynamic analysis

Dynamic analysis - динамический анализ, то есть наблюдение за реально запущенным процессом. В этом проекте точнее говорить dynamic validation - динамическая проверка. Она не исследует все возможные пути программы, а запускает один контролируемый сценарий под Windows debugger и фиксирует выбранные события: исключения, попадания в обработчики и вызовы API динамических таблиц функций.

11. Что такое parser, decoder, heuristic и report

Parser - синтаксический разборщик: он читает байты по правилам формата и создаёт структуры C. Decoder - декодер: он переводит компактное кодирование, например UNWIND_CODE, в более понятные свойства. Heuristic - эвристика: инженерное правило, которое отмечает подозрительное сочетание наблюдений, но не является математическим доказательством вредоносности. Report - отчёт, то есть внешнее представление результата для человека или другой программы.

12. Главная идея проекта

Проект последовательно уменьшает неопределённость. Сначала существуют только неизвестные байты. Затем появляется карта PE. После неё - таблица функций. Затем - расшифрованные unwind metadata. Эвристический слой превращает нарушения в локализованные findings. Динамический слой при наличии Windows добавляет наблюдения о конкретном запуске. Отчёт сохраняет весь путь, чтобы аналитик мог проверить происхождение вывода.

Часть I. Карта проекта в голове

Сначала проект рассматривается как последовательность преобразований. Здесь ещё не требуется помнить имена всех функций. Важно понять, какой вид имеют данные до каждого блока, что блок добавляет к уже известной картине и почему следующий блок не может начать работу раньше.

Одна фраза о назначении каждого блока

main.c организует анализ и удерживает правильный порядок этапов.

util.c превращает файл в безопасно читаемую последовательность байтов и предоставляет общие операции.

pe.c строит карту внешнего PE-контейнера: заголовки, секции и правила адресного отображения.

unwind.c строит внутри PE функцию-ориентированную EH/Unwind-карту: .pdata, .xdata, RUNTIME_FUNCTION и UNWIND_INFO.

heuristics.c проверяет построенную карту по структурным и контекстным правилам и формирует локализованные аналитические выводы.

dynamic_win.c при запуске на Windows наблюдает один реальный маршрут выполнения и добавляет факты, которых нет в файле на диске.

report.c показывает уже полученные модели в CSV, JSON и HTML, не выполняя анализ повторно.

app_info.c описывает происхождение результата: версию, платформу, компилятор, режим сборки и время.

include/*.h задают контракты: какие структуры один блок имеет право передать другому и какие функции доступны извне.

Что означает «строить карту»

Карта в этом проекте - не изображение и не географическая аналогия. Это связанная модель адресов и структур. Для каждой найденной сущности проект сохраняет её координаты, поля, состояние разбора и связь с другими сущностями. Например, RuntimeFunction связывает диапазон машинного кода с RVA структуры UNWIND_INFO; UnwindSummary связывает этот RVA с декодированными полями и диагностикой; finding связывает нарушение с представительным RVA. Поэтому аналитик может пройти обратный путь от вывода к конкретным байтам файла.

```
входные байты
-> карта PE-контейнера
-> карта функций и EH/Unwind metadata
-> карта проверенных нарушений и их адресов
-> наблюдения конкретного запуска
-> отчёт, сохраняющий связи между всеми уровнями
```

Главный принцип

Новый уровень не уничтожает предыдущий. HeuristicReport не заменяет RuntimeFunction и UnwindSummary; DynamicReport не заменяет статическую карту; HTML не заменяет внутренние модели. Каждый уровень добавляет интерпретацию, сохраняя возможность проверить её происхождение.

Полный путь данных: от команды до отчёта

Ниже каждый прямоугольный этап отвечает на четыре вопроса: что уже существует, что делает код, какой новый объект появляется и зачем он нужен следующему уровню.

ЭТАП 1	**main.c принимает запрос**
На входе	Пользователь передал путь к PE-файлу и текстовые параметры командной строки.
Преобразование	parse_args проверяет параметры и записывает их в Args. main.c ещё не читает внутренний формат PE: он только определяет, какие режимы должны быть выполнены.
На выходе	Проверенная конфигурация Args: входной путь, режимы, ограничения и пути выходных файлов.
Зачем	Все дальнейшие блоки получают единое толкование запроса и не разбирают командную строку самостоятельно.

↓

ЭТАП 2	**util.c создаёт безопасный источник байтов**
На входе	Известен путь, но содержимое файла ещё находится на диске.
Преобразование	buf_init_from_file определяет размер, выделяет память, читает файл и создаёт Buf. Поле data указывает на первый байт, size задаёт строгую границу.
На выходе	Buf - неизменяемое представление входных байтов - и owned_mem - адрес памяти, которую main.c должен освободить.
Зачем	Форматные parser-ы получают единый ограниченный диапазон памяти; недоверенное смещение не превращается автоматически в доступ к произвольной памяти.

↓

ЭТАП 3	**pe.c строит карту PE-контейнера**
На входе	Есть Buf, но значения байтов ещё не связаны с заголовками, секциями и адресами PE.
Преобразование	pe_parse подтверждает MZ/PE/AMD64/PE32+, читает Optional Header и section headers, извлекает Exception Directory и строит правила RVA -> file offset.
На выходе	PeImage и массив PeSection: размеры образа, база, секции, Exception Directory и счётчики нарушений контейнера.
Зачем	unwind.c должен знать, где физически лежат байты .pdata/.xdata и какие виртуальные адреса им соответствуют.

↓

ЭТАП 4	**unwind.c строит EH/Unwind-карту**
На входе	PeImage задаёт координаты Exception Directory и секций, Buf содержит сами байты.
Преобразование	Модуль читает RUNTIME_FUNCTION[], затем для каждой записи декодирует UNWIND_INFO, UNWIND_CODE, выравнивание, handler tail и CHAININFO.
На выходе	Связанные массивы RuntimeFunction[], UnwindSummary[] и us_ok[]: диапазон функции, её metadata, результат разбора и точная диагностика.
Зачем	Теперь проект знает не только расположение данных, но и заявленную модель раскрутки стека для каждой функции.

↓

ЭТАП 5	**heuristics.c оценивает построенную карту**
На входе	Есть проверенные поля PE и EH/Unwind; часть записей могла содержать локальные ошибки.
Преобразование	heuristics_analyze проверяет инварианты, размещение, повторное использование и преобладание обработчиков, создаёт findings и агрегирует evidence families.
На выходе	HeuristicReport: локализованные findings, счётчики, score, confidence, verdict и ослабляющие факторы.
Зачем	Аналитик получает приоритеты ручной проверки, а не необъяснимую распечатку сотен низкоуровневых полей.

↓

ЭТАП 6	**dynamic_win.c добавляет факты выполнения**
На входе	Статическая карта уже существует; пользователь явно запросил --dynamic и запуск выполняется на Windows.
Преобразование	Collector создаёт target под debugger, ставит точки наблюдения на статические handler RVA и выбранные Windows API, затем фиксирует события и ограниченный RIP trace.
На выходе	DynamicReport: события конкретной сессии, handler/API hits, регистрации dynamic function tables, исключения и показатели полноты инструментации.
Зачем	Статические возможности отделяются от фактически наблюдавшегося поведения. Отсутствие события не объявляется доказательством невозможности события.

↓

ЭТАП 7	**report.c выдаёт результат**
На входе	В памяти существуют PeImage, EH/Unwind-карта и необязательные HeuristicReport/DynamicReport, а main.c подготовил ReportMetadata.
Преобразование	Writer проверяет согласованность модели, экранирует недоверенные строки и сериализует одни данные в выбранное внешнее представление.
На выходе	CSV, Ghidra CSV, JSON и/или автономный HTML.
Зачем	Человек и внешняя система получают проверяемый результат, сохранивший адреса, статусы разбора, аналитические выводы и происхождение запуска.

↓

Конец общей цепочки

Карта main.c: кто управляет последовательностью

main.c следует понимать как диспетчер жизненного цикла. Он не является самым глубоким анализатором, но только он видит весь запуск целиком: от argc/argv до освобождения последнего массива.

Маршрут

```
argc/argv
-> parse_args
-> Args
-> buf_init_from_file
-> pe_parse
-> unwind_read_runtime_functions_ex
-> цикл unwind_parse для каждого rf[i]
-> heuristics_analyze [если --heuristics]
-> dynamic_collect_windows [если --dynamic]
-> ReportMetadata
-> report_write_*
-> освобождение памяти
-> exit_code
```

Почему main.c не выполняет эти работы параллельно

Большинство этапов образуют зависимость по данным. unwind.c нуждается в PeImage; heuristics.c нуждается в RuntimeFunction и UnwindSummary; dynamic_win.c использует статические handler RVA; report.c получает уже готовые модели. Поэтому порядок задаётся не стилем автора, а необходимостью сначала создать вход для следующего уровня.

Какие ветви необязательны

Эвристический, динамический и отдельные отчётные режимы включаются параметрами. Необязательность означает, что блок можно не запускать; она не означает, что при запуске блок может обойти обязательные уровни подготовки данных.

Что main.c хранит одновременно

В середине анализа main.c владеет Buf, PeImage, массивом RuntimeFunction, массивом UnwindSummary, массивом us_ok и, возможно, двумя отчётами. Такое хранение необходимо, потому что report.c должен увидеть связи между объектами одного индекса. После записи main.c вызывает функции освобождения в обратном порядке.

Карта util.c: как файл становится безопасными байтами

util.c находится ниже предметной логики. Он не знает, что такое .pdata, но обеспечивает условия, при которых pe.c и unwind.c могут читать недоверенный файл без выхода за его физическую границу.

```
путь к файлу
-> fopen / размер
-> malloc
-> fread
-> Buf { data, size }
-> buf_read(off, len)
проверка off <= size
проверка len <= size - off
memcpy только после доказательства границ
-> запрошенные байты или false
```

Две разные роли памяти

Buf не владеет памятью: он только описывает адрес и длину. owned_mem хранит адрес, возвращённый malloc, и нужен main.c для последующего free. Разделение позволяет передавать Buf как неизменяемый вид данных, не передавая право освобождать память каждому parser-у.

Почему little-endian выделен отдельно

PE кодирует многобайтовые числа младшим байтом первым. buf_read_u32 сначала безопасно получает четыре байта, затем le32 собирает их в uint32_t. Таким образом проверка физической доступности и интерпретация порядка байтов выполняются последовательно.

Вторая ветвь util.c

StrBuf обслуживает противоположное направление: не чтение входа, а постепенное построение выходного текста. file_write_atomic принимает уже законченный массив символов, пишет временный файл и только после успеха заменяет итоговый путь.

Карта pe.c: как строится PE-контейнер

PE-контейнер - это согласованная система заголовков, виртуальных диапазонов и физических диапазонов файла. pe.c не ищет аномалии EH/Unwind напрямую: он сначала доказывает систему координат, в которой эти аномалии вообще можно локализовать.

```
Buf
-> DOS header: MZ и e_lfanew
-> NT headers: PE\0\0
-> COFF header: AMD64, число секций, размер Optional Header
-> Optional Header PE32+: ImageBase, SizeOfImage, alignment
-> Data Directory[3]: ExceptionRVA + ExceptionSize
-> Section Headers[]: virtual/raw ranges + characteristics
-> проверки границ, выравнивания и пересечений
-> PeImage + PeSection[]
```

Почему секция имеет два диапазона

virtual_address/virtual_size описывают размещение после загрузки образа. raw_ptr/raw_size описывают байты на диске. RVA относится к первому пространству, Buf - ко второму. pe_rva_to_fileoff находит покрывающую секцию и вычисляет физическое смещение только внутри доказанного raw-диапазона.

```
если RVA принадлежит секции:
delta = RVA - section.virtual_address
file_offset = section.raw_ptr + delta
```

Что именно является картой

PeImage хранит не все байты повторно. Он хранит координатное описание: базу, размеры, секции и адрес Exception Directory. Сами байты остаются в Buf. Поэтому pe.c создаёт модель расположения, а не копию файла.

Что передаётся unwind.c

unwind.c получает одновременно Buf и const PeImage*. PeImage отвечает на вопрос «где читать», Buf предоставляет сами байты. Отсутствие любого из этих объектов делает корректный разбор невозможным.

Карта unwind.c: как внутри PE появляется EH/Unwind

unwind.c строит второй, более предметный уровень карты. Его центральная единица - не секция, а функция: диапазон её кода связывается с metadata, описывающей раскрутку стека и возможный обработчик.

```
PeImage.exception_rva / exception_size
-> RVA переводится в file offset
-> размер / 12 = число RUNTIME_FUNCTION
-> RuntimeFunction[i]
BeginRVA
EndRVA
UnwindRVA
-> UnwindRVA переводится в file offset
-> header UNWIND_INFO
-> CountOfCodes двухбайтовых slots
-> операции и дополнительные operand slots
-> padding до 4-байтовой границы
-> handler RVA ИЛИ chained RUNTIME_FUNCTION
-> UnwindSummary[i] + us_ok[i]
```

Почему три массива связаны одним индексом

rf[i] сообщает, для какой функции выполнялся разбор. us[i] хранит результат её UNWIND_INFO. us_ok[i] сообщает, завершился ли этот разбор успешно. Если us_ok[i] равен false, объект us[i] всё равно полезен: parse_error и diagnostic_* объясняют, где и почему остановился decoder.

Почему одна операция может занимать несколько slots

CountOfCodes измеряет не число смысловых операций, а число двухбайтовых ячеек. Некоторые UWOP хранят дополнительные значения в следующих ячейках. Decoder сначала определяет размер операции в slots, проверяет их доступность и только затем переходит к следующей операции. Благодаря этому tail не принимается за продолжение массива.

Что проект знает и чего не знает после unwind.c

Проект знает заявленные поля metadata, рассчитанный фиксированный эффект на стек, сохранённые регистры, handler/chain и нарушения кодирования. Он пока не доказывает соответствие каждой UWOP реальным инструкциям пролога .text, потому что в текущем ядре нет полного дизассемблирования.

Карта heuristics.c: как карта превращается в findings

heuristics.c не начинает анализ с байтов. Он доверяет parser-ам как владельцам формата и получает готовые модели. Его работа - сравнивать свойства этих моделей с формальными инвариантами и инженерными контекстными правилами.

```
PeImage + RuntimeFunction[] + UnwindSummary[] + us_ok[]
-> проверки каждой записи
диапазон, SizeOfImage, UnwindRVA, parse status
-> проверки всей таблицы
сортировка, overlaps, duplicate begins
-> проверки размещения
секция и права UNWIND_INFO / handler
-> агрегирование
reuse, dominance, проценты нарушений
-> findings с location и affected_count
-> evidence families + score + confidence + verdict
-> HeuristicReport
```

Finding и общий verdict - не одно и то же

Finding описывает одно правило: например, пересечение диапазонов или недоступный UnwindRVA. Verdict описывает совокупность всех finding-ов после весов и ослабляющих факторов. Один finding не обязан означать высокую подозрительность всего файла.

Severity и confidence отвечают на разные вопросы

Severity показывает потенциальную значимость конкретного нарушения. Confidence показывает, насколько разнообразны и сильны основания общего статического вывода. Высокая severity возможна при ограниченной confidence, если потенциально серьёзный сигнал пока не подтверждён другими семействами.

Почему score не является вероятностью

Score - ограниченная сумма инженерных weights, то есть приоритетов правил. Поле suspicion технически выводится из score для шкалы интерфейса, но не является статистически откалиброванной вероятностью вредоносности. Проект не обучает модель на размеченном наборе данных.

Карта dynamic_win.c: две ветви наблюдения

Динамический слой начинает работу после статики и получает два источника адресов. Первая ветвь приходит из уже разобранной UNWIND_INFO. Вторая ветвь строится вокруг системных API, через которые программа регистрирует обработчики и динамические таблицы функций.

Ветвь A: статически найденные unwind handler-ы

```
UnwindSummary[].handler_rva
-> удалить повторяющиеся RVA
-> дождаться CREATE_PROCESS_DEBUG_EVENT
-> получить runtime_image_base
-> VA handler = runtime_image_base + handler_rva
-> заменить первый байт по VA на 0xCC
-> при достижении адреса получить BREAKPOINT event
-> записать unwind_handler_hit
-> при заданном trace_steps начать RIP trace
```

RVA недостаточно, потому что breakpoint ставится в память процесса по VA. Фактическая база становится известна только после создания debuggee - отлаживаемого процесса. Сложение base + RVA переводит статическую координату в адрес конкретного запуска.

Ветвь B: точки наблюдения на Windows API

```
адреса выбранных API в системных модулях target
-> 0xCC на входе API
-> breakpoint hit до выполнения API
-> прочитать аргументы из RCX/RDX/R8/R9
-> если зарегистрирован VEH/VCH/UEF: поставить 0xCC на handler
-> если зарегистрирована function table: сохранить параметры регистрации
-> при последующем handler hit записать VEH/VCH/UEF hit и RIP trace
```

Эти ветви дополняют друг друга

Первая ветвь отвечает: достигался ли handler, на который ссылается статическая x64 unwind metadata? Вторая отвечает: регистрировал ли процесс дополнительные механизмы обработки исключений или runtime function tables? Они наблюдают разные источники EH-поведения и сходятся в одном DynamicReport.

Динамика подробно: почему 0xCC не ломает команду

0xCC - байт инструкции INT3, программной точки останова x86/x64. Collector временно заменяет первый байт наблюдаемой команды. Windows останавливает поток и передаёт событие debugger-у. Чтобы программа продолжила работу корректно, исходную команду необходимо затем исполнить ровно один раз.

```
1. сохранить исходный байт команды
2. записать 0xCC
3. дождаться EXCEPTION_BREAKPOINT
4. восстановить исходный байт
5. установить RIP на начало исходной команды
6. включить Trap Flag в EFlags
7. продолжить поток: выполняется одна исходная команда
8. получить EXCEPTION_SINGLE_STEP
9. снова записать 0xCC
10. снять Trap Flag, если не идёт RIP trace
```

Почему RIP корректируется

После INT3 процессор сообщает адрес после однобайтовой инструкции. Если не вернуть RIP к исходному адресу, восстановленная команда будет пропущена. В коде ctx.Rip устанавливается в адрес breakpoint-события.

Зачем нужен single-step

Если вернуть 0xCC сразу, поток снова попадёт в ту же точку, не выполнив настоящую команду. Trap Flag заставляет процессор остановиться после одной команды. В этот момент Collector может безопасно переустановить breakpoint.

Что означает instrumentation_complete

Это признак технической полноты инструментации конкретной сессии. Ошибка постановки, восстановления или переустановки breakpoint-а делает наблюдение неполным и отражается отдельными счётчиками. Такой статус не является аномалией target-а.

Динамика подробно: какие аргументы читаются

Windows x64 calling convention - соглашение о вызовах - передаёт первые четыре целочисленных или адресных аргумента через RCX, RDX, R8 и R9. Collector останавливается на входе API, то есть до того, как функция изменила эти значения.

Поле	Что хранит и зачем это нужно
`AddVectoredExceptionHandler`	RCX = First, порядок вызова; RDX = адрес регистрируемого VEH handler. После чтения RDX Collector ставит breakpoint на этот handler.
`AddVectoredContinueHandler`	RCX = First; RDX = адрес VCH handler. После регистрации адрес становится новой точкой наблюдения.
`SetUnhandledExceptionFilter`	RCX = адрес top-level filter. Collector сохраняет регистрацию и ставит breakpoint на переданный адрес.
`RtlAddFunctionTable`	RCX = адрес массива RUNTIME_FUNCTION в памяти; RDX = число entries; R8 = base address. Collector фиксирует регистрацию, но в текущей версии не копирует и не декодирует весь runtime-массив.
`RtlInstallFunctionTableCallback`	RCX = table identifier; RDX = base address; R8 = длина диапазона; R9 = адрес callback. Collector сохраняет эти четыре значения как runtime evidence.

Что ещё фиксирует debug loop

Обычные исключения, не созданные собственными INT3/single-step механизмами Collector-а, записываются с exception code, адресом, thread id и first-chance/second-chance. First-chance означает первую передачу события debugger-у до окончательного решения обработчиков; second-chance означает, что исключение не было обработано на предыдущем проходе.

Ограничение вывода

DynamicReport описывает только один запуск с конкретными аргументами, timeout и trace_steps. Если API не вызван или handler не достигнут, корректная формулировка - «в данной сессии событие не наблюдалось», а не «механизм отсутствует».

Карта report.c: как модели становятся документом

report.c работает после анализа. Он не перечитывает Buf и не начисляет новые эвристические веса. Его задача - сохранить уже доказанные связи без потери различия между полем parser-a, finding-ом, runtime event и итоговой оценкой.

```
внутренние структуры C
-> проверка pointer/count и ReportMetadata
-> выбор внешнего формата
CSV: плоская строка на функцию
Ghidra CSV: адреса и комментарии для импорта
JSON: полная машиночитаемая иерархия
HTML: человекочитаемые таблицы и объяснения
-> escaping специальных символов
-> StrBuf
-> file_write_atomic
-> законченный файл
```

Почему форматов несколько

Они предназначены для разных потребителей. CSV удобен для таблиц, но плохо хранит вложенные события. JSON сохраняет структуру для backend-а. HTML показывает доказательства человеку. Ghidra CSV переносит адресную разметку во внешнюю среду анализа.

Что report.c может сопоставлять

Он знает runtime_image_base и статический PeImage, поэтому может представить динамический VA также как RVA и секцию, если адрес принадлежит основному образу. Это отчётное сопоставление координат; оно не превращает наблюдение в новый статический finding.

Карта app_info.c и заголовочных файлов

Эти компоненты не исследуют PE, но обеспечивают воспроизводимость и согласованность программы.

app_info.c

```
compile-time constants и macros + текущее UTC-время
-> tool_version
-> schema_version
-> compiler / platform / build_mode
-> generated_at_utc
-> main.c объединяет их с SHA-256 и командой
-> ReportMetadata
```

Это разделяет свойства объекта и свойства измерительного инструмента. Finding относится к PE, а tool_version - к анализатору, который этот finding сформировал.

include/*.h

Заголовочный файл не выполняется. Он объявляет общий тип и сигнатуру функции, чтобы два отдельно компилируемых .c-файла одинаково понимали контракт. Например, pe.h задаёт PeImage; pe.c заполняет его, а unwind.c читает через const PeImage*. Поля одной структуры имеют один смысл для обоих модулей.

Связь обучающего и пофайлового уровней

После этой части читатель знает смысл каждого преобразования. В последующих главах тот же маршрут раскрывается по файлам: где объявлен тип, где реализована функция, какие выражения C выполняют шаг и как обрабатывается ошибка. Таким образом концептуальная карта не заменяет исходный код, а делает его читаемым.

Часть II. Структуры, которыми обмениваются модули

Функции передают друг другу не абстрактный «результат», а конкретные структуры C. Эта часть объясняет назначение каждого общего объекта, владельца памяти, связь с соседними объектами и смысл полей. После неё пофайловый маршрут можно читать как преобразование одной структуры в другую.

Сначала четыре базовых понятия C

Структура - объединённый тип, в котором несколько именованных полей описывают один объект. Например, PeSection хранит имя секции и оба её диапазона.

Указатель хранит адрес объекта. Запись const PeImage* img означает: функция получает адрес PeImage и обязуется не изменять её через этот указатель.

Массив - последовательность однотипных элементов. RuntimeFunction* rf вместе с size_t n означает массив из n записей; одного указателя без количества недостаточно для безопасного обхода.

Владение отвечает на вопрос, кто обязан освободить динамическую память. PeImage владеет sections после успешного pe_parse; main.c владеет массивами rf/us/us_ok; HeuristicReport и DynamicReport освобождаются собственными free-функциями.

Типы чисел

Поле	Что хранит и зачем это нужно
`bool`	Логическое значение true/false. Оно отвечает на один вопрос состояния, например успешен ли разбор конкретной UNWIND_INFO.
`uint8_t`	Беззнаковое число ровно 8 бит; используется для байтов и полей формата размером в один байт.
`uint16_t`	Беззнаковое 16-битное число; подходит для machine, масок регистров и полей PE шириной два байта.
`uint32_t`	Беззнаковое 32-битное число. RVA и большинство размеров PE x64 metadata хранятся именно в такой ширине.
`uint64_t`	Беззнаковое 64-битное число. Используется для VA, ImageBase, адресов процесса и аргументов x64 API.
`size_t`	Беззнаковый тип размера памяти текущей платформы. Им хранят длины массивов и файловые offsets внутри host-процесса.
`char[N]`	Массив символов фиксированной ёмкости. Последний байт обычно резервируется под NUL - нулевой признак конца строки C.

Ключевая связь массивов

```
rf[i] = координаты функции и RVA metadata
us[i] = результат декодирования metadata этой функции
us_ok[i] = удалось ли считать us[i] полноценным разбором
```

Одинаковый индекс - часть контракта системы. Если перемешать массивы или изменить их длину независимо, finding и строка отчёта будут привязаны не к той функции.

Args - проверенная конфигурация одного запуска

Args существует только внутри main.c и не опубликована в include/*.h. Это внутренний объект orchestration layer, потому что другим модулям не нужно знать синтаксис CLI.

Место структуры в потоке

Кто создаёт или заполняет. parse_args в src/main.c очищает объект, устанавливает значения по умолчанию и заполняет его из argc/argv.

Кто читает. main.c использует поля, чтобы решить, какие блоки вызывать и куда записывать результаты; dynamic_win.c получает выделенную часть аргументов target-a.

Как она связана с другими данными. До Args существует текстовая команда. После Args существует нормализованный набор значений: числовые ограничения уже проверены, switches представлены bool, а пути - const char*.

Поля структуры

Поле	Что хранит и зачем это нужно
`input`	Путь к исследуемому PE. Это обязательный первый позиционный аргумент.
`dump / show_ops`	Нужно ли печатать записи RUNTIME_FUNCTION и текст декодированных операций в терминал.
`heuristics`	Включён ли вызов heuristics_analyze.
`csv_path`	Путь обычного CSV или NULL, если формат не запрошен.
`ghidra_csv_path`	Путь CSV специальной формы для импортера Ghidra.
`json_path / html_path`	Пути полных JSON и HTML отчётов или NULL.
`dump_limit`	Максимальное число печатаемых записей; -1 означает отсутствие лимита.
`follow_chain`	Нужно ли decoder-у следовать UNW_FLAG_CHAININFO.
`strict_abi`	Нужно ли отмечать формальные нарушения Microsoft x64 unwind ABI.
`max_chain_depth`	Предельная глубина chained UNWIND_INFO, проверенная в диапазоне 1..16.
`dynamic`	Запрошена ли Windows-only динамическая сессия.
`timeout_ms`	Жёсткий временной предел debug loop в миллисекундах.
`trace_steps`	Сколько RIP значений собирать после первого handler hit; диапазон 0..256.
`child_argv / child_argc`	Аргументы после разделителя --, передаваемые исследуемому target. argv указывает на строки, argc сообщает их число.

Buf - ограниченное представление входных байтов

Buf не владеет выделенной памятью самостоятельно. owned_mem, возвращённый рядом, освобождает main.c через buf_free_owned. Buf является видом на диапазон памяти.

Место структуры в потоке

Кто создаёт или заполняет. buf_init_from_file в util.c после полной загрузки файла.

Кто читает. pe.c, unwind.c, SHA-256 и тестовые harness-ы. Обычно объект передаётся как const Buf*, поэтому parser читает, но не меняет вход.

Как она связана с другими данными. data задаёт начало координат file offset 0; size задаёт недоступную верхнюю границу. Без обоих полей невозможно доказать безопасность buf_read.

Поля структуры

Поле	Что хранит и зачем это нужно
`data`	Указатель на первый uint8_t файла. Выражение data + off адресует байт с файловым смещением off только после проверки границ.
`size`	Полное число доступных байтов. Допустимый одиночный индекс всегда меньше size.

StrBuf - расширяемая строка с состоянием ошибки

Структура владеет s после успешного sb_init. sb_free освобождает строку. failed запоминает необратимый сбой, чтобы последующие операции не выдавали частичный текст за корректный.

Место структуры в потоке

Кто создаёт или заполняет. sb_init создаёт начальный буфер; sb_append/sb_appendf постепенно увеличивают его содержимое.

Кто читает. util.c, main.c, unwind.c, heuristics.c, dynamic_win.c и report.c используют StrBuf для сообщений, текстовых деталей и сериализации.

Как она связана с другими данными. StrBuf решает противоположную задачу относительно Buf: Buf ограничивает чтение готового массива, а StrBuf безопасно наращивает создаваемый массив символов.

Поля структуры

Поле	Что хранит и зачем это нужно
`s`	Адрес динамического массива char. При успешном состоянии это NUL-terminated строка C.
`len`	Число содержательных байтов до завершающего NUL.
`cap`	Фактическая выделенная ёмкость массива. Всегда должна позволять разместить len байтов и завершающий NUL.
`failed`	Флаг предыдущего сбоя выделения или форматирования. sb_ok возвращает false, если буфер уже недостоверен.

PeSection - одна секция в двух системах координат

PeSection не содержит байты секции. Она описывает, какой виртуальный диапазон и какой диапазон файла относятся к одной логической секции.

Место структуры в потоке

Кто создаёт или заполняет. pe_parse читает одну 40-байтовую запись IMAGE_SECTION_HEADER и создаёт элемент массива sections.

Кто читает. pe_rva_to_fileoff, pe_find_section_for_rva, heuristics.c, dynamic address localization и report.c.

Как она связана с другими данными. Массив PeSection принадлежит PeImage. Поиск секции превращает абстрактный RVA в контекст: имя, физический диапазон и права доступа.

Поля структуры

Поле	Что хранит и зачем это нужно
`name[9]`	До восьми байтов имени PE плюс дополнительный NUL, чтобы безопасно использовать поле как строку C.
`virtual_address`	RVA начала секции после загрузки образа.
`virtual_size`	Логический размер секции в виртуальном образе.
`raw_ptr`	File offset первого физического байта секции на диске.
`raw_size`	Число физически сохранённых байтов секции.
`characteristics`	Битовая маска свойств: содержит ли секция код/данные, разрешены ли чтение, запись и исполнение.

PeImage - карта внешнего PE-контейнера

PeImage связывает Buf с виртуальным образом, но не хранит указатель на Buf. Поэтому вызывающий код обязан сохранять оба объекта, пока продолжается разбор.

Место структуры в потоке

Кто создаёт или заполняет. pe_parse заполняет поля и выделяет sections; pe_free завершает владение массивом.

Кто читает. unwind.c использует адресное отображение и Exception Directory; heuristics.c читает размеры и счётчики; dynamic_win.c/report.c локализуют RVA и VA.

Как она связана с другими данными. Это фундаментальная модель проекта: последующие уровни не должны самостоятельно перечитывать заголовки и создавать другую систему координат.

Поля структуры

Поле	Что хранит и зачем это нужно
ok	Успешно ли построена поддерживаемая базовая модель.
is_pe64	Подтверждён ли Optional Header PE32+.
machine	Архитектура COFF; проект требует IMAGE_FILE_MACHINE_AMD64.
image_base	Предпочтительная виртуальная база образа из файла. В runtime фактическая база может отличаться из-за ASLR.
size_of_image	Полный заявленный размер загруженного образа. Используется как верхняя граница RVA.
size_of_headers	Размер области заголовков, которая также может участвовать в RVA mapping.
section_alignment	Выравнивание секций в виртуальном образе.
file_alignment	Выравнивание физических данных секций в файле.
exception_rva	RVA начала Exception Directory - точки входа в x64 RUNTIME_FUNCTION table.
exception_size	Размер Exception Directory в байтах.
file_size	Фактическая длина входного Buf, сохранённая для диагностик и проверок.
num_sections	Число элементов массива sections.
sections	Owned-массив PeSection, выделенный parser-ом и освобождаемый pe_free.
section_raw_oob	Сколько секций имеют физический диапазон за пределами файла.
section_virtual_oob	Сколько виртуальных диапазонов выходят за SizeOfImage.
section_raw_overlaps	Число обнаруженных пересечений физических диапазонов секций.
section_virtual_overlaps	Число пересечений виртуальных диапазонов.
section_misaligned	Сводный счётчик нарушений выравнивания и связанных header-инвариантов.
exception_directory_oob	Не удалось ли согласованно разместить весь диапазон Exception Directory внутри образа/файла.

RuntimeFunction - связь функции с её unwind metadata

RuntimeFunction[i] связан с UnwindSummary[i] и us_ok[i]. Сама запись не содержит UWOP и handler: она только указывает, где искать UNWIND_INFO.

Место структуры в потоке

Кто создаёт или заполняет. unwind_read_runtime_functions_ex читает три uint32_t из одной 12-байтовой записи Exception Directory.

Кто читает. main.c передаёт unwind_rva в unwind_parse; heuristics.c проверяет диапазоны; report.c показывает координаты; dynamic_win.c косвенно использует handler из связанного summary.

Как она связана с другими данными. Это основная функция-ориентированная единица .pdata для AMD64. Begin/End задают полуоткрытый диапазон [begin, end).

Поля структуры

Поле	Что хранит и зачем это нужно
`begin_rva`	RVA первой инструкции диапазона функции.
`end_rva`	RVA сразу после последней инструкции диапазона; корректный диапазон требует end_rva > begin_rva.
`unwind_rva`	RVA связанной UNWIND_INFO в .xdata или иной отображаемой области.

UnwindParseOpts - политика одного запуска decoder-a

Одинаковый opts передаётся всем функциям, чтобы критерии глубины цепочки и ABI были единообразными в одном отчёте.

Место структуры в потоке

Кто создаёт или заполняет. main.c формирует объект перед циклом unwind_parse из полей Args.

Кто читает. unwind_parse читает параметры, но не владеет структурой и не изменяет её.

Как она связана с другими данными. Это не данные PE, а настройки анализатора. report.c сохраняет эквивалентные значения в ReportMetadata.

Поля структуры

Поле	Что хранит и зачем это нужно
`max_chain_depth`	Максимальное число переходов CHAININFO, после которого разбор останавливается диагностикой.
`follow_chain`	Нужно ли вообще переходить к chained entry и накапливать путь.
`strict_abi`	Нужно ли считать формальные Microsoft x64 ABI deviations диагностическими anomaly bits.

UnwindSummary - результат разбора одной UNWIND_INFO

Summary объединяет не только декодированные поля, но и локальную диагностику. Поэтому неуспешная запись всё равно может объяснить доступные байты и точку остановки.

Место структуры в потоке

Кто создаёт или заполняет. unwind_parse очищает и заполняет us[i] для rf[i].unwind_rva.

Кто читает. heuristics.c, dynamic_win.c, report.c и терминальный dump. us_ok[i] отдельно сообщает, можно ли считать основной разбор успешным.

Как она связана с другими данными. Один объект относится к одному индексу RuntimeFunction. has_handler/handler_rva питают статические проверки и динамические точки наблюдения.

Поля структуры

Поле	Что хранит и зачем это нужно
`valid`	Внутренний признак, что основные данные summary сформированы как допустимый результат.
`version`	Версия формата из младших трёх бит первого байта UNWIND_INFO.
`flags`	Биты EHANDLER, UHANDLER и CHAININFO, определяющие смысл tail после массива codes.
`prolog_size`	Заявленный размер пролога функции в байтах.
`count_codes`	Число двухбайтовых slots UNWIND_CODE, а не число смысловых UWOP.
`frame_reg`	Номер frame register или 0, если отдельный frame register не заявлен.
`frame_off`	Масштабированное смещение frame register; фактический шаг измеряется блоками по 16 байт.
`has_handler`	Есть ли по flags 32-битный RVA обработчика в tail.
`handler_rva`	Статический RVA обработчика; переводится в runtime VA как loaded_base + RVA.
`has_chain`	Содержит ли tail chained RUNTIME_FUNCTION вместо handler RVA.
`chain`	Три поля chained RuntimeFunction, прочитанные из tail.
`fixed_stack_bytes`	Оценка фиксированного изменения стека по push/alloc/machframe. Это интерпретация metadata, не результат дизассемблирования .text.
`pushed_nonvol_count`	Количество операций сохранения nonvolatile-регистров через push по декодированным UWOP.
`saved_nonvol_mask`	Битовая маска сохранённых общих nonvolatile-регистров; номер регистра используется как позиция бита.
`saved_xmm_mask`	Битовая маска сохранённых XMM-регистров.
`anomalies`	Битовая маска UA_*: неизвестная версия, обрыв, неверный opcode/operand, порядок CodeOffset, chain и ABI diagnostics.
`parse_error[160]`	Человекочитаемая причина, когда unwind_parse вернул false.
`diagnostic_field[64]`	Имя точного поля или области, где обнаружена наиболее сильная локальная проблема.

Поле	Что хранит и зачем это нужно
`diagnostic_has_file_offset / diagnostic_file_offset`	Флаг наличия и само физическое смещение проблемных данных.
`diagnostic_has_size / expected_size / available_size`	Ожидаемое и фактически доступное число байтов, если ошибка связана с границей.
`diagnostic_has_slots / expected_slots / available_slots`	Ожидаемое и доступное число UNWIND_CODE slots для ошибки операнда.
`diagnostic_has_bytes`	Есть ли сохранённый фрагмент raw bytes для доказательства.
`diagnostic_bytes_file_offset`	File offset первого сохранённого диагностического байта.
`diagnostic_bytes[32] / diagnostic_bytes_count`	Ограниченная копия фактически прочитанных байтов и её длина.
`chain_path[16] / chain_path_count`	Последовательность RVA, пройденных при CHAININFO, и число сохранённых элементов.
`chain_loop_detected / chain_loop_rva`	Обнаружен ли цикл и на каком RVA произошёл повторный вход.

HeuristicFinding и HeuristicMitigation

Finding хранит одно локализованное аналитическое наблюдение. Mitigation хранит контекст, из-за которого общий вывод должен быть осторожнее.

Место структуры в потоке

Кто создаёт или заполняет. Вспомогательные функции heuristics.c создают элементы фиксированных массивов findings и mitigations.

Кто читает. report.c отображает их; finalize_verdict учитывает количество и семейства findings, а mitigation ослабляет уверенность без удаления наблюдения.

Как она связана с другими данными. В HeuristicReport может быть до 32 findings и до 16 mitigations. Ограничения защищают фиксированную структуру от неограниченного роста.

Поля структуры

Поле	Что хранит и зачем это нужно
`finding.code`	Стабильный machine code правила, например rf_overlaps. Предназначен для JSON и автоматической обработки.
`finding.family`	Семейство доказательств: целостность таблицы, unwind, размещение, reuse или dominance.
`finding.severity`	Текстовая категория потенциальной значимости. В текущих правилах создаются значения medium и high; finalize_verdict считает high/critical сильными, medium - средними, а остальные - слабыми для сводных счётчиков.
`finding.message`	Человекочитаемое описание наблюдения и его агрегированного значения.
`finding.has_location`	Удалось ли привязать вывод к представительному RVA.
`finding.representative_rva`	Адрес одного конкретного места, с которого удобно начать ручную проверку.
`finding.affected_count`	Сколько записей или объектов затронуто этим правилом.
`mitigation.code`	Стабильный код ослабляющего обстоятельства.
`mitigation.message`	Объяснение, почему finding не следует интерпретировать максимально жёстко без дополнительного контекста.

HeuristicReport - сводная статическая оценка

Report сохраняет одновременно исходные агрегаты и итоговую интерпретацию. Благодаря этому score можно объяснить через конкретные счётчики и findings.

Место структуры в потоке

Кто создаёт или заполняет. heuristics_analyze после полного прохода по PE/EH/Unwind-моделям.

Кто читает. report.c и терминальная печать; main.c владеет объектом до heuristics_free.

Как она связана с другими данными. PeImage и массивы не копируются внутрь отчёта. HeuristicReport хранит производные значения; report.c получает и исходные модели, и этот отчёт.

Поля структуры

Поле	Что хранит и зачем это нужно
`score`	Целое 0..100: ограниченная сумма инженерных весов с насыщением.
`suspicion`	Нормированное 0..1 представление score для интерфейса; не статистическая вероятность.
`confidence`	Оценка силы и разнообразия статических evidence после штрафов за неоднозначность.
`requires_manual_review`	Нужна ли ручная проверка по порогам и специальным условиям.
`evidence_families_mask`	Битовая маска семейств, которые реально дали доказательства.
`verdict`	Текстовая категория no/low/moderate/high static suspicion по порогам score.
`total`	Количество RuntimeFunction, реально переданных в эвристический слой.
`pe_section_* / pe_exception_directory_oob`	Скопированные агрегаты нарушений PE-контейнера.
`rf_invalid_ranges / rf_oob / rf_unwind_oob`	Нарушения базовых координат RUNTIME_FUNCTION.
`rf_unsorted / rf_overlaps / rf_duplicate_begins`	Глобальные свойства таблицы диапазонов.
`unwind_parse_fail`	Число записей, где decoder не смог построить полный summary.
`unwind_anomalies / unwind_abi_warnings`	Число структурных и формальных ABI diagnostics.
`unwind_unusual_section / exec / writable`	Счётчики необычного или рискованного размещения metadata.
`handler_nonexec_section / writable_section`	Счётчики обработчиков, указывающих в неисполняемые или изменяемые области.
`unwind_reuse_ratio / unwind_unique_ratio`	Доли максимального повторного использования и уникальных UnwindRVA.
`handler_reuse_ratio / handler_count`	Преобладание одного handler среди всех записей с обработчиком.
`finding_count / findings`	Фактическое число локализованных выводов и их фиксированный массив.
`weak/medium/strong_finding_count`	Распределение выводов по силе evidence.
`evidence_level`	Сводная текстовая характеристика доказательной базы.

Поле	Что хранит и зачем это нужно
`mitigation_count / mitigations`	Ослабляющие факторы и их число.
`details`	Расширяемая строка с дополнительными человекочитаемыми заметками.

DynamicEvent - одно наблюдение debug session

Одинаковые arg0..arg3 имеют разный смысл для разных type. Интерпретировать их можно только вместе с type; report.c должен подписывать аргументы соответствующего события.

Место структуры в потоке

Кто создаёт или заполняет. dynamic_event_add внутри dynamic_win.c при исключении, регистрации, handler hit, timeout или завершении процесса.

Кто читает. DynamicReport хранит events[]; report.c сериализует каждый элемент с учётом type.

Как она связана с другими данными. Это наблюдение, а не finding. Event говорит, что debugger увидел факт в конкретном запуске; аналитическая значимость устанавливается отдельно.

Поля структуры

Поле	Что хранит и зачем это нужно
`type[40]`	Стабильный тип события: exception, veh_registration, rtl_add_function_table, handler_hit и другие.
`module[64]`	Имя загруженного модуля, которому принадлежал адрес, если его удалось определить.
`thread_id`	Идентификатор потока, породившего debug event.
`exception_code`	Код Windows exception для событий исключений; 0 для несвязанных событий.
`first_chance`	Было ли исключение событием первой очереди обработки.
`address`	VA точки события: адрес исключения, API или handler-a.
`arg0..arg3`	До четырёх 64-битных параметров. Для RtlAddFunctionTable это table/count/base; для registration handler находится в определённом arg; смысл задаётся type.

DynamicReport - модель одной динамической сессии

DynamicReport содержит и предметные события, и качество измерения. Счётчики instrumentation необходимы, чтобы отсутствие hit не трактовалось без учёта ошибок наблюдения.

Место структуры в потоке

Кто создаёт или заполняет. dynamic_report_init создаёт начальное состояние; dynamic_collect_windows заполняет; dynamic_report_free освобождает details.

Кто читает. report.c и main.c. Статический HeuristicReport остаётся отдельным объектом.

Как она связана с другими данными. runtime_image_base позволяет report.c сопоставить VA события с RVA/секцией статического PeImage. events и trace имеют фиксированные пределы.

Поля структуры

Поле	Что хранит и зачем это нужно
`enabled / ran / ok`	Запрошен ли режим, была ли сессия реально запущена и завершилась ли инфраструктура без критического сбоя.
`timed_out`	Остановлен ли эксперимент по жёсткому timeout.
`target_exited / target_exit_code`	Завершился ли target сам и с каким кодом.
`timeout_ms / trace_steps`	Фактические ограничения данной сессии.
`runtime_image_base`	VA базы главного модуля из CREATE_PROCESS_DEBUG_EVENT.
`instrumentation_complete`	Удалось ли поддерживать заявленные точки наблюдения без известных потерь.
`modules_observed_peak`	Максимальное число модулей, одновременно видимых в карте target-а.
`module_refresh_failures / module_list_truncations`	Ошибки или усечения при обновлении списка загруженных модулей.
`unwind_handlers_discovered`	Число уникальных handler RVA, полученных из статических summaries.
`breakpoint_set_attempts / failures / capacity_drops`	Качество постановки программных точек останова и потери из-за лимита списка.
`pending_rearm_failures / breakpoint_rearm_failures`	Ошибки механизма восстановить команду и снова поставить 0xCC.
`exceptions_seen / first_chance_seen / second_chance_seen`	Количество обычных наблюдавшихся исключений и распределение по очереди обработки.
`unwind_handler_bp_set / unwind_handler_hits`	Сколько статических handler breakpoints поставлено и сколько раз они достигнуты.
`veh_registrations / veh_hits`	Регистрации vectored exception handlers и фактические попадания в отслеживаемые адреса.
`vch_registrations / vch_hits`	Регистрации vectored continue handlers и hits.
`uef_set_calls / uef_hits`	Установки unhandled exception filter и hits.
`rtl_add_function_table_calls`	Число наблюдавшихся вызовов регистрации готового массива runtime functions.
`rtl_install_function_table_cb_calls`	Число регистраций callback-based dynamic table.

Поле	Что хранит и зачем это нужно
`event_count / dropped_events / events[256]`	Число сохранённых событий, число потерянных из-за лимита и фиксированный массив.
`trace_count / trace[256]`	Число собранных RIP и последовательность VA после первого отслеживаемого handler hit.
`details`	Человекочитаемый журнал сессии в StrBuf.

ReportMetadata - происхождение отчёта

Структура не владеет строками: поля const char* указывают на строки, которые должны оставаться действительными до завершения writer-а.

Место структуры в потоке

Кто создаёт или заполняет. main.c собирает значения из app_info.c, sha256_hex, build_invocation и текущего запуска.

Кто читает. report_write_json_ex и report_write_html валидируют и сериализуют metadata.

Как она связана с другими данными. Metadata описывает анализатор и условия запуска, а не аномалии PE. Это позволяет воспроизвести и сравнить результат.

Поля структуры

Поле	Что хранит и зачем это нужно
`tool_version`	Версия EH/Unwind Analyzer.
`report_schema_version`	Версия внешнего контракта отчёта, текущая 1.2.0.
`generated_at_utc`	Время формирования в UTC/RFC 3339.
`input_sha256`	64 шестнадцатеричных символа SHA-256 конкретных входных байтов.
`input_size`	Размер исследуемого файла.
`command_line`	Безопасное представление запуска; target arguments после -- редактируются в provenance.
`build_compiler`	Компилятор, которым собран анализатор.
`build_platform`	Платформа сборки анализатора.
`build_mode`	Debug или release configuration.
`strict_abi / follow_chain / max_chain_depth`	Ключевые параметры decoder-a, влияющие на повторяемость статического результата.

Как структуры образуют одну модель

Теперь можно прочитать архитектуру не по именам файлов, а по объектам данных. Стрелка означает, что следующий объект создаётся на основе предыдущих и сохраняет связь с ними.

```
Args
-> Buf
-> PeImage { PeSection[] }
-> RuntimeFunction[i]
+ UnwindSummary[i]
+ us_ok[i]
-> HeuristicReport { HeuristicFinding[], Mitigation[] }
-> DynamicReport { DynamicEvent[], trace[] } [необязательно]
+ ReportMetadata
-> CSV / Ghidra CSV / JSON / HTML
```

Почему report.c получает почти все объекты

Производный отчёт не должен терять исходную привязку. HeuristicReport сообщает, что найдено, но PeImage и rf/us сообщают, где именно и из каких полей это следует. DynamicReport сообщает VA runtime, PeImage помогает показать соответствующий RVA. ReportMetadata объясняет происхождение всего документа.

Почему структуры не объединены в один огромный объект

Разделение отражает разные уровни знания и владельцев правил. PE parser не должен зависеть от эвристических весов; динамический collector не должен менять статический summary; writer не должен владеть всеми массивами. Отдельные структуры делают эти границы проверяемыми в коде.

Внутренние структуры dynamic_win.c

Следующие типы объявлены непосредственно в src/dynamic_win.c. Они не входят в публичный dynamic.h и не передаются report.c. Их задача - технически поддерживать наблюдение, из которого затем строятся DynamicEvent и DynamicReport.

ModInfo и ModList

Поле	Что хранит и зачем это нужно
`ModInfo.name`	Нормализованное имя загруженного DLL/EXE в нижнем регистре.
`ModInfo.base`	Фактический VA начала модуля в target process.
`ModInfo.size`	Размер загруженного модуля для проверки принадлежности адреса.
`ModList.mods[128]`	Фиксированный массив наблюдаемых модулей.
`ModList.count`	Число реально заполненных элементов.
`ModList.truncated`	Список был больше 128 и часть модулей не поместилась; DynamicReport отмечает неполноту.

Breakpoint и BpList

Поле	Что хранит и зачем это нужно
`Breakpoint.addr`	VA байта, временно заменяемого на 0xCC.
`Breakpoint.orig`	Сохранённый исходный байт, обязательный для корректного исполнения команды.
`Breakpoint.armed`	Находится ли 0xCC сейчас в памяти target.
`Breakpoint.rearm_pending`	Ожидает ли точка single-step перед повторной постановкой.
`Breakpoint.kinds`	Битовая маска ролей. Один адрес может одновременно быть несколькими типами точки наблюдения.
`BpList.bps[512] / count`	Фиксированный массив точек и фактическое число элементов.

Зачем kinds является маской

Два логических источника могут дать один VA. Вместо двух независимых 0xCC BpList хранит одну физическую точку и объединяет её роли битами BpKind. При hit Collector выполняет действия всех установленных ролей.

Внутренние структуры переустановки breakpoint

После hit исходная команда должна выполниться один раз, а точка затем должна вернуться. В многопоточном процессе нельзя хранить один глобальный pending breakpoint: разные потоки могут остановиться независимо.

Поле	Что хранит и зачем это нужно
`PendingRearmEntry.tid`	Thread id потока, которому разрешено выполнить одну исходную команду.
`PendingRearmEntry.bp`	Указатель на Breakpoint, который необходимо переустановить после single-step этого потока.
`PendingRearmList.e[64]`	Ограниченный набор per-thread ожиданий. Свободный элемент определяется по bp == NULL.
`LocalProcLoc.mod_name`	Реальный модуль, в котором находится экспорт API в процессе анализатора; учитывает forwarded exports.
`LocalProcLoc.offset`	Смещение функции от базы реального локального модуля. К базе одноимённого модуля target прибавляется именно offset, а не локальный абсолютный VA.

Связь с ASLR и forwarded exports

Адрес API в процессе анализатора нельзя прямо ставить в target: базы модулей различаются. local_proc_loc получает пару «реальный модуль + offset». После появления этого модуля в ModList target-а вычисляется target VA = target_module.base + offset.

Внутренние структуры heuristics.c

Эвристический слой должен считать повторения RVA и анализировать диапазоны независимо от исходного порядка. Для этого он создаёт временные рабочие модели, которые не выходят за пределы heuristics_analyze.

Поле	Что хранит и зачем это нужно
`HEntry.key`	RVA, для которого считается частота.
`HEntry.val`	Число появлений RVA.
`HEntry.used`	Занята ли ячейка open-addressing hash map.
`HMap.e`	Динамический массив HEntry.
`HMap.cap / len`	Ёмкость таблицы и число уникальных занятых ключей.
`RfIdx.begin / end`	Минимальная временная запись диапазона функции для qsort и поиска overlaps/duplicate begins.
`FindingLocation.set / rva`	Первый представитель нарушения. set отличает корректный RVA 0 от отсутствия сохранённой координаты.

Почему используется hash map

Для reuse и dominance необходимо узнать, сколько раз встречается каждый UnwindRVA и HandlerRVA. Повторный полный поиск по массиву для каждого элемента дал бы квадратичную работу. HMap агрегирует ключи за один проход с open addressing и linear probing.

Почему диапазоны сортируются в копии

Исходный rf[] нужен отчёту в первоначальном порядке. heuristics.c копирует только begin/end в RfIdx[], сортирует временный массив и проверяет глобальные пересечения, не переставляя основную модель.

Внутренние структуры report.c

report.c не создаёт новых parser findings, но строит сводное представление статического и динамического слоёв. Для этого применяются небольшие внутренние структуры, не публикуемые в report.h.

Поле	Что хранит и зачем это нужно
`AssessmentSummary.correlated`	Есть ли определённая кодом связь между статическим evidence и runtime signals.
`dynamic_signals`	Число категорий динамических сигналов, учитываемых в assessment.
`final_confidence`	Отчётная confidence после ограниченного усиления за полную динамическую сессию и correlation.
`final_verdict`	Строковая сводная категория static/dynamic assessment.
`static_evidence_level`	Ссылка на уровень evidence из HeuristicReport.
`runtime_evidence_level`	Вычисленная текстовая сила runtime-наблюдений.
`strength`	Категория correlated/unconfirmed case.
`ManualValidationSteps.items[16]`	Указатели на заранее определённые рекомендации ручной проверки.
`ManualValidationSteps.count`	Фактическое число выбранных рекомендаций.

Граница этих структур

AssessmentSummary предназначена для представления объединённого отчёта. Она не записывается обратно в HeuristicReport и не изменяет исходный static score. ManualValidationSteps выбирается по уже существующим счётчикам и findings.

Внутренняя структура SHA-256

Sha256Ctx объявлена внутри util.c и существует только во время вычисления цифрового отпечатка входных байтов.

Поле	Что хранит и зачем это нужно
`state[8]`	Восемь 32-битных рабочих слов текущего состояния SHA-256.
`bit_count`	Общее число уже учтённых входных битов; участвует в обязательном финальном padding.
`block[64]`	Текущий 512-битный блок алгоритма.
`block_len`	Сколько байтов текущего блока уже заполнено.

Зачем она не находится в util.h

Другим модулям нужен только готовый sha256_hex. Публикация внутреннего состояния позволила бы внешнему коду зависеть от деталей реализации алгоритма. Поэтому util.h экспортирует функцию, а не устройство Sha256Ctx.

Флаги и перечисления, управляющие смыслом структур

Enum - перечисление именованных целых констант. Bitmask - битовая маска, в которой несколько независимых признаков могут одновременно находиться в одном числе.

UnwindFlags

Поле	Что хранит и зачем это нужно
`UNW_FLAG_EHANDLER = 0x01`	Tail содержит RVA exception handler.
`UNW_FLAG_UHANDLER = 0x02`	Tail содержит RVA termination/unwind handler.
`UNW_FLAG_CHAININFO = 0x04`	Tail содержит chained RUNTIME_FUNCTION; сочетание меняет способ чтения следующих 12 байтов.

UnwindAnom

Поле anomalies объединяет UA_* bits. Группы включают неизвестную version; truncation; opcode/operand; выход CodeOffset; порядок codes; alignment; handler/chain; cycle/depth; ABI register class, reserved OpInfo, push order, chain restrictions и save offsets. Один summary может иметь несколько битов одновременно.

HeuristicEvidenceFamily

Маска evidence_families_mask различает пять источников: целостность таблицы, целостность unwind, размещение metadata, reuse UNWIND_INFO и dominance handler-a. Разнообразие семейств участвует в confidence.

Результаты чтения

RuntimeFunctionReadResult различает OK, MALFORMED и OOM. Это важно: структурно повреждённый вход и нехватка памяти являются разными классами отказа. ParseResult в main.c отдельно различает нормальный запуск, help, version и ошибку CLI.

Часть III. Пофайловое устройство проекта

Концептуальная карта и структуры данных уже объяснены. Теперь тот же анализ рассматривается по исходным файлам: ответственность файла, входные структуры, внутренний маршрут, функции, выражения C, ошибки и передача результата соседнему модулю.

Как пользоваться этой частью

Сначала прочитайте паспорт нужного .c-файла, затем его полный маршрут и только после этого переходите к карточкам функций. При встрече структуры возвращайтесь к Части II: там объяснены её владелец, поля и связь с общей моделью.

Все модули и их взаимосвязь

Эта глава сначала объясняет проект без погружения в отдельные выражения C. Под модулем здесь понимается файл исходного кода с одной областью ответственности. Каждый следующий модуль получает уже подготовленный результат предыдущего уровня и не должен заново толковать исходные байты по собственным правилам.

Главная цепочка данных

```
команда пользователя
-> main.c: конфигурация Args
-> util.c: байты файла в Buf
-> pe.c: структурная карта PeImage
-> unwind.c: RuntimeFunction[] и UnwindSummary[]
-> heuristics.c: HeuristicReport
-> dynamic_win.c: DynamicReport (только по запросу и на Windows)
-> report.c: CSV / Ghidra CSV / JSON / HTML
```

Стрелка означает передачу данных, а не обязательно прямой вызов одного модуля другим. Почти все вызовы организует main.c. Например, pe.c не запускает unwind.c сам: main.c сначала получает PeImage от pe.c, а затем передаёт эту модель функциям unwind.c.

Три типа связей

- Последовательная связь: результат предыдущего этапа необходим следующему. Без PeImage нельзя корректно читать RVA из .pdata и .xdata.
- Сервисная связь: util.c предоставляет безопасные операции нескольким модулям и не является только одним проходным этапом.
- Необязательная связь: dynamic_win.c запускается только при выбранном динамическом режиме; статический анализ и формирование отчёта возможны без него.

Что является общим результатом системы

Проект не передаёт между этапами один текстовый комментарий. Он последовательно строит типизированные структуры C: Buf хранит байты, PeImage описывает PE, RuntimeFunction и UnwindSummary описывают EH/Unwind, HeuristicReport хранит аналитические признаки, DynamicReport хранит наблюдения выполнения, ReportMetadata фиксирует происхождение результата. report.c превращает эти структуры во внешние документы.

main.c - координатор всего анализа

Что это. Это точка входа программы и orchestration layer - координирующий слой. Он определяет порядок работы остальных модулей, но не подменяет их внутренние алгоритмы.

Зачем существует. Без main.c отдельные библиотеки существовали бы как набор несвязанных функций. Его задача - принять запрос пользователя, вызвать этапы в допустимом порядке, сохранить результаты, обработать технические ошибки и завершить жизненный цикл памяти.

Что получает модуль

При запуске среда C передаёт argc - количество аргументов - и argv - массив строк. В них находятся путь к исследуемому PE, выбранные режимы, пути отчётов, ограничения динамической сессии и необязательные аргументы запускаемого target - исследуемого процесса.

Что модуль делает с полученными данными

1. Проверяет параметры командной строки и собирает их в Args - единую структуру конфигурации.

2. Вызывает util.c для загрузки файла, затем pe.c и unwind.c для построения статической модели.

3. По запросу вызывает heuristics.c и dynamic_win.c, сохраняя их результаты отдельно.

4. Собирает ReportMetadata, вызывает report.c, учитывает ошибки записи и формирует exit code - код завершения.

5. В конце освобождает все динамически выделенные массивы, строки и отчёты в определённом порядке.

Что получается на выходе

На выходе самого main.c нет новой предметной структуры. Его результат - выполненный сценарий анализа: созданные файлы отчётов, диагностические сообщения и числовой код завершения, по которому оболочка понимает, был ли запрос технически выполнен.

Связь со следующим этапом

main.c передаёт данные всем следующим этапам. Он является единственным владельцем общей последовательности: Buf -> PeImage -> EH/Unwind model -> аналитические и динамические отчёты -> внешние файлы.

Граница ответственности

main.c не должен самостоятельно декодировать PE, UNWIND_CODE или рассчитывать признаки. Иначе правила формата оказались бы размазаны по программе и перестали бы иметь одного владельца.

Как это выглядит в одном анализе

Пользователь указывает sample.exe и путь HTML. main.c читает параметры, получает от util.c Buf, от pe.c PeImage, от unwind.c массивы функций, от heuristics.c findings, при необходимости от dynamic_win.c события, а затем передаёт всё report.c.

util.c - безопасная основа для байтов, строк и файлов

Что это. Это общий служебный модуль. Он ничего не знает о смысле PE, EH или unwind, но предоставляет операции, без которых остальные модули не смогут безопасно работать с недоверенным файлом.

Зачем существует. Исследуемый файл может быть повреждён и содержать ложные размеры или смещения. util.c централизует проверки границ, владение памятью, сборку строк, атомарную запись и SHA-256, чтобы каждый предметный модуль не реализовывал эти механизмы заново.

Что получает модуль

В зависимости от функции модуль получает путь к файлу, объект Buf, файловое смещение off, длину len, фрагмент строки или массив байтов для хеширования.

Что модуль делает с полученными данными

1. Загружает весь файл в память и создаёт Buf, где data указывает на первый байт, а size задаёт точную длину.

2. Перед каждым чтением проверяет, что требуемый диапазон полностью находится внутри Buf.

3. Собирает числа little-endian - с младшим байтом первым - в uint16_t, uint32_t или uint64_t.

4. Управляет StrBuf - расширяемой строкой - для диагностик и сериализации отчётов.

5. Записывает готовый файл атомарно и вычисляет SHA-256 исследуемых байтов.

Что получается на выходе

Результат зависит от вызванной функции: Buf с байтами, безопасно прочитанное значение, готовая StrBuf, записанный файл либо 64-символьный SHA-256. Успех обычно обозначается true, отказ - false без выхода за границы памяти.

Связь со следующим этапом

Buf получает pe.c. Функциями безопасного чтения пользуются pe.c и unwind.c. StrBuf и атомарная запись применяются при формировании ошибок и отчётов. SHA-256 попадает через main.c в ReportMetadata.

Граница ответственности

util.c не решает, что означает конкретный байт PE, и не формирует аналитические выводы. Он гарантирует только технически контролируемый доступ и инфраструктурные операции.

Как это выглядит в одном анализе

Когда pe.c просит прочитать четыре байта по смещению 0x100, util.c сначала доказывает наличие диапазона 0x100..0x104, затем возвращает число. Значение этого числа уже объясняет pe.c.

pe.c - построение координатной карты PE

Что это. Это форматный parser - синтаксический разборщик - верхнего уровня PE32+ AMD64. Он превращает неизвестные байты Buf в PeImage, то есть проверенную модель заголовков, секций и адресов.

Зачем существует. EH/Unwind-структуры хранят адреса как RVA, а файл читается по file offset. До построения PeImage проект не может доказательно определить, какие физические байты соответствуют конкретному RVA.

Что получает модуль

Модуль получает неизменяемый Buf с байтами, пустую структуру PeImage для результата и StrBuf для понятного сообщения при фатальной ошибке.

Что модуль делает с полученными данными

1. Проверяет MZ, положение PE-заголовка, сигнатуру PE, архитектуру AMD64 и формат PE32+.

2. Извлекает размеры образа, выравнивания, количество секций и запись Exception Directory.

3. Читает section headers - заголовки секций - и сохраняет виртуальные и файловые диапазоны каждой секции.

4. Проверяет границы, переполнения, порядок, выравнивание и пересечения, сохраняя счётчики нарушений.

5. Предоставляет функции поиска секции по RVA и перевода RVA в file offset.

Что получается на выходе

PeImage содержит основные поля PE, массив PeSection, координаты Exception Directory и диагностические счётчики. Это не копия всего файла, а карта, позволяющая другим модулям безопасно локализовать данные.

Связь со следующим этапом

main.c передаёт PeImage в unwind.c, heuristics.c, dynamic_win.c и report.c. Все они используют одну систему координат и не строят собственные несовместимые версии PE-карты.

Граница ответственности

pe.c не читает RUNTIME_FUNCTION и UNWIND_INFO, не оценивает подозрительность и не запускает файл. Его ответственность заканчивается на структуре PE и адресном отображении.

Как это выглядит в одном анализе

Если Exception Directory имеет RVA 0x3000, pe.c определяет покрывающую секцию и вычисляет физическое смещение. unwind.c затем читает таблицу именно из доказанного диапазона байтов.

unwind.c - извлечение и декодирование EH/Unwind

Что это. Это предметный parser и decoder - разборщик и декодер - x64 EH/Unwind metadata. Он строит функцию-ориентированную карту .pdata и .xdata.

Зачем существует. PeImage сообщает, где находятся байты, но ещё не объясняет их смысл как RUNTIME_FUNCTION, UNWIND_INFO и UNWIND_CODE. unwind.c выполняет именно этот переход от адресной карты к структурированной модели раскрутки стека.

Что получает модуль

Модуль получает Buf, PeImage, параметры строгости и ограничения цепочек. Для разбора одной записи он получает unwind_rva из соответствующей RuntimeFunction.

Что модуль делает с полученными данными

1. Находит Exception Directory в файле и читает массив 12-байтовых RUNTIME_FUNCTION.

2. Для каждой записи связывает диапазон BeginRVA..EndRVA с UnwindInfoAddress.

3. Читает заголовок UNWIND_INFO и декодирует Version, Flags, SizeOfProlog, CountOfCodes и frame fields.

4. Разбирает UNWIND_CODE с учётом дополнительных slots - двухбайтовых ячеек операндов - и вычисляет сводные эффекты.

5. Учитывает padding - выравнивающее заполнение - handler tail и CHAININFO; ограничивает циклы и глубину цепочки.

6. При локальном повреждении сохраняет точную parse diagnostic вместо прекращения анализа всех остальных функций.

Что получается на выходе

Первый выход - RuntimeFunction[]: диапазоны функций и ссылки на metadata. Второй выход - UnwindSummary[]: разобранные поля, операции, диагностические счётчики, сведения о handler/chain и parse_error. Массив us_ok[] отдельно отмечает успешность каждой записи.

Связь со следующим этапом

heuristics.c сопоставляет RuntimeFunction[i], UnwindSummary[i] и us_ok[i] по одинаковому индексу. report.c использует ту же связь для строк таблицы и доказательств. dynamic_win.c получает статические handler RVA для точек наблюдения.

Граница ответственности

unwind.c проверяет синтаксис и часть структурной семантики metadata, но не дизассемблирует .text и не доказывает соответствие каждой unwind-операции машинному прологу.

Как это выглядит в одном анализе

Для функции 0x1000..0x1080 модуль находит UnwindInfoAddress 0x4000, читает header, определяет число slots, декодирует операции и сохраняет результат в us[i]. Если хвост обрывается, ошибка остаётся привязанной к этой функции.

heuristics.c - аналитическая интерпретация структурных наблюдений

Что это. Это эвристический слой. Heuristic - инженерное правило, которое связывает проверяемое наблюдение с возможным риском и необходимостью ручной проверки; это не машинное обучение и не доказательство вредоносности.

Зачем существует. После unwind.c проект располагает множеством полей и счётчиков. Аналитику нужен не бесформенный перечень чисел, а локализованные findings - сообщения о том, какое правило сработало, где находится проблема, почему она существенна и насколько сильны доказательства.

Что получает модуль

Модуль получает готовые PeImage, RuntimeFunction[], UnwindSummary[], us_ok[] и количество функций. Исходный файл самостоятельно он не перечитывает.

Что модуль делает с полученными данными

1. Проверяет инварианты диапазонов, сортировку, пересечения, gaps - промежутки - и распространённость нарушений.

2. Оценивает размещение UNWIND_INFO и handler targets по секциям и их правам доступа.

3. Агрегирует reuse - повторное использование - dominance - преобладание - ошибки операций и цепочек.

4. Для каждого сработавшего правила создаёт finding с code, family, severity, location, evidence и рекомендацией.

5. Суммирует инженерные weights - веса - в score, учитывает mitigating factors - ослабляющие обстоятельства - и вычисляет confidence и verdict.

Что получается на выходе

HeuristicReport содержит список findings, число срабатываний, evidence families, score, confidence, verdict, mitigations и человекочитаемые пояснения.

Связь со следующим этапом

report.c отображает HeuristicReport в JSON и HTML. main.c может одновременно передать статический отчёт и DynamicReport, чтобы отчётный слой показал их раздельно и сформировал assessment - сводную оценку.

Граница ответственности

Модуль не изменяет данные parser-ов, не запускает target и не утверждает, что score является вероятностью атаки. Он приоритизирует доказательства для аналитика.

Как это выглядит в одном анализе

Если два RUNTIME_FUNCTION пересекаются, finding хранит оба диапазона и объясняет неоднозначность. Если необычность имеет допустимое объяснение, mitigation снижает уверенность, не стирая исходное наблюдение.

dynamic_win.c - наблюдение одного выполнения под Windows

Что это. Это платформозависимый dynamic collector - сборщик динамических наблюдений. Он создаёт target под Windows debugger и фиксирует ограниченный набор событий, относящихся к исключениям, обработчикам и динамическим таблицам функций.

Зачем существует. Статический слой видит только файл на диске. Динамические function tables могут появиться после запуска, а handler RVA может существовать, но ни разу не достигаться. Динамический слой добавляет факты конкретного выполненного сценария.

Что получает модуль

Модуль получает путь и аргументы target, PeImage, статические RuntimeFunction/UnwindSummary, timeout, число шагов трассировки, пустой DynamicReport и StrBuf ошибки.

Что модуль делает с полученными данными

1. Создаёт процесс под debugger и определяет фактическую базу его образа с учётом ASLR - случайного размещения адресов.

2. Ставит программные breakpoints - точки останова - на найденные handler RVA и выбранные системные API.

3. Обрабатывает debug events: создание процессов и потоков, загрузку модулей, исключения и завершение.

4. После breakpoint hit восстанавливает исходный байт, выполняет одну настоящую команду и переустанавливает точку останова.

5. Считывает аргументы RtlAddFunctionTable и родственных API из регистров x64, фиксирует handler hits и ограниченный RIP trace.

6. Останавливает сессию по завершению или timeout и явно записывает признаки неполноты данных.

Что получается на выходе

DynamicReport содержит статус запуска, исключения, API/handler hits, сведения о регистрациях dynamic function tables, счётчики потерь и массив адресов RIP trace. Отсутствие события означает только, что оно не наблюдалось в данном сценарии.

Связь со следующим этапом

report.c помещает DynamicReport в отдельные динамические разделы и сопоставляет runtime addresses со статической картой. heuristics.c при этом не переписывает свои выводы задним числом.

Граница ответственности

Модуль работает полноценно только на Windows, не обеспечивает изоляцию вредоносной программы и не исследует все возможные пути. Запуск подозрительных образцов требует отдельной лабораторной среды.

Как это выглядит в одном анализе

Если target вызывает RtlAddFunctionTable, breakpoint фиксирует вызов и аргументы регистрации. Если выполнение достигает статически известного handler, отчёт получает handler hit и последующие адреса RIP в пределах заданного лимита.

report.c - преобразование внутренних моделей в отчёты

Что это. Это serialization layer - слой сериализации. Сериализация означает преобразование структур, находящихся в памяти, в устойчивый внешний формат, который можно сохранить, прочитать человеком или передать другой программе.

Зачем существует. Результаты разных этапов имеют разные структуры и вложенность. report.c собирает их без повторного анализа байтов, обеспечивает корректное экранирование и представляет одну доказательную модель в нескольких форматах.

Что получает модуль

Модуль получает путь результата, PeImage, связанные массивы RuntimeFunction/UnwindSummary/us_ok, необязательные HeuristicReport и DynamicReport, а для JSON/HTML также ReportMetadata.

Что модуль делает с полученными данными

1. Проверяет согласованность указателей, количеств элементов и обязательных metadata.
2. Экранирует специальные символы отдельно по правилам CSV, JSON и HTML.
3. Формирует CSV для табличной обработки и Ghidra CSV для импорта адресной разметки.
4. Формирует JSON как машиночитаемый контракт с устойчивыми codes и типами.
5. Формирует автономный HTML с картой секций, матрицей признаков, findings, статическими таблицами и динамическими событиями.
6. Передаёт готовую строку util.c для атомарной записи.

Что получается на выходе

На выходе появляются законченные файлы CSV, Ghidra CSV, JSON и/или HTML. Функция writer возвращает true только после успешного формирования и записи соответствующего файла.

Связь со следующим этапом

CSV может быть открыт в табличном редакторе; JSON получает backend или скрипт; HTML читает аналитик; Ghidra CSV передаётся внешнему скрипту импорта. main.c по результату writer обновляет технический exit code.

Граница ответственности

report.c не начисляет score, не создаёт новые findings и не повторяет PE-разбор. Он отображает уже построенные модели и может вычислять только отчётное сопоставление static/dynamic assessment по определённым правилам представления.

Как это выглядит в одном анализе

Одна RuntimeFunction отображается вместе с соответствующим UnwindSummary. Если us_ok[i] равен false, HTML показывает parse diagnostic, а не придумывает отсутствующие поля.

app_info.c - происхождение и воспроизводимость результата

Что это. Это небольшой модуль provenance. Provenance означает сведения о происхождении: какая версия инструмента, на какой платформе и когда сформировала результат.

Зачем существует. Без таких данных два отчёта невозможно корректно сравнивать: различие может быть связано не с бинарником, а с версией схемы, компилятором или режимом сборки анализатора.

Что получает модуль

Функции обычно не получают предметных структур. Для timestamp - временной отметки - передаются массив символов и его размер.

Что модуль делает с полученными данными

1. Возвращает версию инструмента и версию схемы отчёта.

2. Определяет компилятор, платформу и режим сборки по compile-time macros - условиям компиляции.

3. Формирует время UTC в стандартизованной строковой форме.

4. main.c объединяет эти значения с SHA-256 и командой запуска в ReportMetadata.

Что получается на выходе

На выходе находятся короткие неизменяемые строки и timestamp. Вместе с данными main.c они образуют ReportMetadata, сериализуемую в JSON и HTML.

Связь со следующим этапом

ReportMetadata получает report.c. Пользователь отчёта может установить, каким исполняемым файлом анализатора и при каких условиях сформирован документ.

Граница ответственности

app_info.c не анализирует исследуемый PE и не влияет на score или findings. Он описывает инструмент и запуск, а не объект исследования.

Как это выглядит в одном анализе

В двух отчётах одинаковый SHA-256 входа, но разные tool_version. Это показывает, что исследовались одни байты разными версиями анализатора.

Заголовочные файлы include/*.h - контракты между модулями

Что это. Это не исполняемые модули и не отдельные этапы анализа. Header file - заголовочный файл - объявляет структуры, перечисления, константы и функции, которыми разрешено пользоваться другим .c-файлам.

Зачем существует. Компилятор C обрабатывает каждый .c отдельно. Чтобы main.c мог корректно вызвать pe_parse или heuristics_analyze, он должен заранее знать точную сигнатуру функции и устройство передаваемых типов. Эту общую договорённость задают .h-файлы.

Что получает модуль

Заголовок получает не runtime-данные, а включается препроцессором директивой #include во время сборки программы.

Что модуль делает с полученными данными

1. util.h публикует Buf, StrBuf и общие безопасные операции.
2. pe.h публикует PeSection, PeImage и функции адресного отображения.
3. unwind.h публикует RuntimeFunction, UnwindSummary и интерфейс декодера.
4. heuristics.h и dynamic.h публикуют структуры двух аналитических отчётов.
5. report.h и app_info.h публикуют интерфейсы сериализации и provenance.

Что получается на выходе

После #include компилятор знает типы и проверяет правильность вызовов. В исполняющейся программе заголовочный файл сам по себе ничего не создаёт и не вызывается.

Связь со следующим этапом

Структура, объявленная в одном .h, становится общим языком соседних модулей. Например, PeImage создаётся pe.c, но читается unwind.c, heuristics.c, dynamic_win.c и report.c.

Граница ответственности

В .h следует хранить публичный контракт, а внутренние вспомогательные функции с static остаются внутри соответствующего .c. Заголовок не является местом выполнения алгоритма.

Как это выглядит в одном анализе

main.c включает pe.h и выделяет PeImage. pe.c заполняет её по тому же объявлению, а unwind.c получает const PeImage*, поэтому все три файла одинаково понимают расположение полей.

ghidra/import_unwind_csv.py - внешний мост к Ghidra

Что это. Это вспомогательный скрипт Python для среды Ghidra, а не часть C-ядра и не модуль, вызываемый main.c. Он импортирует специально сформированный Ghidra CSV в уже открытый проект анализа.

Зачем существует. Ядро строит адресную карту и может сохранить её в переносимом файле, но автоматическая разметка интерфейса Ghidra требует кода, выполняемого внутри самой Ghidra. Поэтому интеграция отделена от parser-а.

Что получает модуль

Скрипт получает Ghidra CSV, созданный report.c, и текущую программу, открытую в Ghidra. Пользователь запускает импорт явно.

Что модуль делает с полученными данными

1. Читает строки CSV и проверяет ожидаемые столбцы.
2. Преобразует RVA из отчёта в адреса текущего image base внутри Ghidra.
3. Создаёт или обновляет комментарии и разметку, предусмотренные форматом импорта.
4. Не выполняет заново PE/EH/Unwind-анализ и не пересчитывает эвристики.

Что получается на выходе

На выходе изменяется база анализа Ghidra: адреса получают подготовленную разметку, которую аналитик видит рядом с кодом. Новый HeuristicReport при этом не создаётся.

Связь со следующим этапом

Связь односторонняя: report.c создаёт Ghidra CSV, затем внешний скрипт читает его. C-ядро не зависит от установки Ghidra и продолжает работать без неё.

Граница ответственности

Интеграция не является декомпилятором, не сравнивает автоматически каждую UWOP с прологом и не заменяет ручную интерпретацию аналитика.

Как это выглядит в одном анализе

После статического анализа аналитик импортирует CSV. В Ghidra он переходит к отмеченному RVA и сопоставляет finding из отчёта с дизассемблированной функцией.

Как модули совместно обрабатывают одну функцию

Ниже показана не отдельная функция C, а движение одной записи исследуемого PE через систему. Это позволяет увидеть, почему результаты разных модулей нельзя рассматривать изолированно.

1. pe.c устанавливает координаты

PeImage сообщает, в какой секции находится Exception Directory и как её RVA переводится в file offset. Пока существует только адресная основа; диапазон конкретной функции ещё не прочитан.

2. unwind.c создаёт запись функции

Из .pdata извлекается RuntimeFunction[i]: BeginRVA, EndRVA и UnwindRVA. Затем по UnwindRVA читается .xdata, а результат помещается в UnwindSummary[i]. Индекс i сохраняет связь между диапазоном кода и metadata.

3. heuristics.c интерпретирует наблюдения

Эвристический слой получает уже проверенные поля. Если диапазон пересекается с другим, metadata размещена необычно или декодирование завершилось ошибкой, finding ссылается на конкретный RuntimeFunction[i] и его UnwindSummary[i].

4. dynamic_win.c при необходимости проверяет достижимость

Если summary содержит handler RVA, динамический коллектор может поставить точку наблюдения на фактический VA после загрузки процесса. Попадание фиксируется в DynamicReport, но отсутствие попадания не удаляет статический finding.

5. report.c сохраняет доказательную цепочку

HTML и JSON связывают диапазон функции, UnwindRVA, состояние разбора, finding и доступные runtime events. Поэтому аналитик видит происхождение вывода: PE -> RUNTIME_FUNCTION -> UNWIND_INFO -> проверка -> finding -> возможное динамическое подтверждение.

Главный вывод

Модули не выполняют одну и ту же работу. Каждый повышает уровень представления данных: util.c обеспечивает безопасные байты; pe.c задаёт координаты; unwind.c раскрывает EH/Unwind; heuristics.c приоритизирует нарушения; dynamic_win.c добавляет наблюдения выполнения; report.c делает доказательства доступными человеку и внешним системам; main.c удерживает всю последовательность.

1. Модуль как часть системы

Модуль pe.c отвечает за переход от необработанной последовательности байтов к структурированной модели PE32+ AMD64. До его запуска проект знает лишь адрес первого байта файла и общую длину файла. После успешного завершения проект знает, где располагаются заголовки и секции, каковы границы образа, где заявлена Exception Directory и как переводить RVA в файловые смещения.

PE (Portable Executable, переносимый исполняемый формат) - формат EXE, DLL и ряда других образов Windows. PE32+ - 64-разрядная разновидность формата. AMD64 в данном проекте обозначает архитектуру x86-64, для которой определены анализируемые структуры RUNTIME_FUNCTION и UNWIND_INFO.

Что было до pe.c

В main.c функция buf_init_from_file загружает файл в память. Результат представлен объектом Buf: поле data указывает на байты, а поле size хранит их количество. Эти байты пока не являются картой PE: проект ещё не знает, какие из них означают заголовок, секцию или каталог данных.

Что делает pe.c

- проверяет, что вход действительно имеет поддерживаемую структуру PE32+ AMD64;
- извлекает параметры, необходимые для адресных преобразований;
- строит массив описаний секций;
- фиксирует нарушения размещения и выравнивания секций;
- извлекает RVA и размер Exception Directory;
- формирует объект PeImage, который становится общей картой файла для последующих модулей.

Чего pe.c не делает

Модуль не читает записи RUNTIME_FUNCTION, не декодирует UNWIND_INFO и не начисляет эвристические баллы. Он устанавливает структурную основу. Разбор EH/Unwind начинается в unwind.c, а аналитическая интерпретация выполняется в heuristics.c.

2. Вход, выход и контракт

Функция	pe_parse
Сигнатура	`bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);`
Объявлена	include/pe.h, строка 56. Заголовочный файл публикует функцию другим модулям.
Реализована	src/pe.c, начиная со строки 36. Здесь находится тело функции.
Вызывается	src/main.c, строка 294; также fuzz-тест tests/fuzz/fuzz_parser.c.
Вход	out - адрес будущей структуры PeImage; b - адрес буфера с байтами файла; err - адрес строкового буфера для диагностического сообщения.
Результат	true, если поддерживаемая PE-модель построена; false, если продолжать анализ безопасно или корректно нельзя.
Смысл	Это главная функция модуля. Она последовательно преобразует недоверенные байты в проверенную координатную модель файла.

Почему параметры являются указателями

Запись PeImage* out означает: параметр out хранит адрес объекта PeImage. Функция получает не копию всей структуры, а возможность заполнить объект, созданный вызывающим кодом. Запись const Buf* b также передаёт адрес, но const запрещает pe_parse изменять сам буфер через этот указатель. Это выражает контракт: модуль читает входной файл, но не переписывает его.

```
PeImage img; // main.c создает пустой объект
pe_parse(&img, &b, &err); // &img означает «адрес объекта img»
// внутри функции out указывает на img
out->size_of_image = value; // -> обращается к полю через указатель
```

Что означает bool

Тип bool хранит логическое значение true или false. Здесь он отвечает только на вопрос: удалось ли технически построить поддерживаемую PE-модель. Он не является вердиктом о безопасности файла. PE может успешно разобраться и при этом содержать структурные аномалии, сохранённые в счётчиках PeImage.

3. Две структуры результата

Модель строится на двух уровнях. PeSection описывает одну секцию. PeImage описывает образ целиком и владеет динамически выделенным массивом PeSection.

PeSection - одна строка карты секций

- name[9] - имя секции; девятый байт оставлен для завершающего нуля строки C;
- virtual_address - RVA начала секции при отображении образа в память;
- virtual_size - заявленный размер секции в виртуальной памяти;
- raw_ptr - файловое смещение начала содержимого секции;
- raw_size - количество байтов секции, физически хранящихся в файле;
- characteristics - битовые флаги назначения и прав секции, например чтение, запись или исполнение.

PeImage - общая карта образа

- image_base, size_of_image и size_of_headers задают границы и базовые параметры образа;
- section_alignment и file_alignment задают правила выравнивания в памяти и файле;
- exception_rva и exception_size указывают на Exception Directory;
- num_sections и sections дают доступ к массиву секций;
- section_* counters сохраняют количество найденных структурных нарушений;
- exception_directory_oob отмечает выход каталога исключений за допустимые границы.

Важное различие

PeImage - не копия всего файла и не готовая EH/Unwind-карта. Это проверенная карта размещения PE. На её основе unwind.c строит EH/Unwind-карту: читает таблицу функций и метаданные раскрутки стека.

Этап 1. Принять вход и подготовить пустой PeImage

Что делает модуль

pe_parse проверяет наличие трёх обязательных объектов: места для результата, входного буфера и буфера ошибок. Затем полностью обнуляет будущий PeImage. После этого каждая ещё не заполненная числовая величина равна нулю, указатель sections равен NULL, а логические признаки равны false.

Зачем это делается

Обнуление задаёт известное начальное состояние. Если дальнейший разбор остановится, в структуре не останутся случайные значения из памяти стека. Это особенно важно для указателя sections: случайный адрес нельзя безопасно передавать в free.

```
if (!out || !b || !err) return false;
memset(out, 0, sizeof(*out));
```

Разбор языка C

Оператор ! означает логическое отрицание. В условии указатель трактуется как истина, если он не NULL. Поэтому !out означает «указатель out отсутствует». Оператор || означает «или»: достаточно отсутствия одного параметра, чтобы функция вернула false.

Выражение sizeof(*out) вычисляет размер объекта, на который указывает out. Такой вариант устойчивее, чем ручное написание sizeof(PeImage): если тип параметра изменится, размер останется согласованным с ним.

Функция	memset
Сигнатура	`void* memset(void* dest, int value, size_t count);`
Объявлена	Стандартная библиотека C, заголовок string.h.
Реализована	Реализована стандартной библиотекой среды выполнения.
Вызывается	src/pe.c:38; также используется в pe_free:262.
Вход	dest - начальный адрес области; value - записываемое значение байта; count - число байтов.
Результат	Адрес dest. В pe_parse возвращаемое значение не используется.
Смысл	Заполняет все байты PeImage нулями, создавая предсказуемое начальное состояние.

Что получает следующий этап

Следующий этап получает корректные ссылки на входные объекты и чистый PeImage, готовый к постепенному заполнению.

Этап 2. Проверить DOS-заголовок и получить e_lfanew

Что делает модуль

Модуль начинает с первых байтов файла. Он проверяет сигнатуру MZ, затем читает 32-разрядное поле по файловому смещению 0x3C. Это поле называется e_lfanew и содержит файловое смещение PE-заголовка.

Зачем нужен этот этап

PE-заголовок не обязан начинаться по одному фиксированному адресу во всех файлах. DOS stub может иметь разную длину. Поэтому проект не угадывает позицию PE-заголовка, а получает её из поля формата. Без достоверного e_lfanew все последующие чтения обращались бы к неизвестным байтам.

Функция	read_dos_lfanew
Сигнатура	`static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew);`
Объявлена	Не публикуется в pe.h: static ограничивает видимость текущим src/pe.c.
Реализована	src/pe.c:11-19.
Вызывается	Только pe_parse, src/pe.c:41.
Вход	b - неизменяемый буфер файла; out_lfanew - адрес переменной, куда будет записано найденное смещение.
Результат	true при корректном чтении MZ и e_lfanew; false при коротком файле, неверной сигнатуре или невозможности чтения.
Смысл	Локализует первую проверку формата и передаёт pe_parse координату следующего заголовка.

Как работает read_dos_lfanew по шагам

- Создаёт локальную переменную mz типа uint16_t и задаёт ей ноль.
- Вызывает buf_read_u16 для чтения двух байтов с file offset 0. Смысл: безопасно получить DOS signature, не обращаясь за концом файла.
- Сравнивает mz с PE_DOS_MAGIC, равным 0x5A4D. В little-endian представлении это байты 4D 5A, то есть символы MZ.
- Создаёт lfanew типа uint32_t и через buf_read_u32 читает четыре байта с offset 0x3C.
- Записывает значение в объект вызывающей функции выражением *out_lfanew = lfanew.
- Возвращает true, сообщая, что координата PE-заголовка извлечена.

```
uint32_t lfanew = 0;
if (!buf_read_u32(b, 0x3c, &lfanew)) return false;
*out_lfanew = lfanew;
```

Звёздочка перед out_lfanew означает разыменование: обращение к памяти по адресу, который хранится в указателе. Именно поэтому локальное значение функции оказывается в переменной lfanew внутри pe_parse.

Этап 3. Проверить достижимость PE-заголовка и его сигнатуру

Что делает модуль

Полученное e_lfanew пока является лишь числом из недоверенного файла. pe_parse проверяет, что по этому смещению в файле помещаются как минимум четыре байта PE signature и двадцать байтов COFF File Header. После проверки он читает сигнатуру PE\0\0.

Смысл проверки границ

Злоумышленно или случайно повреждённый файл может записать в e_lfanew очень большое значение. Если использовать его без проверки, программа попытается читать память за пределами загруженного файла. Проверка превращает недоверенное смещение в допустимую координату.

```
if ((size_t)lfanew + 4 + 20 > b->size) return false;
if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) return false;
```

Почему присутствует приведение к size_t

size_t - беззнаковый тип, предназначенный для размеров объектов и индексов памяти. Поле lfanew имеет фиксированный формат uint32_t, а b->size хранится как size_t. Приведение (size_t)lfanew переводит значение в тип, используемый для размеров в текущей системе.

Функция buf_read_u32

Функция	buf_read_u32
Сигнатура	`bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);`
Объявлена	include/util.h:26.
Реализована	src/util.c:59-64.
Вызывается	Многократно вызывается pe_parse для всех 32-разрядных полей PE.
Вход	b - буфер; off - файловое смещение; out - адрес переменной результата.
Результат	true и декодированное little-endian число либо false, если четыре байта недоступны.
Смысл	Централизует безопасное чтение 32-разрядных полей, чтобы pe.c не выполнял небезопасных обращений к байтам напрямую.

Как она работает

Функция создаёт локальный массив из четырёх байтов, вызывает buf_read для проверенного копирования, затем le32 собирает число в порядке little-endian. Little-endian означает, что младший байт числа расположен раньше старшего. Этот порядок используется полями PE на x86/x64.

Что получает следующий этап

Теперь доказано, что файл содержит доступный PE-заголовок с правильной сигнатурой. Можно читать COFF File Header по адресу lfanew + 4.

Этап 4. Прочитать COFF File Header и ограничить архитектуру AMD64

Что делает модуль

После PE signature следуют двадцать байтов COFF File Header. Проект извлекает Machine, NumberOfSections и SizeOfOptionalHeader. Эти поля определяют архитектуру, количество описаний секций и размер следующего заголовка.

```
size_t filehdr = (size_t)lfanew + 4;
buf_read_u16(b, filehdr + 0, &machine);
buf_read_u16(b, filehdr + 2, &nsects);
buf_read_u16(b, filehdr + 16, &opt_size);
```

Смысл смещений +0, +2 и +16

Это не произвольные числа. Они являются позициями полей внутри стандартизованной структуры IMAGE_FILE_HEADER. pe.c не накладывает C-структуру непосредственно на байты файла: он читает каждое нужное поле через bounds-safe функции. Такой подход исключает зависимость от выравнивания структуры компилятором и снижает риск чтения за границами.

Почему принимается только AMD64

Значение Machine должно быть 0x8664. Проект анализирует именно x64 EH/Unwind metadata. PE32 x86 и ARM64 используют другие модели исключений или другое кодирование unwind. Отказ на этом этапе предотвращает ложную интерпретацию чужого формата как AMD64.

Функция	buf_read_u16
Сигнатура	`bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);`
Объявлена	include/util.h:25.
Реализована	src/util.c:53-58.
Вызывается	read_dos_lfanew и pe_parse.
Вход	Буфер, файловое смещение и адрес 16-разрядного результата.
Результат	true и число uint16_t либо false при недостатке байтов.
Смысл	Безопасно извлекает двухбайтовые поля PE и явно учитывает little-endian порядок.

Результат этапа

Модуль знает, что перед ним поддерживаемый AMD64-файл, сколько section headers нужно прочитать и сколько байтов занимает Optional Header.

Этап 5. Проверить Optional Header как PE32+

Что делает модуль

Модуль вычисляет file offset Optional Header как filehdr + 20, проверяет, что заявленный блок полностью помещается в файл, требует минимальный размер 0x70 и проверяет Magic 0x20B.

Почему Optional Header фактически обязателен

Историческое имя optional относится к COFF object files. Для исполняемого PE image этот блок содержит критические параметры загрузки: ImageBase, размеры, выравнивания и Data Directory. Без них невозможно построить карту образа.

```
size_t opthdr = filehdr + 20;
if (opthdr + opt_size > b->size) return false;
if (opt_size < 0x70u) return false;
if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) return false;
```

Зачем две проверки размера

Первая проверка отвечает: находится ли заявленный Optional Header внутри физического файла. Вторая отвечает: достаточно ли он длинный для полей PE32+, которые проект обязан прочитать до смещения 0x6C. Один блок может физически помещаться, но быть слишком коротким по структуре.

Результат этапа

После Magic 0x20B проект имеет основание применять именно смещения полей PE32+. Это нормативная развилка формата, а не эвристика.

Этап 6. Извлечь параметры адресной карты образа

Что делает модуль

Из Optional Header извлекаются ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders и NumberOfRvaAndSizes. Эти значения задают две системы размещения: образ в виртуальной памяти и байты на диске.

Смысл каждого поля

- ImageBase - предпочтительный базовый виртуальный адрес образа. В процессе фактическая база может измениться из-за ASLR, но статические RVA остаются относительными.
- SectionAlignment - шаг выравнивания секций в виртуальном образе.
- FileAlignment - шаг выравнивания секционных данных внутри файла.
- SizeOfImage - общий размер отображённого образа в памяти.
- SizeOfHeaders - область начала образа, соответствующая PE headers.
- NumberOfRvaAndSizes - число доступных элементов Data Directory.

Функция	buf_read_u64
Сигнатура	`bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);`
Объявлена	include/util.h:27.
Реализована	src/util.c:65-70.
Вызывается	pe_parse при чтении ImageBase.
Вход	Буфер, файловое смещение и адрес 64-разрядного результата.
Результат	true и uint64_t либо false при недостатке восьми байтов.
Смысл	Читает 64-разрядное поле ImageBase, характерное для PE32+.

Фатальные проверки

Нулевой SizeOfImage делает невозможной проверку виртуальных границ. Нулевой или превышающий файл SizeOfHeaders разрушает прямое отображение RVA заголовков. Нулевые alignment не позволяют выполнять операции остатка и проверять размещение. Недостаточное NumberOfRvaAndSizes означает отсутствие обязательной для проекта записи Exception Directory.

Почему эти ошибки останавливают анализ

Следующие модули доверяют PeImage как координатной основе. Если базовые границы отсутствуют, продолжение породило бы не аналитическое наблюдение, а недостоверные адресные преобразования. Поэтому pe_parse возвращает false.

Этап 7. Найти запись Exception Directory

Что делает модуль

Data Directory является массивом записей. Каждая запись занимает восемь байтов: четыре байта RVA и четыре байта Size. Exception Directory имеет индекс 3. Для PE32+ массив начинается по относительному смещению 0x70 от начала Optional Header.

```
size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;
size_t dd = opthdr + dd_rel;
buf_read_u32(b, dd + 0, &exc_rva);
buf_read_u32(b, dd + 4, &exc_sz);
```

Смысл вычисления

0x70 задаёт начало массива, индекс 3 выбирает четвёртый элемент, а умножение на 8 переводит номер элемента в смещение в байтах. dd_rel остаётся относительной координатой внутри Optional Header. После прибавления opthdr получается file offset, пригодный для чтения Buf.

Почему RVA пока не переводится

В этот момент секции ещё не прочитаны. pe.c только сохраняет exc_rva и exc_sz. Фактическое отображение Exception Directory в file offset выполнит unwind.c после построения массива секций.

Что означает Exception Directory

Для PE32+ AMD64 этот каталог указывает на таблицу записей RUNTIME_FUNCTION, обычно размещённую в .pdata. Это не список возникших исключений. Это статическая таблица диапазонов функций и ссылок на unwind metadata.

Этап 8. Проверить таблицу section headers и выделить память

Что делает модуль

После Optional Header начинается массив section headers. Каждая запись занимает 40 байтов. Модуль проверяет арифметику размера, убеждается, что весь массив находится в файле, затем выделяет память под nsects объектов PeSection.

```
if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) return false;
size_t needed = secthdr + (size_t)nsects * 40u;
if (needed > b->size) return false;
PeSection* secs = calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection));
```

Почему отдельно проверяется переполнение

Даже если итоговая проверка needed > b->size выглядит достаточной, переполнение size_t могло бы сделать огромное математическое значение маленьким машинным числом. Предварительное неравенство доказывает, что умножение и сложение представимы в size_t.

Функция	calloc
Сигнатура	`void* calloc(size_t count, size_t size);`
Объявлена	Стандартная библиотека C, stdlib.h.
Реализована	Стандартная библиотека среды выполнения.
Вызывается	src/pe.c:150.
Вход	count - число элементов; size - размер одного элемента.
Результат	Указатель на обнулённую память либо NULL при невозможности выделения.
Смысл	Создаёт массив PeSection, размер которого определяется входным файлом, и сразу обнуляет все поля.

Почему nsects ? nsects : 1

Это условный оператор C. Если nsects ненулевой, выделяется столько элементов, сколько заявлено. Если nsects равен нулю, запрашивается один элемент. Такое решение избегает неоднозначного поведения calloc(0, ...), которое может вернуть как NULL, так и уникальный указатель в зависимости от реализации.

Владение памятью

После успешного завершения указатель secs будет передан в out->sections. С этого момента PeImage владеет массивом, а освободить его должна pe_free. До передачи при любой ошибке pe_parse самостоятельно вызывает free(secs).

Этап 9. Прочитать каждую секцию

Что делает модуль

Цикл проходит от i = 0 до i = nsects - 1. На каждой итерации вычисляется файловое смещение очередного 40-байтового section header, читаются имя и пять полей, необходимых проекту.

```
for (uint16_t i = 0; i < nsects; i++) {
size_t sh = secthdr + (size_t)i * 40;
...
}
```

Смысл цикла

Массив section headers хранится непрерывно. Индекс i обозначает номер записи, i * 40 - расстояние от начала массива, а secthdr + i * 40 - file offset конкретной записи. Цикл переводит табличное представление файла в удобный массив C.

Чтение имени

```
uint8_t name8[8];
buf_read(b, sh + 0, name8, 8);
memcpy(secs[i].name, name8, 8);
secs[i].name[8] = 0;
```

Поле имени в PE занимает ровно восемь байтов и может не содержать нулевого терминатора. Строки C завершаются байтом 0. Поэтому PeSection выделяет девять символов: восемь копируются из файла, девятый принудительно обнуляется. Это позволяет безопасно печатать имя как строку.

Функция	buf_read
Сигнатура	`bool buf_read(const Buf* b, size_t off, void* out, size_t len);`
Объявлена	include/util.h:23.
Реализована	src/util.c:42-48.
Вызывается	read helpers и прямое чтение имени секции.
Вход	b - исходный буфер; off - начало чтения; out - адрес назначения; len - число копируемых байтов.
Результат	true при полном чтении либо false, если параметры некорректны или диапазон выходит за файл.
Смысл	Это единая граница безопасного доступа к недоверенным байтам.

Как buf_read защищает проект

- Проверяет b и out на NULL, чтобы memcpy получил действительные адреса.
- Проверяет off <= b->size, чтобы начало чтения не лежало после файла.
- Проверяет len <= b->size - off. Форма с вычитанием избегает переполнения выражения off + len.
- Только после доказательства границ выполняет memcpy(out, b->data + off, len).

Результат этапа

Массив secs содержит структурированное описание всех заявленных секций. Пока это извлечённые факты; далее модуль проверит согласованность их диапазонов и выравнивания.

Этап 10. Проверить глобальные правила выравнивания

Что делает модуль

Перед анализом отдельных секций pe.c проверяет согласованность FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders и ImageBase. Каждое нарушение увеличивает счётчик misaligned.

Почему это не всегда фатальная ошибка

Базовые нулевые значения уже были отклонены. На этом этапе карта всё ещё может быть построена, а странное выравнивание полезно сохранить как аналитическое наблюдение. heuristics.c позже решит, насколько это существенно в сочетании с другими признаками.

Функция	is_power_of_two_u32
Сигнатура	`static bool is_power_of_two_u32(uint32_t value);`
Объявлена	Локальная static-функция, в pe.h отсутствует.
Реализована	src/pe.c:7-9.
Вызывается	pe_parse при проверке FileAlignment.
Вход	Одно 32-разрядное беззнаковое число.
Результат	true, если число является ненулевой степенью двойки.
Смысл	Проверяет нормативную форму выравнивания с помощью битового свойства степеней двойки.

```
return value != 0 && (value & (value - 1u)) == 0;
```

Для степени двойки двоичная запись содержит ровно один установленный бит. Вычитание единицы обнуляет этот бит и устанавливает младшие. Побитовое И исходного значения и value - 1 даёт ноль. Отдельная проверка value != 0 нужна, потому что для нуля выражение также могло бы дать ложноположительный результат из-за беззнакового переполнения.

Что именно учитывает misaligned

- недопустимое FileAlignment;

- SectionAlignment меньше FileAlignment;
- несогласованный компактный режим alignment ниже страницы 0x1000;
- SizeOfImage не кратен SectionAlignment;
- SizeOfHeaders не кратен FileAlignment или не покрывает таблицу заголовков;
- ImageBase не выровнен на 64 KiB.

Смысл счётчика

section_misaligned хранит количество нарушенных условий, а не процент вероятности вредоносности. Это implementation fact проекта, который затем становится evidence для эвристического слоя.

Этап 11. Проверить диапазоны каждой секции

Что делает модуль

Для каждой секции вычисляются конец физического диапазона и конец виртуального диапазона. Модуль проверяет выход за файл, выход за SizeOfImage, выравнивание адресов и порядок секций.

```
uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;
uint64_t virtual_span = u32_max(s->virtual_size, s->raw_size);
uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;
```

Функция	u32_max
Сигнатура	`static uint32_t u32_max(uint32_t a, uint32_t b);`
Объявлена	Локальная static-функция.
Реализована	src/pe.c:5.
Вызывается	pe_parse и pe_find_section_for_rva.
Вход	Два uint32_t.
Результат	Большее из двух значений.
Смысл	Определяет консервативный охват секции как максимум VirtualSize и SizeOfRawData.

Почему расчёт выполняется в uint64_t

Каждое исходное поле занимает 32 бита, но сумма двух корректных uint32_t может превысить 0xFFFFFFFF. Приведение обоих операндов к uint64_t до сложения сохраняет математический результат и позволяет обнаружить выход за границу, а не потерять старшие биты.

Что означают счётчики

- section_raw_oob - число секций, чьи физические данные выходят за конец файла;
- section_virtual_oob - число секций, чей виртуальный диапазон выходит за SizeOfImage;
- section_misaligned - суммарное число нарушений правил alignment и порядка.

Почему секция с raw_size = 0 допустима

Некоторые секции представлены только в виртуальной памяти и заполняются нулями загрузчиком. Поэтому выход raw range проверяется лишь при raw_size > 0. Нулевой физический размер не означает автоматически повреждение.

Этап 12. Найти пересечения секций

Что делает модуль

Для текущей секции i внутренний цикл сравнивает её со всеми ранее прочитанными секциями j. Отдельно исследуются физические диапазоны в файле и виртуальные диапазоны в образе.

Функция	ranges_overlap_u64
Сигнатура	`static bool ranges_overlap_u64(uint64_t a_begin, uint64_t a_size, uint64_t b_begin, uint64_t b_size);`
Объявлена	Локальная static-функция.
Реализована	src/pe.c:28-34.
Вызывается	Вложенный цикл pe_parse.
Вход	Начало и размер двух полуоткрытых диапазонов [begin, begin + size).
Результат	true при наличии общей области; false для непересекающихся или пустых диапазонов.
Смысл	Унифицирует математическое правило пересечения физических и виртуальных секций.

```
return a_begin < b_begin + b_size &&
b_begin < a_begin + a_size;
```

Полуоткрытый диапазон включает начало и не включает конец. Поэтому соседние интервалы [0x100, 0x200) и [0x200, 0x300) не пересекаются. Это соответствует обычной адресной модели: один и тот же байт не принадлежит обоим диапазонам только из-за общей границы.

Аналитический смысл

Пересекающиеся raw ranges означают, что разные section headers претендуют на одни и те же байты файла. Пересекающиеся virtual ranges создают неоднозначную карту RVA. Это не доказывает злонамеренность, но снижает однозначность последующего отображения адресов и требует фиксации в отчёте.

Этап 13. Оценить границы Exception Directory

Что делает модуль

Если хотя бы одно из полей exc_rva или exc_sz ненулевое, модуль вычисляет конец каталога в uint64_t и отмечает exception_directory_oob, когда одно поле отсутствует либо диапазон выходит за SizeOfImage.

```
if (exc_rva != 0 || exc_sz != 0) {
uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;
exception_directory_oob =
exc_rva == 0 || exc_sz == 0 || exc_end > size_of_image;
}
```

Почему оба нуля не считаются этим нарушением

Пара 0/0 означает отсутствие Exception Directory. Сам pe.c может построить PE-карту и без неё. Для исполняемого AMD64-кода отсутствие runtime functions затем оценивается heuristics.c с учётом наличия executable sections. Поэтому pe.c не смешивает отсутствие каталога с повреждённым диапазоном.

Почему неполная пара подозрительна

RVA без размера не задаёт диапазон, а размер без RVA не задаёт начало. Такая запись внутренне противоречива. Флаг сохраняет факт нарушения, чтобы эвристический слой мог сформировать finding.

Ограничение этой проверки

На данном этапе проверяется виртуальная граница SizeOfImage. Возможность фактически прочитать каталог с диска устанавливает unwind_read_runtime_functions_ex через pe_rva_to_fileoff. Это два разных вопроса: допустим ли RVA в образе и обеспечен ли он физическими байтами файла.

Этап 14. Передать готовую карту PeImage

Что делает модуль

После завершения всех проверок pe_parse переносит извлечённые значения, массив секций и счётчики в объект out. Поле ok устанавливается в true, is_pe64 - в true, a sections получает указатель secs.

Почему заполнение выполняется в конце

До этого момента secs принадлежал локальному ходу pe_parse и освобождался при ошибке. Финальное присваивание оформляет передачу владения объекту PeImage. Остальные поля также становятся публичным согласованным результатом только после завершения обязательных стадий.

```
out->sections = secs;
out->num_sections = nsects;
out->exception_rva = exc_rva;
out->exception_size = exc_sz;
out->section_raw_oob = raw_oob;
out->exception_directory_oob = exception_directory_oob;
return true;
```

Что получает main.c

main.c получает PeImage img, содержащий адресную карту и диагностические счётчики. Затем передаёт &img; в unwind_read_runtime_functions_ex. Именно поэтому unwind.c уже не перечитывает PE headers самостоятельно: он использует модель, построенную pe.c.

Состояние данных

```
До pe.c: Buf { data -> необработанные байты, size }
После pe.c: PeImage { image_base, sizes, alignments,
exception_rva/size, PeSection[], counters }
```

15. Сервисная функция pe_find_section_for_rva

Функция	pe_find_section_for_rva
Сигнатура	`const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);`
Объявлена	include/pe.h:63.
Реализована	src/pe.c:265-275.
Вызывается	unwind.c, heuristics.c, report.c, dynamic_win.c и unit tests.
Вход	img - готовая карта PE; rva - относительный виртуальный адрес.
Результат	Адрес PeSection, содержащей RVA, либо NULL, если подходящей секции нет.
Смысл	Связывает адрес с секцией, её именем, диапазоном и characteristics.

Как функция работает по шагам

- Проверяет, что img и img->sections существуют. Без массива поиск невозможен.
- Последовательно рассматривает каждую секцию.
- Вычисляет размер охвата как max(virtual_size, raw_size).
- Преобразует начало и конец в uint64_t, чтобы сложение не переполнило 32 бита.
- Проверяет условие start <= rva < end.
- Возвращает адрес первой подходящей PeSection; если ни одна не подошла, возвращает NULL.

Что означает const PeSection*

Функция возвращает указатель непосредственно на элемент массива img->sections. Слово const запрещает вызывающему коду менять секцию через этот указатель. Отдельная структура не создаётся и память не выделяется.

Почему первая секция

В корректной карте виртуальные диапазоны не пересекаются, поэтому результат единственный. Если sections overlap, pe_parse уже увеличил счётчик section_virtual_overlaps. Поиск остаётся детерминированным, но неоднозначность отдельно отражается как аномалия карты.

16. Сервисная функция pe_rva_to_fileoff

Функция	pe_rva_to_fileoff
Сигнатура	`bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);`
Объявлена	include/pe.h:60.
Реализована	src/pe.c:277-301.
Вызывается	unwind.c и report.c; тестируется в unit_core_edges.c.
Вход	img - карта PE; rva - адрес относительно базы образа; out_off - адрес переменной для file offset.
Результат	true и файловое смещение либо false, если RVA невозможно подкрепить байтами файла.
Смысл	Переводит координату виртуального образа в координату физического файла.

Почему этот перевод необходим

Exception Directory и UnwindInfoAddress хранят RVA. Buf можно читать только по file offset. Числа могут совпасть случайно, но в общем случае секция расположена в памяти и в файле по разным координатам.

```
delta = rva - section.VirtualAddress
file_offset = section.PointerToRawData + delta
```

Пошаговая работа

- Проверяет img и out_off.
- Через pe_find_section_for_rva определяет секцию, виртуальный диапазон которой содержит RVA.
- Вычисляет delta - расстояние от виртуального начала секции до искомого RVA.
- Проверяет delta < raw_size. Если адрес попал в virtual tail, соответствующих байтов в файле нет.
- Через safe_add_u32 складывает raw_ptr и delta без 32-разрядного переполнения.
- Проверяет, что итоговый offset меньше фактического file_size.
- Записывает результат через *out_off и возвращает true.
- Если секция не найдена, отдельно допускает прямое отображение RVA внутри SizeOfHeaders.

Virtual tail

Если VirtualSize больше RawSize, конец секции существует в памяти, но не представлен байтами в файле: загрузчик может заполнить его нулями. Статический анализатор не должен читать несуществующие данные, поэтому функция возвращает false.

Функция	safe_add_u32
Сигнатура	`static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out);`
Объявлена	Локальная static-функция.
Реализована	src/pe.c:21-26.
Вызывается	pe_rva_to_fileoff.
Вход	Два слагаемых uint32_t и адрес результата.
Результат	false при сумме выше 0xFFFFFFFF; иначе true и точная сумма.
Смысл	Не позволяет переполнению превратить недопустимый большой file offset в маленькое число.

```
uint64_t t = (uint64_t)a + (uint64_t)b;
if (t > 0xffffffffu) return false;
*out = (uint32_t)t;
```

17. Освобождение карты: pe_free

Функция	pe_free
Сигнатура	`void pe_free(PeImage* img);`
Объявлена	include/pe.h:57.
Реализована	src/pe.c:259-263.
Вызывается	main.c на всех путях завершения после успешного pe_parse; fuzz_parser.c.
Вход	Адрес PeImage, владеющего массивом sections.
Результат	Ничего: тип void. Побочный эффект - освобождённая память и обнулённая структура.
Смысл	Завершает жизненный цикл динамической памяти, выделенной pe_parse.

Как работает

- Если img равен NULL, немедленно завершает работу. Это делает вызов безопасным для отсутствующего объекта.
- Передаёт img->sections функции free. Стандарт допускает free(NULL), поэтому отдельная проверка поля не нужна.
- Обнуляет весь PeImage через memset, удаляя висячий указатель и старые значения.

Почему одного free недостаточно

После free числовое значение указателя само по себе не обязано стать NULL. Оно продолжало бы указывать на уже освобождённую область и называлось бы dangling pointer. memset делает повторное состояние структуры очевидным и снижает риск случайного использования старых данных.

Правило владения

Тот код, который получил успешно заполненный PeImage, обязан в конце вызвать pe_free ровно один раз. Не следует отдельно освобождать img.sections, а затем вызывать pe_free: это привело бы к двойному освобождению.

18. Ошибочная модель и аналитическая модель

В pe.c существуют два разных способа обращения с проблемой. Их необходимо различать на защите и при чтении кода.

Фатальная структурная ошибка

Если невозможно достоверно определить сам формат или безопасно прочитать обязательную основу, pe_parse записывает сообщение в StrBuf err и возвращает false. Примеры: отсутствует MZ, неверна PE signature, архитектура не AMD64, Optional Header выходит за файл, SizeOfImage равен нулю, section headers физически недоступны.

Аналитическое нарушение

Если базовую модель построить можно, но структура имеет странность, pe_parse сохраняет счётчик или флаг и возвращает true. Примеры: секция выходит за SizeOfImage, секции пересекаются, нарушено выравнивание, Exception Directory выходит за виртуальные границы.

Почему разделение принципиально

Технический успех анализа не равен отсутствию аномалий. Возврат true сообщает: модель создана и пригодна для дальнейшего исследования. Счётчики сообщают: в этой модели есть наблюдения, которые должен оценить heuristics.c. Благодаря этому повреждение одной характеристики не всегда лишает аналитика всей доступной информации.

StrBuf err

StrBuf - динамический строковый буфер из util.c. pe.c вызывает sb_append или sb_appendf, чтобы добавить диагностический текст. Он не печатает сообщение сам: main.c решает, куда вывести ошибку. Такое разделение сохраняет pe.c независимым от пользовательского интерфейса.

19. Как результат используют остальные модули

unwind.c

Получает exception_rva и exception_size, переводит RVA каталога в file offset и читает RUNTIME_FUNCTION. Для каждого UnwindInfoAddress снова вызывает pe_rva_to_fileoff. pe_find_section_for_rva ограничивает чтение пределами физической части секции и определяет размещение metadata.

heuristics.c

Использует счётчики section_raw_oob, section_virtual_oob, overlaps, misaligned и exception_directory_oob как структурные evidence. Также связывает unwind и handler RVA с секциями и их characteristics.

report.c

Добавляет к findings координаты: RVA, file offset, имя секции и права. Благодаря PeImage отчёт способен показать не только утверждение об аномалии, но и её положение в исходном файле.

dynamic_win.c

Использует карту секций при сопоставлении runtime-адресов с основным модулем. Динамический адрес сначала нормализуется относительно загруженной базы, после чего RVA можно связать с PeSection.

Главный архитектурный смысл

Каждый последующий модуль использует одну и ту же координатную модель. Он не интерпретирует PE headers заново. Это снижает дублирование и предотвращает ситуацию, когда разные части проекта по-разному переводят один RVA.

20. Один пример от RVA до байта файла

Предположим, PeSection имеет virtual_address = 0x4000, raw_ptr = 0xA00, virtual_size = 0x600 и raw_size = 0x400. Требуется прочитать UNWIND_INFO по RVA 0x4120.

Шаг 1. Найти секцию

pe_find_section_for_rva вычисляет охват max(0x600, 0x400) = 0x600 и диапазон [0x4000, 0x4600). RVA 0x4120 входит в этот диапазон.

Шаг 2. Вычислить смещение внутри секции

```
delta = 0x4120 - 0x4000 = 0x120
```

Шаг 3. Доказать наличие байтов

delta 0x120 меньше raw_size 0x400, значит искомая позиция представлена в файле, а не только в virtual tail.

Шаг 4. Получить file offset

```
file_offset = 0xA00 + 0x120 = 0xB20
```

Шаг 5. Проверить физический файл

Итоговый 0xB20 должен быть меньше img.file_size. Только после этого unwind.c может безопасно читать Buf по позиции 0xB20.

Контрпример: RVA 0x4500

Он входит в виртуальный диапазон секции, но delta = 0x500, что больше raw_size 0x400. Адрес существует в отображённом образе, однако на диске соответствующих байтов нет. pe_rva_to_fileoff правильно возвращает false.

21. Полный маршрут pe.c простыми словами

1. main.c передаёт модулю байты файла, место для результата и буфер ошибки. pe.c проверяет параметры и очищает будущую модель.

2. Модуль подтверждает сигнатуру MZ и получает e_lfanew, то есть файловое смещение PE-заголовка.

3. Он доказывает, что PE-заголовок физически доступен, и подтверждает сигнатуру PE\0\0.

4. Из COFF header извлекаются архитектура, количество секций и размер Optional Header. Неподдерживаемая архитектура отклоняется.

5. Optional Header подтверждается как PE32+. Из него извлекаются размеры образа, выравнивания и число каталогов.

6. Из Data Directory извлекаются RVA и размер Exception Directory - будущая точка входа к таблице RUNTIME_FUNCTION.

7. Модуль проверяет размер таблицы section headers и выделяет массив PeSection.

8. Каждая 40-байтовая запись секции переводится из байтов в поля C: имя, виртуальный диапазон, физический диапазон и characteristics.

9. Проверяются alignment, выходы за границы, порядок и пересечения. Нарушения считаются, но не всегда прерывают построение карты.

10. Проверяется виртуальная согласованность Exception Directory.

11. Все извлечённые данные и счётчики передаются в PeImage. main.c получает готовую координатную основу.

12. unwind.c использует эту основу, чтобы перейти от RVA к физическим байтам .pdata и .xdata. После завершения всего анализа main.c вызывает pe_free.

Итоговая мысль

pe.c не анализирует смысл unwind operations. Его задача фундаментальнее: доказать, откуда именно берётся каждый последующий байт и в какой системе координат находится каждый адрес. Если этот уровень ошибочен, любой более глубокий вывод об EH/Unwind становится недостоверным.

22. Как объяснить модуль на защите

Краткая связная формулировка

Модуль pe.c строит проверенную структурную и адресную модель PE32+ AMD64. Он подтверждает обязательные заголовки, извлекает параметры виртуального образа, формирует массив секций и находит запись Exception Directory. Одновременно он фиксирует нарушения границ, выравнивания и пересечения диапазонов. Результатом является PeImage - единая координатная основа, которую последующие модули используют для чтения RUNTIME_FUNCTION, UNWIND_INFO, формирования эвристических признаков и локализации доказательств в отчёте.

Если спросят, почему это не просто чтение полей

Чтение поля само по себе даёт недоверенное число. pe.c проверяет происхождение, физическую доступность и согласованность этого числа с другими границами формата. Кроме извлечения значений, модуль строит отображение между RVA и file offset и сохраняет диагностические счётчики. Поэтому его результат является моделью, а не распечаткой заголовка.

Если спросят, почему не используются Windows headers

Проект читает фиксированные поля через собственные bounds-safe функции и типы точной ширины. Это делает статическую часть переносимой между Windows, Linux и macOS и не связывает форматный парсер с выравниванием системных C-структур конкретной платформы.

Если спросят, где начинается EH/Unwind-анализ

pe.c находит координату Exception Directory и создаёт карту секций. unwind.c по этой карте читает RUNTIME_FUNCTION и UNWIND_INFO. heuristics.c интерпретирует обнаруженные структурные и семантические признаки. Следовательно, pe.c является обязательным основанием EH/Unwind-анализа, но не заменяет декодер и эвристический слой.

23. Проверка понимания

1. Почему нельзя читать файл непосредственно по RVA?

Потому что RVA относится к виртуальному образу, а Buf индексируется file offset. Перевод зависит от VirtualAddress и PointerToRawData конкретной секции.

2. Почему pe_parse может вернуть true при ненулевых счётчиках нарушений?

Потому что карта технически построена, а нарушения являются аналитическими наблюдениями для heuristics.c. true не означает, что файл нормален.

3. Зачем используется uint64_t при сложении 32-разрядных полей?

Чтобы сохранить полный математический результат и обнаружить выход за допустимые границы вместо молчаливого переполнения uint32_t.

4. Почему у PeSection.name девять байтов, хотя PE хранит восемь?

Девятый байт используется для нулевого завершения строки C.

5. Кто освобождает массив sections?

После успешного pe_parse им владеет PeImage; main.c завершает его жизненный цикл вызовом pe_free.

6. Что передаёт pe.c модулю unwind.c?

Не прямой вызов, а объект PeImage через main.c: размеры, секции, Exception Directory и функции адресного отображения.

7. Что является главным результатом модуля?

Проверенная координатная модель PE, на которой основано последующее извлечение и локализация EH/Unwind metadata.

Системная архитектура: как модули образуют один анализ

Сначала рассматривается движение данных через систему целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут всей системы

Этап 1. Получить внешний запрос

Состояние перед этапом. Существует команда пользователя: путь к PE-файлу и выбранные режимы.

Техническое выполнение. main.c принимает аргументы командной строки и переводит их в структуру Args.

Смысл этапа и его место в маршруте. Остальные модули не должны самостоятельно разбирать пользовательский текст и по-разному понимать одну option (параметр).

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется единая конфигурация анализа.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Создать контролируемое представление файла». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 2. Создать контролируемое представление файла

Состояние перед этапом. Известен путь, но байты ещё не находятся в памяти анализатора.

Техническое выполнение. util.c загружает файл и создаёт Buf: указатель data и размер size.

Смысл этапа и его место в маршруте. Все последующие чтения должны знать точную границу файла.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Проект получает неизменяемую последовательность байтов с явной длиной.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Доказать структуру PE». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Доказать структуру PE

Состояние перед этапом. Есть байты, но их назначение неизвестно.

Техническое выполнение. pe.c проверяет обязательные заголовки и строит PeImage с секциями и адресными правилами.

Смысл этапа и его место в маршруте. Без карты PE нельзя правильно преобразовать RVA в file offset.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется общая система координат для всех модулей.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Построить карту EH/Unwind». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Построить карту EH/Unwind

Состояние перед этапом. Известно расположение секций и Exception Directory.

Техническое выполнение. unwind.c читает RUNTIME_FUNCTION и UNWIND_INFO и создаёт структурированные summaries (сводки).

Смысл этапа и его место в маршруте. Низкоуровневые байты необходимо превратить в проверяемые поля и операции.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Для каждой функции известен диапазон, unwind RVA, результат разбора и диагностика.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Сформировать аналитические наблюдения». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Сформировать аналитические наблюдения

Состояние перед этапом. Есть структурная карта и результаты декодирования.

Техническое выполнение. heuristics.c применяет инженерные правила и формирует findings - локализованные аналитические сообщения.

Смысл этапа и его место в маршруте. Один необычный байт нельзя автоматически назвать атакой; наблюдения нужно собрать и оценить совместно.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляются code, severity, evidence, score, confidence и рекомендации ручной проверки.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Добавить сведения реального запуска». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Добавить сведения реального запуска

Состояние перед этапом. Статический слой описал, что находится в файле.

Техническое выполнение. dynamic_win.c на Windows запускает один сценарий под debugger и собирает runtime evidence.

Смысл этапа и его место в маршруте. Некоторые механизмы существуют только во время исполнения, например динамические таблицы функций.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется DynamicReport с событиями, счётчиками и RIP trace.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Выдать проверяемый результат». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Выдать проверяемый результат

Состояние перед этапом. В памяти существуют несколько связанных моделей.

Техническое выполнение. report.c сериализует их в CSV, JSON, HTML или Ghidra CSV.

Смысл этапа и его место в маршруте. Человеку и внешним системам нужна одна согласованная версия результата, а не набор несвязанных printf.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Результат анализа становится сохраняемым и проверяемым документом.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. EH/Unwind Analyzer: полная инженерная модель проекта

Что именно должно быть понято во втором томе

Исходный код проекта нельзя полноценно объяснить как независимый список файлов. Функции образуют последовательность преобразований, а структуры данных являются промежуточными состояниями одного анализа.

Полное понимание проекта требует уметь проследить две взаимосвязанные цепочки.

Первая цепочка - цепочка данных:

```
путь к файлу
-> байты файла
-> модель PE image
-> таблица runtime functions
-> summaries unwind metadata
-> статические findings
-> динамические события
-> JSON, HTML и CSV
```

Вторая цепочка - цепочка доверия:

```
недоверенный внешний ввод
-> доказанная доступность диапазона
-> доказанная структурная интерпретация
-> проверенная внутренняя модель
-> эвристическая интерпретация
-> ручная верификация
```

Если читатель понимает только первую цепочку, он знает, куда передаются структуры, но может не понимать надёжность вывода. Если понимает только вторую, но не знает фактический код, он не способен объяснить реализацию. Поэтому каждый модуль будет рассмотрен одновременно как преобразователь данных и как граница доверия.

Предметная область проекта

Проект называется `EH/Unwind Analyzer`.

`EH` - сокращение от `Exception Handling`, обработка исключений.

`Unwind` - процесс восстановления состояния предыдущих вызовов функции, в частности значения stack pointer и сохранённых nonvolatile registers.

`Analyzer` - программный инструмент, который извлекает свойства исследуемого объекта и формирует аналитическое представление.

Полное назначение проекта:

> Проект статически разбирает Windows PE32+ AMD64 images, извлекает таблицу функций и x64 unwind metadata, проверяет структурную согласованность, формирует объяснимые эвристические признаки и отчёты, а в Windows-сборке может дополнительно наблюдать exception-related runtime events при запуске target под debugger.

Каждая часть формулировки ограничивает область продукта.

`PE32+` означает 64-битную разновидность PE optional header. Это не означает поддержку любого 64-битного формата.

`AMD64` означает конкретную архитектуру и её модель unwind. ARM64 использует другой формат.

`Статически разбирает` означает, что основное ядро читает файл как данные и не выполняет target.

`Эвристические признаки` означает, что проект не выдаёт математическое доказательство вредоносности.

`Windows-сборка` ограничивает настоящий dynamic collector, потому что он использует Windows Debug API.

Граница продукта

Проект не выполняет следующие задачи:

- полное дизассемблирование `.text`;
- декомпиляцию программы;
- восстановление исходного C/C++;
- универсальную классификацию malware;
- эмуляцию всех возможных путей;
- kernel-mode tracing;
- поддержку PE32 x86;
- поддержку ARM64 unwind;
- поддержку ELF или Mach-O;
- автоматическое доказательство соответствия каждого `UNWIND_INFO` машинному прологу;
- полное декодирование всех language-specific exception tables.

Это не список недостатков, а определение архитектурной границы. Завершённость продукта оценивается относительно заявленного scope.

Архитектурный тип

Основные алгоритмы собираются в статическую библиотеку:

```
add_library(ehua_core STATIC ...)
```

CLI собирается отдельным executable и связывается с ядром:

```
add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
```

Такую систему корректно называть модульным монолитом с библиотечным ядром.

`Монолит` означает, что штатный анализ выполняется одним процессом и компоненты взаимодействуют через вызовы функций и C-структуры, а не через сеть.

`Модульный` означает, что ответственность разделена между исходными модулями с явными интерфейсами.

`Библиотечное ядро` означает, что основная логика не находится исключительно внутри `main`, а собрана в переиспользуемый target `ehua_core`.

Это решение соответствует характеру задачи. Между чтением PE header и декодированием unwind передаются маленькие структуры и массивы. Выделение этих стадий в отдельные сетевые сервисы не повысило бы точность, но добавило бы сериализацию и failure modes.

Над модульным монолитом существует отдельный слой поставки. Он не участвует в алгоритме анализа и не превращает ядро в микросервисную систему:

```
аналитическое ядро и CLI
-> CMake/CTest/CPack
-> release manifest и quality passport
-> Windows, macOS, source и evidence packages
```

`scripts/create_release_bundles.py` собирает воспроизводимые архивы для разных получателей. Это release orchestration, а не новый analytical module. Поэтому архитектурный тип самого анализатора остаётся прежним: один CLI process, статическая библиотека `ehua_core` и модульные границы внутри процесса.

Модули и их строгие контракты

`util.c`

Предоставляет примитивы, на которых основана безопасность остальных модулей:

- загрузку файла;
- bounds-safe чтение;
- little-endian decoding;
- динамический строковый буфер;
- атомарную запись;
- SHA-256.

`util.c` не знает формат PE. Это уровень общих операций над байтами и выходными файлами.

`pe.c`

Преобразует `Buf` в `PeImage`.

Вход: неизменяемая последовательность байтов.

Выход: проверенные ключевые поля PE и массив секций, а также счётчики структурных нарушений.

`pe.c` определяет систему координат, необходимую всем последующим модулям.

`unwind.c`

Использует `PeImage`, чтобы:

- найти exception directory;
- прочитать массив `RUNTIME_FUNCTION`;
- преобразовать `UnwindInfoAddress` в file offset;
- декодировать `UNWIND_INFO`;

- вычислить сводные свойства;
- зафиксировать аномалии и parse failures.

`heuristics.c`

Не читает исходный файл самостоятельно. Он получает уже построенные модели и агрегирует наблюдения.

Это принципиальная граница: heuristic rule не должна обходить parser и заново интерпретировать недоверенные байты по своим правилам.

`dynamic_win.c`

В Windows-сборке создаёт target process под debugger и собирает bounded runtime evidence. Он использует статические handler RVA для установки breakpoints после определения loaded image base.

В не-Windows сборке экспортирует тот же интерфейс, но возвращает контролируемое сообщение о недоступности режима.

`report.c`

Сериализует внутренние модели. Он не должен менять score, повторно анализировать PE или владеть переданными массивами.

`app_info.c`

Создаёт provenance:

- tool version;
- schema version;
- compiler;
- platform;
- build mode;
- timestamp.

`main.c`

Является orchestration layer:

- разбирает CLI;
- определяет требуемые режимы;
- вызывает модули в правильном порядке;
- управляет временем жизни объектов;
- агрегирует ошибки в exit code;
- инициирует запись outputs.

Почему CLI и ядро разделены

CLI является одним способом использования ядра. Он решает вопросы пользовательского интерфейса:

- как записываются options;
- какие defaults применяются;
- что печатается в терминал;
- какие файлы создавать;
- какой вернуть exit code.

Ядро решает предметную задачу:

- как разобрать PE;
- как декодировать unwind;
- как вычислить findings;
- как сформировать report model.

Такое разделение позволяет backend запускать CLI как worker. Сетевой API между backend и ядром необязателен. Backend может передать путь процессу и получить JSON.

Если в будущем потребуется прямой library API, `ehua_core` уже отделён, но публичный стабильный ABI потребует дополнительного проектирования opaque handles и versioning structures.

Главные структуры как состояния анализа

`Buf`

```
typedef struct Buf {
const uint8_t* data;
size_t size;
} Buf;
```

Состояние: файл загружен, но его формат ещё не доказан.

`PeImage`

Состояние: обязательные PE32+ AMD64 headers разобраны, sections известны, RVA mapping возможен.

`RuntimeFunction[]`

Состояние: exception directory интерпретирован как последовательность троек begin/end/unwind RVA.

`UnwindSummary[]` и `us_ok[]`

Состояние: для каждой runtime function выполнена попытка декодирования. `us_ok[i]` отделяет успешную структурную интерпретацию от failure. `UnwindSummary` сохраняет и результат, и причину ошибки.

`HeuristicReport`

Состояние: низкоуровневые наблюдения агрегированы в findings, score, confidence, verdict и mitigations.

`DynamicReport`

Состояние: один ограниченный runtime experiment завершён либо не выполнен. Содержит counters, events, trace и status.

`ReportMetadata`

Состояние: результат связан с конкретной версией инструмента, временем и SHA-256 входа.

Почему используются параллельные массивы

Для `N` runtime functions проект выделяет:

```
UnwindSummary* us;
bool* us_ok;
```

Индекс связывает:

```
rf[i] <-> us[i] <-> us_ok[i]
```

`rf[i]` описывает диапазон функции и RVA metadata.

`us[i]` содержит декодированную сводку либо заполненную ошибку.

`us_ok[i]` сообщает, допустимо ли использовать основные decoded fields.

Почему success не определяется только `us[i].valid`? Внутренняя структура действительно содержит `valid`, но внешний массив success обеспечивает простой единообразный контракт для heuristics и reports. Нулевая структура после `calloc` не должна ошибочно считаться успешным результатом.

Недостаток модели - необходимость поддерживать одинаковую длину и порядок массивов. `main` создаёт их одновременно на основании `rf_count`, а report validation проверяет наличие pointers при ненулевом количестве.

Ownership graph

Память в C не освобождается автоматически по выходу из всей программы, если она выделена в heap. Для краткоживущего CLI ОС освободила бы адресное пространство после exit, но корректное явное освобождение необходимо для:

- unit tests;
- sanitizer checks;
- библиотечного использования;
- ранних ошибок;
- предотвращения накопления ресурсов.

Граф владения:

```
main
├── owned input bytes
```

```
├── PeImage.sections
├── RuntimeFunction[]
├── UnwindSummary[]
├── us_ok[]
├── HeuristicReport.details
├── DynamicReport.details
└── локальные StrBuf
```

Конкретные release functions:

```
buf_free_owned
pe_free
free
heuristics_free
dynamic_report_free
sb_free
```

Report writers получают borrowed pointers. Они не освобождают модели и не хранят их после возврата.

Error model

Проект различает несколько уровней ошибки.

CLI error

Неверная option или значение. Exit code `2`.

Input or structural fatal error

Файл не читается, не является поддерживаемым PE или exception directory невозможно прочитать. Exit code `1`.

Per-record parse failure

Одна `UNWIND_INFO` не декодируется, но анализ остальных records продолжается. Ошибка становится частью отчёта и heuristics.

Analytical finding

Наблюдение подозрительно, но анализ технически успешен. Exit code не обязан становиться ненулевым.

Output error

Не удалось сформировать metadata или записать requested report. Exit code `1`.

Dynamic session failure

Collector не запустился или не завершил требуемую операцию. Exit code `1`, но при возможности status отражается в `DynamicReport`.

Критически важно: score выше нуля не является технической ошибкой выполнения инструмента.

Почему analysis продолжается при локальной ошибке

Если одна unwind structure повреждена, прекращение всего анализа уничтожило бы информацию о тысячах остальных функций. Проект использует устойчивую модель:

```
ошибка конкретной записи
-> us_ok[i] = false
-> parse_error сохраняется
-> цикл продолжается
-> heuristic counter увеличивается
-> отчёт содержит failure
```

Такой подход называется fault containment на уровне записи: локальная ошибка не должна разрушать весь результат, если верхнеуровневая структура всё ещё позволяет безопасное продолжение.

Доказательная модель

Проект не переходит непосредственно от байтов к verdict. Между ними существуют уровни.

```
байты
-> parsed fields
-> validation counters
-> findings
-> evidence families
-> score/confidence
-> static/dynamic assessment
-> manual validation
```

`Parsed field` - значение, извлечённое по формату.

`Validation counter` - число нарушений конкретного условия.

`Finding` - структурированное аналитическое сообщение с code, family, severity и location.

`Score` - сумма инженерных weights.

`Confidence` - оценка силы и разнообразия evidence с weakening rules.

`Assessment` - объединение static и dynamic levels.

`Manual validation` - действия, которые должен выполнить человек.

Чем выше уровень, тем больше интерпретации и тем меньше допустимо выдавать результат за нормативный факт.

Сквозной инвариант неизменности входа

Static core объявляет байты:

```
const uint8_t* data;
```

Functions принимают:

```
const Buf* b;
```

`const` выражает контракт: parser не модифицирует вход. Это позволяет:

- вычислить SHA-256 тех же байтов;

- сравнивать результаты разных стадий;
- безопасно повторять parsing;
- исключить случайное patching target;
- использовать вход как immutable evidence artifact.

Динамический target является отдельным process и может изменять собственную memory. Это не меняет сохранённый input buffer анализатора.

Связь модулей с теоретическим фундаментом

Теоретическое понятие	Реализация проекта
конечная последовательность байтов	`Buf`
файловое смещение	параметр `off`, section raw pointers
little-endian integer	`le16`, `le32`, `le64`
виртуальный образ	`PeImage`
RVA	поля sections/runtime functions
ABI и stack frame	decoded unwind operations
exception dispatch	handlers и dynamic events
множество и частота	hash maps heuristics
сериализация	`report.c`
процесс и поток	`dynamic_win.c`
жизненный цикл ресурса	ownership и cleanup
контракт	headers в `include/`
доказательство корректности	unit/regression/schema tests
воспроизводимая поставка	`scripts/create_release_bundles.py`, `release/`

Как читать последующие главы

Каждый модуль будет разобран в одинаковом порядке:

1. зачем он нужен в общей цепочке;
2. какие предпосылки уже доказаны;
3. публичные структуры и функции;
4. внутренний алгоритм;
5. обработка каждого класса ошибки;
6. ownership;
7. передача результата следующему модулю;
8. соответствующие tests;
9. отражение в report;
10. ограничения и возможные улучшения.

Модуль main.c: управление полным жизненным циклом анализа

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Разобрать команду

Состояние перед этапом. C runtime передал argc - число аргументов - и argv - массив строк аргументов.

Техническое выполнение. parse_args проверяет порядок options, значения чисел, пути отчётов и аргументы target после разделителя --.

Смысл этапа и его место в маршруте. Неверное значение timeout или неизвестная option должны быть обнаружены до запуска анализа.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Структура Args содержит нормализованные параметры и defaults - значения по умолчанию.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Загрузить исследуемый файл». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. При ошибке main печатает usage и возвращает exit code 2, обозначающий ошибку командной строки.

Этап 2. Загрузить исследуемый файл

Состояние перед этапом. Args.input содержит путь к target, но содержимое файла ещё недоступно.

Техническое выполнение. main вызывает buf_init_from_file(&b;, a.input, &owned;). Символ & передаёт адрес переменной, которую функция должна заполнить.

Смысл этапа и его место в маршруте. Парсеры работают с памятью, а не выполняют произвольные операции чтения файла на каждом поле.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: b описывает байты, owned хранит адрес памяти, которую main позже освободит.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Построить PeImage». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. Если файл нельзя открыть или прочитать, анализ прекращается с exit code 1.

Этап 3. Построить PeImage

Состояние перед этапом. Есть Buf, но проект ещё не доказал, что это поддерживаемый PE32+ AMD64.

Техническое выполнение. main создаёт PeImage img и передаёт его адрес в pe_parse.

Смысл этапа и его место в маршруте. Следующие модули должны использовать одну проверенную карту секций и адресов.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: img содержит базовые параметры PE, секции, Exception Directory и счётчики нарушений.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Прочитать таблицу функций». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. Фатальная ошибка PE приводит к освобождению Buf и завершению процесса.

Этап 4. Прочитать таблицу функций

Состояние перед этапом. PeImage указывает, где заявлена Exception Directory.

Техническое выполнение. unwind_read_runtime_functions_ex создаёт динамический массив RuntimeFunction и записывает число элементов в rf_count.

Смысл этапа и его место в маршруте. Нужно получить список диапазонов функций до разбора индивидуальных UNWIND_INFO.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: rf[i] содержит begin_rva, end_rva и unwind_rva одной записи.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Подготовить результаты для каждой функции». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. При нехватке памяти анализ прекращается; при структурно повреждённом каталоге main сохраняет partial report - частичный отчёт - и помечает техническую ошибку.

Этап 5. Подготовить результаты для каждой функции

Состояние перед этапом. Известно N = rf_count записей RUNTIME_FUNCTION.

Техническое выполнение. main через calloc выделяет массив UnwindSummary[N] и массив bool us_ok[N].

Смысл этапа и его место в маршруте. Summary хранит данные разбора, а отдельный bool показывает, можно ли считать разбор успешным. Один индекс связывает три объекта: rf[i], us[i], us_ok[i].

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляются пустые места для N независимых результатов.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Разобрать все UNWIND_INFO». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Разобрать все UNWIND_INFO

Состояние перед этапом. Для каждой функции известен unwind_rva.

Техническое выполнение. Цикл for вызывает unwind_parse для каждого индекса. Опции follow_chain, strict_abi и max_chain_depth передаются через UnwindParseOpts.

Смысл этапа и его место в маршруте. Каждая функция может ссылаться на отдельную metadata; сбой одной записи не должен скрыть результаты остальных.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: После цикла main располагает полной таблицей успешных summaries и parse failures.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Выполнить выбранные виды анализа». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Выполнить выбранные виды анализа

Состояние перед этапом. Статические структуры уже построены.

Техническое выполнение. Если включён --heuristics, вызывается heuristics_analyze. Если включён --dynamic, вызывается dynamic_collect_windows.

Смысл этапа и его место в маршруте. CLI должен запускать только явно запрошенные дорогостоящие или платформозависимые этапы.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: hr_ptr и dr_ptr либо указывают на готовые отчёты, либо остаются NULL, означая отсутствие соответствующего слоя.

Передача дальше. Полученный результат становится исходным состоянием для этапа 8 «Подготовить происхождение результата». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 8. Подготовить происхождение результата

Состояние перед этапом. Нужно записать JSON или HTML, но ещё не зафиксированы версия и идентичность входа.

Техническое выполнение. main строит безопасную строку команды, вычисляет SHA-256, получает UTC timestamp и сведения о сборке.

Смысл этапа и его место в маршруте. Два внешне одинаковых имени файла могут содержать разные байты; отчёт должен указывать, что именно анализировалось.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: ReportMetadata связывает результат с входом и версией инструмента.

Передача дальше. Полученный результат становится исходным состоянием для этапа 9 «Записать отчёты». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 9. Записать отчёты

Состояние перед этапом. Все выбранные модели находятся в памяти.

Техническое выполнение. main последовательно вызывает report_write_csv, report_write_json_ex и report_write_html для заданных путей.

Смысл этапа и его место в маршруте. Writer - функция записи - должен получить согласованный набор указателей и размеров.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: На диске появляются законченные отчёты; ошибка записи отражается в exit_code.

Передача дальше. Полученный результат становится исходным состоянием для этапа 10 «Освободить ресурсы». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 10. Освободить ресурсы

Состояние перед этапом. Анализ закончен, но массивы и строки занимают динамическую память.

Техническое выполнение. main вызывает free-функции в обратном порядке: dynamic/heuristic reports, массивы, PeImage, StrBuf и owned bytes.

Смысл этапа и его место в маршруте. Память принадлежит процессу только до конца её жизненного цикла; забытый free создаёт leak - утечку памяти.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Процесс завершает работу без оставленных ресурсов и возвращает итоговый exit code.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. `main.c`: полный жизненный цикл одного анализа

Роль `main.c`

`main.c` не реализует формат PE или unwind encoding. Его задача - orchestration: создать правильную последовательность вызовов, сохранить результаты, сформировать requested outputs и освободить ресурсы.

Эта роль часто недооценивается. Даже корректные отдельные модули образуют некорректный продукт, если orchestration:

- вызывает их в неверном порядке;
- игнорирует failure;
- передаёт несогласованные pointers;
- создаёт отчёт до вычисления metadata;
- освобождает объект слишком рано;
- возвращает неверный exit code.

Поэтому `main.c` является формальным описанием полного user workflow.

Входной интерфейс процесса

Сигнатура:

```
int main(int argc, char** argv)
```

`argc` содержит число аргументов.

`argv` указывает на массив pointers на null-terminated byte strings.

Для команды:

```
eh_unwind_analyzer benign_richer.exe --heuristics --json report.json
```

логическая модель:

```
argc = 5
argv[0] = "eh_unwind_analyzer"
argv[1] = "benign_richer.exe"
argv[2] = "--heuristics"
argv[3] = "--json"
argv[4] = "report.json"
```

Операционная система и C runtime формируют эти значения до вызова `main`.

Внутренняя структура `Args`

```
typedef struct Args {
const char* input;
bool dump;
bool show_ops;
bool heuristics;
const char* csv_path;
const char* ghidra_csv_path;
const char* json_path;
const char* html_path;
int dump_limit;
bool follow_chain;
int max_chain_depth;
bool dynamic;
uint32_t timeout_ms;
uint32_t trace_steps;
const char** child_argv;
int child_argc;
} Args;
```

Эта структура является нормализованным представлением command line.

Поля `const char*` не владеют строками. Они указывают на memory, предоставленную через `argv`, и действительны в течение `main`.

Boolean fields определяют включённые режимы.

Числовые fields уже должны быть проверены по диапазонам.

`child_argv` и `child_argc` описывают arguments target после разделителя `--`.

Почему parsing arguments отделён

Функция:

```
static ParseResult parse_args(int argc, char** argv, Args* a)
```

обнуляет output:

```
memset(a, 0, sizeof(*a));
```

а затем задаёт nonzero defaults:

```
a->dump_limit = -1;
a->follow_chain = true;
a->max_chain_depth = 8;
a->timeout_ms = 15000;
a->trace_steps = 64;
```

Обнуление создаёт доказуемое начальное состояние всех optional pointers и flags. Defaults задаются явно после него.

Возвращаемый `ParseResult` различает:

```
PARSE_OK
PARSE_HELP
PARSE_VERSION
PARSE_ERR
```

Boolean результата было бы недостаточно: help и version являются успешным завершением пользовательского запроса, а malformed CLI - ошибкой.

Позиционный input

Если первый аргумент равен `--help` или `--version`, input file не требуется.

Иначе:

```
a->input = argv[1];
```

Проект использует первый позиционный аргумент как путь target. Остальные аргументы до `--` должны быть известными options.

Это означает, что текущий CLI не поддерживает произвольное расположение input после options. Backend должен следовать документированному contract.

Разделитель `--`

Option:

```
--
```

завершает options анализатора. Все последующие strings передаются target в dynamic mode.

Пример:

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat
```

До separator:

```
--dynamic
```

принадлежит анализатору.

После separator:

```
--input sample.dat
```

принадлежит target.

Без separator невозможно однозначно определить, чья option `--input`.

Проверка чисел

`parse_long_range` использует `strtol`.

Она проверяет:

- null/empty input;
- `ERANGE`;
- отсутствие цифр;
- наличие trailing characters;
- minimum и maximum.

Условие:

```
end == text
```

означает, что не была распознана ни одна цифра.

Условие:

```
*end != '\0'
```

означает, что после числа остался непотреблённый текст, например `10abc`.

Для unsigned values `parse_u32_range` дополнительно отклоняет начальный `-`, потому что `strtoul` имеет поведение, которое могло бы преобразовать отрицательную запись в большое unsigned value.

Диапазоны options

`--max-chain-depth` допускает `1..16`.

Нижняя граница исключает бессмысленную нулевую глубину при включённом follow chain.

Верхняя граница ограничивает рекурсивную или итеративную работу на malicious chain.

`--trace-steps` допускает `0..256`. Ноль отключает trace. Верхняя граница соответствует фиксированному массиву:

```
uint64_t trace[256];
```

CLI validation и размер модели согласованы.

`--timeout-ms` требует значение больше нуля и допускает диапазон `uint32_t`.

Exit code CLI parsing

После:

```
ParseResult pres = parse_args(argc, argv, &a);
```

`main` обрабатывает результат:

```
if (pres != PARSE_OK) {
if (pres == PARSE_VERSION) {
print_version();
return 0;
}
usage(argv[0]);
return (pres == PARSE_HELP) ? 0 : 2;
}
```

Help и version возвращают 0. Invalid command line возвращает 2.

Различие важно для automated caller:

```
0: запрос выполнен
1: runtime/input/output failure
2: неправильный способ вызова
```

Чтение input

Объявляются:

```
uint8_t* owned = NULL;
Buf b;
```

После успешного `buf_init_from_file`:

```
owned == b.data
b.size == file size
```

но roles различаются. `owned` используется для освобождения, `b` - для чтения.

Если чтение не удалось, другие объекты ещё не созданы, поэтому cleanup минимален.

Error buffers

Проект использует `StrBuf` для накопления подробных diagnostics:

```
StrBuf err;
sb_init(&err, 1024);
```

В текущем `main` return value `sb_init` в некоторых местах не проверяется немедленно. `StrBuf` сохраняет `failed`, а последующие append возвращают false. Для критических output metadata проверки выполняются составным условием.

Инженерное улучшение могло бы унифицировать проверку каждого `sb_init`, но test/sanitizer coverage снижает риск незамеченного поведения.

PE parsing

```
PeImage img;
if (!pe_parse(&img, &b, &err)) {
...
return 1;
}
```

До вызова `img` не инициализирован вызывающим кодом. Контракт `pe_parse` начинается с:

```
memset(out, 0, sizeof(*out));
```

При failure `main` не вызывает `pe_free`, потому что parser сам освобождает локально выделенные sections на соответствующих failure paths и не возвращает успешное owned state.

При success ownership `img.sections` переходит вызывающему, и позднее требуется `pe_free`.

Runtime function table

```
RuntimeFunction* rf = NULL;
size_t rf_count = 0;
```

Использование null и zero перед вызовом обеспечивает определённое состояние даже при failure.

`unwind_read_runtime_functions` выделяет массив. После success:

```
rf[0..rf_count-1]
```

доступен caller. Освобождение выполняется `free(rf)`.

Если exception directory корректно отсутствует, допустим success с `rf_count == 0`. Код последующих allocations учитывает ноль.

Allocations для per-function state

```
UnwindSummary* us =
(UnwindSummary*)calloc(rf_count ? rf_count : 1,
sizeof(UnwindSummary));
bool* us_ok =
(bool*)calloc(rf_count ? rf_count : 1,
sizeof(bool));
```

Почему при `rf_count == 0` выделяется один элемент?

Стандарт C допускает implementation-defined поведение `malloc(0)`/`calloc(0, ...)`, включая null. Проект хочет различать реальный out-of-memory и пустую таблицу, поэтому запрашивает минимум один элемент.

Логическая длина всё равно равна `rf_count`; фиктивный элемент не обрабатывается.

`calloc` обнуляет память:

- `us_ok[i]` изначально false;
- все fields `UnwindSummary` изначально zero;
- `parse_error` содержит empty string.

Options unwind parser

```
UnwindParseOpts uopts = {
.max_chain_depth = a.max_chain_depth,
.follow_chain = a.follow_chain
};
```

Designated initializers явно связывают значения с именами fields.

Эта структура отделяет policy пользователя от обязательных параметров parser. В будущем в неё можно добавить controlled options, но изменение public structure потребует проверки ABI/library use.

Основной цикл unwind

```
for (size_t i = 0; i < rf_count; i++) {
StrBuf perr;
sb_init(&perr, 256);
bool ok = unwind_parse(
&b,
&img,
rf[i].unwind_rva,
&uopts,
&us[i],
a.show_ops ? &ops : NULL,
&perr
);
us_ok[i] = ok;
sb_free(&perr);
}
```

На каждой итерации:

1. выбирается `rf[i].unwind_rva`;

2. parser использует общие immutable `b` и `img`;

3. результат записывается точно в `us[i]`;

4. success записывается в `us_ok[i]`;

5. локальная diagnostic освобождается;

6. цикл продолжается независимо от failure одной записи.

`perr` не печатается в этом цикле, потому что `unwind_parse` копирует конкретную причину в `us[i].parse_error`. Локальный buffer используется для внутреннего diagnostic contract.

Почему операции могут разбираться повторно

В первом цикле текст operations строится только при `a.show_ops`, но общий `ops` не хранит отдельный текст для всех functions.

В dump loop при необходимости `unwind_parse` вызывается повторно для конкретной записи и строит `ops2`.

Преимущество:

- не хранится большой массив текстов для всех функций;
- detailed output создаётся только для отображаемых entries.

Недостаток:

- выбранные structures разбираются второй раз.

Для CLI dump это разумный обмен memory на computation. Parser детерминирован относительно immutable input и options.

Heuristic stage

`HeuristicReport` объявляется на stack:

```
HeuristicReport hr;
HeuristicReport* hr_ptr = NULL;
```

Pointer остаётся null, пока анализ не завершится успешно.

После success:

```
hr_ptr = &hr;
```

Reports получают null, если heuristics не запрошены или failed. Это позволяет сериализатору различать отсутствие этапа и заполненную модель.

Если `heuristics_analyze` failed, `exit_code` становится 1, но код может продолжить формировать другие requested outputs. Такая стратегия позволяет пользователю получить частичный результат, не скрывая failure через process status.

Dynamic stage

`DynamicReport dr` предварительно обнуляется:

```
memset(&dr, 0, sizeof(dr));
```

Флаг:

```
bool dr_initialized = false;
```

нужен для корректного вызова `dynamic_report_free`. После вызова collector считается, что report прошёл init path, даже если session не ran успешно.

`dr_ptr` передаётся reports, если dynamic mode был запрошен. Тогда JSON способен показать failure/status, а не просто отсутствие блока.

Outputs CSV

CSV writers вызываются независимо:

```
report_write_csv(..., false, ...)
report_write_csv(..., true, ...)
```

Последний boolean выбирает full CSV или Ghidra-oriented variant.

Каждый output имеет собственный error buffer и собственную обработку. Failure одного файла устанавливает `exit_code = 1`, но не запрещает попытку создать остальные.

Подготовка provenance

JSON и HTML требуют metadata. `main` строит:

```
StrBuf invocation;
char timestamp[32];
char input_sha256[65];
ReportMetadata metadata;
```

Составное условие требует success всех операций:

```
sb_init
build_invocation
app_utc_timestamp
sha256_hex
```

Только после этого:

```
metadata_ptr = &metadata;
```

Если metadata неполна, JSON/HTML не записываются как будто полноценные. Это сохраняет report invariant: каждый полный report должен быть привязан к tool build и input hash.

Redaction arguments target

`build_invocation` проходит исходный `argv`. При достижении `--` он не копирует последующие значения, а записывает:

```
-- [N target arguments redacted]
```

Причина не связана с синтаксисом JSON. Target arguments могут содержать:

- credentials;
- tokens;
- персональные данные;
- внутренние пути;
- содержимое запроса.

Для воспроизводимости сохраняется факт и количество, но не значения.

SHA-256

`sha256_hex(b.data, b.size, input_sha256)` вычисляет digest всех входных байтов.

Если файл изменится хотя бы в одном бите, ожидается другое digest. SHA-256 связывает отчёт с конкретным содержимым, но не доказывает происхождение или безопасность файла.

JSON и HTML

Writers получают одинаковый analytical model:

```
path
PeImage
RuntimeFunction[]
UnwindSummary[]
us_ok[]
HeuristicReport optional
DynamicReport optional
ReportMetadata
```

Различается presentation:

- JSON предназначен для machine integration;
- HTML предназначен для interactive human review.

Логические значения должны согласовываться между форматами. Unit tests проверяют оба serializers.

Финальный cleanup

Порядок:

```
heuristics_free
dynamic_report_free
sb_free(&ops)
free(us_ok)
free(us)
free(rf)
pe_free(&img)
```

```
sb_free(&err)
sb_free(&err2)
buf_free_owned(owned)
```

Порядок следует зависимостям. Reports уже записаны, поэтому borrowed pointers больше не используются.

`PeImage.sections` освобождается до input bytes, но мог бы освобождаться и после: sections является отдельной allocation и не указывает внутрь input. Главное - не использовать models после release.

Полный автомат состояний `main`

```
START
-> CLI_PARSED
-> INPUT_LOADED
-> PE_PARSED
-> RF_TABLE_READ
-> UNWIND_ATTEMPTED_FOR_ALL
-> OPTIONAL_HEURISTICS
-> OPTIONAL_DYNAMIC
-> OPTIONAL_OUTPUTS
-> CLEANUP
-> EXIT
```

Fatal transitions до построения основной модели немедленно ведут в cleanup соответствующего уровня и exit 1.

Локальные analytical failures могут позволять продолжение, но изменяют final exit code.

Tests, относящиеся к `main`

`test_cli_contract.py` проверяет:

- help/version;
- invalid options;
- numeric ranges;
- exit codes;
- output combinations.

`run_regression_samples.py` проверяет end-to-end pipeline на шести PE.

`test_report_semantics.py` запускает CLI и проверяет отношения полей.

`test_release_metadata.py` проверяет согласованность version/schema declarations.

Mutation test многократно вызывает CLI на изменённых входах и требует controlled result вместо crash.

Что можно улучшить

Текущий `main` является последовательным orchestration layer. Возможные улучшения:

- единый cleanup block для уменьшения повторения ранних failure paths;
- structured machine-readable error output;
- более детальные exit subcodes;
- `--quiet`;
- `--output-dir`;
- сохранение duration каждого этапа;
- явное различие `not_requested`, `failed`, `completed` для каждого stage;
- streaming или batching для очень больших reports.

Эти изменения не требуют менять предметные алгоритмы PE/unwind.

Проверка полного понимания `main`

Читатель должен суметь для любой строки основного pipeline ответить:

1. какие объекты уже существуют;
2. кто ими владеет;
3. какие инварианты доказаны;
4. что произойдёт при failure;
5. продолжится ли анализ;
6. какой exit code сформируется;
7. какие outputs ещё могут быть созданы;
8. где объект освобождается.

Модуль util.c: безопасные байты, строки, файлы и SHA-256

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Открыть файл

Состояние перед этапом. Есть путь как строка, но размер и содержимое неизвестны.

Техническое выполнение. buf_init_from_file вызывает fopen в бинарном режиме, перемещается к концу через fseek и узнаёт длину через ftell.

Смысл этапа и его место в маршруте. До выделения памяти нужно знать, сколько байтов требуется.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Известен точный размер входа.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Выделить и заполнить память». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. Любой сбой открытия или позиционирования закрывает FILE и возвращает false.

Этап 2. Выделить и заполнить память

Состояние перед этапом. Размер известен, но байты ещё находятся в файле на диске.

Техническое выполнение. Функция выделяет uint8_t-массив через malloc и полностью читает его через fread.

Смысл этапа и его место в маршруте. Остальным модулям необходим непрерывный буфер с быстрым произвольным доступом.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Buf.data указывает на байты, Buf.size хранит длину, owned_mem фиксирует владельца памяти.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Проверять каждое чтение». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Проверять каждое чтение

Состояние перед этапом. Parser хочет прочитать len байтов с позиции off.

Техническое выполнение. buf_read проверяет указатели, условие off <= size и len <= size - off, затем вызывает memcpy.

Смысл этапа и его место в маршруте. Недоверенный файл может заявить смещение за своим концом. Проверка до memcpy предотвращает чтение чужой памяти.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Вызывающая функция либо получает ровно запрошенные байты, либо false без частичного результата.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Собрать числа little-endian». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Собрать числа little-endian

Состояние перед этапом. Получены отдельные байты поля PE.

Техническое выполнение. buf_read_u16/u32/u64 читают временный массив и функции le16/le32/le64 собирают число, помещая младший байт в младшие разряды.

Смысл этапа и его место в маршруте. PE хранит числа в порядке little-endian; прямое копирование в native integer было бы зависимо от архитектуры host.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Проект получает переносимое целое фиксированной ширины.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Создать расширяемую строку». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Создать расширяемую строку

Состояние перед этапом. Модулю нужно постепенно собирать ошибку, JSON или HTML неизвестной заранее длины.

Техническое выполнение. sb_init создаёт StrBuf, sb_reserve увеличивает capacity, sb_appendn копирует фрагмент, sb_appendf сначала вычисляет размер форматирования.

Смысл этапа и его место в маршруте. Обычный массив char имеет фиксированную ёмкость; переполнение строки повреждает память.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: StrBuf хранит строку, её текущую длину len, доступную ёмкость cap и признак failed.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Записать файл атомарно». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Записать файл атомарно

Состояние перед этапом. Полный текст отчёта уже собран в памяти.

Техническое выполнение. file_write_atomic сначала пишет временный файл рядом с итоговым, закрывает его и только затем выполняет rename.

Смысл этапа и его место в маршруте. Если запись оборвётся, старый законченный отчёт не должен быть заменён частичным.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Итоговый путь появляется только после успешной полной записи.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Вычислить SHA-256». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Вычислить SHA-256

Состояние перед этапом. Нужно идентифицировать конкретные байты входа.

Техническое выполнение. sha256_hex обрабатывает данные блоками, выполняет раунды SHA-256, дополняет последний блок и переводит 32-байтовый digest в 64 шестнадцатеричных символа.

Смысл этапа и его место в маршруте. Имя файла не является надёжным идентификатором содержимого.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Metadata получает устойчивый fingerprint - цифровой отпечаток входа.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. util.c: основание безопасного parser и генерации отчётов

Почему util.c является частью доказательства корректности

pe.c и unwind.c содержат предметные знания о формате. Но все их проверки бессмысленны, если базовая операция чтения способна выйти за границу или неверно собрать integer.

util.c формирует минимальный trusted computing base проекта на уровне пользовательского кода. Ошибка здесь распространялась бы на все последующие conclusions.

Buf как read-only view

```
typedef struct Buf {
const uint8_t* data;
size_t size;
} Buf;
```

Buf не содержит capacity, mutable pointer или ownership flag. Он описывает только доступное для чтения содержимое.

const uint8_t* запрещает изменение байтов через этот pointer на уровне type system C. Это shallow const: сама структура Buf может быть присвоена, но pointed bytes через неё не модифицируются.

size задаёт единственную допустимую границу. Parser не использует null terminator, потому что binary input может содержать нулевые байты в любой позиции.

buf_init_from_file

Контракт:

```
bool buf_init_from_file(
Buf* out,
const char* path,
uint8_t** owned_mem
);
```

Preconditions:

- все три pointers ненулевые;
- path указывает на корректную C string.

Postcondition success:

```
out->data == *owned_mem
out->size == фактическое число прочитанных байтов
*owned_mem != NULL
```

Postcondition failure:

```
*owned_mem == NULL
нет открытого FILE
нет неосвобождённой allocation, созданной функцией
```

Функция сначала выполняет:

```
*owned_mem = NULL;
```

Это важно для failure paths caller. Даже если открытие не удалось, caller не получает неопределённый pointer.

Определение размера

Используются `fseek` и `ftell`. `ftell` возвращает `long`, поэтому текущая реализация имеет практическое ограничение для файлов, размер которых не представим `long` на host.

На современных 64-битных POSIX и Windows toolchains это обычно достаточно для анализируемых PE, но строгая поддержка очень больших файлов потребовала бы platform-specific 64-bit file APIs.

Отрицательный результат `ftell` считается failure:

```
if (sz < 0) ...
```

Пустой файл

Для размера zero:

```
size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
```

Выделяется один байт, хотя логический `Buf.size` равен нулю.

Причина: `malloc(0)` может вернуть null или уникальный pointer в зависимости от реализации. Проект хочет, чтобы failure allocation не смешивался с пустым input.

Ни один байт фиктивной allocation не считается частью input, потому что bounds определяет `size`.

Полнота `fread`

```
size_t rd = fread(mem, 1, (size_t)sz, f);
```

Функция требует:

```
rd == sz
```

Частичное чтение не принимается. Если файл изменился, возник I/O error или чтение прервано, allocation освобождается.

Это сохраняет invariant: SHA-256 и parser работают с полной последовательностью заявленного размера.

buf_free_owned

```
void buf_free_owned(uint8_t* owned_mem) {
free(owned_mem);
}
```

Отдельная wrapper function документирует ownership pair:

```
buf_init_from_file -> buf_free_owned
```

`free(NULL)` допустим стандартом C, поэтому функция безопасна для null.

buf_read

Контракт:

```
bool buf_read(
const Buf* b,
size_t off,
void* out,
size_t len
);
```

`void* out` означает, что destination type не фиксирован интерфейсом. Функция копирует raw bytes.

Проверка:

```
if (!b || !out) return false;
```

Даже при `len == 0` проект требует non-null `out`. Это упрощает contract.

Далее:

```
if (off > b->size) return false;
```

`off == size` допустим только для zero-length read.

Затем:

```
if (len > b->size - off) return false;
```

После доказательства диапазона вызывается:

```
memcpy(out, b->data + off, len);
```

`memcpy` требует, чтобы source и destination ranges были валидны и не перекрывались. Destination предоставляется caller. Source доказан `Buf`.

Typed read functions

`buf_read_u16` не записывает прямо в `uint16_t` из arbitrary file address:

```
uint8_t tmp[2];
if (!buf_read(b, off, tmp, 2)) return false;
*out = le16(tmp);
```

Причины:

- file offset может быть невыровнен;
- host endianness не должен влиять;
- чтение C-object через cast может нарушать aliasing/alignment rules;

- bounds должны быть проверены до interpretation.

Такой подход немного многословнее direct cast, но создаёт переносимый parser.

`le16`, `le32`, `le64`

`le64` строится из двух `le32`:

```
return (uint64_t)le32(p) |
((uint64_t)le32(p + 4) << 32);
```

Cast к `uint64_t` выполняется до сдвига на 32. Если сдвигать 32-битное значение на 32 без расширения, операция была бы недопустимой.

Эти helpers являются pure functions: результат зависит только от входных байтов, state не изменяется.

`StrBuf`

```
typedef struct StrBuf {
char* s;
size_t len;
size_t cap;
bool failed;
} StrBuf;
```

`s` указывает на allocation.

`len` - число содержательных bytes без завершающего null.

`cap` - вместимость allocation в bytes.

`failed` - sticky error state.

Invariant успешного initialized buffer:

```
s != NULL
len < cap
s[len] == '\0'
failed == false
```

Наличие null terminator позволяет передавать `s` функциям C strings, а `len` позволяет не вычислять `strlen` после каждого append.

Почему error state sticky

Если одна операция append не смогла выделить память, дальнейшее продолжение могло бы сформировать усечённый JSON. Поэтому:

```
if (!sb || sb->failed) return false;
```

После failure buffer остаётся failed до `sb_free`/новой initialization.

Caller может проверить итог:

```
sb_ok(&buffer)
```

Это transactional-like property на уровне построения текста: нельзя принять partial output как complete.

Рост capacity

`sb_reserve` увеличивает capacity:

```
increment = newcap / 2 + 64
newcap += increment
```

Рост больше линейного снижает число `realloc` при последовательных append.

Перед сложением:

```
if (increment > SIZE_MAX - newcap)
```

проверяется overflow.

Если geometric growth не может быть вычислен, функция использует точно `need_cap`. Затем `realloc` либо возвращает новую allocation, либо оставляет старую действительной и возвращает null. При null устанавливается `failed`.

sb_appendn

Проверка required capacity:

```
if (sb->len == SIZE_MAX ||
n > SIZE_MAX - sb->len - 1u)
```

`+1` нужен для null terminator.

После reserve:

```
memcpy(sb->s + sb->len, s, n);
sb->len += n;
sb->s[sb->len] = 0;
```

Invariant восстанавливается после каждой успешной операции.

sb_appendf

Variadic function принимает printf format.

Первый `vsnprintf(NULL, 0, ...)` вычисляет необходимую длину. Для повторного использования аргументов создаётся `va_copy`, потому что проход `vsnprintf` изменяет состояние `va_list`.

После reserve второй `vsnprintf` пишет строку. Проверяется:

```
wrote == need
```

Compiler attribute `format(printf, ...)` для GCC/Clang позволяет статически проверять соответствие format и аргументов.

Атомарная запись

`file_write_atomic` создаёт temporary sibling path:

```
<path>.tmp.<pid>
```

Пишет полное содержимое, выполняет `fflush`, закрывает stream, затем заменяет destination.

До replace существующий report остаётся неизменным. Если write failed, temp удаляется.

На Windows:

```
MoveFileExA(
temp_path,
path,
MOVEFILE_REPLACE_EXISTING |
MOVEFILE_WRITE_THROUGH
)
```

На POSIX:

```
rename(temp_path, path)
```

Соседнее расположение важно для атомарности rename в пределах файловой системы.

Ограничение: текущая реализация не выполняет POSIX `fsync` для file descriptor и parent directory. `fflush` переносит данные из user-space stdio buffer, но не является полной гарантией durability при system crash. Для CLI report это разумный уровень, но high-assurance storage мог бы усилить protocol.

SHA-256

SHA-256 преобразует сообщение произвольной конечной длины в 256-bit digest.

Проект хранит результат как 64 lowercase hex characters:

```
256 bits / 4 bits per hex digit = 64 digits
```

Плюс null terminator:

```
char out_hex[65];
```

Digest используется для identity input bytes. Он не:

- шифрует файл;
- подтверждает автора;
- является digital signature;
- определяет maliciousness.

Связь `util.c` с отчётом и tests

`Buf` используется всеми parsers.

`StrBuf` используется diagnostics и serializers.

Atomic writer используется CSV/JSON/HTML.

SHA-256 попадает в report metadata.

`unit_core_edges` проверяет boundary conditions.

Mutation fuzz проходит через `Buf` operations на изменённых inputs.

JSON/HTML tests косвенно проверяют `StrBuf` и atomic output.

Sanitizers проверяют memory safety на runtime test paths.

Полная роль модуля

`util.c` обеспечивает четыре свойства всего продукта:

1. input bytes имеют явную длину;

2. каждое чтение доказывает границы;

3. многобайтные значения декодируются независимо от host layout;

4. output либо формируется полностью, либо отмечается failure.

Следующий модуль `pe.c` опирается на эти свойства и добавляет семантику формата.

Модуль unwind.c: от Exception Directory к UNWIND_INFO

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Найти физические байты Exception Directory

Состояние перед этапом. PeImage содержит exception_rva и exception_size - координаты в виртуальном образе.

Техническое выполнение. unwind_read_runtime_functions_ex вызывает pe_rva_to_fileoff и получает файловое смещение.

Смысл этапа и его место в маршруте. Buf можно читать только по file offset, а каталог хранит RVA.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Проект знает, с какого байта файла начинается таблица функций.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Определить количество записей». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. Если RVA невозможно сопоставить с физическими байтами, возвращается MALFORMED, а main формирует частичный структурный отчёт.

Этап 2. Определить количество записей

Состояние перед этапом. Известен размер каталога в байтах.

Техническое выполнение. Функция проверяет кратность 12, потому что AMD64 RUNTIME_FUNCTION содержит три 32-разрядных поля, и вычисляет count = size / 12.

Смысл этапа и его место в маршруте. Неполный хвост не должен интерпретироваться как законченная запись.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Известно точное число элементов массива.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Прочитать RUNTIME_FUNCTION». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Прочитать RUNTIME_FUNCTION

Состояние перед этапом. Выделен массив нужной длины.

Техническое выполнение. Цикл читает BeginAddress, EndAddress и UnwindInfoAddress через buf_read_u32 и сохраняет их как begin_rva, end_rva, unwind_rva.

Смысл этапа и его место в маршруте. Эта тройка связывает диапазон машинного кода с metadata его стека.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется RuntimeFunction[] - основная функция-ориентированная карта .pdata.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Найти UNWIND_INFO конкретной функции». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Найти UNWIND_INFO конкретной функции

Состояние перед этапом. main передал rf[i].unwind_rva в unwind_parse.

Техническое выполнение. parse_unwindinfo_header переводит RVA в file offset, проверяет наличие четырёх байтов header и читает их.

Смысл этапа и его место в маршруте. Нельзя декодировать операции, пока не доказана доступность заголовка.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Известны исходные четыре байта и физическая граница записи.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Разобрать поля заголовка». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Разобрать поля заголовка

Состояние перед этапом. Получены четыре байта UNWIND_INFO.

Техническое выполнение. unwind_parse битовыми операциями извлекает Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister и FrameOffset.

Смысл этапа и его место в маршруте. Несколько логических полей упакованы в общие байты; их нужно разделить по спецификации.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: UnwindSummary получает основные свойства функции и число двухбайтовых slots.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Декодировать все UNWIND_CODE». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Декодировать все UNWIND_CODE

Состояние перед этапом. CountOfCodes сообщает количество slots, но не количество человеческих операций.

Техническое выполнение. Цикл читает CodeOffset и пару UnwindOp/OpInfo. slots_for_code определяет, сколько дополнительных slots занимает operand. Decoder не увеличивает индекс на единицу вслепую, а перескакивает через всю операцию.

Смысл этапа и его место в маршруте. UWOP_ALLOC_LARGE и SAVE-операции занимают несколько slots. Ошибка здесь сдвинула бы чтение всего хвоста.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Summary содержит stack effect, маски сохранённых регистров и anomaly bits.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Учесть выравнивание хвоста». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Учесть выравнивание хвоста

Состояние перед этапом. Массив кодов закончился, но следующий 32-разрядный элемент должен начинаться на правильной границе.

Техническое выполнение. Если CountOfCodes нечётный, decoder пропускает один двухбайтовый padding slot.

Смысл этапа и его место в маршруте. Без padding handler RVA или chained entry были бы прочитаны со сдвигом на два байта.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Получено правильное начало tail - хвостовой части UNWIND_INFO.

Передача дальше. Полученный результат становится исходным состоянием для этапа 8 «Интерпретировать tail по Flags». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 8. Интерпретировать tail по Flags

Состояние перед этапом. Известны flags и адрес начала хвоста.

Техническое выполнение. При EHANDLER/UHANDLER читается handler RVA; при CHAININFO читается chained RUNTIME_FUNCTION. Эти варианты взаимоисключающе определяют смысл следующих байтов.

Смысл этапа и его место в маршруте. Одинаковые четыре байта нельзя интерпретировать без учёта Flags.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Summary получает handler или chain и точные diagnostics при невозможности чтения.

Передача дальше. Полученный результат становится исходным состоянием для этапа 9 «Проследовать по цепочке безопасно». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 9. Проследовать по цепочке безопасно

Состояние перед этапом. CHAININFO указывает на другую запись metadata.

Техническое выполнение. Decoder хранит посещённые RVA, ограничивает глубину max_chain_depth и прекращает цикл при повторе.

Смысл этапа и его место в маршруте. Повреждённая цепочка не должна привести к бесконечной рекурсии или чтению без границ.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: В отчёт попадает chain path, depth и признак loop/depth violation.

Передача дальше. Полученный результат становится исходным состоянием для этапа 10 «Сохранить объяснимый сбой». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 10. Сохранить объяснимый сбой

Состояние перед этапом. Любой этап может обнаружить обрыв или недопустимое поле.

Техническое выполнение. unwind_parse_fail и diagnostic helpers записывают имя поля, file offset, expected/available bytes или slots и небольшой фрагмент исходных байтов.

Смысл этапа и его место в маршруте. Одного сообщения «parse failed» недостаточно для проверки результата аналитиком.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: us_ok[i] становится false, но UnwindSummary сохраняет доказательство причины и остальные функции продолжают анализироваться.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. `unwind.c`: от таблицы функций к модели восстановления стека

Почему Windows x64 хранит таблицу функций

Исключение может возникнуть в произвольной инструкции функции. В этот момент обычный epilog не выполнен. Значения `RSP` и nonvolatile registers отражают частично или полностью созданный stack frame.

Windows должна:

- определить, какой function range содержит текущий `RIP`;
- найти описание изменений prolog;
- восстановить caller context;
- определить наличие language-specific handler;
- продолжить поиск обработчика или stack walk.

Для nonleaf functions AMD64 image содержит function table. PE exception directory указывает на массив `RUNTIME_FUNCTION`.

`RuntimeFunction`

```
typedef struct RuntimeFunction {
uint32_t begin_rva;
uint32_t end_rva;
uint32_t unwind_rva;
} RuntimeFunction;
```

Каждое поле 4 bytes. Размер записи 12.

Диапазон code:

```
[BeginAddress, EndAddress)
```

`UnwindInfoAddress` указывает на `UNWIND_INFO` либо связанную encoding.

Fields являются image-relative, поэтому хранятся как RVA.

Чтение exception directory

`unwind_read_runtime_functions` сначала обнуляет outputs:

```
*out_rf = NULL;
*out_count = 0;
```

Если RVA и size оба zero, отсутствие таблицы считается корректным:

```
return true;
```

Если zero только одно field, directory malformed.

Далее exception RVA преобразуется через `pe_rva_to_fileoff`.

Проверяется полный файловый диапазон:

```
off + exception_size <= b->size
```

и structural size:

```
size >= 12
size % 12 == 0
```

Текущая policy strict: неполный хвост отклоняет всю таблицу. Альтернативный forensic parser мог бы прочитать complete entries и сохранить remainder anomaly, но проект предпочитает однозначный contract directory.

Количество:

```
n = exception_size / 12;
```

Для sample:

```
0x24 / 0x0C = 3
```

Почему reader не проверяет begin/end сразу

Функция безопасно извлекает все triples. Semantics ranges проверяются в `heuristics.c`.

Разделение позволяет сохранить malformed values в report model и агрегировать counts. Reader отвечает на вопрос «можно ли физически прочитать таблицу», heuristics - «насколько согласованы значения».

`UnwindSummary`

Структура является нормализованным результатом декодирования:

```
valid
version
flags
prolog_size
count_codes
frame_reg
frame_off
handler
chain
fixed_stack_bytes
pushed_nonvol_count
saved register masks
anomalies
parse_error
```

Она не сохраняет все raw slots. Подробный текст может строиться отдельно через `ops_text`.

Это design choice: массовый analysis хранит компактную summary, а `--ops` повторно декодирует выбранные entries для presentation.

Register encoding

Unwind format кодирует general-purpose register числом `0..15`.

`unwind_reg_name` отображает:

```
0 RAX
1 RCX
2 RDX
3 RBX
4 RSP
5 RBP
6 RSI
7 RDI
8 R8
...
15 R15
```

Unknown value возвращает `REG?`, хотя 4-bit field физически ограничен 0..15.

Flags

Поддерживаемые:

```
UNW_FLAG_EHANDLER = 0x01
UNW_FLAG_UHANDLER = 0x02
UNW_FLAG_CHAININFO = 0x04
```

`EHANDLER` указывает exception handler.

`UHANDLER` указывает termination/unwind handler.

`CHAININFO` означает, что tail содержит chained `RUNTIME_FUNCTION`.

CHAININFO не должен одновременно сочетаться с handler flags в той же record. Проект отмечает такое состояние `UA_BAD_OPERAND`.

Вход `unwind_parse`

```
bool unwind_parse(
const Buf* b,
const PeImage* img,
uint32_t unwind_rva,
const UnwindParseOpts* opts,
UnwindSummary* out,
StrBuf* ops_text,
StrBuf* err
);
```

`b` предоставляет bytes.

`img` предоставляет mapping и `SizeOfImage`.

`unwind_rva` выбирает structure.

`opts` задаёт chain policy.

`out` получает summary.

`ops_text` optional.

`err` получает diagnostic.

Инициализация и локализация ошибки

```
memset(out, 0, sizeof(*out));
size_t err_start = err->len;
```

`err` может уже содержать старый текст. `err_start` позволяет `unwind_parse_fail` скопировать только diagnostic текущего вызова в `out->parse_error`.

Trailing newline удаляется, потому что JSON field должен содержать compact string.

Options

Defaults:

```
max_depth = 8
follow_chain = true
```

Caller может передать options. Значение depth выше 16 clamp-ится, потому что local array:

```
uint32_t seen[16];
```

не способен хранить больше уникальных chain RVA.

Alignment `UNWIND_INFO`

RVA должен быть 4-byte aligned:

```
if ((unwind_rva & 3u) != 0)
UA_ALIGNMENT
```

Нарушение не делает header физически нечитаемым, поэтому parser продолжает и сохраняет anomaly.

Это пример distinction:

```
structurally suspicious, but decodable
```

Mapping header

`parse_unwindinfo_header`:

1. преобразует RVA в file offset;

2. проверяет четыре bytes;

3. читает header.

Failure добавляет `UA_TRUNCATED` и возвращает false.

Термин `TRUNCATED` используется расширенно: structure физически недоступна или не помещается в file-backed range.

Header layout

Четыре байта:

```
byte 0: Version:3 | Flags:5
byte 1: SizeOfProlog
byte 2: CountOfCodes
byte 3: FrameRegister:4 | FrameOffset:4
```

Extraction:

```
ver = hdr[0] & 0x7;
flags = hdr[0] >> 3;
fr = hdr[3] & 0x0F;
foff = hdr[3] >> 4;
```

`FrameOffset` измеряется units of 16 bytes. Actual offset:

```
foff * 16
```

Version policy

Поддерживаются V1 и V2.

V3 для Intel APX имеет другую структуру и не должен интерпретироваться как V1/V2. Проект:

```
UA_VERSION_UNKNOWN
return false
```

Test `test_unsupported_version_is_fatal` закрепляет это поведение.

Отказ лучше ложного успешного decode: неправильная версия сместила бы slots и tail, создав фиктивные handlers.

Frame register checks

Если `frame_reg == 0`, `frame_off` должен быть zero.

Если frame register nonzero, он должен быть допустимым GPR.

После parsing codes проект также требует наличие `UWOP_SET_FPREG`, если frame register задан и record не является chained.

Несогласованность помечается `UA_BAD_OPERAND`, но structure может оставаться valid.

CountOfCodes

`CountOfCodes` - количество двухбайтовых slots, а не логических operations.

Одна operation может занимать:

```
1, 2 или 3 slots
```

Array физически padded до чётного числа slots, чтобы tail был 4-byte aligned:

```
slots_even = (count + 1) & ~1
codes_bytes = slots_even * 2
```

Если `count = 1`, stored slots = 2, bytes = 4.

Второй slot является padding и не входит в semantic operation count.

Structure slot

Первый slot operation:

```
byte 0: CodeOffset
byte 1 low nibble: UnwindOp
byte 1 high nibble: OpInfo
```

`CodeOffset` указывает position в prolog после выполнения соответствующей operation.

Codes хранятся в порядке убывания offsets, потому что unwind применяет действия в обратном направлении.

Если очередной `CodeOffset` больше предыдущего, проект ставит:

```
UA_CODEOFFSET_ORDER
```

Если offset больше `SizeOfProlog`:

```
UA_CODEOFFSET_OUTSIDE
```

V2 epilog descriptors имеют другую семантику, поэтому исключаются из обычной проверки prolog offset.

Определение длины operation

`slots_for_code` получает:

```
version
op
opinfo
v2_seen_epilog
```

и возвращает физическое число slots.

Эта функция должна быть выполнена до чтения operands. Затем проверяется:

```
i + slots <= count
```

Неизвестный opcode получает `UA_BAD_OPCODE`, но parser безопасно продвигается на один slot. Это позволяет собрать остальные anomalies без чтения за границей.

Если известная operation заявляет operands за CountOfCodes, parsing становится fatal.

UWOP_PUSH_NONVOL

Семантика prolog:

```
push register
```

Stack pointer уменьшается на 8, значение register сохраняется.

Summary:

```
fixed_stack += 8
pushed_nonvol_count += 1
register bit added to saved_nonvol_mask
```

Проект допускает некоторые volatile register operands из-за наблюдаемого real-world toolchain behavior, исключая `RSP`. Это implementation compatibility policy, а не простое повторение имени opcode.

UWOP_ALLOC_SMALL

Размер:

```
OpInfo * 8 + 8
```

Диапазон:

```
8..128 bytes
```

Для sample slot:

```
CodeOffset = 4
Op = 2
OpInfo = 4
```

размер:

```
4 * 8 + 8 = 40
```

Поэтому report показывает:

```
prolog_size = 4
fixed_stack_bytes = 40
```

UWOP_ALLOC_LARGE

Имеет два modes.

Mode 0, `OpInfo == 0`:

- следующий slot содержит size / 8;
- actual size = operand * 8;
- total 2 slots.

Mode 1, `OpInfo == 1`:

- следующие два slots содержат полный 32-bit size;
- total 3 slots.

Другой `OpInfo` недопустим.

Project sanity rules отмечают использование large form для размеров, которые выглядят несогласованно с ожидаемым encoding choice. Это anomaly, не parse failure.

UWOP_SET_FPREG

Указывает установку frame pointer register, заданного header.

Operation сама не добавляет fixed stack bytes. Она изменяет способ вычисления frame base.

Повторная operation или отсутствие header frame register создаёт `UA_BAD_OPERAND`.

UWOP_SAVE_NONVOL

Описывает сохранение register в ранее выделенной области stack без `push`.

Следующий slot кодирует scaled offset.

Проект:

- проверяет наличие operand;
- отмечает register в mask;
- не добавляет fixed stack, потому что allocation описывается отдельной operation.

Это различие критично. `SAVE_NONVOL` сообщает расположение сохранённого значения внутри frame, но не означает дополнительное изменение `RSP`.

UWOP_SAVE_NONVOL_FAR

Использует два operand slots для 32-bit offset.

Far form предназначена для больших offsets. Малое или неверно выровненное значение получает `UA_OPERAND_SANITY`.

Parser всё равно сохраняет register mask, если field декодируемо.

XMM saves

`UWOP_SAVE_XMM128` и FAR variant описывают сохранение 128-bit XMM register.

Offsets масштабируются согласно specification, хотя compact summary проекта в основном сохраняет register mask и sanity.

Отдельный `saved_xmm_mask` не смешивает vector state с GPR.

V1 opcodes 6 и 7

Для V1 код поддерживает observed undocumented/common encodings `SAVE_XMM` и `SAVE_XMM_FAR`. Это compatibility behavior, которое должно явно отличаться от официальных V2 meanings.

V2 epilog descriptors

В V2 opcode 6 используется для epilog descriptors, opcode 7 - spare.

Первый descriptor может иметь short или long form; последующие описывают offsets from end.

Parser:

- определяет slot count с учётом first descriptor;
- строит readable `ops_text`;
- не применяет обычную CodeOffset check;
- продолжает декодировать последующие prolog operations.

Unit test содержит short и long records и проверяет `offset_from_end`.

UWOP_PUSH_MACHFRAME

Описывает machine frame, сформированный hardware/exception mechanism.

`OpInfo` определяет наличие error code.

Summary добавляет:

```
40 bytes без error code
48 bytes с error code
```

Другой `OpInfo` помечается `UA_BAD_OPERAND`.

Защита суммы fixed stack

`add_fixed_stack` проверяет:

```
UINT32_MAX - total < amount
```

При overflow:

```
total = UINT32_MAX
UA_OPERAND_SANITY
```

Saturation сохраняет upper-bound signal и не позволяет wrap к малому размеру.

Tail после slots

Tail начинается:

```
after_codes = codes_off + codes_bytes;
```

`codes_bytes` уже включает padding.

Содержимое зависит от flags.

Handler tail

При EHANDLER/UHANDLER первые 4 bytes tail - handler RVA.

Проект проверяет physical read и:

```
handler_rva < SizeOfImage
```

Выход за image ставит `UA_HANDLER_OOB`, но read остаётся successful.

Данные после handler RVA являются language-specific. Текущая summary их не декодирует.

Chain tail

При CHAININFO tail содержит ещё один 12-byte `RUNTIME_FUNCTION`.

Проверяются:

- `end > begin`;
- range в `SizeOfImage`;
- nonzero unwind RVA;
- unwind RVA в image.

Нарушение даёт `UA_CHAIN_INVALID`.

Следование по chain

Если enabled, проект начинает с:

```
cur = out->chain.unwind_rva;
```

Массив `seen[16]` хранит уже посещённые RVA. Повтор означает loop.

Внутренний recursive-like вызов:

```
unwind_parse(... follow_chain = false)
```

декодирует один следующий record без собственного продолжения chain. Внешний цикл контролирует depth и loop detection.

Summary parent объединяется:

```
fixed stack sum
pushed count sum
OR register masks
inherited handler
frame consistency check
```

Это compact analytical aggregation. Оно не заменяет точную реализацию `RtlVirtualUnwind` для любого instruction offset.

`valid` и anomalies

В конце:

```
out->valid = true;
return true;
```

Anomalies могут быть nonzero. `valid` означает, что structure была декодирована достаточно для summary, а не что metadata идеальна.

Fatal conditions:

- header unmappable/truncated;
- unsupported version;
- code array за file;
- operation operands за CountOfCodes;
- truncated handler/chain tail.

Nonfatal anomalies:

- alignment;
- bad flags/operands;
- order;
- code offset outside;
- handler OOB;
- chain invalid;
- suspicious operand form.

parse_error

При failure `unwind_parse_fail`:

- устанавливает `valid = false`;
- выбирает diagnostic текущего invocation;
- удаляет newline;
- ограничивает длину 159 characters;
- null-terminates field.

Report всегда может показать конкретную причину для function.

Сквозной sample

Runtime table:

```
[0] 1000..1010 -> 4000
[1] 1010..1029 -> 4004
[2] 1029..1040 -> 400C
```

Для record 1:

```
Unwind RVA 0x4004
section .xdata VA 0x4000
raw ptr 0xA00
delta 0x4
file offset 0xA04
```

Header и slot приводят к:

```
Version 1
Flags 0
Prolog 4
Count 1
Frame register 0
ALLOC_SMALL 40
```

JSON function object сохраняет эти значения.

Связь с heuristics

`unwind_parse` не назначает weight.

Он создаёт:

```
us_ok[i]
us[i].anomalies
handler_rva
unwind_rva
```

`heuristics.c` считает:

- parse failures;
- anomaly-bearing records;
- placement unwind RVA;
- placement handler RVA;
- reuse.

Так decode semantics отделена от detection policy.

Tests

`unit_unwind` строит synthetic byte buffers для точной проверки:

- alloc small;
- push nonvol;
- code offset order;
- operand sanity;
- real-world compatible operands;
- V2 epilogs;
- chain inheritance;
- invalid flags/frame;
- preserved parse error;
- unsupported V3;
- truncated handler;
- chain validation/alignment.

Synthetic tests необходимы, потому что позволяют задать отдельные bits без влияния linker.

Regression tests необходимы дополнительно, потому что проверяют полный PE path.

Ограничения decoder

Текущий decoder:

- не выполняет полный `RtlVirtualUnwind`;
- не вычисляет caller context для заданного instruction offset;
- не декодирует language-specific handler data;
- не поддерживает V3;
- агрегирует chain эвристически;
- не сопоставляет operations с disassembly;
- сохраняет masks, но не полные offsets каждого register save в JSON summary.

Это определяет направления развития, но не нарушает заявленный contract summary analyzer.

Модуль heuristics.c: от наблюдений к объяснимым findings

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Получить готовые факты

Состояние перед этапом. PeImage, RuntimeFunction[] и UnwindSummary[] уже построены parsers.

Техническое выполнение. heuristics_analyze обнуляет HeuristicReport и проверяет согласованность указателей и количества элементов.

Смысл этапа и его место в маршруте. Эвристический слой не должен повторно читать недоверенные байты по собственным правилам.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Он работает только с проверенными моделями и явными parse statuses.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Проверить карту функций». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 2. Проверить карту функций

Состояние перед этапом. Каждая RUNTIME_FUNCTION задаёт begin, end и unwind RVA.

Техническое выполнение. Модуль считает недопустимые диапазоны, отсутствие сортировки, одинаковые начала, overlaps и gaps.

Смысл этапа и его место в маршруте. Эти свойства влияют на однозначность поиска функции по адресу инструкции.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляются счётчики range-map family - семейства карты диапазонов.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Проверить размещение metadata». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Проверить размещение metadata

Состояние перед этапом. Для каждого unwind RVA можно определить секцию и её права.

Техническое выполнение. pe_find_section_for_rva связывает адрес с PeSection; heuristics проверяет необычные имена, executable или writable placement.

Смысл этапа и его место в маршруте. Метаданные обычно являются данными только для чтения; изменяемое или исполняемое размещение требует большего внимания.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется placement evidence с representative RVA.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Проверить результаты декодера». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Проверить результаты декодера

Состояние перед этапом. us_ok и UnwindSummary сообщают успех, anomalies, handler и chain.

Техническое выполнение. Модуль агрегирует parse failures, unsupported versions, truncation, operand anomalies, chain loops и подозрительную сложность операций.

Смысл этапа и его место в маршруте. Одиночная ошибка должна быть отличима от массового систематического нарушения.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Для каждого класса известно число затронутых функций и первая локализация.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Оценить reuse и dominance». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Оценить reuse и dominance

Состояние перед этапом. Handler RVA и UnwindInfoAddress могут повторяться у многих функций.

Техническое выполнение. Внутренняя hash map - хеш-таблица - считает частоты значений и находит наиболее распространённую запись.

Смысл этапа и его место в маршруте. Массовое повторное использование может быть нормальным свойством runtime, но необычная концентрация усиливает другие признаки.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляются ratios - доли reuse/dominance - и возможные findings.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Создать finding». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Создать finding

Состояние перед этапом. Конкретное правило превысило заданный порог.

Техническое выполнение. add_finding записывает code, family, severity, affected_count и location; annotate_finding добавляет описание и evidence.

Смысл этапа и его место в маршруте. Finding должен быть стабильным структурированным объектом, а не свободной строкой.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Report сможет показать, какое правило сработало, где и на скольких записях.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Начислить score». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Начислить score

Состояние перед этапом. Сработало одно или несколько правил.

Техническое выполнение. add_score прибавляет заданные implementation weights и ограничивает итог диапазоном.

Смысл этапа и его место в маршруте. Score нужен для приоритизации ручной проверки, но не является вероятностью вредоносности и не происходит из спецификации Microsoft.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: HeuristicReport получает воспроизводимый инженерный балл.

Передача дальше. Полученный результат становится исходным состоянием для этапа 8 «Применить факторы ослабления». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 8. Применить факторы ослабления

Состояние перед этапом. Некоторые наблюдения имеют нормальные объяснения: стандартный handler, допустимое имя секции, небольшой padding gap.

Техническое выполнение. add_mitigation сохраняет причину, по которой уверенность должна быть ниже, не удаляя исходный finding.

Смысл этапа и его место в маршруте. Честный анализ должен показывать не только подозрение, но и факторы неопределённости.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Пользователь видит и evidence, и причины не делать чрезмерный вывод.

Передача дальше. Полученный результат становится исходным состоянием для этапа 9 «Сформировать итог». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 9. Сформировать итог

Состояние перед этапом. Известны findings, families, score и mitigations.

Техническое выполнение. finalize_verdict считает разнообразие evidence, confidence, evidence_level, verdict и requires_manual_review.

Смысл этапа и его место в маршруте. Severity показывает серьёзность наблюдения, confidence - уверенность в достаточности доказательств; это разные величины.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется законченный HeuristicReport, который report.c только отображает.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. Глава 6. `heuristics.c`: от структурных фактов к оценке свидетельств

Место модуля в системе

`pe.c` и `unwind.c` отвечают на вопросы формата: какие поля присутствуют, какие значения прочитаны, нарушены ли локальные invariants. Они не должны решать, насколько совокупность наблюдений подозрительна.

`heuristics.c` получает уже построенные модели:

```
const PeImage* img
const RuntimeFunction* rf
const UnwindSummary* us
const bool* us_ok
size_t n
```

и формирует `HeuristicReport`.

Это переход от parsing к assessment. Входные данные модуля не являются raw bytes. Они представляют нормализованные факты предыдущих этапов.

Поток данных:

```
PE bytes
-> PeImage + structural counters
-> RuntimeFunction[]
-> UnwindSummary[] + us_ok[]
-> HeuristicReport
```

Если parser неверно извлёк поле, heuristic engine получит неверный факт. Поэтому качество assessment зависит от correctness всех предыдущих contracts.

Что означает слово heuristic

Heuristic, далее эвристика, - правило практического вывода, которое использует наблюдаемые признаки, но не гарантирует единственно правильную классификацию.

Проект не содержит обученной statistical model, malware classifier или probability calibration. Все weights и thresholds заданы константами в source code. Комментарий в начале `heuristics.c` прямо называет модель conservative engineering model, а не mathematically validated classifier.

Следствия:

- `score = 70` не означает 70% вероятности вредоносности;
- `confidence = 0.8` не является частотой правильных предсказаний на независимой выборке;
- threshold `1%` не выведен из стандарта PE;
- weight `25` не является нормативным уровнем Microsoft;
- одинаковый score может быть получен разными наборами evidence.

`score` - ограниченная инженерная сумма весов. `suspicion` - её линейная нормализация `score / 100`. `confidence` - отдельная rule-based оценка полноты и взаимного подтверждения evidence.

Выходная модель

`HeuristicReport` содержит четыре класса данных.

Первый класс - summary assessment:

```
int score;
double suspicion;
double confidence;
bool requires_manual_review;
uint32_t evidence_families_mask;
char verdict[48];
char evidence_level[16];
```

Второй класс - counters наблюдаемых нарушений и ratios.

Третий класс - structured findings:

```
typedef struct HeuristicFinding {
char code[40];
char family[32];
char severity[16];
char message[192];
bool has_location;
uint32_t representative_rva;
size_t affected_count;
} HeuristicFinding;
```

Четвёртый класс - weakening rules, названные `mitigations` в C model:

```
typedef struct HeuristicMitigation {
char code[40];
char message[192];
} HeuristicMitigation;
```

В JSON они выводятся как `weakening_rules_applied`, что точнее отражает назначение: правило не устраняет факт, а снижает confidence вывода.

Валидация arguments и initialization

`heuristics_analyze` требует ненулевые `img`, `out`, `err`. Если `n > 0`, обязаны существовать `rf`, `us` и `us_ok`.

Затем весь output обнуляется:

```
memset(out, 0, sizeof(*out));
```

После этого инициализируется `out->details`. Если allocation fails, функция возвращает false. Это важно для ownership: после успешного init `HeuristicReport` должен быть освобождён через `heuristics_free`.

`out->total = n`, а PE counters копируются из `PeImage`. Таким образом report сохраняет snapshot входных structural facts.

Evidence families

Проект группирует findings в пять семейств.

`table_integrity` включает PE section structure и `RUNTIME_FUNCTION` ranges.

`unwind_integrity` включает parse failures, opcode/operand/chain anomalies и комбинации placement с metadata corruption.

`metadata_placement` включает section context unwind records и handlers.

`unwind_reuse` включает concentration одного `UnwindInfoRVA`.

`handler_dominance` включает concentration одного handler target.

Семейство представлено одним bit в `evidence_families_mask`. `heuristics_evidence_family_count` считает установленные bits.

Family count влияет на confidence, потому что независимые классы evidence считаются более убедительными, чем несколько variants одного pattern. Это engineering assumption. Реальная независимость признаков не доказана статистически.

Frequency map

Для подсчёта повторов используется собственная небольшая hash map с open addressing и linear probing.

`HEntry` хранит:

```
uint32_t key;
uint32_t val;
bool used;
```

Для `unwind_counts` key равен `UnwindInfoRVA`, value - числу records с этим RVA.

Для `handler_counts` key равен `HandlerRVA`, value - числу unwind records с этим handler.

Initial capacity выбирается не меньше 64. При load factor около 70% map растёт:

```
(len + 1) * 10 >= cap * 7
```

`hmap_grow` выделяет новый array и повторно вставляет entries. Это необходимо, потому что index зависит от capacity.

Если map insertion fails после init, loop игнорирует return value:

```
(void)hmap_inc(...)
```

Практически failure возможен главным образом при grow allocation. В таком случае frequency results могут быть неполны, но функция не сигнализирует failure. Это остаточная техническая слабость: для production-grade evidence engine следовало бы propagate OOM и не формировать potentially incomplete reuse ratios. В учебной защите это нужно назвать честно.

Первый проход по RUNTIME_FUNCTION

Для каждой записи извлекаются:

```
b = BeginRVA
e = EndRVA
u = UnwindRVA
```

Invalid range

Если:

```
e <= b
```

увеличивается `rf_invalid_ranges`. Первый RVA сохраняется как representative location.

`End == Begin` описывает пустой range. `End < Begin` описывает обратный range. Оба нарушают ожидаемый полуинтервал.

Function range out of image

Если:

```
b >= SizeOfImage || e > SizeOfImage
```

увеличивается `rf_oob`.

Для `begin` используется `>=`, потому что RVA ровно `SizeOfImage` уже находится за последним byte. Для `end` используется `>`, потому что exclusive end может быть равен верхней границе.

Unwind RVA out of image

Если:

```
u >= SizeOfImage || u == 0
```

увеличивается `rf_unwind_oob`.

Zero рассматривается как missing. Даже если header bytes существуют по file offset zero, `UnwindData = 0` не является поддерживаемой valid link на `UNWIND_INFO`.

Unsorted input table

Модуль сравнивает текущий `BeginRVA` с предыдущим:

```
current_begin < previous_begin
```

Каждое такое место считается inversion. Это не полное число инверсий математического массива; это число локальных нарушений nondecreasing order между соседними entries. Поле `rf_unsorted` и сообщение «inversions» поэтому следует понимать в реализации проекта, а не как полный inversion count $O(n^2)$.

Reuse counts

Каждый `UnwindRVA`, включая malformed values, добавляется в `unwind_counts`. Это означает, что множество zero или одинаковых OOB values может увеличить reuse ratio. Одновременно они создадут integrity findings. Combination отражает реальную концентрацию указателей, но analyst должен учитывать, что dominant key может быть invalid.

Handler добавляется только если:

```
us_ok[i] && us[i].has_handler
```

Parse-failed record не участвует в handler dominance, потому что handler field не считается надёжно извлечённым.

Placement unwind metadata

Для успешно decoded record проект находит section по `UnwindRVA`.

Если section отсутствует, увеличивается `unwind_unusual_section`.

Если section существует, имя оценивается двумя policy.

Строгий allowlist:

```
.xdata
.rdata
.pdata
.gnu_xdata
.mrdata
xdata
rdata
pdata
```

Дополнительная plausible check принимает имя, содержащее case-insensitive `xdata`, `pdata` или `rdata`.

Это снижает false positives для toolchain-specific names. Но section names не являются гарантией semantics. Специально созданная executable+writable section с именем `.xdata` всё равно будет обнаружена по permissions.

Независимо от имени проверяются flags:

```
IMAGE_SCN_MEM_EXECUTE
IMAGE_SCN_MEM_WRITE
```

Исполняемая section увеличивает `unwind_exec_section`. Writable section увеличивает `unwind_writable_section`.

Metadata обычно является read-only data. Writable или executable placement создаёт повышенный review interest, но JIT/runtime и нестандартный linker layout остаются возможными benign explanations.

Placement handler

Если record имеет handler, проект находит section по `HandlerRVA`.

Если section отсутствует или не имеет execute flag:

```
handler_nonexec_section++
```

Если section writable:

```
handler_writable_section++
```

Handler - адрес code, поэтому non-executable placement противоречит ожидаемому runtime use. Writable+executable memory возможна, но нарушает распространённую W^X discipline и требует объяснения.

Здесь есть важная модельная граница. Section characteristics в файле не всегда полностью совпадают с runtime page protections после loader или защитного runtime. Поэтому static finding должен коррелироваться с dynamic memory map при серьёзном выводе.

Parse failure и anomaly

Если `us_ok[i] == false`, увеличивается `unwind_parse_fail`.

Если parse succeeded, но `us[i].anomalies != 0`, увеличивается `unwind_anomalies`.

Эти категории различаются.

Parse failure означает, что decoder не смог надёжно завершить поддерживаемую интерпретацию record.

Anomaly означает, что summary построен, но присутствуют нарушения: неизвестная version, bad opcode, code offset order, chain issue, alignment и другие.

Разделение позволяет report сохранять partial data для anomalous-but-decodable records.

Overlap и duplicate begin

Input table может быть unsorted, поэтому непосредственное сравнение соседей недостаточно для overlap detection. Проект создаёт `RfIdx[]`, копирует begin/end, сортирует через `qsort`, затем проходит sorted array.

`duplicate_begin`:

```
current.begin == previous.begin
```

`overlap`:

```
current.begin < max_end всех предыдущих records
```

Использование `max_end`, а не только previous end, корректно обнаруживает nested overlap:

```
A = [0x1000, 0x2000)
B = [0x1100, 0x1200)
C = [0x1300, 0x1400)
```

Хотя B закончилась до C, C всё ещё пересекает A. `max_end` остаётся `0x2000`.

Sort array содержит только begin/end и не меняет original arrays. Поэтому parallel relation `rf[i] <-> us[i] <-> us_ok[i]` сохраняется.

Ratios

После counting:

```
unwind_reuse_ratio = max_count_same_unwind / total_records
unwind_unique_ratio = unique_unwind_rvas / total_records
handler_reuse_ratio = max_count_same_handler / handler_count
```

При denominator zero `safe_ratio` возвращает 0.

`unwind_reuse_ratio` отвечает на вопрос о концентрации наиболее частого unwind RVA. Он не равен доле всех reused records. Например, два разных RVA могут встречаться по 20% каждый; max ratio будет 20%.

`unwind_unique_ratio` отражает diversity. Значение 1.0 означает unique RVA для каждого record. Низкое значение показывает reuse, но само по себе не создаёт finding.

`handler_reuse_ratio` считается среди records, которые имеют handler, а не среди всех runtime functions. Это важное различие denominator.

Score saturation

`add_score` вычисляет:

```
score = clamp(score + weight, 0, 100)
```

После достижения 100 дальнейшие findings сохраняются, но числовой score не растёт. Поэтому score 100 не означает одинаковую evidence mass для всех cases. Для анализа насыщенных cases нужно читать findings, affected counts и families.

PE structural findings и веса

Точные implementation weights:

```
| Finding | Weight | Severity |
|---|---:|---|
| `pe_section_raw_oob` | 25 | high |
| `pe_section_virtual_oob` | 20 | high |
| `pe_section_raw_overlap` | 15 | medium |
| `pe_section_virtual_overlap` | 20 | high |
| `pe_section_misaligned` | 8 | medium |
| `pe_exception_directory_oob` | 25 | high |
```

Для каждого finding сохраняется первый representative section RVA и полный affected count.

Representative RVA не означает, что только один объект нарушен. Это адрес первого найденного примера для навигации. Полный count показывает масштаб.

Empty exception table

Если `n == 0` и image имеет непустую executable section, добавляется:

```
code: no_runtime_functions
family: table_integrity
severity: medium
weight: 10
```

Message прямо допускает leaf-only image.

Если executable sections нет, score не начисляется. В details появляется info note о resource-only или forwarding image.

Это context-sensitive rule: одинаковое отсутствие `.pdata` оценивается по-разному в зависимости от наличия executable content.

Процентные integrity thresholds

Для invalid ranges, function OOB, unwind RVA OOB и parse failures threshold high равен строго больше 1%:

```
THRESHOLD_HIGH_PERCENT = 0.01;
```

Сравнение использует >, не >=.

Веса:

```
| Signal | Low/medium prevalence | High prevalence |
|---|---:|---:|
| invalid range | 10 | 20 |
| function OOB | 10 | 25 |
| unwind RVA OOB | 10 | 25 |
| unwind parse fail | 10 | 25 |
```

При высоком варианте severity становится high; иначе medium.

На малом sample один record почти всегда превышает 1%. Например, 1 из 3 равен 33.3%. Поэтому позже confidence получает small-sample penalty, если strong findings отсутствуют. Однако если finding уже high по prevalence, penalty `small_sample_size` не применяется, поскольку condition требует `strong_count == 0`.

Это демонстрирует, что score и confidence решают разные задачи.

Unwind anomaly threshold

Для decoded records с anomaly high threshold равен строго больше 5%:

```
THRESHOLD_UNWIND_ANOM_HIGH = 0.05;
```

Weight:

```
10 при <= 5%
20 при > 5%
```

Severity medium или high соответственно.

Table order findings

Точные weights:

```
rf_unsorted 15, medium
rf_overlaps 25, high
duplicate_begin 10, medium
```

Overlap получает более высокий weight, потому что нарушает однозначность coverage ranges. Unsorted table может быть восстановлена сортировкой для анализа, но runtime contract ожидает sorted entries.

Strong unwind reuse

Strong reuse finding появляется только если:

```
unwind_reuse_ratio > 0.30
n > 200
```

Weight 20, severity high.

Ограничение `n > 200` защищает от малых samples, где несколько одинаковых records дали бы высокий ratio. Strict `>` означает, что ровно 200 records недостаточно, а ровно 30% не проходит.

Representative RVA равен dominant `UnwindInfoRVA`; affected count равен max frequency.

Reuse может быть legitimate из-за shared epilog/prolog patterns, chained info или runtime layout. Поэтому даже strong reuse остаётся heuristic.

Handler dominance с benign-context suppression

Candidate требует:

```
handler_count > 50
handler_reuse_ratio > 0.80
```

Затем проект определяет `has_static_context`: имеются ли integrity issues, placement issues или moderate reuse candidate.

Также проверяется `dominant_runtime_like`:

- dominant handler находится в section;
- section executable;
- section name code-like: `.text`, `text`, содержит `text`, `code` или `init`;
- нет non-executable handler findings.

Если handler выглядит как обычный code runtime helper и других static signals нет, finding подавляется.

Именно это проверяется unit test, где 60 entries законно используют один handler в executable `.text`: score остаётся zero. При добавлении suspicious context dominance появляется.

Если finding создаётся:

```
weight: 15
severity: medium
family: handler_dominance
```

Это важная защита от false positive для central language runtime handlers.

Moderate unwind reuse требует corroboration

Moderate candidate:

```
unwind_reuse_ratio > 0.10
n > 200
```

Finding не создаётся автоматически. Требуется хотя бы один другой static signal: handler dominance, integrity issue, placement issue или anomaly.

Если strong reuse уже создан, moderate finding не добавляется.

Weight 10, severity medium.

Правило реализует принцип corroboration: умеренный repetition без context не считается самостоятельным evidence.

Placement weights

`unwind_in_exec_section`:

```
weight 20
severity high
```

`unwind_unusual_section`:

```
weight 8, severity medium
weight 15, severity high, если count > n/20
```

`n/20` использует integer division. Для `n < 20`, `n/20 == 0`, поэтому любое positive count проходит high branch. Это implementation nuance. Small-sample confidence penalties могут смягчить case только если strong finding отсутствует; здесь high severity создаёт strong count и penalty не применяется. Это потенциально резкое поведение на малых samples и кандидат для будущей calibration.

`unwind_writable_section`:

```
weight 10, severity medium
weight 18, severity high, если count > n/20
```

`handler_nonexec_section`:

```
weight 15
severity high
```

`handler_writable_section`:

```
weight 10
severity medium
```

Combination findings

Модуль начисляет дополнительные weights, когда два разных свойства встречаются вместе.

Executable unwind metadata и non-executable handler

Condition:

```
unwind_exec_section > 0
handler_nonexec_section > 0
```

Finding:

```
placement_exec_nonexec_handler_combo
weight 10
severity high
family metadata_placement
```

Placement и integrity

Condition:

```
(unwind_exec_section || handler_nonexec_section)
&&
(unwind_parse_fail || unwind_anomalies || rf_unwind_oob)
```

Finding:

```
placement_integrity_combo
weight 8
severity medium
family unwind_integrity
```

Reuse и dominance

Condition:

```
unwind_reuse_ratio > 0.10
handler_dominance finding exists
```

Finding:

```
reuse_dominance_combo
weight 8
severity medium
```

Writable metadata и writable handler

Condition:

```
unwind_writable_section > 0
handler_writable_section > 0
```

Finding:

```
writable_metadata_handler_combo
weight 10
severity high
```

Combination weights intentionally double-count related raw observations in aggregate score. Это не статистически независимое суммирование; это explicit policy, что сочетание труднее объяснить benignly.

Finding storage limit

Maximum:

```
HEURISTICS_MAX_FINDINGS = 32
```

Если findings больше, `add_finding` продолжает добавлять human-readable line в `details` и family bit, но structured array больше не растёт.

Consequences:

- score продолжает учитывать rule;

- `details` может содержать больше findings, чем `finding_count`;
- severity counts вычисляются только по stored structured findings;
- evidence level и confidence могут недоучесть findings за пределом 32.

Текущий набор rules меньше limit в обычном run, но model limitation существует. Production version могла бы использовать dynamic array или explicit truncation flag.

Severity counts и evidence level

High и critical считаются strong. Medium считается medium. Остальные stored findings относятся к weak.

Evidence level:

```
strong:
strong_count >= 2 или score >= 70

medium:
strong_count >= 1
или medium_count >= 2
или score >= 30

weak:
существует finding или score > 0

none:
иначе
```

Evidence level не совпадает с verdict thresholds. Например, один high finding при score 15 может дать `evidence_level = medium`, но `verdict = low_suspicion`.

Это не ошибка: поля описывают разные projections. Однако пользователю необходимо объяснять обе шкалы.

Confidence calculation

Initial:

```
confidence = 0.15
```

Additions:

```
+0.11 за evidence family
+0.08 за strong finding
+0.04 за medium finding
+0.08 при hard table issue
+0.08 при parse failure или unwind anomaly
```

Hard table issue:

```
rf_invalid_ranges
rf_oob
rf_unwind_oob
rf_overlaps
```

Если `score == 0`, confidence заменяется на `0.20`. Это означает умеренную уверенность только в ограниченном выводе «сильных static anomalies не обнаружено», а не уверенность в безопасности файла.

Final confidence ограничивается:

```
0.05 <= confidence <= 0.90
```

Static model никогда не выдаёт выше 0.90.

Weakening rules

`single_signal_only`

Применяется, если:

```
finding_count == 1
strong_count == 0
```

Penalty `0.12`.

`small_sample_size`

Применяется, если:

```
total < 32
strong_count == 0
finding_count > 0
```

Penalty `0.06`.

`placement_only_signal`

Применяется, если есть unusual section, но отсутствуют executable/writable placement, handler placement, range issues, parse failures и anomalies.

Penalty `0.10`.

Это правило не применится к `unwind_writable_section`, потому что writable placement рассматривается сильнее простого unusual name.

`single_family_only`

Применяется, если:

```
strong_finding_count == 0
family_count <= 1
finding_count > 0
```

Penalty `0.08`.

Mitigations добавляются в report после findings и не уменьшают score. Следовательно:

```
score описывает weighted anomaly mass
confidence описывает уверенность в статическом verdict
```

Manual review flag

`requires_manual_review` становится true, если:

- score не меньше 20;
- unwind metadata executable;
- unwind metadata writable;
- handler non-executable;
- handler writable;
- есть parse failure;
- применено хотя бы одно weakening rule.

Score 15 с единственным high placement finding также может потребовать review из-за конкретного counter. И наоборот, low score zero не требует review в автоматической модели, хотя общий reverse engineering может быть нужен по внешним причинам.

Verdict

Thresholds:

```
score >= 70 -> high_suspicion
score >= 40 -> moderate_suspicion
score >= 15 -> low_suspicion
иначе -> no_strong_static_anomalies
```

Название последнего verdict выбрано осторожно. Оно не утверждает `benign`, `clean` или `safe`. Оно утверждает только отсутствие сильных anomalies в реализованном наборе static rules.

Фактический benign sample

Для `benign_richer.exe`:

```
total = 3
all range counters = 0
unwind_parse_fail = 0
unwind_anomalies = 0
placement counters = 0
reuse conditions не проходят n > 200
score = 0
suspicion = 0.0
confidence = 0.2
verdict = no_strong_static_anomalies
evidence_level = none
requires_manual_review = false
```

Confidence 0.2 не означает, что sample на 20% benign. Это base confidence в узком static conclusion.

Фактический suspicious sample `suspicious_unwind_oob.exe`

Manifest требует:

```
rf_unwind_oob = 1
unwind_parse_fail = 1
score >= 40
verdict = moderate_suspicion
case_strength = strong_unconfirmed_case
```

При малом total один OOB pointer превышает 1%, поэтому:

```
unwind_rva_oob: +25, high
unwind_parse_fail: +25, high
```

Score достигает не менее 50. Два strong findings дают static evidence strong. Dynamic corroboration отсутствует, поэтому report-level case называется strong unconfirmed.

Фактический suspicious sample `suspicious_unwind_writable.exe`

Manifest требует writable unwind metadata и score не менее 15. На малом sample condition `count > n/20` может выбрать high branch weight 18.

Finding говорит, что metadata находится в section с write permission. Он не доказывает tampering. Manual validation должна проверить:

- section characteristics;
- runtime memory protection;
- toolchain;
- наличие JIT/runtime scenario;
- correspondence metadata и code.

Тесты модуля

`unit_heuristics` проверяет базовые score, placement, verdict и confidence.

`unit_heuristics_mitigations` покрывает weakening rules.

`unit_heuristics_context` проверяет benign dominance suppression, suspicious context и writable combinations.

`unit_heuristics_combos` проверяет additional weights при corroborating patterns.

`unit_core_edges` проверяет empty input, resource-only case, nested overlaps и PE structural findings.

Regression manifest связывает synthetic PE samples с ожидаемыми JSON paths.

Тесты подтверждают implementation contract. Они не валидируют weights как научно оптимальные для реального world corpus.

Что следует улучшать в будущей версии

Наиболее значимые направления:

1. Версионировать heuristic policy отдельно от report schema и tool version.
2. Добавить calibration corpus с размеченными compiler/runtime families.
3. Публиковать score contribution каждого finding, а не только total.
4. Добавить truncation flag для fixed finding array.
5. Propagate hash-map allocation failures.
6. Исправить percentage comparison для малых `n`, используя ratio вместо `count > n/20`.
7. Отделить invalid dominant keys от valid reuse statistics.
8. Сохранять all representative locations, а не только первый.
9. Добавить explicit rationale и standard/nonstandard tag каждому finding.
10. Не называть `confidence` вероятностью до empirical calibration.

Итоговая модель главы

`heuristics.c` не обнаруживает «зло» как бинарное свойство. Он:

```
получает проверяемые structural observations
-> агрегирует counts и concentration
-> создаёт named findings
-> группирует findings по evidence families
-> начисляет bounded engineering score
-> оценивает rule-based confidence
-> применяет weakening rules
-> формирует осторожный verdict
-> требует manual validation там, где автоматический вывод недостаточен
```

Именно так модуль должен быть представлен заказчику и на защите: как explainable static evidence engine для EH/unwind anomalies, а не как сертифицированный malware detector.

Модуль dynamic_win.c: Windows debugger и runtime evidence

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Проверить платформу и подготовить команду

Состояние перед этапом. Пользователь запросил --dynamic и передал путь и аргументы target.

Техническое выполнение. На Windows build_cmdline превращает отдельные аргументы в строку по правилам Windows quoting. На macOS/Linux stub сразу возвращает controlled unsupported result.

Смысл этапа и его место в маршруте. CreateProcessA принимает изменяемую строку, а не готовый argv; неправильные кавычки изменили бы аргументы программы.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Получена точная command line для одного запуска либо явное сообщение о недоступной платформе.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Создать процесс под управлением debugger». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 2. Создать процесс под управлением debugger

Состояние перед этапом. Команда готова, но target ещё не выполняется.

Техническое выполнение. dynamic_collect_windows вызывает CreateProcessA с DEBUG_ONLY_THIS_PROCESS. Windows создаёт процесс и направляет его debug events анализатору.

Смысл этапа и его место в маршруте. Только debugger получает уведомления об исключениях и может безопасно останавливать процесс на breakpoints.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: DynamicReport.ran становится true после успешного создания процесса.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Получить базу загруженного образа». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Получить базу загруженного образа

Состояние перед этапом. Статические адреса представлены как RVA, а debugger сообщает полные VA.

Техническое выполнение. При CREATE_PROCESS_DEBUG_EVENT collector получает фактический base address основного модуля. Модули процесса дополнительно перечисляются через Toolhelp snapshot.

Смысл этапа и его место в маршруте. Из-за ASLR образ может загрузиться не по preferred ImageBase. Формула handler VA = loaded base + handler RVA должна использовать фактическую базу.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Статическая карта и runtime-адреса теперь можно связать.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Найти адреса наблюдаемых API». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Найти адреса наблюдаемых API

Состояние перед этапом. Нужно остановиться на RtlAddFunctionTable и регистрациях handlers, но их VA различаются между процессами.

Техническое выполнение. local_proc_loc находит функцию в локальном системном модуле, определяет реальный модуль forwarded export и вычисляет offset. add_api_bps_if_ready прибавляет offset к базе такого же модуля target.

Смысл этапа и его место в маршруте. Нельзя переносить абсолютный адрес функции из процесса анализатора в target из-за ASLR.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Получены target VA системных API, пригодные для breakpoint.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Поставить breakpoint». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Поставить breakpoint

Состояние перед этапом. Известен target VA handler или API.

Техническое выполнение. bp_set читает исходный байт команды, сохраняет его, записывает 0xCC через WriteProcessMemory и вызывает FlushInstructionCache. При защищённой странице write_one_byte временно меняет protection через VirtualProtectEx.

Смысл этапа и его место в маршруте. 0xCC является инструкцией INT3. Когда CPU достигает её, Windows создаёт breakpoint exception и отдаёт управление debugger.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Breakpoint помечен armed, а self-check подтверждает, что в памяти действительно находится 0xCC.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Обрабатывать debug event loop». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Если этап не выполнен. Если breakpoint не установлен, счётчик failures и instrumentation_complete показывают неполноту наблюдения.

Этап 6. Обрабатывать debug event loop

Состояние перед этапом. Target продолжает исполняться и периодически останавливается на событиях.

Техническое выполнение. Цикл WaitForDebugEvent получает создание/завершение процесса и потоков, загрузку DLL, exceptions и single-step. После каждого события вызывается ContinueDebugEvent с выбранным статусом.

Смысл этапа и его место в маршруте. Debugger обязан явно решить, передавать ли исключение приложению и можно ли продолжить поток.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: DynamicReport постепенно накапливает события и состояние сессии.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Отличить обычное исключение от собственного breakpoint». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Отличить обычное исключение от собственного breakpoint

Состояние перед этапом. При EXCEPTION_DEBUG_EVENT известны exception code, address и first-chance flag.

Техническое выполнение. Collector ищет address в BpList. Неизвестное исключение записывается как runtime event и передаётся target. Известное означает попадание в установленную точку наблюдения.

Смысл этапа и его место в маршруте. Собственный INT3 не должен ошибочно считаться исходным поведением target.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: exceptions_seen отражает наблюдаемые исключения, а handler/API hits учитываются отдельно.

Передача дальше. Полученный результат становится исходным состоянием для этапа 8 «Выполнить исходную команду после breakpoint hit». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 8. Выполнить исходную команду после breakpoint hit

Состояние перед этапом. CPU остановился после исполнения INT3, исходный байт временно заменён.

Техническое выполнение. bp_clear восстанавливает orig byte. Context.Rip уменьшается на один, чтобы снова указывать на начало исходной команды. В EFlags устанавливается Trap Flag, создающий single-step после одной команды. PendingRearmList запоминает breakpoint для конкретного thread.

Смысл этапа и его место в маршруте. Если просто продолжить процесс, исходная команда будет пропущена; если сразу вернуть 0xCC, процесс снова остановится в той же точке.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Поток готов ровно один раз выполнить настоящую команду.

Передача дальше. Полученный результат становится исходным состоянием для этапа 9 «Переустановить breakpoint после single-step». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 9. Переустановить breakpoint после single-step

Состояние перед этапом. Исходная команда выполнилась и Windows прислала EXCEPTION_SINGLE_STEP.

Техническое выполнение. pending_rearm_take находит breakpoint по thread id, bp_set снова записывает 0xCC, Trap Flag снимается, процесс продолжается.

Смысл этапа и его место в маршруте. Точка наблюдения должна оставаться активной для последующих вызовов и других потоков.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Breakpoint снова armed, поведение target продолжено с минимальным вмешательством.

Передача дальше. Полученный результат становится исходным состоянием для этапа 10 «Считать параметры вызова API». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 10. Считать параметры вызова API

Состояние перед этапом. Breakpoint сработал на входе в системную функцию, а thread context содержит регистры x64.

Техническое выполнение. Collector читает RCX, RDX, R8 и R9 - первые четыре целочисленных аргумента Windows x64 calling convention - и сохраняет их как arg0..arg3.

Смысл этапа и его место в маршруте. Для RtlAddFunctionTable важен не только факт вызова, но и адрес массива, число entries и base address динамического кода.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: DynamicEvent содержит структурированные параметры регистрации или handler API.

Передача дальше. Полученный результат становится исходным состоянием для этапа 11 «Зафиксировать handler hit и RIP trace». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 11. Зафиксировать handler hit и RIP trace

Состояние перед этапом. Breakpoint сработал на RVA обработчика, найденного статикой.

Техническое выполнение. start_trace записывает адрес попадания и включает ограниченное single-step наблюдение. add_trace сохраняет последовательные RIP - адреса выполняемых команд - до trace_steps или capacity.

Смысл этапа и его место в маршруте. Статика показывает потенциальный handler, а hit доказывает достижение этого адреса в данном запуске. Trace показывает дальнейшее движение, но не объясняет семантику всех команд.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: DynamicReport получает unwind_handler_hits и массив trace.

Передача дальше. Полученный результат становится исходным состоянием для этапа 12 «Остановить эксперимент контролируемо». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 12. Остановить эксперимент контролируемо

Состояние перед этапом. Процесс может завершиться сам, зависнуть или породить слишком много событий.

Техническое выполнение. Collector проверяет hard timeout, ограничения массивов, dropped_events и состояние instrumentation. При timeout процесс завершается контролируемо.

Смысл этапа и его место в маршруте. Динамический анализ не должен работать бесконечно и скрывать потерю данных.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Поля ok, timed_out, target_exited, dropped_events и instrumentation_complete честно описывают качество конкретной сессии.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. Глава 7. `dynamic_win.c`: Windows dynamic evidence collector

Назначение и граница платформы

Статическая часть проекта компилируется и работает на macOS, Linux и Windows, потому что читает PE как массив bytes. Динамическая часть использует Windows Debug API и должна исполнять Windows x64 target. Поэтому она доступна только в Windows build.

Это различие реализовано compile-time branch:

```
#ifdef _WIN32
/* реальный collector */
#else
/* unsupported-platform stub */
#endif
```

На non-Windows система не пытается эмулировать process execution. `dynamic_collect_windows` инициализирует report, записывает понятную ошибку и возвращает false.

Следовательно:

```
Linux/macOS host:
static analysis Windows PE32+ AMD64 = поддерживается
dynamic Windows exception validation = не поддерживается

Windows x64 host:
static analysis = поддерживается
dynamic validation = поддерживается
```

Wine или virtual machine могут быть отдельной средой исследования, но текущий код напрямую не реализует и не тестирует Wine-specific backend.

Defensive intent

Header `dynamic.h` определяет цель как сбор runtime evidence:

- hits handlers из `UNWIND_INFO`;
- registration и hits VEH/VCH/UEF;
- registration dynamic function tables;
- first-chance и second-chance exception flow.

Collector не заявляет обход защит, извлечение скрытых данных или автоматическую деобфускацию. Он является ограниченным debugger, созданным под конкретные event types.

Модель результата

`DynamicReport` хранит session state, counters, bounded events, bounded trace и human-readable log.

Состояние:

```
bool enabled;
bool ran;
bool ok;
bool timed_out;
bool target_exited;
uint32_t target_exit_code;
```

`enabled` означает, что dynamic mode был запрошен и report инициализирован.

`ran` означает, что `CreateProcessA` succeeded и target действительно был создан.

`ok` означает:

```
CREATE_PROCESS event был получен
&& debug loop не встретил internal failure
&& timeout не наступил
```

`target_exited` отдельно показывает normal observation `EXIT_PROCESS_DEBUG_EVENT`. Session может быть завершена принудительно по timeout, тогда target не считается self-exited.

Events ограничены:

```
DYNAMIC_MAX_EVENTS = 256
```

После заполнения array новые события не перезаписывают старые; увеличивается `dropped_events`.

Trace также ограничен 256 instruction pointers. Input `trace_steps` clamp до 256.

Module map

Breakpoints на system APIs нельзя ставить по address, полученному в process анализатора. Из-за ASLR module bases отличаются.

`ModList` содержит максимум 128 modules:

```
typedef struct ModList {
ModInfo mods[128];
size_t count;
} ModList;
```

`mods_refresh_toolhelp`:

1. сбрасывает count;

2. вызывает `CreateToolhelp32Snapshot`;

3. перечисляет `MODULEENTRY32`;

4. сохраняет basename, base и size;

5. приводит name к lowercase.

Module list fixed-size. Если target загрузил более 128 modules, остаток не сохраняется. Truncation flag отсутствует. Это residual limit: API resolution для позднего module может не сработать без

явной ошибки.

Разрешение forwarded API

Windows export может быть forwarded. Например, symbol запрашивается через `kernel32.dll`, но фактический code address принадлежит `kernelbase.dll`.

`local_proc_loc` выполняет:

1. `GetModuleHandleA(preferred_mod)`;
2. `GetProcAddress(symbol)`;
3. `GetModuleHandleExA(...FROM_ADDRESS...)`, чтобы определить real local module;
4. получает basename real module;
5. вычисляет offset:

```
offset = LocalFunctionVA - LocalRealModuleBase
```

После этого `add_api_bps_if_ready` ищет module с таким basename в target:

```
TargetFunctionVA = TargetModuleBase + offset
```

Эта схема корректно переносит symbol location между процессами при одинаковой версии загруженного module. Она предполагает, что local и target используют совместимый module binary с тем же internal offset. Для обычного запуска на одной Windows installation это разумно; cross-architecture или module substitution потребовали бы remote export parsing.

Наблюдаемые API

Collector ставит breakpoints на:

```
AddVectoredExceptionHandler
AddVectoredContinueHandler
SetUnhandledExceptionFilter
RtlAddFunctionTable
RtlInstallFunctionTableCallback
```

Kinds хранятся как bit mask. Один address может соответствовать нескольким logical kinds, поэтому existing breakpoint получает:

```
ex->kinds |= kind;
```

Это предотвращает дублирование `INT3` по одному address.

Breakpoint model

```
typedef struct Breakpoint {
uint64_t addr;
uint8_t orig;
bool armed;
bool rearm_pending;
uint32_t kinds;
} Breakpoint;
```

`addr` - target VA.

`orig` - исходный byte, который был заменён.

`armed` - по address ожидается `0xCC`.

`rearm_pending` - исходная instruction временно восстановлена, breakpoint должен быть поставлен снова после single-step.

`kinds` - logical meanings address.

List имеет capacity 512. При переполнении `bp_add_and_arm` возвращает false, но dedicated overflow counter отсутствует.

Запись bytes в remote process

`write_one_byte` сначала вызывает `WriteProcessMemory`. Если write failed, code сохраняет error и пробует временно изменить page protection:

```
VirtualProtectEx(..., PAGE_EXECUTE_READWRITE)
WriteProcessMemory
VirtualProtectEx(..., oldProt)
```

После successful write вызывается `FlushInstructionCache`.

Причина cache flush: processor может иметь ранее полученные instruction bytes. После изменения executable memory debugger должен обеспечить согласованность instruction fetch.

Временный `PAGE_EXECUTE_READWRITE` расширяет permissions целевой страницы. Проект возвращает previous protection, но на короткий период изменяет security state target. Это unavoidable вмешательство выбранной software-breakpoint technique и должно учитываться при anti-debug-sensitive samples.

Установка breakpoint

`bp_set`:

1. читает один byte по `addr`;

2. сохраняет его в `orig`;

3. пишет `0xCC`;

4. устанавливает `armed = true`;

5. очищает `rearm_pending`.

Здесь имеется subtle risk при повторной постановке: `bp_set` каждый раз заново читает current byte и записывает его в `orig`. Correct flow вызывает re-arm после того, как original instruction уже восстановлена, поэтому сохраняется правильный byte. Если функцию ошибочно вызвать на уже armed breakpoint, `orig` мог бы стать `0xCC`; guard в `bp_add_and_arm` предотвращает этот path.

`bp_clear` восстанавливает `orig` только если `armed`.

Self-check

`bp_add_and_arm_selfcheck` после первой попытки установки снова читает byte и добавляет deterministic log line:

```
[SelfCheck] ... OK (byte=0xcc)
```

Если byte не `0xCC`, появляется warning. Self-check также форматирует address как:

```
module.dll+offset (absolute VA)
```

Это улучшает auditability: report показывает не только желание поставить breakpoint, но и результат remote memory verification.

Self-check log не является structured event. Он находится в `DynamicReport.details`, поэтому backend должен сохранять это поле, если требуется полный audit.

Per-thread pending re-arm

После hit breakpoint временно снимается. Original instruction должна выполниться в том же потоке, который hit breakpoint. После single-step этот конкретный breakpoint ставится обратно.

`PendingRearmList` имеет 64 slots:

```
typedef struct PendingRearmEntry {
DWORD tid;
Breakpoint* bp;
} PendingRearmEntry;
```

`pending_rearm_set` сначала обновляет existing entry для `tid`, затем ищет free slot. Если 64 threads одновременно находятся между breakpoint hit и single-step, insertion fails и session считается failed.

Один thread может иметь только один pending breakpoint. Это соответствует normal sequence: thread stopped on one breakpoint cannot hit another до выполнения original instruction и single-step.

При `EXIT_THREAD_DEBUG_EVENT` pending breakpoint также re-arm, чтобы завершение потока не оставило shared code без breakpoint.

Командная строка target

Windows `CreateProcessA` принимает mutable command-line string, а не готовый `argv[]`. Поэтому collector строит строку по Microsoft C runtime quoting rules.

`append_cmd_arg`:

- добавляет quotes для empty string, spaces, tabs или internal quotes;
- удваивает backslashes перед quote;
- удваивает trailing backslashes перед closing quote.

Это существенно. Простое соединение arguments через spaces изменило бы boundaries. Например path с пробелом стал бы несколькими arguments.

CLI отделяет target arguments после `--`:

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat
```

Collector получает `child_argv` и сохраняет boundaries при построении Windows command line.

Report metadata по умолчанию redacts child arguments, потому что они могут содержать secrets. Это относится к `main.c`; actual CreateProcess command line всё равно включает переданные target arguments.

Создание процесса

Используется:

```
CreateProcessA(
NULL,
cmd.s,
NULL,
NULL,
FALSE,
DEBUG_ONLY_THIS_PROCESS,
NULL,
NULL,
&si,
&pi
)
```

`DEBUG_ONLY_THIS_PROCESS` означает, что дочерние процессы target автоматически не становятся debuggees этого collector.

Code намеренно не использует `CREATE_SUSPENDED`. Комментарий фиксирует обнаруженную практическую проблему: на некоторых configurations suspended debuggee не выдавал ожидаемый create event, что создавало deadlock. Debug API сам предоставляет ранний `CREATE_PROCESS_DEBUG_EVENT` до обычного user-mode execution.

После success:

```
out->ran = true
hProcess = pi.hProcess
pid = pi.dwProcessId
```

Static handler addresses как input dynamics

До debug loop collector строит unique list `handler_rvas` из успешно parsed records с `has_handler`.

List выделяется размером `rf_count`, минимум один element. Unique check линейный:

```
для каждого handler
сравнить со всеми уже сохранёнными
```

Worst-case complexity $O(n^2)$. Для типичных handler counts это приемлемо, но большое PE с unique handler для каждого record может замедлить preparation. Hash set был бы масштабируемее.

На `CREATE_PROCESS_DEBUG_EVENT`:

```
HandlerVA = base_main + HandlerRVA
```

и ставится `BP_UNWIND_HANDLER`.

Если RVA invalid, write may fail; failure считается best effort и не завершает session.

Debug loop и timeout

Start time:

```
uint64_t start = GetTickCount64();
```

На каждой iteration:

```
elapsed = current - start
if elapsed > timeout_ms -> timeout
```

Сравнение строго `>`. Timeout event добавляется в structured events с `arg0 = timeout_ms`.

`WaitForDebugEvent` вызывается с 100 ms. `ERROR_SEM_TIMEOUT` означает отсутствие event в интервале и приводит к следующей iteration. Другой error завершает loop как failure.

Granularity timeout поэтому примерно 100 ms плюс время обработки события.

Continuation status

Для каждого event initial:

```
DWORD cont = DBG_CONTINUE;
```

Для ordinary exception, не принадлежащего breakpoint machinery:

```
cont = DBG_EXCEPTION_NOT_HANDLED;
```

Это передаёт исключение обычному target dispatch.

Для собственных breakpoint и single-step:

```
cont = DBG_CONTINUE;
```

потому что debugger полностью обработал служебное exception.

Неверный status здесь способен либо поглотить target exception, либо передать приложению `INT3`, вставленный analyzer.

CREATE_PROCESS_DEBUG_EVENT

Collector:

1. устанавливает `got_create = true`;

2. сохраняет `base_main = lpBaseOfImage`;

3. refresh modules;

4. пытается поставить API breakpoints;

5. ставит breakpoints на static handler RVA;

6. закрывает `hFile` из event.

Закрытие file handle обязательно по Windows Debug API contract.

`base_main` - actual base. Он используется для статических handler RVA и address-to-section formatting.

LOAD_DLL_DEBUG_EVENT

При каждой DLL load:

1. refresh module list;

2. повторно вызвать `add_api_bps_if_ready`;

3. закрыть `hFile`.

Повторная попытка нужна, потому что real module API мог отсутствовать раньше. Existing breakpoints deduplicate по address.

`UNLOAD_DLL_DEBUG_EVENT` только refresh list. Breakpoints внутри выгруженного module не удаляются из `BpList`. Их remote pages уже могут быть unmapped. На final cleanup `bp_clear` best-effort игнорирует failure. Если module выгружен и затем другой module загружен по тому же address, stale breakpoint model может стать проблемой. Это limitation текущего collector.

Ordinary exception event

Для code, отличного от breakpoint и single-step:

```
exceptions_seen++
first_chance_seen++ или second_chance_seen++
```

В event сохраняются:

- type `exception`;
- thread ID;
- exception address;
- exception code;
- first_chance flag;
- module by address.

После этого `DBG_EXCEPTION_NOT_HANDLED`.

Collector не сериализует `NumberParameters` и `ExceptionInformation[]`. Для access violation поэтому не сохраняются attempted address и operation type. Это направление расширения evidence model.

Initial system breakpoint

После создания debuggee Windows генерирует initial breakpoint. Он не принадлежит `BpList`.

Code допускает один неизвестный breakpoint как initial:

```
if !bp && !initial_break_ignored:
initial_break_ignored = true
DBG_CONTINUE
```

Следующие unknown breakpoints передаются как not handled.

Это pragmatic rule. Target может выполнить собственный early `INT3` до system breakpoint order, а system behavior может отличаться. Более строгая identification могла бы учитывать module/address и event sequence.

Hit собственного `INT3`

По exception address выполняется `bps_find`.

Если breakpoint найден:

1. `bp_clear` восстанавливает original byte;
2. `rearm_pending = true`;
3. `pending_rearm_set(tid, bp)`;
4. открывается event thread;
5. читается context;
6. `ctx.Rip = addr`;
7. `ctx.EFlags |= 0x100`;
8. анализируются kinds;
9. записывается context;
10. event продолжается.

Комментарий «RIP points to INT3+1; rewind» реализуется присваиванием address exception. Поскольку `addr` равен адресу breakpoint, это возвращает execution на original instruction.

Если clear, pending storage, thread open, context read или context write fails, debug loop помечается failed и завершается после продолжения текущего event.

Capture регистрации VEH

На entry `AddVectoredExceptionHandler` Windows x64 ABI:

```
RCX = First
RDX = Handler
```

Collector:

```
veh_registrations++
event type = veh_registration
arg0 = First
arg1 = Handler
```

Если Handler nonzero, ставится breakpoint `BP_CUSTOM_VEH_HANDLER`.

Collector не наблюдает return value API, поэтому не знает, была ли регистрация successful. Он фиксирует вызов и arguments. Чтобы подтвердить success, нужен return breakpoint или анализ последующего hit.

Capture VCH

Аналогично:

```
RCX = First
RDX = Handler
```

Counters и event types отделены:

```
vch_registrations
vch_registration
BP_CUSTOM_VCH_HANDLER
vch_hits
```

Это сохраняет различие VEH и VCH phases.

Capture UEF

На entry `SetUnhandledExceptionFilter`:

```
RCX = Handler
```

Collector увеличивает `uef_set_calls`, создаёт `uef_registration` и ставит breakpoint на nonzero handler.

Как и для VEH, return value предыдущего filter не записывается.

Microsoft указывает, что top-level filter вызывается, когда exception достигает unhandled filter и process не debugged. Наличие active debugger может менять UEF behavior. Поэтому hit UEF под debugger нельзя считать гарантированным для всех scenarios; bundled smoke target и validation должны учитывать actual Windows semantics.

Capture `RtlAddFunctionTable`

Arguments:

```
RCX = FunctionTable
RDX = EntryCount
R8 = BaseAddress
```

Structured event:

```
type = rtl_add_function_table
arg0 = table pointer
arg1 = count
arg2 = base
```

Collector не читает remote entries. Для полноценного dynamic table analysis нужно:

1. validate count upper bound;

2. compute byte size safely;

3. read remote array;

4. validate `RUNTIME_FUNCTION`;

5. resolve unwind records against supplied base;

6. snapshot page protections;

7. account for table lifetime and `RtlDeleteFunctionTable`.

Текущая реализация подтверждает API call, но не content.

Capture callback table

`RtlInstallFunctionTableCallback`:

```
RCX = TableIdentifier
RDX = BaseAddress
R8 = Length
R9 = Callback
```

Event stores all four arguments. Callback table is more dynamic: system may call supplied callback to obtain entries on demand. Full analysis потребовал бы breakpoint on callback и decoding return behavior.

Handler hits

Kinds:

```
BP_UNWIND_HANDLER
BP_CUSTOM_VEH_HANDLER
BP_CUSTOM_VCH_HANDLER
BP_CUSTOM_UEF
```

При hit соответствующий counter увеличивается и создаётся event:

```
unwind_handler_hit
veh_handler_hit
vch_handler_hit
uef_handler_hit
```

Для static unwind handler address formatter показывает absolute VA, RVA и section.

Если `trace_steps > 0` и трасса ещё не собиралась, включается single-step trace:

```
tracing = true
trace_tid = current tid
add_trace(current RIP)
trace_left = min(trace_steps, 256) - 1
```

После rewind `current RIP` равен handler entry. Поэтому первым trace element является address original handler instruction до её исполнения.

Трасса запускается только при первом handler hit. Последующие попадания продолжают учитываться отдельными counters и events, но не накапливают новую RIP trace поверх первой. Благодаря этому `trace_count` никогда не превышает заданный `trace_steps`.

EXCEPTION_SINGLE_STEP

Сначала:

```
pr_bp = pending_rearm_take(tid)
bp_set(pr_bp)
```

Это возвращает `0xCC` после выполнения original instruction.

Затем читается context.

Если active tracing принадлежит тому же thread:

```
add_trace(ctx.Rip)
trace_left--
TF remains set
```

Когда `trace_left == 0`, TF снимается.

Если tracing не active для этого thread, TF снимается сразу.

`trace_count` включает initial handler RIP в общий бюджет. При `trace_steps = 32` сохраняется не более 32 addresses: entry point и до 31 последующего single-step addresses.

Collector поддерживает только один active trace session global to report. Повторный handler hit не перезапускает и не перезаписывает трассу, потому что `start_trace` возвращается при `trace_count != 0`. Counters и events повторных hits продолжают записываться. Отдельные одновременные per-thread traces не поддерживаются.

Thread exit

Если thread завершился с pending re-arm, collector повторно ставит breakpoint. Это важно, потому что code memory принадлежит процессу, а не thread.

Если exiting thread был traced, trace state сбрасывается.

Process exit

На `EXIT_PROCESS_DEBUG_EVENT`:

```
process_exited = true
target_exited = true
target_exit_code = event code
```

Добавляется `process_exit` event с exit code в `arg0`. Loop завершится только после `ContinueDebugEvent`, что соответствует Debug API lifecycle.

Exit code zero не означает отсутствие anomalies. Это только status target process.

Session finalization

После loop:

```
timed_out = local timed_out
target_exited = process_exited
ok = got_create && debug_loop_ok && !timed_out
```

Затем collector пытается снять все breakpoints.

Если target сам не завершился, вызывается `TerminateProcess`. После этого collector до одной секунды принимает и продолжает завершающие debug events, включая `EXIT_PROCESS_DEBUG_EVENT`. Такой drain не добавляет фиксированные пять секунд к каждому timeout-case и корректно завершает debug lifecycle.

Закрываются initial thread и process handles, освобождается handler list и command-line buffer.

Если `ok == false`, функция добавляет одну из summary errors:

- timeout;

- failure до create event;
- failure внутри event loop.

Даже при false `DynamicReport` остаётся доступным. `main.c` передаёт его в JSON/HTML, чтобы partial events и failure state не терялись.

Semantics dynamic signal count

Report layer считает не raw events, а категории:

```
1: any unwind handler hits
1: any VEH registration or hit
1: any VCH registration or hit
1: any UEF set or hit
1: any dynamic function table API call
1: any second-chance exception
```

Maximum category count шесть. Десять VEH hits всё равно дают один dynamic signal category.

Это summary diversity, а не event volume.

Runtime evidence level

Strong runtime component:

```
any handler hit
```

Medium components:

```
dynamic table registration
second chance
```

Weak components:

```
handler registrations
first chance
```

Result:

```
strong, если strong>=1 и medium>=1
medium, если strong>=1 или medium>=2
weak, если medium>=1 или weak>=1
none иначе
```

Опять же это engineering taxonomy, не probability.

Structured event limitations

`DynamicEvent` содержит generic `arg0..arg3`. Их meaning зависит от `type`.

Плюс: model compact и позволяет serializing разные APIs.

Минус: schema не выражает types напрямую. Backend должен знать mapping:

```
veh_registration:
arg0 First
arg1 Handler

rtl_add_function_table:
arg0 Table
arg1 EntryCount
```

```
arg2 BaseAddress
```

Для долгосрочной API лучше использовать typed subobjects или event-specific fields, сохранив backward-compatible base.

Safety and trust boundary

Target является untrusted executable. Dynamic mode действительно запускает его. Он может:

- читать и изменять доступные user files;
- обращаться к сети;
- создавать child processes;
- обнаруживать debugger;
- зависать;
- потреблять ресурсы;
- атаковать parser/debugger through event volume.

`DEBUG_ONLY_THIS_PROCESS` не sandbox. Timeout не sandbox. Поэтому production deployment должен запускать dynamic worker в disposable Windows VM или hardened sandbox с resource limits и restricted network.

Backend нельзя запускать collector от имени privileged service account над arbitrary upload.

Bundled dynamic smoke targets

Project содержит Windows test programs:

```
dyn_veh.c
dyn_vch.c
dyn_uef.c
dyn_rtl_ftable.c
```

Они компилируются в:

```
15_dynamic_veh.exe
16_dynamic_vch.exe
17_dynamic_uef.exe
18_dynamic_rtl_function_table.exe
```

Smoke script запускает analyzer с timeout 8000 ms, trace 32 для handlers и JSON validation.

Targets подтверждают ожидаемые counters:

- VEH registration/hit;
- VCH registration/hit;
- UEF set/hit и exception flow;
- dynamic function table calls.

На macOS эти tests не могут проверить реальный Debug API branch. Нужен Windows CI или физический Windows x64 host.

Фактическая Windows validation 8 июня 2026 года

Финальная MinGW-w64 сборка dynamic collector была запущена на Windows x64. Проверенный executable имеет:

```
SHA-256
489e7571827693cd555c1958f9aec7694d1de717f896d33ecfebe8f13e843624
```

Корпус содержал 18 синтетических и 106 реальных PE-файлов:

```
124/124 JSON reports прошли schema и semantic validation
86 dynamic sessions были фактически запущены
54 target завершились самостоятельно
32 target корректно остановлены hard timeout
30 DLL/PYD получили ran=false, потому что CreateProcess не запускает library
8 EXE потребовали elevation
4/4 focused VEH/VCH/UEF/runtime-table scenarios прошли
0 dynamic events потеряно
maximum trace_count = 32 при trace_steps = 32
```

Эта проверка подтверждает не только synthetic smoke, но и устойчивость collector contract на разнообразных реальных EXE: normal exit, nonzero target exit, GUI/long-running timeout и launch failure. DLL/PYD всё равно не были исполнены: для них требуется отдельный host process.

Артефакты сохранены в `reports/dynamic_validation_2026-06-08/`. Скрипт `tests/validate_dynamic_corpus_evidence.py` повторно проверяет 124 индивидуальных reports, aggregate JSON/CSV, trace budget, dropped events, source hashes и четыре expected-signal scenarios без повторного запуска target.

Эмпирическая граница результата: подтверждена MinGW-w64 сборка. MSVC остаётся отдельным compiler gate и должен проходить в Windows CI на финальной revision.

Что сделано правильно

Сильные стороны implementation:

- exact separation Windows/non-Windows;
- forwarded API resolution через real module offset;
- bounded event and trace storage;
- command-line quoting;
- instruction cache flush;
- temporary protection restoration;
- per-thread pending re-arm;
- preservation ordinary exception dispatch;
- repeated API breakpoint setup after DLL load;
- partial report retention on failure;
- timeout and cleanup;
- self-check breakpoint bytes.

Что остаётся улучшить

Наиболее значимые improvements:

1. Remote parse dynamic function tables.
2. Наблюдать removal API и lifetime tables.
3. Typed event schema вместо generic args.
4. Сохранять exception parameters и context snapshots.
5. Поддержать concurrent per-thread traces.
6. Track module unload and invalidate breakpoints.
7. Dynamic capacity или truncation flags для module, breakpoint и pending lists.
8. Return breakpoints для registration API success.
9. Unicode command-line path через `CreateProcessW`.
10. Isolated worker protocol и VM orchestration.
11. Runtime memory protection map.
12. Hardware-breakpoint backend для anti-tamper-sensitive cases.
13. Завершить независимый MSVC gate и расширять Windows version matrix.

Итоговая state machine

Collector можно представить как finite state machine:

```
UNINITIALIZED
-> REPORT_INITIALIZED
-> COMMAND_BUILT
-> PROCESS_CREATED
-> WAITING_EVENT
-> HANDLING_EVENT
-> CONTINUING_EVENT
-> WAITING_EVENT
-> EXITED | TIMED_OUT | FAILED
-> BREAKPOINT_CLEANUP
-> HANDLE_CLEANUP
-> REPORT_READY
```

Внутри breakpoint branch существует вложенная state machine:

```
ARMED
-> HIT
-> ORIGINAL_RESTORED
-> RIP_REWOUND_AND_TF_SET
-> ORIGINAL_INSTRUCTION_EXECUTED
-> SINGLE_STEP
-> REARMED
```

Понимание этих состояний необходимо, чтобы объяснить dynamic part проекта. Она не «просто запускает EXE». Она реализует специализированный user-mode debugger и строит ограниченный, воспроизводимый набор runtime evidence.

Модуль report.c: доказательная модель и внешние отчёты

Сначала рассматривается движение данных через модуль целиком. Маршрут ниже не является кратким перечнем команд: для каждого этапа объясняются исходное состояние, техническое действие, смысл, полученный результат и передача данных дальше. После маршрута разбираются структуры, функции, проверки и выражения C.

Полный маршрут модуля

Этап 1. Проверить входную модель

Состояние перед этапом. Writers получили указатели на PeImage, arrays, reports и ReportMetadata.

Техническое выполнение. report_model_valid проверяет согласованность pointer/count и bounded capacities; report_metadata_valid проверяет версию schema, timestamp и SHA-256.

Смысл этапа и его место в маршруте. Ошибка сериализации не должна превращать повреждённую внутреннюю модель в внешне корректный отчёт.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Writer либо принимает согласованную модель, либо возвращает false до записи.

Передача дальше. Полученный результат становится исходным состоянием для этапа 2 «Подготовить безопасные строки». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 2. Подготовить безопасные строки

Состояние перед этапом. Пути, имена секций и error messages могут содержать специальные символы.

Техническое выполнение. CSV cells заключаются в кавычки, JSON использует json_escape, HTML использует html_escape.

Смысл этапа и его место в маршруте. Один и тот же символ имеет разные правила в разных форматах; отсутствие escaping может сломать синтаксис или создать HTML injection.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Каждое внешнее представление получает корректно закодированный текст.

Передача дальше. Полученный результат становится исходным состоянием для этапа 3 «Записать CSV». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 3. Записать CSV

Состояние перед этапом. Нужна плоская таблица по функциям.

Техническое выполнение. report_write_csv проходит по rf[i]/us[i], выводит адреса, поля summary, anomalies и parse_error. В ghidra_mode выбирается специальный набор columns и комментарий.

Смысл этапа и его место в маршруте. CSV удобен для электронных таблиц и импорта, но плохо выражает вложенные события.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется обычный CSV или Ghidra CSV.

Передача дальше. Полученный результат становится исходным состоянием для этапа 4 «Собрать JSON». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 4. Собрать JSON

Состояние перед этапом. Нужен machine-readable - машиночитаемый - полный результат.

Техническое выполнение. report_write_json_ex последовательно добавляет metadata, image, sections, functions, findings, mitigations, dynamic events, trace и assessment.

Смысл этапа и его место в маршруте. Backend должен получать типизированные поля, стабильные codes и null там, где значение неизвестно.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется документ JSON schema version 1.2.0.

Передача дальше. Полученный результат становится исходным состоянием для этапа 5 «Связать статические и динамические данные». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 5. Связать статические и динамические данные

Состояние перед этапом. HeuristicReport и DynamicReport могут присутствовать одновременно или по отдельности.

Техническое выполнение. assessment helpers определяют полноту dynamic session, наличие correlation и итоговую strength/confidence, не переписывая исходный static score.

Смысл этапа и его место в маршруте. Один слой не должен выдавать предположение другого слоя за собственный факт.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Отчёт отдельно показывает static evidence, runtime evidence и сводную экспертную оценку.

Передача дальше. Полученный результат становится исходным состоянием для этапа 6 «Собрать HTML». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 6. Собрать HTML

Состояние перед этапом. Нужен отчёт, который человек может открыть без отдельного сервера.

Техническое выполнение. report_write_html формирует CSS, карточки, секционную карту, diagnostics, матрицу 27 признаков, findings с evidence chains, dynamic events, RIP trace и таблицу RUNTIME_FUNCTION.

Смысл этапа и его место в маршруте. Аналитику требуется видеть не только итоговый балл, но и адрес, секцию, поле, байты и причину вывода.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: Появляется self-contained HTML - один автономный файл.

Передача дальше. Полученный результат становится исходным состоянием для этапа 7 «Записать результат атомарно». Такая связь объясняет порядок этапов. Необязательные ветви, например динамический режим или отдельный формат отчёта, выполняются только при соответствующем параметре, но при запуске всё равно получают уже подготовленные данные предыдущих обязательных уровней.

Этап 7. Записать результат атомарно

Состояние перед этапом. Полная строка отчёта собрана в StrBuf.

Техническое выполнение. Writer проверяет sb_ok и вызывает file_write_atomic.

Смысл этапа и его место в маршруте. При нехватке памяти или ошибке диска частичный документ не должен выглядеть завершённым.

Как изменяются данные. До этого существовало состояние, описанное выше. После выполнения появляется следующий проверяемый результат: На итоговом пути находится только полностью сформированный отчёт, а main получает true/false.

Передача дальше. Это конечный результат маршрута данного модуля. main.c либо передаёт его следующему модулю общей системы, либо завершает соответствующую ветвь анализа.

Как читать дальнейший разбор

Сначала удерживайте в голове маршрут целиком: какие данные существовали, какая функция их преобразовала и кто получил результат. Затем переходите к сигнатуре функции и выражениям C. Функция объясняется не изолированно, а как технический исполнитель конкретного этапа маршрута.

Том II. Глава 8. `report.c`: отчёты, итоговая assessment и интеграционный контракт

Отчёт не является форматированным `printf`

После анализа существуют несколько внутренних моделей:

```
PeImage
RuntimeFunction[]
UnwindSummary[]
us_ok[]
HeuristicReport
DynamicReport
ReportMetadata
```

`report.c` преобразует их в четыре внешних представления:

- brief console output;
- CSV;
- JSON;
- self-contained HTML.

Это serialization boundary. Ошибка здесь может:

- потерять evidence;
- создать invalid JSON;
- внедрить untrusted text в HTML;
- нарушить report schema;
- записать частичный файл как будто он полный;
- исказить числовую semantics.

Поэтому reporting layer проверяет model bounds, metadata, encoding и atomic write.

Brief console report

`report_print_brief` выводит:

```
input path
machine
ImageBase
SizeOfImage
SizeOfHeaders
section count
exception directory
RUNTIME_FUNCTION count
```

Это human-readable summary для terminal. Он не содержит все functions и не предназначен как stable machine contract.

Terminal output может меняться без schema migration. Backend не должен parsing console text.

CSV report

Обычный CSV содержит строку на runtime function:

```
index
begin_rva
end_rva
size
unwind_rva
unwind_ver
unwind_flags
prolog_size
fixed_stack
frame_reg
frame_off
handler_rva
has_chain
anomalies
parse_error
```

При parse failure поля, которые нельзя считать надёжными, получают `NA`, а error сохраняется.

CSV escaping реализован согласно базовым правилам:

- cell с comma, quote или newline оборачивается quotes;
- internal quote удваивается.

CSV подходит для spreadsheet и простого exchange, но не выражает nested structures, typed events или null semantics так полно, как JSON.

Ghidra CSV

При `ghidra_mode` columns:

```
begin_rva,end_rva,name,comment
```

Name строится:

```
sub_%08X
```

Comment содержит summary:

```
version
flags
prolog size
stack bytes
frame register
saved GP/XMM
handler
chain
anomalies
```

Этот report создаёт bridge к Ghidra script. Он не меняет source binary и не является отдельной копией core.

Anomaly text

Bit mask `UnwindSummary.anomalies` преобразуется в compact labels:

```
ver
trunc
op
arg
off
chain_loop
chain_depth
handler_oob
order
operand_sanity
alignment
chain_invalid
```

Если mask zero, записывается `"0"`.

Compact text удобен в table, a numeric mask сохраняется в JSON. Consumer может использовать stable numeric bits, но для долгосрочной schema лучше также публиковать array full anomaly codes.

Report metadata

Полный JSON/HTML требует:

```
typedef struct ReportMetadata {
const char* tool_version;
const char* report_schema_version;
const char* generated_at_utc;
const char* input_sha256;
uint64_t input_size;
const char* command_line;
const char* build_compiler;
const char* build_platform;
const char* build_mode;
} ReportMetadata;
```

Эти поля обеспечивают provenance.

Tool version говорит, какая implementation создала report.

Schema version говорит, как consumer должен интерпретировать JSON structure.

Generated timestamp фиксирует время serialization в UTC.

Input SHA-256 и size идентифицируют bytes.

Command line фиксирует analyzer options, но child arguments могут быть redacted.

Compiler, platform и build mode позволяют различить binaries tool.

Metadata validation

`report_metadata_valid` требует:

- nonempty tool version;
- exact supported schema version;
- timestamp format длиной 20: `YYYY-MM-DDTHH:MM:SSZ`;
- SHA-256 length 64;
- только lowercase hex;
- nonnull command line;
- nonempty compiler/platform;
- build mode `debug` или `release`.

Проверка timestamp валидацирует syntax positions и digits, но не календарную корректность. Например, month 99 формально прошёл бы. Timestamp генерируется internally, поэтому risk ограничен, но reusable API могла бы выполнять полную date validation.

Schema version comparison strict. Старый или будущий version metadata не записывается этим writer. Это защищает от случайного mislabeled output.

Model validation

`report_model_valid` проверяет:

- если есть sections, pointer не null;
- finding count не выше capacity;
- mitigation count не выше capacity;
- dynamic event count не выше capacity;
- trace count и requested steps не выше array capacity.

Функция не перепроверяет все semantic relationships. Она является boundary sanity guard.

JSON escaping

Input path, error text, section names, command line и dynamic details являются untrusted strings. Они должны быть escaped.

`json_escape` обрабатывает:

```
\ -> \\
" -> \"
newline -> \n
carriage return -> \r
tab -> \t
control bytes -> \u00xx
```

Для bytes >= 0x80 функция проверяет valid UTF-8 sequence lengths, включая:

- исключение overlong sequences;
- исключение surrogate range;
- ограничение code points до U+10FFFF.

Valid sequence копируется целиком. Invalid byte кодируется как `\u00xx`.

Это гарантирует syntactically valid JSON string, хотя invalid multibyte source преобразуется byte-wise, а не в Unicode replacement character.

JSON numeric representation

Addresses сериализуются strings с `0x`:

```
"begin_rva": "0x00001010"
```

Причины:

- сохраняется hexadecimal representation;
- 64-bit VA не теряется в JavaScript number;
- адрес явно отличается от обычного count.

Counts и ratios сериализуются numbers:

```
"score": 18,
"confidence": 0.43
```

Consumer должен знать type contract. Backend не должен преобразовывать address string в signed 32-bit integer.

Top-level JSON model

Основные sections:

```
input
report_metadata
image_base
size_of_image
size_of_headers
section_alignment
file_alignment
exception_directory
pe_validation
sections
heuristic_analysis если запрошена и успешна
dynamic_validation если dynamic был запрошен
assessment если есть static или dynamic report
functions
```

Optional blocks отражают выполненные stages. Отсутствие `heuristic_analysis` отличается от score zero.

`pe_validation`

Этот block содержит parser-level structural facts:

```
section_raw_oob
section_virtual_oob
section_raw_overlaps
section_virtual_overlaps
section_misaligned
exception_directory_oob
```

Они доступны независимо от heuristic score. Это полезно для backend, который хочет собственную policy.

sections

Для каждой section:

```
name
virtual_address
virtual_size
raw_offset
raw_size
characteristics
readable
writable
executable
```

Permissions как booleans дублируют bit field для удобства consumer. `characteristics` сохраняет original value.

heuristic_analysis

Block включает:

- score, suspicion, confidence;
- manual review flag;
- verdict и evidence level;
- severity counts;
- все raw counters;
- reuse ratios;
- evidence families;
- structured findings;
- weakening rules;
- human-readable details.

Каждый finding содержит:

```
code
family
severity
message
affected_count
representative_rva или null
representative_section
```

Для database нормализованный key - `code`, а message является display text. Business logic не должна зависеть от точной английской строки message.

dynamic_validation

Block сохраняет не только successful session:

```
enabled
ran
ok
timed_out
target_exited
target_exit_code
timeout_ms
trace_steps
counters
dynamic_signal_count
dropped_events
details
events
trace
```

Если `ok=false`, partial events всё равно могут быть значимы. Backend должен хранить session status и не отбрасывать block целиком.

Каждый event:

```
type
module
thread_id
exception_code
first_chance
address
arg0..arg3
```

Meaning args определяется event type. Это нужно документировать в API adapter.

Functions

Для каждой table entry JSON содержит original range, section context, parse status, decoded record и comment.

Внутренний `unwind` object:

```
version
flags
prolog_size
count_codes
fixed_stack_bytes
frame_reg
frame_off
has_handler
handler_rva
handler_section
has_chain
anomalies
anomalies_text
chain
```

Даже при `unwind_ok=false` object присутствует с initialized partial/default values, а `unwind_error` объясняет failure.

Это упрощает schema: array shape постоянна.

Итоговая static/dynamic correlation

`has_static_dynamic_correlation` использует две группы rules.

Первая:

```
suspicious static handler context
&&
any unwind/VEH/VCH/UEF handler hit
```

Suspicious handler context:

- handler non-executable;
- handler writable;
- handler dominance family.

Вторая:

```
any placement anomaly
&&
dynamic function table registration
```

Placement anomaly включает unusual/executable/writable unwind metadata или handler placement.

Correlation boolean означает совпадение агрегированных classes evidence. Она не всегда доказывает exact address identity. Поэтому detailed analyst report должен выполнять более строгий address-level join.

Dynamic signal count

Function считает категории, а не события:

```
unwind handler
VEH
VCH
UEF
dynamic function table
second chance
```

Registration и hit одной категории не дают два points.

Это снижает влияние repeated events и отражает diversity runtime mechanisms.

Runtime evidence level

Handler hit относится к strong component, dynamic table call и second chance - medium, registrations и first chance - weak.

Итог `none`, `weak`, `medium`, `strong` строится rule-based. Он не учитывает benign identity handler.

Case strength

`case_strength` комбинирует static и runtime evidence levels:

```
strong_correlated_case
medium_correlated_case
strong_unconfirmed_case
medium_case
weak_case
no_case
```

Если есть correlation и хотя бы одна сторона strong - strong correlated.

Если нет correlation, но одна сторона strong - strong unconfirmed.

Слово case означает аналитический набор evidence, а не юридически доказанный malicious verdict.

Final confidence

Начальная final confidence:

```
hr->confidence, если static report есть
0.10 иначе
```

Добавляется:

```
+0.10 если есть dynamic signals
+0.10 если есть static/dynamic correlation
```

Maximum `0.98`.

Это не Bayesian update. Runtime signal и correlation могут быть зависимы, а increments не калиброваны. Поле следует трактовать как rule-based report confidence.

Final verdict

Если static score:

```
>=70 high_suspicion_correlated/static
>=40 moderate_suspicion_correlated/static
>=15 low_suspicion_correlated/static
```

Если static score ниже 15, но dynamic signals существуют:

```
runtime_signals_only
```

Иначе:

```
no_strong_correlated_evidence
```

Dynamic events не повышают static score. Они влияют на final suffix, confidence, case strength и отдельный evidence level.

Manual validation plan

`collect_manual_validation_steps` строит context-dependent список.

Примеры:

- проверить `.pdata` ranges в PE viewer;
- открыть suspicious `UNWIND_INFO` в hex view;
- сравнить codes с Ghidra prolog;
- проверить section permissions;
- дизассемблировать handler;
- исследовать JIT explanation;
- capture dynamic table arguments;
- single-step registered handlers;
- выполнить dynamic run при наличии только static findings.

Это сильная часть report design: output не заканчивается score, а сообщает следующий проверочный шаг.

Steps являются English display strings, не stable IDs. Для backend localization и workflow automation лучше добавить code каждому step.

HTML security model

HTML self-contained: CSS и JavaScript встроены, external resources отсутствуют.

Content Security Policy:

```
default-src 'none'
style-src 'unsafe-inline'
script-src 'unsafe-inline'
img-src 'none'
base-uri 'none'
form-action 'none'
```

Такой CSP запрещает network loads, images, base URL и forms. Inline style/script разрешены, потому что report содержит собственный dashboard и filter.

Все untrusted strings проходят `html_escape`:

```
& < > " '
```

Invalid UTF-8 bytes превращаются в numeric entities.

Если path содержал `<script>`, он выводится text, а не исполняется.

HTML sections

Dashboard показывает:

- function count;
- parse failures;
- anomalous entries;
- static score/verdict/confidence;
- dynamic status.

Далее:

- provenance;
- PE image fields;
- structural validation;
- sections;
- final assessment;
- manual plan;
- findings;
- weakening rules;
- dynamic events and log;
- filterable runtime functions.

Client-side filter работает только по уже встроенной table и не отправляет данные наружу.

HTML status nuance

Dynamic card class green только если:

```
dr && dr->ran && dr->ok && !dr->timed_out
```

Text различает:

```
not requested
timed out
not run
completed
failed
```

Это предотвращает превращение «dynamic не запрошен» в «dynamic anomalies не найдены».

Atomic file write

CSV, JSON и HTML сначала полностью строятся в `StrBuf`, затем передаются `file_write_atomic`.

Atomic strategy:

1. записать temporary file;
2. flush/close;
3. rename в final path.

Consumer либо видит старый complete file, либо новый complete file, а не partial serialization.

Atomic rename не равен полной crash durability: без directory fsync внезапное отключение питания может иметь platform-dependent outcome. Для обычного report generation гарантия достаточна, но archival evidence storage требует отдельной durability policy.

Memory failure behavior

`StrBuf` имеет sticky `ok` state. Если append failed, дальнейшие operations не превращают truncated buffer в successful report.

Перед write проверяется:

```
sb_ok(all working buffers)
```

Если false, final file не записывается.

Это важнее частичного convenience output: invalid JSON не должен выдаваться как report.

JSON schema

`docs/report.schema.json` формализует expected types and required fields. Tests валидируют generated reports.

Schema version находится и в metadata, и в application constants. Изменение field contract требует:

1. обновить schema;

2. изменить schema version;

3. добавить migration/compatibility notes;

4. обновить tests;

5. уведомить backend consumer.

Нельзя silently переименовать `rf_unwind_oob`, если коллега уже сохраняет это поле в database.

Как backend должен вызывать core

Текущее ядро предоставляет CLI executable и static library. Для другого разработчика самый безопасный initial integration:

```
backend receives uploaded binary
-> saves immutable job input
-> starts analyzer subprocess with explicit output paths
-> enforces timeout/resource limits
-> checks process exit code
-> parses JSON
-> validates JSON schema
-> stores raw JSON and normalized indexes
```

API между frontend и backend нужен. Между backend и core отдельный network API необязателен: subprocess protocol уже является boundary.

Отдельный core service нужен только если:

- анализ выполняется на другом host;

- нужен Windows-only dynamic worker;
- требуется queue and isolation;
- много concurrent jobs;
- deployment policy запрещает запуск subprocess в backend.

Рекомендуемая database model

Не следует разбивать каждый JSON field в таблицы до потери original report. Надёжная схема:

`analysis_job`:

```
id
status
input_sha256
input_size
original_filename
requested_static
requested_dynamic
created_at
started_at
finished_at
tool_version
schema_version
exit_code
failure_reason
```

`analysis_report`:

```
job_id
raw_json
json_hash
html_object_key
created_at
```

Нормализованные tables для search:

```
finding
runtime_function
dynamic_event
section
assessment
```

Raw JSON остаётся source of truth для backward compatibility и повторной migration.

Сравнение analyses

Сравнивать можно:

- один hash в разных tool versions;
- разные versions одного binary;
- static-only и static+dynamic runs;
- runs с разными target arguments.

Join keys:

```
input_sha256
tool_version
schema_version
function begin_rva
```

```
finding code + representative_rva
dynamic event type + module + offset
```

Absolute runtime VA нельзя напрямую сравнивать между runs из-за ASLR. Его нужно нормализовать к module+offset или RVA.

API adapter не должен менять core

Коллега может написать backend adapter без изменения parser/decoder:

```
HTTP upload
-> job queue
-> subprocess execution
-> JSON schema validation
-> persistence
-> REST response
```

Core следует менять только при необходимости нового analytical field, stable library API или structured streaming. Web concerns, authentication, database transactions и UI не должны входить в C analyzer.

Что сделано правильно

- несколько outputs для разных consumers;
- machine-readable JSON;
- provenance and hash;
- schema validation;
- UTF-8-aware escaping;
- HTML escaping and restrictive CSP;
- atomic writes;
- explicit partial dynamic status;
- structured findings and events;
- manual validation plan;
- static/dynamic assessment без изменения raw evidence.

Что улучшить

1. Добавить explicit policy version heuristics.
2. Typed dynamic event payloads.
3. Stable codes manual steps.
4. Full anomaly code arrays.
5. Score contribution per finding.
6. JSON writer abstraction, уменьшающий риск comma errors.
7. Streaming for extremely large function tables.
8. Optional gzip or JSON Lines for events.

9. Localization layer, не меняющий machine codes.
10. Digital signing of reports для chain of custody.
11. Separate raw observation schema from assessment policy.
12. Compatibility tests against previous schema versions.

Итог

`report.c` является публичным контрактом продукта. Он соединяет C core с analyst, Ghidra, backend, database и frontend. Полноценность проекта определяется не только тем, может ли parser прочитать `.pdata`, но и тем, может ли downstream system точно понять:

```
что анализировалось
какой версией
какие этапы реально выполнены
какие факты обнаружены
какие выводы построены
какова их ограниченность
что проверять вручную
```

В текущем проекте этот контракт уже достаточно развит для subprocess integration. Сетевую API-прослойку можно строить вне ядра, не изменяя аналитические modules.

Модуль app_info.c: происхождение результата

Этот небольшой модуль не анализирует PE, но связывает отчёт с конкретной версией программы, схемой данных, платформой, компилятором, режимом сборки и временем формирования.

Маршрут модуля

1. main.c запрашивает строковые характеристики сборки.

2. app_version и app_report_schema_version возвращают compile-time constants из app_info.h.

3. app_build_compiler, app_build_platform и app_build_mode выбирают строку через preprocessor branches.

4. app_utc_timestamp получает текущее UTC-время, преобразует его в struct tm и формирует ISO 8601 строку.

5. main.c помещает значения в ReportMetadata, а report.c валидирует и сериализует их.

Развёрнутое объяснение маршрута

Исходное состояние. Статический и, возможно, динамический анализ уже выполнены, однако результат пока не содержит сведений о том, какой версией инструмента он создан. Одного имени исследуемого файла для воспроизводимости недостаточно.

Этап 1 - получить постоянные характеристики. main.c вызывает app_version и app_report_schema_version. Функции возвращают строки, встроенные в программу во время компиляции. Первая обозначает версию анализатора, вторая - версию структуры внешнего JSON/HTML-контракта.

Этап 2 - описать конкретную сборку. app_build_compiler, app_build_platform и app_build_mode проверяют compile-time macros - признаки, определённые компилятором. Так устанавливаются компилятор, операционная платформа и режим debug/release именно этого исполняемого файла анализатора.

Этап 3 - зафиксировать время. app_utc_timestamp получает текущее время, переводит его в UTC, помещает компоненты даты и времени в struct tm и формирует строку ISO 8601. Переданный размер out_size не позволяет записать больше байтов, чем доступно в выходном массиве.

Этап 4 - передать сведения дальше. app_info.c возвращает значения main.c. Сам модуль не создаёт ReportMetadata: main.c объединяет эти строки с SHA-256 входа, размером файла и безопасным представлением команды запуска.

Конечный результат маршрута. Заполненная ReportMetadata передаётся report.c. Поэтому читатель отчёта может отличить свойства исследуемого PE от свойств инструмента, которым выполнено исследование.

Граница маршрута. app_info.c не анализирует PE, не меняет findings и не вычисляет score. Если получение timestamp завершается ошибкой, это техническая проблема формирования provenance, а

не аналитическая аномалия исследуемого файла.

Функции версии и схемы

Функция	app_version / app_report_schema_version
Сигнатура	`const char* app_version(void); const char* app_report_schema_version(void);`
Объявлена	include/app_info.h:13-14.
Реализована	src/app_info.c:6-12.
Вызывается	main.c, report.c и вывод --version.
Вход	Не получают параметров.
Результат	Указатель на неизменяемый строковый литерал.
Смысл	Делают версию программы и версию внешней схемы явной частью provenance.

Возвращаемый const char* указывает на строковый литерал. Его не нужно и нельзя освобождать через free. Вызвавший код только читает эту строку.

Функции параметров сборки

Функция	app_build_compiler / app_build_platform / app_build_mode
Сигнатура	`const char* app_build_compiler(void); и аналогичные функции.`
Объявлена	include/app_info.h:15-17.
Реализована	src/app_info.c:14-54.
Вызывается	main.c при формировании ReportMetadata и --version.
Вход	Параметров нет; выбор зависит от macros, заданных компилятором и CMake.
Результат	Нормализованные строки compiler, platform и debug/release.
Смысл	Позволяют отличить результаты разных сборок и объяснить доступность Windows dynamic mode.

Preprocessor выполняется до компиляции. Ветви #if не являются runtime if: в бинарник попадает только ветвь, соответствующая текущей платформе и компилятору.

Функция app_utc_timestamp

Функция	app_utc_timestamp
Сигнатура	`bool app_utc_timestamp(char* out, size_t out_size);`
Объявлена	include/app_info.h:20.
Реализована	src/app_info.c:56-68.
Вызывается	main.c перед записью JSON и HTML.
Вход	out - предоставленный массив символов; out_size - его фактическая ёмкость.
Результат	true и строка YYYY-MM-DDTHH:MM:SSZ либо false при ошибке времени/форматирования.
Смысл	Фиксирует время генерации отчёта в UTC независимо от локального часового пояса.

Сначала проверяются out и минимальная ёмкость. Затем time(NULL) получает календарное время. На Windows используется gmtime_s с порядком параметров Microsoft, на POSIX - gmtime_r. strftime записывает строку фиксированного формата. Различие API изолировано внутри модуля и не распространяется на report.c.

Заголовочные файлы: публичные контракты модулей

Файл .h не является второстепенной копией .c. Он описывает типы и функции, которые разрешено использовать другим единицам трансляции. Реализация остаётся в .c, а заголовок задаёт договор между модулями.

Как компилятор связывает .h и .c

Директива #include текстуально подставляет объявления заголовка до компиляции. Компилятор проверяет вызов по сигнатуре, а linker позднее связывает его с единственным внешним определением из object file. static-функции в заголовке не объявлены, потому что являются внутренними деталями одного .c.

include/util.h

Публикует Buf, StrBuf, безопасные функции чтения, little-endian inline helpers, атомарную запись и SHA-256. Это базовый контракт хранения байтов и динамических строк.

include/pe.h

Публикует PeSection, PeImage, константы PE32+ AMD64 и функции построения/освобождения карты, поиска секции и перевода RVA в file offset.

include/unwind.h

Публикует RuntimeFunction, UnwindSummary, UnwindParseOpts, битовые коды аномалий и интерфейсы чтения Exception Directory и декодирования UNWIND_INFO. Поля UnwindSummary являются договором между decoder, heuristics и reports.

include/heuristics.h

Определяет FindingLocation, HeuristicFinding, HeuristicMitigation и HeuristicReport. Здесь зафиксированы capacity массивов, evidence family bits и публичный вход heuristics_analyze.

include/dynamic.h

Определяет bounded DynamicEvent и DynamicReport. Заголовок одинаков для Windows и non-Windows сборки; различается реализация dynamic_collect_windows в dynamic_win.c.

include/report.h

Определяет ReportMetadata и writers. Report writer получает const pointers: он сериализует модель, но не владеет ею и не должен изменять аналитический результат.

include/app_info.h

Фиксирует tool version, report schema version и интерфейс получения provenance. Изменение schema version является внешним контрактным изменением, а не декоративной правкой строки.

#pragma once и extern C

#pragma once предотвращает повторное включение одного заголовка в единицу трансляции. Блок extern "C" отключает C++ name mangling, если заголовок подключается из C++ кода, сохраняя имена функций совместимыми с C linker contract.

Полный навигатор функций

Этот раздел гарантирует полноту обзора исходного кода. Сначала функцию следует найти здесь, затем вернуться к пошаговому маршруту её модуля, чтобы увидеть место функции в общей последовательности.

Функции src/main.c

В текущей реализации обнаружено определений: 11. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

dup_cstr

```
static char* dup_cstr(const char* s);
```

Где реализована: src/main.c:47. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:60, src/main.c:65, src/main.c:70, src/main.c:80, src/main.c:85

argv_text_to_utf8

```
static char* argv_text_to_utf8(const char* s);
```

Где реализована: src/main.c:56. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:145, src/main.c:458

print_version

```
static void print_version(void);
```

Где реализована: src/main.c:89. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:276

usage

```
static void usage(const char* argv0);
```

Где реализована: src/main.c:95. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:279

append_invocation_arg

```
static bool append_invocation_arg(StrBuf* out, const char* arg);
```

Где реализована: src/main.c:118. Видимость: static - внутренняя.

Зачем существует: Добавляет форматированный фрагмент в строковый буфер, соблюдая правила конкретного внешнего формата.

Где участвует: src/main.c:147

build_invocation

```
static bool build_invocation(int argc, char** argv, StrBuf* out);
```

Где реализована: src/main.c:132. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:461

parse_long_range

```
static bool parse_long_range(const char* text, long min_value, long max_value, long* out);
```

Где реализована: src/main.c:154. Видимость: static - внутренняя.

Зачем существует: Проверяет вход и переводит текстовое или бинарное представление в структурированное значение.

Где участвует: src/main.c:206, src/main.c:214

parse_u32_range

```
static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value,
uint32_t* out);
```

Где реализована: src/main.c:165. Видимость: static - внутренняя.

Зачем существует: Проверяет вход и переводит текстовое или бинарное представление в структурированное значение.

Где участвует: src/main.c:221, src/main.c:228

parse_args

```
static ParseResult parse_args(int argc, char** argv, Args* a);
```

Где реализована: src/main.c:176. Видимость: static - внутренняя.

Зачем существует: Переводит строки командной строки в проверенную структуру Args.

Где участвует: src/main.c:273

print_one

```
static void print_one(size_t idx, const RuntimeFunction* r, const UnwindSummary* u, bool
ok);
```

Где реализована: src/main.c:245. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/main.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:368

main

```
int main(int argc, char** argv);
```

Где реализована: src/main.c:270. Видимость: external - межмодульная.

Зачем существует: Управляет полным запуском: вызывает модули в правильном порядке, обрабатывает ошибки, пишет отчёты и освобождает ресурсы.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

Функции src/util.c

В текущей реализации обнаружено определений: 21. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

buf_init_from_file

```
bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
```

Где реализована: src/util.c:13. Видимость: external - межмодульная.

Зачем существует: Загружает весь входной файл в память и создаёт Buf с явной длиной.

Где участвует: src/main.c:285

buf_free_owned

```
void buf_free_owned(uint8_t* owned_mem);
```

Где реализована: src/util.c:38. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/util.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:297, src/main.c:314, src/main.c:338, src/main.c:525

buf_read

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len);
```

Где реализована: src/util.c:42. Видимость: external - межмодульная.

Зачем существует: Проверяет границы и копирует заданное число байтов из Buf.

Где участвует: src/util.c:51, src/util.c:55, src/util.c:61, src/util.c:67, src/pe.c:159, src/unwind.c:67

buf_read_u8

```
bool buf_read_u8(const Buf* b, size_t off, uint8_t* out);
```

Где реализована: src/util.c:50. Видимость: external - межмодульная.

Зачем существует: Читает число фиксированной ширины из Buf с проверкой границ и little-endian decoding.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

buf_read_u16

```
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
```

Где реализована: src/util.c:53. Видимость: external - межмодульная.

Зачем существует: Читает число фиксированной ширины из Buf с проверкой границ и little-endian decoding.

Где участвует: src/pe.c:13, src/pe.c:60, src/pe.c:61, src/pe.c:62, src/pe.c:83

buf_read_u32

```
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
```

Где реализована: src/util.c:59. Видимость: external - межмодульная.

Зачем существует: Читает число фиксированной ширины из Buf с проверкой границ и little-endian decoding.

Где участвует: src/pe.c:16, src/pe.c:51, src/pe.c:101, src/pe.c:102, src/pe.c:103, src/pe.c:104

buf_read_u64

```
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);
```

Где реализована: src/util.c:65. Видимость: external - межмодульная.

Зачем существует: Читает число фиксированной ширины из Buf с проверкой границ и little-endian decoding.

Где участвует: src/pe.c:100

sb_reserve

```
static bool sb_reserve(StrBuf* sb, size_t need_cap);
```

Где реализована: src/util.c:72. Видимость: static - внутренняя.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/util.c:101, src/util.c:128, src/util.c:172

sb_init

```
bool sb_init(StrBuf* sb, size_t initial_cap);
```

Где реализована: src/util.c:94. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/main.c:291, src/main.c:304, src/main.c:349, src/main.c:353, src/main.c:372, src/main.c:373

sb_free

```
void sb_free(StrBuf* sb);
```

Где реализована: src/util.c:109. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/main.c:296, src/main.c:312, src/main.c:313, src/main.c:336, src/main.c:337, src/main.c:360

sb_appendn

```
bool sb_appendn(StrBuf* sb, const char* s, size_t n);
```

Где реализована: src/util.c:118. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/main.c:125, src/main.c:127, src/main.c:137, src/main.c:144, src/util.c:141, src/dynamic_win.c:465

sb_append

```
bool sb_append(StrBuf* sb, const char* s);
```

Где реализована: src/util.c:135. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/main.c:121, src/main.c:122, src/main.c:129, src/pe.c:42, src/pe.c:46, src/pe.c:52

sb_appendf

```
bool sb_appendf(StrBuf* sb, const char* fmt, ...);
```

Где реализована: src/util.c:144. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/main.c:138, src/pe.c:68, src/pe.c:88, src/unwind.c:60, src/unwind.c:247, src/unwind.c:411

sb_ok

```
bool sb_ok(const StrBuf* sb);
```

Где реализована: src/util.c:186. Видимость: external - межмодульная.

Зачем существует: Управляет StrBuf - расширяемой строкой с контролем ёмкости и ошибки памяти.

Где участвует: src/heuristics.c:1033, src/dynamic_win.c:673, src/dynamic_win.c:1147, src/dynamic_win.c:1243, src/report.c:579, src/report.c:680

file_write_atomic

```
bool file_write_atomic(const char* path, const char* data, size_t len);
```

Где реализована: src/util.c:190. Видимость: external - межмодульная.

Зачем существует: Записывает файл через временный путь и заменяет итог только после полного успеха.

Где участвует: src/report.c:681, src/report.c:1656, src/report.c:3435

sha256_rotr

```
static uint32_t sha256_rotr(uint32_t value, unsigned count);
```

Где реализована: src/util.c:240. Видимость: static - внутренняя.

Зачем существует: Выполняет внутренний этап алгоритма SHA-256.

Где участвует: src/util.c:264, src/util.c:265, src/util.c:272, src/util.c:275

sha256_transform

```
static void sha256_transform(Sha256Ctx* ctx, const uint8_t block[64]);
```

Где реализована: src/util.c:244. Видимость: static - внутренняя.

Зачем существует: Выполняет внутренний этап алгоритма SHA-256.

Где участвует: src/util.c:306, src/util.c:316, src/util.c:323

sha256_init

```
static void sha256_init(Sha256Ctx* ctx);
```

Где реализована: src/util.c:285. Видимость: static - внутренняя.

Зачем существует: Выполняет внутренний этап алгоритма SHA-256.

Где участвует: src/util.c:339

sha256_update

```
static void sha256_update(Sha256Ctx* ctx, const uint8_t* data, size_t size);
```

Где реализована: src/util.c:295. Видимость: static - внутренняя.

Зачем существует: Выполняет внутренний этап алгоритма SHA-256.

Где участвует: src/util.c:340

sha256_final

```
static void sha256_final(Sha256Ctx* ctx, uint8_t digest[32]);
```

Где реализована: src/util.c:312. Видимость: static - внутренняя.

Зачем существует: Выполняет внутренний этап алгоритма SHA-256.

Где участвует: src/util.c:341

sha256_hex

```
bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);
```

Где реализована: src/util.c:333. Видимость: external - межмодульная.

Зачем существует: Вычисляет шестнадцатеричный SHA-256 входных байтов.

Где участвует: src/main.c:463

Функции src/pe.c

В текущей реализации обнаружено определений: 9. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

u32_max

```
static uint32_t u32_max(uint32_t a, uint32_t b);
```

Где реализована: src/pe.c:5. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/pe.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/pe.c:201, src/pe.c:222, src/pe.c:224, src/pe.c:269

is_power_of_two_u32

```
static bool is_power_of_two_u32(uint32_t value);
```

Где реализована: src/pe.c:7. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/pe.c:183

read_dos_lfanew

```
static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew);
```

Где реализована: src/pe.c:11. Видимость: static - внутренняя.

Зачем существует: Безопасно читает определённое поле или элемент из уже ограниченного диапазона.

Где участвует: src/pe.c:41

safe_add_u32

```
static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out);
```

Где реализована: src/pe.c:21. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/pe.c:287

ranges_overlap_u64

```
static bool ranges_overlap_u64(uint64_t a_begin, uint64_t a_size, uint64_t b_begin, uint64_t b_size);
```

Где реализована: src/pe.c:28. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/pe.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/pe.c:217, src/pe.c:220

pe_parse

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
```

Где реализована: src/pe.c:36. Видимость: external - межмодульная.

Зачем существует: Строит PeImage - проверенную карту PE32+ AMD64.

Где участвует: src/main.c:294

pe_free

```
void pe_free(PeImage* img);
```

Где реализована: src/pe.c:259. Видимость: external - межмодульная.

Зачем существует: Выполняет операцию над картой PE: построение, поиск, преобразование адреса или освобождение.

Где участвует: src/main.c:311, src/main.c:335, src/main.c:522

pe_find_section_for_rva

```
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);
```

Где реализована: src/pe.c:265. Видимость: external - межмодульная.

Зачем существует: Выполняет операцию над картой PE: построение, поиск, преобразование адреса или освобождение.

Где участвует: src/pe.c:279, src/unwind.c:98, src/heuristics.c:693, src/heuristics.c:715, src/heuristics.c:927, src/dynamic_win.c:650

pe_rva_to_fileoff

```
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);
```

Где реализована: src/pe.c:277. Видимость: external - межмодульная.

Зачем существует: Переводит RVA виртуального образа в позицию байта внутри файла.

Где участвует: src/unwind.c:59, src/unwind.c:283, src/unwind.c:786, src/report.c:87, src/report.c:492, src/report.c:861

Функции src/unwind.c

В текущей реализации обнаружено определений: 28. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

unwind_reg_name

```
const char* unwind_reg_name(uint8_t reg);
```

Где реализована: src/unwind.c:5. Видимость: external - межмодульная.

Зачем существует: Выполняет операцию декодирования или представления x64 unwind metadata.

Где участвует: src/main.c:263, src/unwind.c:411, src/unwind.c:414, src/unwind.c:415, src/unwind.c:416, src/report.c:17

unwind_flags_to_string

```
void unwind_flags_to_string(uint8_t flags, char out[64]);
```

Где реализована: src/unwind.c:15. Видимость: external - межмодульная.

Зачем существует: Выполняет операцию декодирования или представления x64 unwind metadata.

Где участвует: src/main.c:248, src/report.c:553, src/report.c:652, src/report.c:3391

slots_for_code

```
static bool slots_for_code(uint8_t version, uint8_t op, uint8_t opinfo, bool v2_seen_epilog,
uint8_t* out_slots);
```

Где реализована: src/unwind.c:24. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:374

parse_unwindinfo_header

```
static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err);
```

Где реализована: src/unwind.c:56. Видимость: static - внутренняя.

Зачем существует: Проверяет вход и переводит текстовое или бинарное представление в структурированное значение.

Где участвует: src/unwind.c:287

read_u16_slot

```
static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out);
```

Где реализована: src/unwind.c:75. Видимость: static - внутренняя.

Зачем существует: Безопасно читает определённое поле или элемент из уже ограниченного диапазона.

Где участвует: src/unwind.c:361, src/unwind.c:422, src/unwind.c:471, src/unwind.c:483, src/unwind.c:484, src/unwind.c:513

clamp_size_to_u32

```
static uint32_t clamp_size_to_u32(size_t value);
```

Где реализована: src/unwind.c:83. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/unwind.c:124, src/unwind.c:147, src/unwind.c:150, src/unwind.c:151

min_size

```
static size_t min_size(size_t a, size_t b);
```

Где реализована: src/unwind.c:87. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:102, src/unwind.c:109

safe_add_size

```
static size_t safe_add_size(size_t a, size_t b);
```

Где реализована: src/unwind.c:91. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/unwind.c:101

unwind_record_limit

```
static size_t unwind_record_limit(const Buf* b, const PeImage* img, uint32_t unwind_rva);
```

Где реализована: src/unwind.c:96. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию декодирования или представления x64 unwind metadata.

Где участвует: src/unwind.c:296

available_until_limit

```
static size_t available_until_limit(const Buf* b, size_t off, size_t limit);
```

Где реализована: src/unwind.c:107. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:170, src/unwind.c:292, src/unwind.c:337

unwind_diag_capture_bytes

```
static void unwind_diag_capture_bytes(UnwindSummary* out, const Buf* b, size_t file_offset,
size_t max_count);
```

Где реализована: src/unwind.c:114. Видимость: static - внутренняя.

Зачем существует: Сохраняет точную диагностику ошибки UNWIND_INFO: поле, offset, размеры, slots и байты.

Где участвует: src/unwind.c:160

unwind_diag_set

```
static void unwind_diag_set(UnwindSummary* out, const Buf* b, const char* field, bool
has_file_offset, size_t file_offset, bool has_size, size_t expected_size, size_t
available_size, bool has_slots, uint32_t expected_slots, uint32_t available_slots);
```

Где реализована: src/unwind.c:128. Видимость: static - внутренняя.

Зачем существует: Сохраняет точную диагностику ошибки UNWIND_INFO: поле, offset, размеры, slots и байты.

Где участвует: src/unwind.c:171, src/unwind.c:289, src/unwind.c:314, src/unwind.c:340, src/unwind.c:381, src/unwind.c:687

unwind_diag_set_read_failure

```
static void unwind_diag_set_read_failure(UnwindSummary* out, const Buf* b, const char*
field, size_t file_offset, size_t expected_size, size_t limit);
```

Где реализована: src/unwind.c:164. Видимость: static - внутренняя.

Зачем существует: Сохраняет точную диагностику ошибки UNWIND_INFO: поле, offset, размеры, slots и байты.

Где участвует: src/unwind.c:363, src/unwind.c:473, src/unwind.c:486, src/unwind.c:515, src/unwind.c:535, src/unwind.c:557

read_u32_from_slots

```
static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi);
```

Где реализована: src/unwind.c:175. Видимость: static - внутренняя.

Зачем существует: Безопасно читает определённое поле или элемент из уже ограниченного диапазона.

Где участвует: src/unwind.c:491, src/unwind.c:540, src/unwind.c:584, src/unwind.c:626

is_unwindable_gpr

```
static bool is_unwindable_gpr(uint8_t reg);
```

Где реализована: src/unwind.c:179. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/unwind.c:326, src/unwind.c:462, src/unwind.c:520, src/unwind.c:542

is_unwindable_xmm

```
static bool is_unwindable_xmm(uint8_t reg);
```

Где реализована: src/unwind.c:184. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/unwind.c:562, src/unwind.c:586, src/unwind.c:606, src/unwind.c:628

is_abi_nonvolatile_gpr

```
static bool is_abi_nonvolatile_gpr(uint8_t reg);
```

Где реализована: src/unwind.c:188. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/unwind.c:329, src/unwind.c:463, src/unwind.c:521, src/unwind.c:543

is_abi_nonvolatile_xmm

```
static bool is_abi_nonvolatile_xmm(uint8_t reg);
```

Где реализована: src/unwind.c:192. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/unwind.c:563, src/unwind.c:587, src/unwind.c:607, src/unwind.c:629

add_fixed_stack

```
static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies);
```

Где реализована: src/unwind.c:196. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:460, src/unwind.c:478, src/unwind.c:492, src/unwind.c:501, src/unwind.c:640, src/unwind.c:738

alloc_large_mode0_suspicious

```
static bool alloc_large_mode0_suspicious(uint16_t scaled8);
```

Где реализована: src/unwind.c:206. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:479

alloc_large_mode1_suspicious

```
static bool alloc_large_mode1_suspicious(uint32_t size);
```

Где реализована: src/unwind.c:211. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:493

save_nonvol_far_suspicious

```
static bool save_nonvol_far_suspicious(uint32_t off);
```

Где реализована: src/unwind.c:215. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:541

save_xmm_far_suspicious

```
static bool save_xmm_far_suspicious(uint32_t off);
```

Где реализована: src/unwind.c:219. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/unwind.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/unwind.c:585, src/unwind.c:627

parse_chain_rf

```
static bool parse_chain_rf(const Buf* b, const PeImage* img, size_t fileoff_after_codes,
RuntimeFunction* out_chain);
```

Где реализована: src/unwind.c:223. Видимость: static - внутренняя.

Зачем существует: Проверяет вход и переводит текстовое или бинарное представление в структурированное значение.

Где участвует: src/unwind.c:660

unwind_parse_fail

```
static bool unwind_parse_fail(UnwindSummary* out, StrBuf* err, size_t err_start, const char*
fallback);
```

Где реализована: src/unwind.c:236. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию декодирования или представления x64 unwind metadata.

Где участвует: src/unwind.c:294, src/unwind.c:315, src/unwind.c:344, src/unwind.c:365, src/unwind.c:385, src/unwind.c:476

unwind_parse

```
bool unwind_parse(const Buf* b, const PeImage* img, uint32_t unwind_rva, const
UnwindParseOpts* opts, UnwindSummary* out, StrBuf* ops_text, StrBuf* err);
```

Где реализована: src/unwind.c:258. Видимость: external - межмодульная.

Зачем существует: Декодирует одну UNWIND_INFO и сохраняет summary либо точную диагностику ошибки.

Где участвует: src/main.c:355, src/main.c:374, src/unwind.c:731

unwind_read_runtime_functions_ex

```
RuntimeFunctionReadResult unwind_read_runtime_functions_ex( const Buf* b, const PeImage*
img, RuntimeFunction** out_rf, size_t* out_count, StrBuf* err);
```

Где реализована: src/unwind.c:766. Видимость: external - межмодульная.

Зачем существует: Расширенный вариант чтения, различающий успех, повреждение структуры и нехватку памяти.

Где участвует: src/main.c:306, src/unwind.c:834

unwind_read_runtime_functions

```
bool unwind_read_runtime_functions(const Buf* b, const PeImage* img, RuntimeFunction**
out_rf, size_t* out_count, StrBuf* err);
```

Где реализована: src/unwind.c:829. Видимость: external - межмодульная.

Зачем существует: Читает массив RUNTIME_FUNCTION из Exception Directory.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

Функции src/heuristics.c

В текущей реализации обнаружено определений: 35. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

sec_name_eq

```
static bool sec_name_eq(const char* a, const char* b);
```

Где реализована: src/heuristics.c:86. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:120, src/heuristics.c:121, src/heuristics.c:122, src/heuristics.c:130

sec_name_contains_ci

```
static bool sec_name_contains_ci(const char* haystack, const char* needle);
```

Где реализована: src/heuristics.c:99. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:126, src/heuristics.c:131

sec_name_allowed_for_unwind

```
static bool sec_name_allowed_for_unwind(const char* name8);
```

Где реализована: src/heuristics.c:118. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:698

sec_name_plausibly_unwind_named

```
static bool sec_name_plausibly_unwind_named(const char* name8);
```

Где реализована: src/heuristics.c:125. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:699

sec_name_code_like

```
static bool sec_name_code_like(const char* name8);
```

Где реализована: src/heuristics.c:129. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:930

image_has_executable_section

```
static bool image_has_executable_section(const PeImage* img);
```

Где реализована: src/heuristics.c:134. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:836

map

```
/* Very small u32->count hash map (open addressing, linear probing). */ typedef struct HEntry;
```

Где реализована: src/heuristics.c:146. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

h32

```
static uint32_t h32(uint32_t x);
```

Где реализована: src/heuristics.c:159. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:192, src/heuristics.c:231

hmap_init

```
static bool hmap_init(HMap* m, size_t cap);
```

Где реализована: src/heuristics.c:169. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:637, src/heuristics.c:638

hmap_free

```
static void hmap_free(HMap* m);
```

Где реализована: src/heuristics.c:177. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:640, src/heuristics.c:641, src/heuristics.c:673, src/heuristics.c:674, src/heuristics.c:683, src/heuristics.c:684

hmap_inc

```
static bool hmap_inc(HMap* m, uint32_t key, uint32_t* out_newval);
```

Где реализована: src/heuristics.c:187. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:671, src/heuristics.c:681

hmap_grow

```
static bool hmap_grow(HMap* m);
```

Где реализована: src/heuristics.c:214. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:185, src/heuristics.c:190

hmap_max_val

```
static uint32_t hmap_max_val(const HMap* m);
```

Где реализована: src/heuristics.c:249. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:775, src/heuristics.c:782

hmap_max_entry

```
static bool hmap_max_entry(const HMap* m, uint32_t* out_key, uint32_t* out_val);
```

Где реализована: src/heuristics.c:258. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:777, src/heuristics.c:784

hmap_unique

```
static size_t hmap_unique(const HMap* m);
```

Где реализована: src/heuristics.c:275. Видимость: static - внутренняя.

Зачем существует: Выполняет операцию над внутренней хеш-таблицей частот, используемой для reuse и dominance.

Где участвует: src/heuristics.c:778

remember_location

```
static void remember_location(FindingLocation* location, uint32_t rva);
```

Где реализована: src/heuristics.c:287. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:336, src/heuristics.c:653, src/heuristics.c:657, src/heuristics.c:661, src/heuristics.c:666, src/heuristics.c:696

first_pe_section_issue

```
static FindingLocation first_pe_section_issue(const PeImage* img, enum PeSectionIssue
issue);
```

Где реализована: src/heuristics.c:301. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:794, src/heuristics.c:801, src/heuristics.c:808, src/heuristics.c:815, src/heuristics.c:823

cmp_rfidx

```
static int cmp_rfidx(const void* a, const void* b);
```

Где реализована: src/heuristics.c:343. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

clamp_int

```
static int clamp_int(int v, int lo, int hi);
```

Где реализована: src/heuristics.c:353. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/heuristics.c:361

add_score

```
static void add_score(int* score, int add);
```

Где реализована: src/heuristics.c:359. Видимость: static - внутренняя.

Зачем существует: Безопасно прибавляет инженерный вес к эвристическому score.

Где участвует: src/heuristics.c:790, src/heuristics.c:797, src/heuristics.c:804, src/heuristics.c:811, src/heuristics.c:818, src/heuristics.c:827

safe_ratio

```
static double safe_ratio(double a, double b);
```

Где реализована: src/heuristics.c:364. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/heuristics.c:517, src/heuristics.c:779, src/heuristics.c:780, src/heuristics.c:785, src/heuristics.c:844, src/heuristics.c:845

heuristics_evidence_family_name

```
const char* heuristics_evidence_family_name(uint32_t bit);
```

Где реализована: src/heuristics.c:367. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:990, src/report.c:1835

heuristics_evidence_family_count

```
size_t heuristics_evidence_family_count(uint32_t mask);
```

Где реализована: src/heuristics.c:378. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:519, src/report.c:1222, src/report.c:1408

family_bit_from_name

```
static uint32_t family_bit_from_name(const char* family);
```

Где реализована: src/heuristics.c:387. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:422

safe_copy

```
static void safe_copy(char* dst, size_t dst_cap, const char* src);
```

Где реализована: src/heuristics.c:397. Видимость: static - внутренняя.

Зачем существует: Ограничивает арифметическую операцию или значение так, чтобы переполнение не изменило смысл проверки.

Где участвует: src/heuristics.c:428, src/heuristics.c:429, src/heuristics.c:430, src/heuristics.c:431, src/heuristics.c:492, src/heuristics.c:493

add_finding

```
static HeuristicFinding* add_finding(HeuristicReport* out, const char* code, const char* family, const char* severity, const char* fmt, ...);
```

Где реализована: src/heuristics.c:408. Видимость: static - внутренняя.

Зачем существует: Создаёт новый структурированный finding в HeuristicReport.

Где участвует: src/heuristics.c:792, src/heuristics.c:799, src/heuristics.c:806, src/heuristics.c:813, src/heuristics.c:820, src/heuristics.c:829

annotate_finding

```
static void annotate_finding(HeuristicFinding* finding, FindingLocation location, size_t affected_count);
```

Где реализована: src/heuristics.c:442. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/heuristics.c:791, src/heuristics.c:798, src/heuristics.c:805, src/heuristics.c:812, src/heuristics.c:819, src/heuristics.c:828

append_note

```
static void append_note(HeuristicReport* out, const char* fmt, ...);
```

Где реализована: src/heuristics.c:451. Видимость: static - внутренняя.

Зачем существует: Добавляет форматированный фрагмент в строковый буфер, соблюдая правила конкретного внешнего формата.

Где участвует: src/heuristics.c:841, src/heuristics.c:1026

count_findings_with_severity

```
static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev);
```

Где реализована: src/heuristics.c:463. Видимость: static - внутренняя.

Зачем существует: Подсчитывает элементы, удовлетворяющие заданному условию, для последующей агрегации.

Где участвует: src/heuristics.c:520, src/heuristics.c:521

has_finding_code

```
static bool has_finding_code(const HeuristicReport* out, const char* code);
```

Где реализована: src/heuristics.c:472. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/heuristics.c:947, src/heuristics.c:948, src/heuristics.c:1014

add_mitigation

```
static void add_mitigation(HeuristicReport* out, const char* code, const char* fmt, ...);
```

Где реализована: src/heuristics.c:480. Видимость: static - внутренняя.

Зачем существует: Добавляет фактор, снижающий уверенность аналитического вывода.

Где участвует: src/heuristics.c:542, src/heuristics.c:547, src/heuristics.c:561, src/heuristics.c:566

evidence_level_from_counts

```
static const char* evidence_level_from_counts(size_t weak_count, size_t medium_count, size_t
strong_count, int score);
```

Где реализована: src/heuristics.c:504. Видимость: static - внутренняя.

Зачем существует: Строит или проверяет evidence - конкретное доказательное наблюдение, связанное с finding.

Где участвует: src/heuristics.c:529

finalize_verdict

```
static void finalize_verdict(HeuristicReport* out);
```

Где реализована: src/heuristics.c:514. Видимость: static - внутренняя.

Зачем существует: Завершает агрегацию и вычисляет итоговые поля отчёта.

Где участвует: src/heuristics.c:1029

heuristics_analyze

```
bool heuristics_analyze(const PeImage* img, const RuntimeFunction* rf, const UnwindSummary*
us, const bool* us_ok, size_t n, HeuristicReport* out, StrBuf* err);
```

Где реализована: src/heuristics.c:590. Видимость: external - межмодульная.

Зачем существует: Агрегирует структурные наблюдения в findings, score, confidence и verdict.

Где участвует: src/main.c:391

heuristics_free

```
void heuristics_free(HeuristicReport* r);
```

Где реализована: src/heuristics.c:1040. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/heuristics.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/main.c:515

Функции src/dynamic_win.c

В текущей реализации обнаружено определений: 40. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

mods_init

```
static void mods_init(ModList* ml);
```

Где реализована: src/dynamic_win.c:31. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:721

basename_lower

```
static const char* basename_lower(const char* path);
```

Где реализована: src/dynamic_win.c:33. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:100, src/dynamic_win.c:551

strlower_inplace

```
static void strlower_inplace(char* s);
```

Где реализована: src/dynamic_win.c:41. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:56, src/dynamic_win.c:66, src/dynamic_win.c:549, src/dynamic_win.c:554

mods_add

```
static void mods_add(ModList* ml, const char* name, uint64_t base, uint64_t size);
```

Где реализована: src/dynamic_win.c:47. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:101

mods_find

```
static const ModInfo* mods_find(const ModList* ml, const char* name);
```

Где реализована: src/dynamic_win.c:61. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:580

mods_find_by_addr

```
static const ModInfo* mods_find_by_addr(const ModList* ml, uint64_t addr);
```

Где реализована: src/dynamic_win.c:73. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:267, src/dynamic_win.c:437

mods_refresh_toolhelp

```
static bool mods_refresh_toolhelp(ModList* ml, DWORD pid);
```

Где реализована: src/dynamic_win.c:85. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:109

mods_refresh_accounted

```
static bool mods_refresh_accounted(ModList* ml, DWORD pid, DynamicReport* dr);
```

Где реализована: src/dynamic_win.c:108. Видимость: static - внутренняя.

Зачем существует: Управляет картой модулей, загруженных в процесс target.

Где участвует: src/dynamic_win.c:796, src/dynamic_win.c:816, src/dynamic_win.c:821, src/dynamic_win.c:914, src/dynamic_win.c:930, src/dynamic_win.c:945

bps_init

```
static void bps_init(BpList* bl);
```

Где реализована: src/dynamic_win.c:155. Видимость: static - внутренняя.

Зачем существует: Управляет списком breakpoints в пределах одной динамической сессии.

Где участвует: src/dynamic_win.c:724

bps_find

```
static Breakpoint* bps_find(BpList* bl, uint64_t addr);
```

Где реализована: src/dynamic_win.c:157. Видимость: static - внутренняя.

Зачем существует: Управляет списком breakpoints в пределах одной динамической сессии.

Где участвует: src/dynamic_win.c:225, src/dynamic_win.c:290, src/dynamic_win.c:854

write_one_byte

```
static bool write_one_byte(HANDLE hProcess, uint64_t addr, uint8_t val, StrBuf* err);
```

Где реализована: src/dynamic_win.c:165. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:209, src/dynamic_win.c:217

bp_set

```
static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err);
```

Где реализована: src/dynamic_win.c:200. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:229, src/dynamic_win.c:238, src/dynamic_win.c:1051, src/dynamic_win.c:1117

bp_clear

```
static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err);
```

Где реализована: src/dynamic_win.c:215. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:862, src/dynamic_win.c:1154

bp_add_and_arm

```
static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf*
err);
```

Где реализована: src/dynamic_win.c:222. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:297

bp_peek_byte

```
static bool bp_peek_byte(HANDLE hProcess, uint64_t addr, uint8_t* outb, StrBuf* err);
```

Где реализована: src/dynamic_win.c:251. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:311

format_bp_location

```
static void format_bp_location(const ModList* mods, uint64_t addr, char* out, size_t outsz);
```

Где реализована: src/dynamic_win.c:264. Видимость: static - внутренняя.

Зачем существует: Преобразует внутреннее значение в понятное текстовое представление.

Где участвует: src/dynamic_win.c:307

bp_add_and_arm_selfcheck

```
static bool bp_add_and_arm_selfcheck(BpList* bl, HANDLE hProcess, const ModList* mods,
DynamicReport* dr, uint64_t addr, BpKind kind, const char* what, StrBuf* err);
```

Где реализована: src/dynamic_win.c:280. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:591, src/dynamic_win.c:804, src/dynamic_win.c:915, src/dynamic_win.c:931, src/dynamic_win.c:946

pending_rearm_init

```
static void pending_rearm_init(PendingRearmList* pr);
```

Где реализована: src/dynamic_win.c:357. Видимость: static - внутренняя.

Зачем существует: Связывает поток с breakpoint, который нужно переустановить после single-step.

Где участвует: src/dynamic_win.c:760

pending_rearm_set

```
static bool pending_rearm_set(PendingRearmList* pr, DWORD tid, Breakpoint* bp);
```

Где реализована: src/dynamic_win.c:362. Видимость: static - внутренняя.

Зачем существует: Связывает поток с breakpoint, который нужно переустановить после single-step.

Где участвует: src/dynamic_win.c:865

pending_rearm_take

```
static Breakpoint* pending_rearm_take(PendingRearmList* pr, DWORD tid);
```

Где реализована: src/dynamic_win.c:382. Видимость: static - внутренняя.

Зачем существует: Связывает поток с breakpoint, который нужно переустановить после single-step.

Где участвует: src/dynamic_win.c:1048, src/dynamic_win.c:1113

open_event_thread

```
static HANDLE open_event_thread(DWORD tid);
```

Где реализована: src/dynamic_win.c:395. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:879, src/dynamic_win.c:1063

close_event_thread

```
static void close_event_thread(HANDLE h);
```

Где реализована: src/dynamic_win.c:399. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:894, src/dynamic_win.c:1039, src/dynamic_win.c:1078, src/dynamic_win.c:1105

dr_append

```
static void dr_append(DynamicReport* dr, const char* fmt, ...);
```

Где реализована: src/dynamic_win.c:403. Видимость: static - внутренняя.

Зачем существует: Добавляет текстовую или структурированную информацию в DynamicReport.

Где участвует: src/dynamic_win.c:249, src/dynamic_win.c:315, src/dynamic_win.c:322, src/dynamic_win.c:327, src/dynamic_win.c:582, src/dynamic_win.c:592

dynamic_event_add

```
static void dynamic_event_add(DynamicReport* dr, const ModList* mods, const char* type,
DWORD thread_id, uint64_t address, uint32_t exception_code, bool first_chance, uint64_t
arg0, uint64_t arg1, uint64_t arg2, uint64_t arg3);
```

Где реализована: src/dynamic_win.c:416. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/dynamic_win.c:770, src/dynamic_win.c:828, src/dynamic_win.c:846, src/dynamic_win.c:910, src/dynamic_win.c:926, src/dynamic_win.c:941

dynamic_report_init

```
void dynamic_report_init(DynamicReport* dr);
```

Где реализована: src/dynamic_win.c:449. Видимость: external - межмодульная.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/dynamic_win.c:672, src/dynamic_win.c:1214, src/dynamic_win.c:1242

dynamic_report_free

```
void dynamic_report_free(DynamicReport* dr);
```

Где реализована: src/dynamic_win.c:457. Видимость: external - межмодульная.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/main.c:516, src/dynamic_win.c:1222

append_backslashes

```
static bool append_backslashes(StrBuf* out, size_t count);
```

Где реализована: src/dynamic_win.c:463. Видимость: static - внутренняя.

Зачем существует: Добавляет форматированный фрагмент в строковый буфер, соблюдая правила конкретного внешнего формата.

Где участвует: src/dynamic_win.c:485, src/dynamic_win.c:489, src/dynamic_win.c:491

append_cmd_arg

```
static bool append_cmd_arg(StrBuf* out, const char* arg);
```

Где реализована: src/dynamic_win.c:471. Видимость: static - внутренняя.

Зачем существует: Добавляет форматированный фрагмент в строковый буфер, соблюдая правила конкретного внешнего формата.

Где участвует: src/dynamic_win.c:505, src/dynamic_win.c:509

build_cmdline

```
static bool build_cmdline(const char* path, const char* const* argv, int argc, StrBuf* out);
```

Где реализована: src/dynamic_win.c:500. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:686

to

```
/* Resolve a symbol to (real module basename, offset) in *this* process. Important: many
APIs are forwarded (e.g. kernel32 -> kernelbase). */ typedef struct LocalProcLoc;
```

Где реализована: src/dynamic_win.c:514. Видимость: external - межмодульная.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: прямых вызовов в других строках не найдено; функция может быть platform-specific или вызываться как entry point.

local_proc_loc

```
static bool local_proc_loc(const char* preferred_mod, const char* sym, LocalProcLoc* out);
```

Где реализована: src/dynamic_win.c:521. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:578

add_api_bps_if_ready

```
static bool add_api_bps_if_ready(BpList* bl, HANDLE hProcess, const ModList* mods,
DynamicReport* dr, StrBuf* err);
```

Где реализована: src/dynamic_win.c:561. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:797, src/dynamic_win.c:817

bp_kind_name

```
static const char* bp_kind_name(BpKind k);
```

Где реализована: src/dynamic_win.c:602. Видимость: static - внутренняя.

Зачем существует: Управляет software breakpoint: ищет, устанавливает, снимает, проверяет или регистрирует точку останова.

Где участвует: src/dynamic_win.c:248, src/dynamic_win.c:316, src/dynamic_win.c:323, src/dynamic_win.c:328

add_trace

```
static void add_trace(DynamicReport* dr, uint64_t rip);
```

Где реализована: src/dynamic_win.c:617. Видимость: static - внутренняя.

Зачем существует: Сохраняет очередной RIP в ограниченный массив трассировки.

Где участвует: src/dynamic_win.c:636, src/dynamic_win.c:1084

start_trace

```
static void start_trace(DynamicReport* dr, uint32_t trace_steps, DWORD tid, uint64_t rip,
bool* tracing, DWORD* trace_tid, uint32_t* trace_left);
```

Где реализована: src/dynamic_win.c:623. Видимость: static - внутренняя.

Зачем существует: Начинает сбор RIP trace после интересующего runtime hit.

Где участвует: src/dynamic_win.c:993, src/dynamic_win.c:1004, src/dynamic_win.c:1015, src/dynamic_win.c:1026

addr_to_loc

```
static void addr_to_loc(const PeImage* img, uint64_t base, uint64_t addr, char* out, size_t outsz);
```

Где реализована: src/dynamic_win.c:642. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/dynamic_win.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/dynamic_win.c:988

dynamic_collect_windows

```
bool dynamic_collect_windows(const char* pe_path, const char* const* child_argv, int child_argc, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t rf_count, uint32_t timeout_ms, uint32_t trace_steps, DynamicReport* out, StrBuf* err);
```

Где реализована: src/dynamic_win.c:657. Видимость: external - межмодульная.

Зачем существует: Запускает target под Windows debugger и собирает ограниченные runtime evidence.

Где участвует: src/main.c:405, src/dynamic_win.c:1228

dynamic_report_init

```
void dynamic_report_init(DynamicReport* dr);
```

Где реализована: src/dynamic_win.c:1214. Видимость: external - межмодульная.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/dynamic_win.c:449, src/dynamic_win.c:672, src/dynamic_win.c:1242

dynamic_report_free

```
void dynamic_report_free(DynamicReport* dr);
```

Где реализована: src/dynamic_win.c:1222. Видимость: external - межмодульная.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/main.c:516, src/dynamic_win.c:457

dynamic_collect_windows

```
bool dynamic_collect_windows(const char* pe_path, const char* const* child_argv, int child_argc, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t rf_count, uint32_t timeout_ms, uint32_t trace_steps, DynamicReport* out, StrBuf* err);
```

Где реализована: src/dynamic_win.c:1228. Видимость: external - межмодульная.

Зачем существует: Запускает target под Windows debugger и собирает ограниченные runtime evidence.

Где участвует: src/main.c:405, src/dynamic_win.c:657

Функции src/report.c

В текущей реализации обнаружено определений: 118. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

append_regmask

```
static void append_regmask(StrBuf* sb, uint16_t mask, bool xmm);
```

Где реализована: src/report.c:11. Видимость: static - внутренняя.

Зачем существует: Добавляет форматированный фрагмент в строковый буфер, соблюдая правила конкретного внешнего формата.

Где участвует: src/report.c:562, src/report.c:564

anomalies_to_str

```
static void anomalies_to_str(StrBuf* sb, uint32_t anom);
```

Где реализована: src/report.c:23. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:577, src/report.c:1597, src/report.c:2095, src/report.c:3394

section_name_or_dash

```
static const char* section_name_or_dash(const PeImage* img, uint32_t rva);
```

Где реализована: src/report.c:53. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:878, src/report.c:880, src/report.c:884, src/report.c:968, src/report.c:1449, src/report.c:1617

section_permissions

```
static void section_permissions(uint32_t characteristics, char out[4]);
```

Где реализована: src/report.c:58. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:497, src/report.c:866, src/report.c:867, src/report.c:939, src/report.c:1424, src/report.c:2249

report_add_u32_size_mul

```
static bool report_add_u32_size_mul(uint32_t base, size_t index, uint32_t item_size,
uint32_t* out);
```

Где реализована: src/report.c:71. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:81

pdata_entry_rva_for_index

```
static bool pdata_entry_rva_for_index(const PeImage* img, size_t index, uint32_t* out_rva);
```

Где реализована: src/report.c:79. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:86, src/report.c:859, src/report.c:935, src/report.c:1604, src/report.c:2198, src/report.c:2244

pdata_entry_fileoff_for_index

```
static bool pdata_entry_fileoff_for_index(const PeImage* img, size_t index, uint32_t*
out_off);
```

Где реализована: src/report.c:84. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:860, src/report.c:936, src/report.c:1605, src/report.c:2199, src/report.c:2245, src/report.c:2291

report_sec_name_eq

```
static bool report_sec_name_eq(const char* a, const char* b);
```

Где реализована: src/report.c:90. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:124, src/report.c:125, src/report.c:126, src/report.c:127

report_sec_name_contains_ci

```
static bool report_sec_name_contains_ci(const char* haystack, const char* needle);
```

Где реализована: src/report.c:104. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:131, src/report.c:132, src/report.c:133

report_sec_name_allowed_for_unwind

```
static bool report_sec_name_allowed_for_unwind(const char* name8);
```

Где реализована: src/report.c:123. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:171

report_sec_name_plausibly_unwind_named

```
static bool report_sec_name_plausibly_unwind_named(const char* name8);
```

Где реализована: src/report.c:130. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:172

report_non_abi_anomaly_mask

```
static uint32_t report_non_abi_anomaly_mask(void);
```

Где реализована: src/report.c:136. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:212, src/report.c:235

report_rf_range_valid

```
static bool report_rf_range_valid(const RuntimeFunction* r);
```

Где реализована: src/report.c:143. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:148

report_rf_ranges_overlap

```
static bool report_rf_ranges_overlap(const RuntimeFunction* a, const RuntimeFunction* b);
```

Где реализована: src/report.c:147. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:163

report_rf_has_duplicate_begin

```
static bool report_rf_has_duplicate_begin(const RuntimeFunction* rf, size_t n, size_t
index);
```

Где реализована: src/report.c:152. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:210

report_rf_has_overlap

```
static bool report_rf_has_overlap(const RuntimeFunction* rf, size_t n, size_t index);
```

Где реализована: src/report.c:160. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:209

report_unwind_section_is_unusual

```
static bool report_unwind_section_is_unusual(const PeImage* img, uint32_t unwind_rva);
```

Где реализована: src/report.c:168. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:214, src/report.c:230

report_unwind_section_has_flag

```
static bool report_unwind_section_has_flag(const PeImage* img, uint32_t unwind_rva, uint32_t
flag);
```

Где реализована: src/report.c:175. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:213, src/report.c:215, src/report.c:225, src/report.c:229, src/report.c:231, src/report.c:239

report_handler_section_nonexec

```
static bool report_handler_section_nonexec(const PeImage* img, const UnwindSummary* u);
```

Где реализована: src/report.c:180. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:216, src/report.c:226, src/report.c:232

report_handler_section_writable

```
static bool report_handler_section_writable(const PeImage* img, const UnwindSummary* u);
```

Где реализована: src/report.c:186. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:217, src/report.c:233, src/report.c:240

evidence_record_matches

```
static bool evidence_record_matches(const HeuristicFinding* finding, const PeImage* img,
const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n, size_t
index);
```

Где реализована: src/report.c:192. Видимость: static - внутренняя.

Зачем существует: Строит или проверяет evidence - конкретное доказательное наблюдение, связанное с finding.

Где участвует: src/report.c:253, src/report.c:911, src/report.c:2286

evidence_record_count

```
static size_t evidence_record_count(const HeuristicFinding* finding, const PeImage* img,
const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:245. Видимость: static - внутренняя.

Зачем существует: Строит или проверяет evidence - конкретное доказательное наблюдение, связанное с finding.

Где участвует: src/report.c:903, src/report.c:2271

evidence_record_reason_ru

```
static void evidence_record_reason_ru(const HeuristicFinding* finding, const PeImage* img,
const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n, size_t
index, char* out, size_t out_size);
```

Где реализована: src/report.c:258. Видимость: static - внутренняя.

Зачем существует: Строит или проверяет evidence - конкретное доказательное наблюдение, связанное с finding.

Где участвует: src/report.c:868, src/report.c:2297

finding_rule_text

```
static const char* finding_rule_text(const char* code);
```

Где реализована: src/report.c:324. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1477, src/report.c:2344, src/report.c:2417

finding_expected_text

```
static const char* finding_expected_text(const char* code);
```

Где реализована: src/report.c:356. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1479, src/report.c:2348

finding_impact_text

```
static const char* finding_impact_text(const char* code);
```

Где реализована: src/report.c:388. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:438, src/report.c:1480

finding_title_ru

```
static const char* finding_title_ru(const char* code);
```

Где реализована: src/report.c:403. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1434, src/report.c:2340

finding_meaning_ru

```
static const char* finding_meaning_ru(const char* code);
```

Где реализована: src/report.c:423. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1435, src/report.c:2342

finding_why_it_matters_ru

```
static const char* finding_why_it_matters_ru(const char* code);
```

Где реализована: src/report.c:437. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1436, src/report.c:2350

finding_next_step_ru

```
static const char* finding_next_step_ru(const char* code);
```

Где реализована: src/report.c:441. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1437, src/report.c:2352

finding_related_function_index

```
static size_t finding_related_function_index(const HeuristicFinding* finding, const
RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:454. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:489, src/report.c:926, src/report.c:1419, src/report.c:2232

finding_observed_text

```
static void finding_observed_text(const HeuristicFinding* finding, const PeImage* img, const
RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n, char* out, size_t
out_size);
```

Где реализована: src/report.c:474. Видимость: static - внутренняя.

Зачем существует: Связывает finding с правилом, наблюдаемым значением, функцией или объяснением для отчёта.

Где участвует: src/report.c:1427, src/report.c:2338

runtime_function_size

```
static uint32_t runtime_function_size(const RuntimeFunction* r);
```

Где реализована: src/report.c:527. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:656, src/report.c:877, src/report.c:1616, src/report.c:3399

csv_cell

```
static bool csv_cell(StrBuf* out, const char* s);
```

Где реализована: src/report.c:532. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:637, src/report.c:638, src/report.c:639, src/report.c:640, src/report.c:662, src/report.c:675

build_unwind_comment

```
static bool build_unwind_comment(const UnwindSummary* u, StrBuf* sb);
```

Где реализована: src/report.c:548. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:625, src/report.c:1590

report_print_brief

```
void report_print_brief(const char* path, const PeImage* img, size_t rf_count);
```

Где реализована: src/report.c:582. Видимость: external - межмодульная.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/main.c:326

report_write_csv

```
bool report_write_csv(const char* path, const PeImage* img, const RuntimeFunction* rf, const
UnwindSummary* us, const bool* us_ok, size_t n, bool ghidra_mode, StrBuf* err);
```

Где реализована: src/report.c:593. Видимость: external - межмодульная.

Зачем существует: Сериализует функции и unwind summaries в CSV или Ghidra CSV.

Где участвует: src/main.c:430, src/main.c:441

valid_utf8_sequence_length

```
static size_t valid_utf8_sequence_length(const unsigned char* s, size_t remaining);
```

Где реализована: src/report.c:688. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:739, src/report.c:1686

json_escape

```
static void json_escape(StrBuf* sb, const char* s);
```

Где реализована: src/report.c:724. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:754, src/report.c:760

json_key

```
static void json_key(StrBuf* sb, const char* k);
```

Где реализована: src/report.c:752. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:765, src/report.c:771, src/report.c:807, src/report.c:810, src/report.c:813, src/report.c:816

json_string

```
static void json_string(StrBuf* sb, const char* s);
```

Где реализована: src/report.c:758. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:766, src/report.c:990, src/report.c:1193, src/report.c:1617, src/report.c:1619, src/report.c:1629

json_string_field

```
static void json_string_field(StrBuf* sb, const char* k, const char* v, bool
trailing_comma);
```

Где реализована: src/report.c:764. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:806, src/report.c:825, src/report.c:878, src/report.c:880, src/report.c:882, src/report.c:884

json_bool_field

```
static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma);
```

Где реализована: src/report.c:770. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:837, src/report.c:887, src/report.c:1329, src/report.c:1330, src/report.c:1350, src/report.c:1364

json_nullable_hex32

```
static void json_nullable_hex32(StrBuf* sb, bool has_value, uint32_t value);
```

Где реализована: src/report.c:777. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:808, src/report.c:823, src/report.c:839, src/report.c:872, src/report.c:873, src/report.c:875

json_nullable_count

```
static void json_nullable_count(StrBuf* sb, bool has_value, uint32_t value);
```

Где реализована: src/report.c:782. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:811, src/report.c:814, src/report.c:817, src/report.c:820

bytes_to_hex

```
static void bytes_to_hex(const uint8_t* bytes, size_t count, char* out, size_t out_size);
```

Где реализована: src/report.c:787. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:803, src/report.c:2007, src/report.c:2118

json_unwind_diagnostic

```
static void json_unwind_diagnostic(StrBuf* sb, const UnwindSummary* u);
```

Где реализована: src/report.c:799. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:892, src/report.c:1648

json_finding_evidence_record

```
static void json_finding_evidence_record(StrBuf* sb, const HeuristicFinding* finding, const
PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t
n, size_t index);
```

Где реализована: src/report.c:843. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:913

json_finding_evidence_records

```
static void json_finding_evidence_records(StrBuf* sb, const HeuristicFinding* finding, const
PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t
n);
```

Где реализована: src/report.c:896. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:1484

json_finding_evidence_chain

```
static void json_finding_evidence_chain(StrBuf* sb, const HeuristicFinding* finding, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:919. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:1482

json_heuristic_families

```
static void json_heuristic_families(StrBuf* sb, uint32_t mask);
```

Где реализована: src/report.c:977. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:1409

dynamic_signal_count

```
static unsigned dynamic_signal_count(const DynamicReport* dr);
```

Где реализована: src/report.c:996. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:1079, src/report.c:1181, src/report.c:1537

dynamic_session_complete

```
static bool dynamic_session_complete(const DynamicReport* dr);
```

Где реализована: src/report.c:1008. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:1019, src/report.c:1043, src/report.c:1086, src/report.c:1105, src/report.c:1108

has_static_dynamic_correlation

```
static bool has_static_dynamic_correlation(const HeuristicReport* hr, const DynamicReport* dr);
```

Где реализована: src/report.c:1017. Видимость: static - внутренняя.

Зачем существует: Отвечает на один логический вопрос и возвращает true или false без изменения основной модели.

Где участвует: src/report.c:1078

runtime_evidence_level

```
static const char* runtime_evidence_level(const DynamicReport* dr);
```

Где реализована: src/report.c:1032. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1049, src/report.c:1083

case_strength

```
static const char* case_strength(const HeuristicReport* hr, const DynamicReport* dr, bool
correlated);
```

Где реализована: src/report.c:1047. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1084

assessment_summary

```
static AssessmentSummary assessment_summary(const HeuristicReport* hr, const DynamicReport*
dr);
```

Где реализована: src/report.c:1075. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1570, src/report.c:3054, src/report.c:3212

heuristics_has_finding_code

```
static bool heuristics_has_finding_code(const HeuristicReport* hr, const char* code);
```

Где реализована: src/report.c:1114. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1160

heuristics_has_family

```
static bool heuristics_has_family(const HeuristicReport* hr, const char* family);
```

Где реализована: src/report.c:1122. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1148, src/report.c:1151, src/report.c:1160

collect_manual_validation_steps

```
static ManualValidationSteps collect_manual_validation_steps(const HeuristicReport* hr,
const DynamicReport* dr);
```

Где реализована: src/report.c:1135. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:1189, src/report.c:3213

json_manual_validation_steps

```
static void json_manual_validation_steps(StrBuf* sb, const HeuristicReport* hr, const
DynamicReport* dr);
```

Где реализована: src/report.c:1188. Видимость: static - внутренняя.

Зачем существует: Добавляет или экранирует определённую часть JSON, сохраняя корректный синтаксис и тип значения.

Где участвует: src/report.c:1582

report_print_heuristics

```
void report_print_heuristics(const HeuristicReport* hr);
```

Где реализована: src/report.c:1198. Видимость: external - межмодульная.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/main.c:392

report_metadata_valid

```
static bool report_metadata_valid(const ReportMetadata* metadata);
```

Где реализована: src/report.c:1226. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:1304, src/report.c:3026

report_model_valid

```
static bool report_model_valid(const PeImage* img, const HeuristicReport* hr, const
DynamicReport* dr);
```

Где реализована: src/report.c:1266. Видимость: static - внутренняя.

Зачем существует: Выполняет проверку, адресное вычисление или подготовку данных, необходимую reporting layer - слою отчётов.

Где участвует: src/report.c:1304, src/report.c:3026

report_write_json_ex

```
bool report_write_json_ex(const char* path, const char* input_path, const PeImage* img,
const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n, const
HeuristicReport* hr, const DynamicReport* dr, const ReportMetadata* metadata, StrBuf* err);
```

Где реализована: src/report.c:1292. Видимость: external - межмодульная.

Зачем существует: Сериализует полную модель анализа в машиночитаемый JSON.

Где участвует: src/main.c:488

html_escape

```
static void html_escape(StrBuf* sb, const char* text);
```

Где реализована: src/report.c:1671. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:1702, src/report.c:1735, src/report.c:1787, src/report.c:1789, src/report.c:1932, src/report.c:1994

html_text_cell

```
static void html_text_cell(StrBuf* sb, const char* text);
```

Где реализована: src/report.c:1700. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:1784, src/report.c:1785, src/report.c:2406, src/report.c:2407, src/report.c:2408, src/report.c:3005

html_hex32_cell

```
static void html_hex32_cell(StrBuf* sb, uint32_t value);
```

Где реализована: src/report.c:1706. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2414, src/report.c:3003, src/report.c:3202, src/report.c:3203, src/report.c:3204, src/report.c:3205

html_hex64_cell

```
static void html_hex64_cell(StrBuf* sb, uint64_t value);
```

Где реализована: src/report.c:1710. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:3002, src/report.c:3359, src/report.c:3360, src/report.c:3361, src/report.c:3362, src/report.c:3363

html_frame_saved_cell

```
static void html_frame_saved_cell(StrBuf* sb, const UnwindSummary* u);
```

Где реализована: src/report.c:1714. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:3411

html_handler_chain_cell

```
static void html_handler_chain_cell(StrBuf* sb, const UnwindSummary* u);
```

Где реализована: src/report.c:1753. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:3412

html_issue_class

```
static const char* html_issue_class(bool issue);
```

Где реализована: src/report.c:1773. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:1786

html_metric_row

```
static void html_metric_row(StrBuf* sb, const char* group, const char* check, const char*
value, bool issue, const char* meaning);
```

Где реализована: src/report.c:1777. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:1800, src/report.c:1808, src/report.c:1818, src/report.c:1839, src/report.c:2551, src/report.c:2591

html_count_metric_row

```
static void html_count_metric_row(StrBuf* sb, const char* group, const char* check, size_t
value, const char* meaning);
```

Где реализована: src/report.c:1793. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2536, src/report.c:2538, src/report.c:2540, src/report.c:2542, src/report.c:2544, src/report.c:2554

html_bool_metric_row

```
static void html_bool_metric_row(StrBuf* sb, const char* group, const char* check, bool value, const char* meaning);
```

Где реализована: src/report.c:1803. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2546

html_ratio_metric_row

```
static void html_ratio_metric_row(StrBuf* sb, const char* group, const char* check, double value, const char* meaning);
```

Где реализована: src/report.c:1811. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2585, src/report.c:2587, src/report.c:2593

html_evidence_families_metric_row

```
static void html_evidence_families_metric_row(StrBuf* sb, const HeuristicReport* hr);
```

Где реализована: src/report.c:1821. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2602

html_score_class

```
static const char* html_score_class(const HeuristicReport* hr);
```

Где реализована: src/report.c:1843. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:3110, src/report.c:3117

html_level_class

```
static const char* html_level_class(const char* level);
```

Где реализована: src/report.c:1850. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:3117, src/report.c:3132

ru_yes_no

```
static const char* ru_yes_no(bool value);
```

Где реализована: src/report.c:1859. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:3160, src/report.c:3161, src/report.c:3188, src/report.c:3230, src/report.c:3330

dynamic_platform_unsupported

```
static bool dynamic_platform_unsupported(const DynamicReport* dr);
```

Где реализована: src/report.c:1863. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:1870, src/report.c:1879, src/report.c:2667, src/report.c:3128, src/report.c:3233

ru_status_text

```
static const char* ru_status_text(const DynamicReport* dr);
```

Где реализована: src/report.c:1868. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:3141, src/report.c:3317

dynamic_status_class

```
static const char* dynamic_status_class(const DynamicReport* dr);
```

Где реализована: src/report.c:1876. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:3139, src/report.c:3315

ru_level_text

```
static const char* ru_level_text(const char* value);
```

Где реализована: src/report.c:1883. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:3122, src/report.c:3124, src/report.c:3223, src/report.c:3225

ru_severity_text

```
static const char* ru_severity_text(const char* value);
```

Где реализована: src/report.c:1892. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:2408, src/report.c:3266, src/report.c:3301

ru_verdict_text

```
static const char* ru_verdict_text(const char* value);
```

Где реализована: src/report.c:1902. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:3119, src/report.c:3216

ru_case_strength_text

```
static const char* ru_case_strength_text(const char* value);
```

Где реализована: src/report.c:1919. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:3218

html_append_manual_step_ru

```
static void html_append_manual_step_ru(StrBuf* sb, const char* step);
```

Где реализована: src/report.c:1930. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3243

html_append_finding_message_ru

```
static void html_append_finding_message_ru(StrBuf* sb, const HeuristicFinding* finding);
```

Где реализована: src/report.c:1935. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3276, src/report.c:3305

html_append_unwind_diagnostic_inline

```
static void html_append_unwind_diagnostic_inline(StrBuf* sb, const UnwindSummary* u);
```

Где реализована: src/report.c:1998. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2261, src/report.c:3418

html_unwind_has_key_diagnostic

```
static bool html_unwind_has_key_diagnostic(bool ok, const UnwindSummary* u);
```

Где реализована: src/report.c:2048. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2178, src/report.c:2193

html_key_diagnostic_interpretation_ru

```
static const char* html_key_diagnostic_interpretation_ru(bool ok, const UnwindSummary* u);
```

Где реализована: src/report.c:2056. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2218

html_append_key_diagnostic_reason

```
static void html_append_key_diagnostic_reason(StrBuf* sb, bool ok, const UnwindSummary* u);
```

Где реализована: src/report.c:2082. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2214

html_append_key_diagnostic_data

```
static void html_append_key_diagnostic_data(StrBuf* sb, const UnwindSummary* u);
```

Где реализована: src/report.c:2109. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2216

html_append_key_diagnostics_panel

```
static void html_append_key_diagnostics_panel(StrBuf* sb, const PeImage* img, const
RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:2169. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3251

html_append_finding_chain_ru

```
static void html_append_finding_chain_ru(StrBuf* sb, const HeuristicFinding* finding, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:2225. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2353

html_append_finding_evidence_records

```
static void html_append_finding_evidence_records(StrBuf* sb, const HeuristicFinding* finding, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:2264. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2354

html_append_finding_evidence_block

```
static void html_append_finding_evidence_block(StrBuf* sb, const HeuristicFinding* finding, const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n);
```

Где реализована: src/report.c:2329. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3277

html_append_mitigation_message_ru

```
static void html_append_mitigation_message_ru(StrBuf* sb, const char* code, const char* message);
```

Где реализована: src/report.c:2358. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3289

html_find_finding_by_code

```
static const HeuristicFinding* html_find_finding_by_code(const HeuristicReport* hr, const char* code);
```

Где реализована: src/report.c:2375. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2399, src/report.c:2456, src/report.c:2465, src/report.c:2491

html_matrix_status_class

```
static const char* html_matrix_status_class(bool active, const char* severity);
```

Где реализована: src/report.c:2384. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2402

html_heuristic_matrix_row

```
static void html_heuristic_matrix_row(StrBuf* sb, size_t number, const HeuristicReport* hr,
const char* code, const char* family, const char* default_severity, bool active, const char*
value);
```

Где реализована: src/report.c:2391. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2430, src/report.c:2441, src/report.c:2455, src/report.c:2466, src/report.c:2494

html_heuristic_matrix_count_row

```
static void html_heuristic_matrix_count_row(StrBuf* sb, size_t* number, const
HeuristicReport* hr, const char* code, const char* family, const char* default_severity,
size_t count);
```

Где реализована: src/report.c:2421. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2484, src/report.c:2485, src/report.c:2486, src/report.c:2487, src/report.c:2488, src/report.c:2497

html_heuristic_matrix_bool_row

```
static void html_heuristic_matrix_bool_row(StrBuf* sb, size_t* number, const
HeuristicReport* hr, const char* code, const char* family, const char* default_severity,
bool flag);
```

Где реализована: src/report.c:2434. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2489

html_heuristic_matrix_ratio_row

```
static void html_heuristic_matrix_ratio_row(StrBuf* sb, size_t* number, const
HeuristicReport* hr, const char* code, const char* family, const char* default_severity,
double ratio, const char* label);
```

Где реализована: src/report.c:2445. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2507, src/report.c:2508, src/report.c:2509

html_heuristic_matrix_combo_row

```
static void html_heuristic_matrix_combo_row(StrBuf* sb, size_t* number, const
HeuristicReport* hr, const char* code, const char* family, const char* default_severity);
```

Где реализована: src/report.c:2459. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2517, src/report.c:2518, src/report.c:2519, src/report.c:2520

html_append_static_heuristic_matrix

```
static void html_append_static_heuristic_matrix(StrBuf* sb, const HeuristicReport* hr,
size_t runtime_function_count);
```

Где реализована: src/report.c:2470. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3255

html_append_static_coverage_panel

```
static void html_append_static_coverage_panel(StrBuf* sb, const HeuristicReport* hr, size_t
runtime_function_count);
```

Где реализована: src/report.c:2525. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3254

dynamic_event_type_ru

```
static const char* dynamic_event_type_ru(const char* type);
```

Где реализована: src/report.c:2609. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:3354

dynamic_event_meaning_ru

```
static const char* dynamic_event_meaning_ru(const char* type);
```

Где реализована: src/report.c:2626. Видимость: static - внутренняя.

Зачем существует: Создаёт, заполняет или освобождает структуру динамического наблюдения.

Где участвует: src/report.c:3365

html_dynamic_evidence_row

```
static void html_dynamic_evidence_row(StrBuf* sb, const char* signal, const char* observed,
const char* rule, const char* interpretation);
```

Где реализована: src/report.c:2643. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2668, src/report.c:2675, src/report.c:2690, src/report.c:2705, src/report.c:2721, src/report.c:2736

html_append_dynamic_evidence_table

```
static void html_append_dynamic_evidence_table(StrBuf* sb, const DynamicReport* dr);
```

Где реализована: src/report.c:2659. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3343

html_trace_addr_to_rva

```
static bool html_trace_addr_to_rva(const PeImage* img, const DynamicReport* dr, uint64_t
addr, uint32_t* out_rva, const char** out_base_kind);
```

Где реализована: src/report.c:2777. Видимость: static - внутренняя.

Зачем существует: Подготавливает HTML-представление значения, статуса, текста или таблицы.

Где участвует: src/report.c:2888, src/report.c:2889, src/report.c:2904, src/report.c:2990, src/report.c:2997

html_append_trace_interpretation

```
static void html_append_trace_interpretation(StrBuf* sb, uint32_t index, uint64_t addr, bool
mapped, uint32_t rva, const PeSection* section, const char* base_kind, bool has_prev,
uint64_t prev_addr, bool prev_mapped, const PeSection* prev_section);
```

Где реализована: src/report.c:2806. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3007

trace_segment_same

```
static bool trace_segment_same(const PeImage* img, const DynamicReport* dr, uint64_t a,
uint64_t b);
```

Где реализована: src/report.c:2881. Видимость: static - внутренняя.

Зачем существует: Вспомогательная функция модуля src/report.c; изолирует одну повторяемую операцию, чтобы основной маршрут не дублировал её реализацию.

Где участвует: src/report.c:2973

html_append_trace_segment_row

```
static void html_append_trace_segment_row(StrBuf* sb, const PeImage* img, const
DynamicReport* dr, uint32_t first, uint32_t last);
```

Где реализована: src/report.c:2895. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:2974, src/report.c:2978

html_append_dynamic_trace_table

```
static void html_append_dynamic_trace_table(StrBuf* sb, const PeImage* img, const
DynamicReport* dr);
```

Где реализована: src/report.c:2940. Видимость: static - внутренняя.

Зачем существует: Добавляет в HTML законченный смысловой блок, строку таблицы или доказательную секцию.

Где участвует: src/report.c:3344

report_write_html

```
bool report_write_html(const char* path, const char* input_path, const PeImage* img, const
RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok, size_t n, const
HeuristicReport* hr, const DynamicReport* dr, const ReportMetadata* metadata, StrBuf* err);
```

Где реализована: src/report.c:3014. Видимость: external - межмодульная.

Зачем существует: Создаёт автономный HTML-отчёт для аналитика.

Где участвует: src/main.c:502

Функции src/app_info.c

В текущей реализации обнаружено определений: 6. Static означает внутреннюю функцию, доступную только внутри этого .c; external означает функцию, которую может вызвать другой модуль через объявление в .h.

app_version

```
const char* app_version(void);
```

Где реализована: src/app_info.c:6. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:90, src/main.c:96, src/main.c:468

app_report_schema_version

```
const char* app_report_schema_version(void);
```

Где реализована: src/app_info.c:10. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:91, src/main.c:469

app_build_compiler

```
const char* app_build_compiler(void);
```

Где реализована: src/app_info.c:14. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:92, src/main.c:474

app_build_platform

```
const char* app_build_platform(void);
```

Где реализована: src/app_info.c:30. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:92, src/main.c:475

app_build_mode

```
const char* app_build_mode(void);
```

Где реализована: src/app_info.c:48. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:92, src/main.c:476

app_utc_timestamp

```
bool app_utc_timestamp(char* out, size_t out_size);
```

Где реализована: src/app_info.c:56. Видимость: external - межмодульная.

Зачем существует: Возвращает сведения о версии, сборке или времени для provenance отчёта.

Где участвует: src/main.c:462

Сквозной прогон: один файл от команды до отчёта

Этот раздел собирает модули в одну причинную цепочку. Он нужен, чтобы объяснять проект не как перечень файлов, а как последовательность преобразований одного исследуемого PE.

Шаг 1. Принять команду

C runtime вызывает main(argc, argv). parse_args переводит текстовые options в структуру Args и отклоняет неизвестные параметры до чтения файла.

Как объяснить этот шаг на защите. Анализ начинается с команды пользователя. Пользователь указывает, какой файл требуется исследовать, какие виды отчёта сформировать и нужно ли выполнять динамическое наблюдение. Главный модуль проверяет эти параметры до обращения к файлу: распознаёт допустимые команды, контролирует числовые ограничения и отклоняет противоречивый или неполный запрос. Поэтому все последующие модули получают уже однозначное задание и не должны самостоятельно угадывать намерения пользователя.

Что происходит внутри. Операционная система передаёт функции main количество аргументов argc и массив argv, содержащий текст команды. parse_args последовательно сопоставляет эти элементы с поддерживаемыми параметрами: путём к входному файлу, режимами статического и динамического анализа, форматами отчёта, ограничением времени и параметрами запуска исследуемой программы. Числовые значения преобразуются не напрямую, а через функции с проверкой допустимого диапазона.

Почему шаг обязателен. Один и тот же набор модулей может работать в разных режимах, поэтому до начала анализа требуется получить единую конфигурацию Args. Если пропустить такую проверку, ошибочный параметр может быть замечен уже после выделения памяти, запуска процесса или частичной записи отчёта. Раннее отклонение неверной команды уменьшает неоднозначность и не позволяет пользовательской ошибке превратиться в ошибку анализа.

Что передаётся дальше. Следующий этап получает не исходный текст команды, а проверенные поля Args. На защите важно подчеркнуть: main не исследует EH/Unwind самостоятельно. Он является координатором, который определяет последовательность модулей, передаёт им данные и отвечает за завершение всего сценария.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/main.c. Это orchestration layer: он принимает управление от C runtime и определяет конфигурацию всего запуска.

Какие функции выполняют шаг. Функции main и parse_args в src/main.c. parse_long_range и parse_u32_range проверяют числовые значения; build_invocation позднее создаёт безопасную запись команды для отчёта.

Где объявлены функции и типы. main и внутренние parse-функции публично в .h не объявлены. Структура Args объявлена в src/main.c:20-37; parse_args - src/main.c:176; main - src/main.c:270.

Где искать реализацию и вызов. src/main.c:154 - parse_long_range; :165 - parse_u32_range; :176-253 - parse_args; :270-281 - вход в main и обработка help/version/CLI error.

Как меняются данные. До шага существуют только argc и argv. После шага существует Args: input, включённые режимы, пути outputs - выходных файлов, timeout, trace_steps и аргументы запускаемой программы.

Что считается ошибкой. Неизвестная option, отсутствующее значение или число вне диапазона останавливают запуск до доступа к target.

Как результат используется дальше. Следующий шаг использует Args.input, а поздние стадии проверяют flags, чтобы решить, какие модули запускать.

Смысл шага: нормализовать пользовательский запрос и не допустить неоднозначной конфигурации.

Шаг 2. Загрузить файл

buf_init_from_file открывает target, определяет размер, выделяет память и создаёт Buf. На этом этапе известны только bytes и length; формат ещё не доказан.

Как объяснить этот шаг на защите. После проверки команды программа загружает исследуемый файл в память. Пока это только последовательность байтов: анализатор ещё не утверждает, что перед ним корректный PE-файл. Модуль общих операций определяет точный размер файла, выделяет достаточную область памяти и считывает в неё содержимое целиком. Одновременно он фиксирует, кто владеет этой памятью и должен освободить её после анализа. Этот шаг создаёт стабильный и контролируемый источник данных для всех последующих проверок.

Что происходит внутри. buf_init_from_file открывает файл в двоичном режиме, определяет его длину, проверяет возможность представить эту длину в памяти, выделяет непрерывный массив байтов и считывает файл. Структура Buf хранит указатель data на начало массива и size - число доступных байтов. Отдельный указатель owned фиксирует владение выделенной областью, то есть обязанность позднее вызвать освобождение памяти.

Почему шаг обязателен. Все дальнейшие адресные проверки зависят от точного знания границ входа. Анализатор не должен читать поле только потому, что спецификация предполагает его наличие: сначала требуется доказать, что соответствующие байты действительно находятся внутри Buf. Загрузка всего файла создаёт неизменяемый снимок данных, поэтому разные модули исследуют одну и ту же последовательность байтов.

Что передаётся дальше. pe.c получает const Buf *, где const означает запрет изменять байты через этот интерфейс. На этом этапе ещё нет ни секций, ни функций, ни UNWIND_INFO. Есть только достоверно известные размер и содержимое файла; смысл этим байтам назначит следующий модуль.

Какой модуль отвечает за шаг. Исполняющий модуль - src/util.c; вызов организует src/main.c. util.c владеет правилами загрузки, main.c - жизненным циклом полученной памяти.

Какие функции выполняют шаг. buf_init_from_file в src/util.c. Она заполняет Buf b и возвращает через owned адрес выделенной памяти, которую обязан освободить main.

Где объявлены функции и типы. Публичное объявление buf_init_from_file: include/util.h:20. Тип Buf объявлен в include/util.h:15-18.

Где искать реализацию и вызов. src/util.c:13-36 - buf_init_from_file; src/main.c:283-288 - место вызова и обработка ошибки загрузки.

Как меняются данные. До шага есть путь. После шага Buf.data указывает на первый байт, Buf.size сообщает точную длину, owned хранит тот же адрес как объект владения.

Что считается ошибкой. Ошибка fopen/fseek/ftell/malloc/fread приводит к false. Частично созданные ресурсы закрываются внутри функции.

Как результат используется дальше. pe_parse получает const Buf*: он читает байты, но по контракту не изменяет их.

Смысл шага: создать единственную контролируемую копию недоверенных входных данных.

Шаг 3. Построить PE-карту

pe_parse проверяет MZ, PE signature, AMD64 и PE32+, извлекает sections и Exception Directory, считает structural counters и формирует PeImage.

Как объяснить этот шаг на защите. Далее анализатор устанавливает внутреннее устройство PE-файла. Он проверяет обязательные сигнатуры и архитектуру AMD64, читает заголовки и описания секций, определяет размер образа и находит каталог исключений. Результатом становится карта PE-контейнера: программа понимает, какие диапазоны относятся к машинному коду, служебным данным и конкретным участкам файла. Эта карта необходима потому, что метаданные Windows используют относительные виртуальные адреса, тогда как физическое чтение выполняется по смещениям внутри файла.

Что происходит внутри. pe_parse начинает с формальных признаков формата: проверяет DOS-сигнатуру MZ, находит PE-заголовок, проверяет PE signature, тип процессора AMD64 и формат PE32+. Затем извлекает ImageBase, SizeOfImage, параметры выравнивания, массив секций и запись Exception Directory. Для каждой секции сохраняются виртуальный адрес, виртуальный размер, физическое смещение в файле, размер хранимых байтов и права доступа.

Почему шаг обязателен. RVA - относительный виртуальный адрес внутри загруженного образа - нельзя использовать как обычный индекс файла. Между виртуальным размещением секции и её физическим положением на диске может быть разница. PeImage становится системой координат: по ней можно установить, какой секции принадлежит RVA и в какой file offset, то есть физическое смещение, следует обратиться для чтения.

Что передаётся дальше. unwind.c получает проверенную структуру PeImage и больше не разбирает PE-заголовки самостоятельно. Это важная архитектурная граница: правила отображения адресов сосредоточены в pe.c, поэтому все последующие модули используют одну и ту же карту и не получают противоречащие друг другу смещения.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/pe.c. Безопасные чтения он выполняет через публичный контракт include/util.h; main.c только вызывает parser.

Какие функции выполняют шаг. pe_parse в src/pe.c; внутри участвуют read_dos_lfanew, buf_read_u16/u32/u64, is_power_of_two_u32 и ranges_overlap_u64.

Где объявлены функции и типы. Публичное объявление pe_parse: include/pe.h:56. Типы PeSection и PeImage находятся в include/pe.h:24-53.

Где искать реализацию и вызов. src/pe.c:36-256 - pe_parse; src/main.c:290-299 - создание PeImage, вызов parser-а и фатальная ветвь ошибки.

Как меняются данные. До шага байты не имеют доказанного назначения. После шага PeImage связывает поля заголовков, секции, виртуальные диапазоны и физические диапазоны файла.

Что считается ошибкой. Неверные MZ/PE signatures, неподдерживаемая архитектура или недоступные обязательные заголовки являются фатальными. Некоторые нарушения секций сохраняются как counters и допускают продолжение.

Как результат используется дальше. unwind.c получает &img; и использует pe_rva_to_fileoff вместо самостоятельного повторного разбора PE.

Смысл шага: установить систему координат, без которой адреса EH/Unwind нельзя локализовать.

Шаг 4. Прочитать .pdata

unwind_read_runtime_functions_ex переводит exception_rva в file offset, проверяет размер каталога и создаёт RuntimeFunction[]. Каждая запись связывает диапазон кода с UnwindInfoAddress.

Как объяснить этот шаг на защите. Используя построенную PE-карту, анализатор переходит к каталогу исключений и читает таблицу RUNTIME_FUNCTION. Каждая её запись описывает диапазон машинного кода одной функции и содержит адрес связанной с ним информации раскрутки стека. В результате программа получает перечень функций, для которых Windows располагает EH/Unwind-метаданными. На этом этапе ещё не интерпретируются отдельные операции раскрутки: сначала создаётся надёжная связь между диапазоном функции и адресом её служебного описания.

Что происходит внутри. unwind_read_runtime_functions_ex берёт RVA и размер Exception Directory из PeImage, переводит начало каталога в file offset и проверяет, что заявленный диапазон доступен в файле. На AMD64 одна запись RUNTIME_FUNCTION состоит из трёх 32-битных относительных адресов: BeginAddress, EndAddress и UnwindInfoAddress. Функция читает доступные записи и формирует массив RuntimeFunction.

Почему шаг обязателен. Эта таблица связывает машинный код с метаданными, которыми Windows пользуется при раскрутке стека и обработке исключений. BeginAddress и EndAddress задают полуоткрытый диапазон функции: начало включается, конец не включается. UnwindInfoAddress указывает на описание пролога и правил восстановления состояния. Без этой связи отдельный блок .xdata нельзя надёжно отнести к конкретной функции.

Что передаётся дальше. main получает указатель rf и число rf_count. Теперь анализ организован по функциям: индекс i обозначает конкретную запись таблицы и остаётся её идентификатором на последующих этапах. Если каталог повреждён, инструмент не придумывает отсутствующие записи, а сохраняет диагностируемую ошибку или доступную частичную картину.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/unwind.c. Для преобразования RVA в file offset он использует сервис src/pe.c; main.c получает готовый массив.

Какие функции выполняют шаг. unwind_read_runtime_functions_ex в src/unwind.c и pe_rva_to_fileoff в src/pe.c.

Где объявлены функции и типы. Объявление unwind_read_runtime_functions_ex: include/unwind.h:117-122. pe_rva_to_fileoff объявлена в include/pe.h:60.

Где искать реализацию и вызов. src/unwind.c:766-827 - unwind_read_runtime_functions_ex; src/pe.c:277-306 - pe_rva_to_fileoff; src/main.c:301-324 - вызов, различение MALFORMED/OOM и partial report.

Как меняются данные. До шага известна только координата каталога. После шага rf указывает на массив, rf_count сообщает его длину, а rf[i] описывает одну функцию.

Что считается ошибкой. Нехватка памяти является фатальной. Структурно нечитаемая Exception Directory сохраняется как ошибка и допускает частичный отчёт без выдуманных записей.

Как результат используется дальше. Цикл main использует rf_count как границу и rf[i].unwind_rva как вход каждой попытки unwind_parse.

Смысл шага: получить функцию-ориентированную карту exception directory.

Шаг 5. Подготовить результаты .xdata

main.c выделяет UnwindSummary[] и us_ok[]. Два массива разделяют сами данные разбора и факт его успешности для каждого индекса RuntimeFunction.

Как объяснить этот шаг на защите. Перед подробным разбором программа подготавливает отдельное место для результата каждой функции. Для одной записи необходимо хранить не только извлечённые сведения, но и независимый признак того, завершился ли её разбор успешно. Поэтому используются два согласованных массива: один содержит результаты, второй сообщает их достоверность. Одинаковый индекс сохраняет связь между исходной RUNTIME_FUNCTION, разобранной UNWIND_INFO и статусом разбора, даже если отдельная повреждённая запись не смогла быть прочитана полностью.

Что происходит внутри. main выделяет массив UnwindSummary размером rf_count и параллельный массив логических значений us_ok той же длины. calloc не только резервирует память, но и заполняет её нулями. Поэтому до разбора каждая сводка находится в определённом исходном состоянии, а каждый флаг успешности равен false.

Почему нужны два массива. UnwindSummary хранит сами извлечённые поля и диагностические сведения. Однако частично заполненная структура ещё не доказывает, что разбор завершился успешно. us_ok отдельно отвечает на этот вопрос. Такое разделение не позволяет эвристическому или отчётному слою случайно принять остаточные либо частичные данные повреждённой записи за полностью достоверный результат.

Что передаётся дальше. Для каждой RuntimeFunction[i] заранее существуют UnwindSummary[i] и us_ok[i]. Индекс является связью между тремя сущностями. Благодаря этому отказ разбора одной функции не сдвигает массив и не разрушает соответствие результатов всем следующим функциям.

Какой модуль отвечает за шаг. Ответственный за организацию шага - src/main.c. Здесь нет отдельного предметного модуля: main резервирует связанные места для будущих результатов decoder-а.

Какие функции выполняют шаг. Стандартная функция calloc вызывается из main.c. Первый массив имеет тип UnwindSummary, второй - bool.

Где объявлены функции и типы. calloc является функцией стандартной библиотеки C и объявляется через ; отдельного проектного .h для этого шага нет. Тип UnwindSummary объявлен в include/unwind.h:25-78.

Где искать реализацию и вызов. src/main.c:328-340 - два calloc и cleanup при нехватке памяти; :342-346 - создание общих UnwindParseOpts.

Как меняются данные. До шага результатов нет. После шага для каждого i зарезервированы us[i] и us_ok[i], изначально заполненные нулями calloc.

Что считается ошибкой. Если один из calloc вернул NULL, main освобождает второй массив, rf, PeImage, buffers и завершает работу.

Как результат используется дальше. unwind_parse заполняет us[i], а main записывает возвращённый bool в us_ok[i].

Смысл шага: сохранить однозначное соответствие индекс i: RuntimeFunction -> результат/статус разбора.

Шаг 6. Декодировать UNWIND_INFO

Цикл main.c вызывает unwind_parse для rf[i].unwind_rva. Decoder учитывает slots, operands, padding, handler tail и CHAININFO и сохраняет summary либо диагностику ошибки.

Как объяснить этот шаг на защите. Затем для каждой функции анализатор переходит по адресу UNWIND_INFO и декодирует её содержимое. Он читает версию, флаги, размер пролога, число двухбайтовых слотов и отдельные unwind-операции; учитывает дополнительные слоты сложных операций, выравнивание, адрес обработчика исключений и цепочки CHAININFO. Одновременно контролируются границы файла и логические ограничения формата. В результате компактная бинарная запись превращается в понятную структурированную модель. Ошибка одной функции фиксируется локально и не прекращает анализ остальных функций.

Что происходит внутри. unwind_parse получает PeImage, RVA конкретной UNWIND_INFO и параметры безопасного разбора. Сначала RVA переводится в file offset и читается четырёхбайтовый заголовок. Затем декодируются CountOfCodes двухбайтовых слотов. Одна логическая операция UWOP может занимать один, два или три слота, поэтому slots_for_code определяет её реальный размер. После массива учитывается выравнивание до четырёхбайтовой границы.

Как интерпретируется хвост. Флаги определяют назначение данных после unwind-кодов. При EHANDLER или UHANDLER там ожидается RVA обработчика; при CHAININFO - связанная RUNTIME_FUNCTION-подобная запись. Decoder проверяет допустимость версии, opcode и OpInfo, CodeOffset, дополнительные операнды, границы каждого чтения и ограниченность цепочки. Он не выполняет универсальное декодирование всех language-specific data, потому что их формат зависит от конкретного обработчика и runtime языка.

Что передаётся дальше. Успешный результат содержит структурированную сводку: параметры пролога, операции, вычисленный эффект на стек, используемые регистры, handler или chain и локальные аномалии. При ошибке сохраняется место и причина остановки, а цикл продолжает обработку остальных записей. Поэтому итоговая карта остаётся полезной даже для частично повреждённого файла.

Какой модуль отвечает за шаг. Предметный исполнитель - src/unwind.c; цикл по всем функциям и сохранение bool-статуса организует src/main.c.

Какие функции выполняют шаг. unwind_parse в src/unwind.c; внутри работают parse_unwindinfo_header, slots_for_code, read_u16_slot, parse_chain_rf и diagnostic helpers - функции формирования диагностики.

Где объявлены функции и типы. Публичное объявление unwind_parse: include/unwind.h:107-114. UnwindParseOpts объявлена в include/unwind.h:94-98.

Где искать реализацию и вызов. src/main.c:351-361 - цикл и связь rf[i] -> us[i] -> us_ok[i]; src/unwind.c:258-763 - unwind_parse; внутренние helpers: slots_for_code :24, parse_unwindinfo_header :56, read_u16_slot :75, parse_chain_rf :223.

Как меняются данные. До шага unwind_rva является только адресом. После шага us[i] содержит Version, Flags, prolog size, count codes, stack effect, registers, handler/chain, anomalies и parse diagnostics.

Что считается ошибкой. Ошибка одной UNWIND_INFO не останавливает цикл. us_ok[i] становится false, а диагностика объясняет, на каком поле и каких границах остановился decoder.

Как результат используется дальше. heuristics.c и report.c используют одинаковый индекс для связи function, summary и статуса разбора.

Смысл шага: превратить компактное бинарное кодирование в проверяемую структурированную модель.

Шаг 7. Построить статическую интерпретацию

heuristics_analyze получает готовые модели, вычисляет 27 признаков, создаёт findings с evidence, применяет weights/mitigations и формирует HeuristicReport.

Как объяснить этот шаг на защите. После структурного разбора начинает работать эвристический слой. Он не перечитывает исходные байты, а исследует уже проверенную карту: ищет нарушения границ и порядка, пересечения диапазонов, необычное размещение UNWIND_INFO, проблемы обработчиков, цепочек и другие предусмотренные признаки. Для каждого срабатывания создаётся finding - структурированное аналитическое наблюдение с причиной, адресом и доказательными данными. Затем признаки агрегируются в оценку, уровень уверенности и итоговую категорию. Это не автоматическое объявление файла вредоносным, а приоритизация участков, которые аналитику следует проверить вручную.

Что происходит внутри. heuristics_analyze получает PeImage, RuntimeFunction[], UnwindSummary[] и us_ok[]. Он проверяет предусмотренные проектом признаки на нескольких уровнях: целостность PE-координат, корректность диапазонов функций, синтаксис UNWIND_INFO, размещение данных и обработчиков, повторное использование структур и согласованность связанных записей. Каждое установленное наблюдение оформляется через add_finding, а не остаётся безымянным числом.

Как формируется оценка. Finding содержит код правила, семейство, серьёзность, местоположение и evidence - конкретные данные, на которых основано срабатывание. add_score применяет инженерный вес правила; add_mitigation учитывает обстоятельства, ослабляющие интерпретацию; finalize_verdict формирует итоговую категорию и confidence. Вес является свойством реализации анализатора, а не нормативным числом из документации Microsoft.

Как это объяснять комиссии. Эвристика не доказывает вредоносность и не заменяет аналитика. Она автоматизирует однообразную проверку большого числа структур, локализует отклонения и ранжирует их для ручного исследования. Высокая severity означает потенциальную значимость наблюдения, а высокая confidence - надёжность доказательства его наличия; эти характеристики отвечают на разные вопросы.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/heuristics.c. Он получает готовые модели и не перечитывает входной файл самостоятельно.

Какие функции выполняют шаг. heuristics_analyze в src/heuristics.c; add_finding создаёт finding, add_score начисляет балл, add_mitigation фиксирует ослабляющий фактор, finalize_verdict строит итог.

Где объявлены функции и типы. Публичное объявление heuristics_analyze: include/heuristics.h:95-101. HeuristicReport объявлена в include/heuristics.h:42-93.

Где искать реализацию и вызов. src/heuristics.c:590-1038 - heuristics_analyze; внутренние этапы: add_score :359, add_finding :408, add_mitigation :480, finalize_verdict :514; вызов из main - src/main.c:388-399.

Как меняются данные. До шага имеются низкоуровневые факты. После шага HeuristicReport содержит findings, evidence families, score, confidence, verdict и mitigations.

Что считается ошибкой. Нехватка памяти при построении отчёта эвристик возвращает false. Обычное наличие findings не является технической ошибкой.

Как результат используется дальше. report.c использует HeuristicReport как готовую аналитическую модель и не пересчитывает правила.

Смысл шага: перейти от отдельных наблюдений к объяснимой приоритизации ручной проверки.

Шаг 8. При необходимости выполнить динамический эксперимент

На Windows dynamic_collect_windows запускает target под debugger, ставит breakpoints, фиксирует exceptions, handler/API hits, dynamic table registrations и RIP trace в DynamicReport.

Как объяснить этот шаг на защите. Если выбран динамический режим и программа запущена в Windows, анализатор запускает исследуемый файл под управлением отладочного механизма. Статическая часть заранее предоставляет известные адреса обработчиков, а динамический модуль наблюдает реальные исключения, вызовы важных Windows API, регистрацию динамических таблиц функций и изменение адреса выполняемой инструкции. Так проверяется не только то, что записано в файле, но и то, какие механизмы действительно проявились в конкретном запуске. Отсутствие события означает лишь, что оно не наблюдалось в данном сценарии, а не что такой путь принципиально невозможен.

Что происходит внутри. dynamic_collect_windows создаёт исследуемый процесс в режиме отладки и получает от Windows последовательность debug events - событий запуска, загрузки модулей, исключений и завершения. После определения фактической базы загрузки модуля статический RVA преобразуется в реальный виртуальный адрес как base + RVA. На интересующие адреса устанавливаются программные точки останова, временно заменяющие первый байт инструкцией 0xCC.

Какие две группы адресов наблюдаются. Первая группа поступает из статического анализа: это адреса unwind-handler-ов, достижение которых фиксируется как handler hit. Вторая группа относится к Windows API: регистрации VEH/VCH/UEF и динамических function tables. При входе в соответствующий API модуль читает регистры вызова AMD64, извлекает переданный адрес обработчика, фильтра или таблицы и при возможности устанавливает следующую точку останова уже на этот адрес.

Как понимать результат. DynamicReport хранит только фактически наблюдавшиеся события данного запуска: исключения, точки срабатывания, вызовы API, регистрации таблиц и ограниченную последовательность RIP после интересного события. Динамика подтверждает либо дополняет статическую гипотезу, но не исследует неисполненные ветви. Поэтому отсутствие hit нельзя интерпретировать как доказательство отсутствия механизма во всех возможных сценариях.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/dynamic_win.c; публичный контракт находится в include/dynamic.h. main.c запускает ветвь только по --dynamic.

Какие функции выполняют шаг. dynamic_collect_windows в src/dynamic_win.c; build_cmdline готовит команду, bp_set/bp_clear управляют breakpoints, dynamic_event_add и add_trace сохраняют наблюдения.

Где объявлены функции и типы. Публичное объявление dynamic_collect_windows: include/dynamic.h:113-125. DynamicEvent и DynamicReport объявлены в include/dynamic.h:28-96.

Где искать реализацию и вызов. Windows-реализация dynamic_collect_windows: src/dynamic_win.c:657-1209; non-Windows stub: :1228-1250. Внутри: bp_set :200, bp_clear :215, dynamic_event_add :416, build_cmdline :500, add_trace :617. Вызов из main: src/main.c:401-425.

Как меняются данные. До шага известны потенциальные адреса. После конкретного запуска DynamicReport содержит фактические debug events, API/handler hits, dynamic table calls и RIP trace.

Что считается ошибкой. Недоступная платформа, CreateProcess failure, timeout или неполная установка breakpoints отражаются в статусе и exit_code; они не заменяются фиктивными runtime данными.

Как результат используется дальше. report.c сопоставляет DynamicReport со статикой, но помечает отсутствие или неполноту runtime evidence.

Смысл шага: проверить, какие статические гипотезы проявляются в конкретном runtime execution.

Шаг 9. Сформировать provenance

main.c вычисляет SHA-256 входа, UTC timestamp, безопасное отображение команды и сведения о сборке. Так результат связывается с конкретными bytes и версией инструмента.

Как объяснить этот шаг на защите. Чтобы результат можно было воспроизвести и проверить, программа формирует сведения о происхождении отчёта. Она вычисляет SHA-256 исследованного файла, записывает его размер, время анализа, параметры запуска, платформу, компилятор и режим сборки инструмента. Благодаря этому отчёт можно однозначно связать с конкретной версией входных байтов и конкретными условиями эксперимента. Если файл позднее изменится хотя бы на один байт, его контрольная сумма, как правило, станет другой.

Что происходит внутри. После анализа main собирает ReportMetadata - метаданные самого исследования. sha256_hex вычисляет криптографический отпечаток входного файла; app_utc_timestamp фиксирует время в единой временной шкале UTC; app_info сообщает версию схемы отчёта, платформу, компилятор и режим сборки; build_invocation формирует безопасное текстовое представление команды.

Почему одной фамилии файла недостаточно. Два разных файла могут иметь одинаковое имя, а один файл может измениться между запусками. SHA-256 связывает вывод с точной

последовательностью байтов, а параметры запуска объясняют, почему два эксперимента могли дать разные динамические события. Версия схемы сообщает backend-у или другому потребителю, как интерпретировать поля JSON.

Что передаётся дальше. ReportMetadata поступает в отчётный слой вместе со статическими и динамическими моделями. В результате читатель видит не обезличенный набор срабатываний, а проверяемое свидетельство: что именно исследовалось, каким инструментом, когда и в каком режиме.

Какой модуль отвечает за шаг. Координатор - src/main.c; SHA-256 реализует src/util.c, сведения о версии/сборке/времени - src/app_info.c.

Какие функции выполняют шаг. sha256_hex из util.c, app_utc_timestamp и app_build_* из app_info.c, build_invocation и argv_text_to_utf8 из main.c.

Где объявлены функции и типы. sha256_hex объявлена в include/util.h:66; app_version/app_report_schema_version/app_build_*/app_utc_timestamp - include/app_info.h:13-20. build_invocation и argv_text_to_utf8 являются static и в .h не публикуются.

Где искать реализацию и вызов. src/util.c:333-347 - sha256_hex; src/app_info.c:6-68 - версия, схема, compiler/platform/mode и UTC; src/main.c:56 - argv_text_to_utf8, :132 - build_invocation, :450-480 - сборка ReportMetadata.

Как меняются данные. После шага ReportMetadata содержит версию, schema version, UTC-время, SHA-256, размер файла, команду, platform, compiler и build mode.

Что считается ошибкой. Если timestamp, SHA-256 или invocation не созданы, JSON/HTML не записываются как будто provenance полный.

Как результат используется дальше. JSON и HTML writers получают metadata вместе с аналитическими моделями.

Смысл шага: обеспечить воспроизводимость и проверяемость происхождения результата.

Шаг 10. Сериализовать результаты

report.c преобразует одну внутреннюю модель в выбранные представления. JSON предназначен для backend, HTML - для аналитика, CSV - для таблиц, Ghidra CSV - для импорта разметки.

Как объяснить этот шаг на защите. Когда внутренние модели готовы, модуль отчётности преобразует их в требуемые внешние форматы. JSON предназначен для программной обработки и последующей интеграции, HTML - для изучения человеком в браузере, CSV - для табличного анализа, а специальный CSV - для переноса разметки в Ghidra. Все представления создаются из одной и той же модели, поэтому модуль отчёта не должен заново анализировать файл или изменять оценки. Его задача - полно и однозначно показать уже полученный результат.

Что происходит внутри. report.c последовательно сериализует внутренние структуры. Сериализация означает преобразование объектов программы в устойчивое файловое представление. JSON сохраняет иерархию и типы данных для программной интеграции; HTML организует те же сведения в визуальные разделы; CSV представляет выбранные сущности строками и столбцами; Ghidra CSV содержит координаты, пригодные для импорта разметки.

Почему отчётный модуль отделён. Он не имеет права менять score, повторно декодировать UNWIND_INFO или создавать новые факты. Аналитика заканчивается до начала сериализации.

Такое разделение гарантирует, что разные форматы являются представлениями одного результата, а не независимыми анализами с потенциально различающимися выводами.

Как обеспечивается целостность файла. Готовое содержимое записывается через file_write_atomic. Сначала создаётся временный файл, и только после успешной записи он заменяет итоговый. Если операция прервётся, пользователь не должен получить обрезанный HTML или JSON, который выглядит завершённым.

Какой модуль отвечает за шаг. Главный ответственный модуль - src/report.c. Физическую атомарную запись готовой строки выполняет src/util.c.

Какие функции выполняют шаг. report_write_csv, report_write_json_ex и report_write_html из report.c. file_write_atomic из util.c завершает физическую запись.

Где объявлены функции и типы. report_write_csv/report_write_json_ex/report_write_html объявлены в include/report.h:35-65. file_write_atomic объявлена в include/util.h:63.

Где искать реализацию и вызов. src/report.c:593 - report_write_csv; :1292 - report_write_json_ex; :3014 - report_write_html; src/util.c:190-231 - file_write_atomic. Точки вызова: src/main.c:427-510.

Как меняются данные. После шага внутренние структуры превращены в файлы, которые можно открыть, передать backend или загрузить в Ghidra.

Что считается ошибкой. Invalid metadata/model, нехватка памяти или ошибка диска приводят к false. Atomic write предотвращает появление частичного итогового файла.

Как результат используется дальше. Пользователь, backend или Ghidra работают уже с отчётом, не обращаясь напрямую к внутренней памяти процесса анализатора.

Смысл шага: выдать данные человеку и внешним системам без повторной интерпретации байтов.

Шаг 11. Зафиксировать технический статус

Ошибка output или dynamic session меняет exit_code. Наличие finding само по себе не обязано делать exit code ненулевым: аналитическое наблюдение не равно техническому сбою программы.

Как объяснить этот шаг на защите. После формирования результатов программа определяет технический статус запуска. Здесь принципиально разделяются две ситуации. Обнаруженная аномалия означает, что анализ успешно выявил объект для проверки; это нормальный аналитический результат. Техническая ошибка означает, что запрос не удалось корректно выполнить, например файл не прочитан, отчёт не записан или динамическая сессия не запущена. Код завершения позволяет внешней системе отличить исправную работу анализатора от сбоя, не смешивая этот вопрос с подозрительностью исследуемого файла.

Что происходит внутри. main хранит переменную exit_code и меняет её при технических отказах: невозможности прочитать поддерживаемый вход, построить обязательную модель, запустить запрошенную динамическую сессию или записать отчёт. В конце это число возвращается операционной системе. Значение 0 означает технически успешное выполнение запроса, 1 - ошибку исполнения, 2 - неверное использование командного интерфейса.

Чем это отличается от verdict. Verdict находится внутри аналитического отчёта и характеризует исследуемый файл. Exit code характеризует работу самого инструмента. Файл с многочисленными аномалиями может быть успешно проанализирован с кодом 0. И наоборот, заведомо нормальный файл может дать код 1, если, например, не удалось записать

запрошенный отчёт.

Почему разделение важно. Backend, сценарий автоматизации или система непрерывного тестирования могут отдельно проверить исправность запуска по exit code и отдельно прочитать аналитический результат из JSON. Это предотвращает ложное представление, будто любое срабатывание является аварией программы.

Какой модуль отвечает за шаг. Ответственный модуль - src/main.c. Именно main отличает технический failure от аналитического finding и накапливает exit_code.

Какие функции выполняют шаг. Переменная exit_code в main.c аккумулирует технические failures - сбои. Findings остаются данными анализа и сами по себе не меняют exit code.

Где объявлены функции и типы. exit_code - локальная переменная main, поэтому объявления в .h нет. Её создание следует смотреть в src/main.c:272.

Где искать реализацию и вызов. Смотреть все присваивания exit_code = 1 в src/main.c:323, 396, 413, 432, 443, 466, 492 и 506; окончательный return находится в src/main.c:526. Ошибка CLI возвращает 2 на строке 280.

Как меняются данные. После шага операционная система получает 0 при технически успешном запуске, 1 при ошибке ввода/анализа/вывода или dynamic session и 2 при ошибке CLI.

Что считается ошибкой. Ненулевой exit code отвечает на вопрос «исправно ли выполнен запрос», а verdict - на вопрос «какие наблюдения обнаружены». Эти вопросы не смешиваются.

Как результат используется дальше. Shell, CI или backend могут проверить exit code отдельно от содержимого JSON verdict.

Смысл шага: разделить аналитический вывод и исправность запуска инструмента.

Шаг 12. Освободить ресурсы

main.c освобождает metadata strings, reports, arrays, PeImage, StrBuf и owned bytes в обратном порядке. После этого процесс возвращает exit code операционной системе.

Как объяснить этот шаг на защите. В заключение программа освобождает всю память и закрывает ресурсы, созданные во время анализа. Освобождение выполняется в контролируемом порядке: сначала удаляются отчётные и динамические структуры, затем массивы функций и результатов, PE-карта, строки и исходный буфер файла. Это обязательная часть корректной программы на языке C, поскольку память не освобождается автоматически. После завершения этого шага анализ одного файла закончен, а операционная система получает итоговый технический код выполнения.

Что происходит внутри. Все созданные во время запуска объекты имеют владельца. heuristics_free освобождает внутренние массивы эвристического отчёта, dynamic_report_free - события динамической сессии, pe_free - массив секций, sb_free - динамические строки, buf_free_owned - исходный снимок файла. Обычные массивы освобождаются стандартной функцией free. main вызывает эти операции централизованно.

Почему порядок имеет значение. Структура может содержать указатели на другие выделенные области. Сначала освобождаются вложенные и производные данные, затем базовые массивы и входной буфер. После free указатель больше нельзя разыменовывать: память уже не принадлежит программе. Повторное освобождение той же области также является ошибкой.

Чем завершается прогон. После очистки main возвращает накопленный exit_code, процесс завершается, а созданные отчёты остаются на диске. Таким образом, полный маршрут одного файла имеет определённое начало, контролируемую передачу данных между модулями и корректно закрытый жизненный цикл ресурсов.

Какой модуль отвечает за шаг. Координатор освобождения - src/main.c. Каждый владеющий модуль предоставляет свою free-функцию, а обычные массивы освобождаются стандартной free.

Какие функции выполняют шаг. heuristics_free, dynamic_report_free, free, pe_free, sb_free и buf_free_owned. Каждая освобождает только тот ресурс, которым владеет соответствующая структура.

Где объявлены функции и типы. heuristics_free: include/heuristics.h:103; dynamic_report_free: include/dynamic.h:99; pe_free: include/pe.h:57; sb_free и buf_free_owned: include/util.h:57 и 21. Стандартная free объявляется в .

Где искать реализацию и вызов. Центральный normal cleanup: src/main.c:512-526. Реализации: heuristics_free - src/heuristics.c:1040; dynamic_report_free - src/dynamic_win.c:457 для Windows и :1222 для stub-ветви; pe_free - src/pe.c:259; sb_free - src/util.c:109; buf_free_owned - src/util.c:38.

Как меняются данные. После шага динамически выделенные области больше не принадлежат программе; процесс не должен к ним обращаться.

Что считается ошибкой. Повторный free или использование указателя после free являются ошибками C. Поэтому cleanup централизован и выполняется один раз.

Как результат используется дальше. Последним действием main является return exit_code; жизненный цикл одного запуска полностью завершён.

Смысл шага: закрыть жизненный цикл памяти и файлов без утечек и висячих указателей.

Итоговая модель проекта

Проект представляет собой модульный монолит с библиотечным ядром. Его основная ценность заключается не в отдельном parser-вызове, а в сохранении доказательной цепочки от конкретных байтов PE до локализованного finding и, при наличии Windows runtime experiment, до динамического события.

```
bytes
-> parsed fields
-> validation counters
-> RuntimeFunction / UnwindSummary
-> findings and evidence families
-> score and confidence
-> static/dynamic assessment
-> manual validation
```

Каждая стрелка обозначает отдельную границу ответственности. util.c отвечает за безопасный доступ, pe.c - за координаты, unwind.c - за синтаксис EH/Unwind, heuristics.c - за аналитическую агрегацию, dynamic_win.c - за runtime evidence, report.c - за объяснимое внешнее представление, app_info.c - за provenance, main.c - за правильный порядок и жизненный цикл.

Как формируется аналитический вывод

Finding, evidence, severity, score, confidence, mitigation и verdict

Эта глава объясняет заключительную часть статического анализа. После pe.c и unwind.c проект уже располагает проверенной картой PE/EH/Unwind. heuristics.c не читает файл заново: он получает PeImage, RuntimeFunction[], UnwindSummary[] и us_ok[] и преобразует низкоуровневые факты в структурированный результат для аналитика.

```
поля и счётчики
-> условие эвристического правила
-> finding
-> evidence families и severity
-> score и suspicion
-> confidence и mitigations
-> evidence level, verdict и manual review
```

Важно: это последовательность разных понятий, а не несколько названий одного процента. Каждое поле отвечает на отдельный вопрос.

1. От факта к finding

Сначала проект устанавливает наблюдаемый факт. Например, две записи RUNTIME_FUNCTION имеют пересекающиеся диапазоны, UNWIND_INFO невозможно полностью прочитать либо handler расположен в записываемой секции. Сам факт хранится в счётчике и может быть привязан к конкретному RVA.

Когда условие правила выполнено, heuristics_analyze вызывает add_finding. Finding переводится как «обнаруженное аналитически значимое наблюдение». Это не окончательный диагноз файла и не утверждение о вредоносности. Это оформленная причина, по которой определённый участок следует показать аналитику.

Поле	Что хранит и зачем это нужно
`code`	Стабильный машинный код правила, например rf_overlaps. По нему backend, JSON-потребитель или тест отличает одно правило от другого независимо от русского текста.
`family`	Семейство доказательств - уровень или тип механизма, к которому относится наблюдение.
`severity`	Потенциальная серьёзность наблюдения, если оно действительно присутствует и существенно для исследуемого сценария.
`message`	Человекочитаемое описание того, что установил анализатор.
`has_location`	Признак того, что finding удалось привязать к конкретному RVA.
`representative_rva`	Представительный адрес, с которого удобно начинать ручную проверку.
`affected_count`	Число функций, записей или других объектов, затронутых правилом.

add_finding одновременно сохраняет структуру в findings[], добавляет текстовую строку в details и устанавливает бит семейства в evidence_families_mask. annotate_finding позднее добавляет location и affected_count. В текущей структуре хранится до 32 findings; это технический предел массива, а не число правил.

2. Почему 27 правил не означают 27 findings в каждом отчёте

В текущей реализации присутствуют 27 стабильных кодов finding. Это 27 возможных условий срабатывания, а не обязательные 27 строки в каждом отчёте. Нормальный файл может не активировать ни одного правила. Один файл может активировать несколько правил, причём одно правило агрегирует множество однотипных объектов через affected_count.

Например, если обнаружено десять пересекающихся диапазонов, проект создаёт finding rf_overlaps с affected_count = 10, а не десять несвязанных сообщений. Так отчёт остаётся обозримым, но не теряет масштаб проблемы.

В коде правила сгруппированы вокруг целостности PE и RUNTIME_FUNCTION, декодирования UNWIND_INFO, размещения metadata и handler-ов, повторного использования UNWIND_INFO, доминирования обработчика и корреляционных комбинаций. Комбинационное правило срабатывает, когда несколько разных наблюдений образуют более значимую совокупность.

Следовательно, finding_count отвечает на вопрос «сколько видов значимых наблюдений сработало», а affected_count внутри finding отвечает на вопрос «сколько объектов затронул этот вид наблюдения».

3. Evidence и семейства доказательств

Evidence означает конкретное доказательное основание finding: поля структуры, адреса, диапазоны, секции, счётчики и результаты безопасного разбора. Семейство evidence показывает, из какой независимой части карты получен сигнал. Проект хранит семейства как биты evidence_families_mask, поэтому одновременно могут быть отмечены несколько семейств.

Поле	Что хранит и зачем это нужно
`table_integrity`	Целостность PE-секций, Exception Directory и таблицы RUNTIME_FUNCTION: границы, порядок, пересечения, дубликаты и доступность адресов.
`unwind_integrity`	Возможность корректно декодировать UNWIND_INFO и соблюдение проверяемых структурных и ABI-ограничений.
`metadata_placement`	Размещение UNWIND_INFO и handler-ов по секциям и правам read/write/execute.
`unwind_reuse`	Степень повторного использования одного UnwindInfoAddress множеством функций.
`handler_dominance`	Доля функций с обработчиком, которые ссылаются на один доминирующий handler.

Несколько независимых семейств повышают уверенность сильнее, чем несколько вариаций одного и того же сигнала. Поэтому проект отдельно считает число установленных битов, а не только общее количество findings.

4. Severity: насколько значимо наблюдение

Severity переводится как «серьёзность». Она отвечает на вопрос: насколько существенными могут быть последствия finding, если наблюдение корректно и относится к важному пути программы. Severity не показывает, насколько анализатор уверен в самом наблюдении. Для уверенности существует отдельное поле confidence.

Поле	Что хранит и зачем это нужно
`high / critical`	Сильное наблюдение. В текущих правилах реально создаётся high; finalize_verdict также умеет учитывать critical как сильное значение.
`medium`	Средняя потенциальная значимость: отклонение требует внимания, но чаще нуждается в контексте и корреляции.
`info / другие`	Слабое либо информационное наблюдение. При сводном подсчёте всё, что не high, critical или medium, относится к weak.

Пример различия: точное обнаружение небольшой misalignment может иметь высокую достоверность факта, но среднюю серьёзность. Напротив, редкое необычное размещение может иметь потенциально важное последствие, однако без динамической или ручной проверки уверенность интерпретации остаётся ограниченной.

5. Score: инженерная сумма весов

Score - целочисленная инженерная оценка от 0 до 100. При срабатывании правила add_score прибавляет заранее заданный вес, после чего clamp_int ограничивает результат диапазоном 0..100. Поэтому score выше 100 невозможен, даже если сумма отдельных вкладов больше.

Вес не является вероятностью, статистически обученным коэффициентом или нормативным числом Microsoft. Это прозрачная консервативная калибровка реализации: нарушения границ и целостности получают более сильные веса, размещение - средние или сильные в зависимости от распространённости, reuse и dominance - умеренные, а сочетания независимых проблем дают дополнительный вклад.

Поле	Что хранит и зачем это нужно
`25 баллов`	Выход raw section или Exception Directory за границы, RF out of bounds при высокой распространённости, unwind RVA out of bounds при высокой распространённости, массовый parse failure, пересечение RUNTIME_FUNCTION.
`20 баллов`	Virtual section out of bounds/overlap, много invalid ranges, высокая доля unwind anomalies, сильный unwind reuse, размещение UNWIND_INFO в executable section.
`15 баллов`	Raw section overlap, несортированная RUNTIME_FUNCTION table, handler dominance, handler в non-executable section, широко распространённая необычная секция.
`8-10 баллов`	Пустая .pdata, единичные или малораспространённые нарушения, misalignment, умеренный reuse и некоторые placement-сигналы.
`8-10 дополнительных`	Корреляционные комбинации: размещение плюс целостность, reuse плюс dominance, writable metadata плюс writable handler и другие предусмотренные сочетания.

Один finding не обязательно равен одному фиксированному весу: для ряда правил вклад зависит от доли затронутых записей. Например, единичная ошибка и систематическая ошибка получают разные веса.

6. Suspicion: нормированное представление score

Suspicion вычисляется строго как score / 100. Если score равен 35, suspicion равна 0.35. Это удобно для JSON и интерфейсов, которым требуется шкала 0..1.

Suspicion нельзя называть вероятностью вредоносности или точностью модели. Проект не обучает статистический классификатор и не располагает калиброванным утверждением «0.35 означает 35 процентов вредоносности». Корректная формулировка: это нормированная инженерная степень статической подозрительности по правилам проекта.

```
score = 35
suspicion = score / 100 = 0.35

Неверно: вероятность вредоносности 35%
Верно: нормированная эвристическая оценка 0.35
```

7. Confidence: насколько убедительна доказательная база

Confidence переводится как «уверенность». Она отвечает не на вопрос, насколько файл подозрителен, а на вопрос, насколько сильна, разнообразна и непротиворечива доказательная база статического вывода. Поэтому score и confidence могут двигаться по-разному.

Расчёт начинается с базы 0.15. Добавляется 0.11 за каждое независимое evidence family, 0.08 за каждый strong finding и 0.04 за каждый medium finding. Жёсткие нарушения таблицы и жёсткие нарушения unwind дополнительно дают по 0.08. При score = 0 используется отдельная база 0.20. После бонусов применяются ослабляющие правила, а результат ограничивается диапазоном 0.05..0.90.

```
confidence = 0.15
+ 0.11 * число семейств
+ 0.08 * число strong findings
+ 0.04 * число medium findings
+ hard-table / hard-unwind bonuses
- mitigation penalties

после расчёта: 0.05 <= confidence <= 0.90
```

Высокий score при низком confidence означает: потенциально серьёзные сигналы есть, но их доказательная база изолирована или ограничена. Низкий score при сравнительно высокой confidence означает: проект достаточно уверен, что сильных статических аномалий не обнаружено в пределах реализованной модели, но это не доказывает полную безопасность файла.

8. Mitigation: почему вывод следует ослабить

Mitigation означает ослабляющее обстоятельство. Оно не удаляет finding и не утверждает, что finding ложный. Оно фиксирует причину, по которой итог нельзя интерпретировать максимально жёстко без дополнительной проверки.

Поле	Что хранит и зачем это нужно
`single_signal_only`	Имеется только один не-сильный сигнал. Confidence уменьшается на 0.12.
`small_sample_size`	В .pdata меньше 32 записей, нет strong finding, но findings имеются. Отношения на малой выборке могут быть завышены; штраф 0.06.
`placement_only_signal`	Есть только необычное размещение без исполняемых/записываемых секций и без нарушений целостности. Возможна редкая нормальная компоновка; штраф 0.10.
`single_family_only`	Все наблюдения принадлежат одному семейству и нет strong finding. Требуется межслойное подтверждение; штраф 0.08.

Mitigation влияет на confidence, но в текущей реализации напрямую не вычитает баллы из score. Это осознанное разделение: факт подозрительности сохраняется, а уверенность в широкой интерпретации уменьшается.

9. Evidence level

Evidence level - словесная сводка силы набора findings. Он вычисляется по количеству сильных и средних наблюдений и по score.

Поле	Что хранит и зачем это нужно
`strong`	Не менее двух strong findings или score не ниже 70.
`medium`	Хотя бы один strong finding, не менее двух medium findings или score не ниже 30.
`weak`	Есть хотя бы один finding либо score выше нуля, но условия medium/strong не выполнены.
`none`	Findings отсутствуют и score равен нулю.

Evidence level и verdict не дублируют друг друга. Evidence level описывает силу доказательного набора, а verdict - диапазон итоговой подозрительности по score.

10. Verdict и requires_manual_review

Verdict - итоговая текстовая категория статической оценки. Она вычисляется только после завершения всех правил, подсчёта score, confidence и mitigations.

Поле	Что хранит и зачем это нужно
`high_suspicion`	score от 70 до 100.
`moderate_suspicion`	score от 40 до 69.
`low_suspicion`	score от 15 до 39.
`no_strong_static_anomalies`	score от 0 до 14. Формулировка намеренно не означает «файл безопасен».

requires_manual_review устанавливается, если score не ниже 20, обнаружены опасные варианты размещения metadata/handler, имеются parse failures либо хотя бы одна mitigation. Поэтому ручная проверка может быть рекомендована даже при невысоком score: например, когда данных недостаточно для уверенной автоматической интерпретации.

Вердикт не является диагнозом вредоносности. Он сообщает, насколько статическая EH/Unwind-карта отклоняется от ожидаемой модели и насколько приоритетно аналитику следует исследовать найденные участки.

11. Полный расчётный пример

Предположим, проект обнаружил пересечение диапазонов RUNTIME_FUNCTION и небольшую долю UNWIND_INFO, которые невозможно разобрать. Первое правило создаёт high finding rf_overlaps и добавляет 25 баллов. Второе создаёт medium finding unwind_parse_fail и добавляет 10 баллов. Итоговый score равен 35, suspicion равна 0.35.

```
score = 25 + 10 = 35
suspicion = 35 / 100 = 0.35
verdict = low_suspicion, потому что 15 <= 35 < 40
evidence_level = medium, потому что есть один strong finding
```

Отмечены два семейства: table_integrity и unwind_integrity. Confidence начинается с 0.15. Два семейства добавляют 0.22, strong finding добавляет 0.08, medium finding - 0.04, жёсткое нарушение таблицы - 0.08, жёсткое нарушение unwind - ещё 0.08. Получается 0.65. Ослабляющие правила не применяются: сигналов больше одного, присутствует strong finding и evidence получен из двух семейств.

```
confidence = 0.15 + 0.22 + 0.08 + 0.04 + 0.08 + 0.08 = 0.65
```

Как читать результат: суммарная подозрительность пока относится к диапазону low_suspicion, но доказательство обнаруженных структурных проблем достаточно убедительно. Аналитику следует начать с representative RVA пересекающихся функций и диагностик неразобранных UNWIND_INFO. Это хороший пример того, почему verdict, severity и confidence нельзя заменять одним словом «опасно».

12. Пример слабого изолированного сигнала

Пусть в небольшом файле UNWIND_INFO расположена в необычно названной, но не исполняемой и не записываемой секции. Других нарушений нет. Правило добавляет 8 баллов и один medium finding семейства metadata_placement.

```
score = 8
suspicion = 0.08
verdict = no_strong_static_anomalies
evidence_level = weak
```

Предварительная confidence: 0.15 + 0.11 за семейство + 0.04 за medium finding = 0.30. Затем действуют single_signal_only (-0.12), small_sample_size (-0.06), placement_only_signal (-0.10) и single_family_only (-0.08). Расчёт уходит ниже минимума, поэтому confidence ограничивается значением 0.05.

```
0.30 - 0.12 - 0.06 - 0.10 - 0.08 = -0.06
после ограничения confidence = 0.05
```

Finding сохраняется, но отчёт прямо показывает, что одиночное отклонение размещения на малой выборке не позволяет делать сильный вывод. requires_manual_review остаётся true из-за mitigations. Это механизм снижения ложной категоричности, а не механизм сокрытия наблюдения.

13. Как объяснить результат комиссии

Последовательность ответа должна сохранять смысл каждого уровня.

```
1. Проект установил конкретный структурный факт.
2. Факт оформлен как finding с кодом, семейством, серьёзностью и адресом.
3. Вес finding внёс вклад в score.
4. Score показывает инженерную степень подозрительности, но не вероятность malware.
5. Confidence показывает силу и разнообразие доказательств.
6. Mitigations уменьшают категоричность при изолированных сигналах.
7. Verdict задаёт диапазон score.
8. Manual review сообщает, требуется ли проверка аналитика.
```

Корректная формулировка: «Инструмент обнаружил конкретные нарушения EH/Unwind-карты, локализовал их, оценил потенциальную серьёзность, агрегировал прозрачными весами и отдельно оценил уверенность с учётом разнообразия доказательств и ослабляющих факторов».

Некорректная формулировка: «Проект с вероятностью 73 процента определил вредоносный файл». Код не выполняет такую статистическую классификацию, а confidence и suspicion имеют другое назначение.

Где искать реализацию: HeuristicFinding и HeuristicReport объявлены в include/heuristics.h:27-93; add_score - src/heuristics.c:359; add_finding - :408; add_mitigation - :480; evidence level - :504; finalize_verdict - :514; полный проход heuristics_analyze - :590.